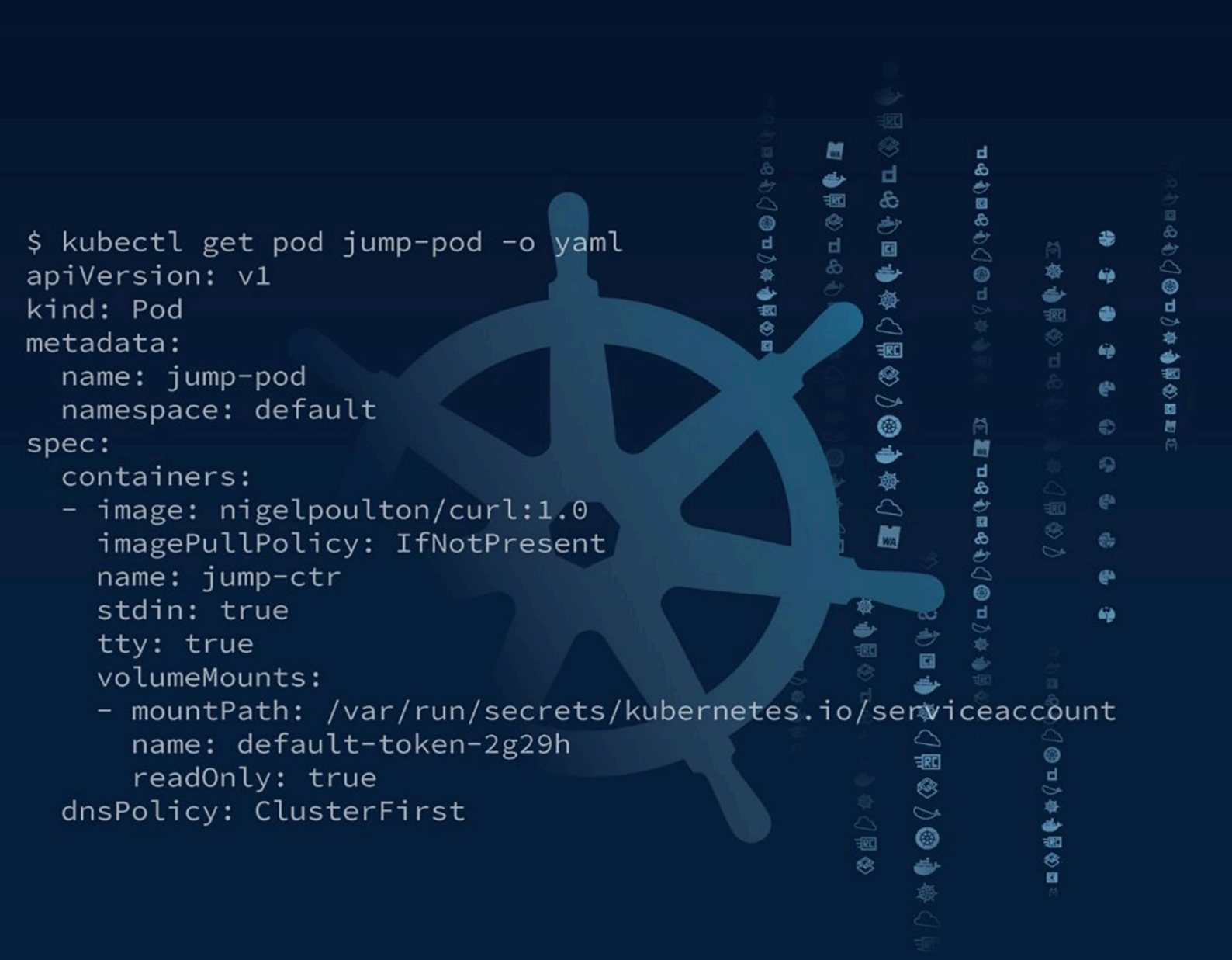


THE KUBERNETES BOOK

2025 Edition

Nigel Poulton
& Pushkar Joglekar

درباره این ویرایش (۲۰۲۵)

این نسخه بر مبنای ویرایش منتشر شده در فوریه ۲۰۲۵ کتاب **The Kubernetes Book** اثر نایجل پولتون تهیه شده است. در این ویرایش، تمام واژگان، تئوری‌ها و کدهای عملیاتی با جدیدترین نسخه استاندارد کوبرنتیز (**v1.32**) مطابقت داده شده‌اند. ویژگی‌های مدرن معماری مانند **Native Sidecar Containers**، اجرای کدهای **WebAssembly (Wasm)** بر روی کلاستر، و مدل‌سازی امنیتی تهدیدات با چارچوب **STRIDE** در این ویرایش به طور تفسیری و دقیق تشریح شده‌اند.

مشخصات و دسترسی:

- **مخزن کدهای رسمی کتاب:** `https://github.com/nigelpoulton/TKB.git` (شاخه `2025`)
- **رویکرد این اثر:** فراتر از یک ترجمه تحت‌اللفظی؛ انتقال شفاف تجربیات، کالبدشکافی سناریوهای واقعی مهندسی و مستندسازی گام‌به‌گام دستورات و مانیفست‌های YAML.

فهرست مطالب

پیش‌گفتار (Preface) 7
مخزن نمونه‌کدها و تمرین‌های عملی (GitHub Repository) 7
راهنمای ویژه کاربران سیستم‌عامل ویندوز (Windows) 8
اصطلاحات استاندارد و زبان مسئولانه (Terminology) 8
فصل ۱: الفبای کوبرنتیز (Kubernetes Primer) 10
۱. مفاهیم و پیش‌نیازهای حیاتی کوبرنتیز 10
۲. کوبرنتیز از کجا آمد؟ (ریشه‌های تاریخی) 12
۳. رابطه کوبرنتیز و داکر: چه اتفاقی افتاد؟ 13
۴. ریشه‌شناسی نام و رازهای لوگو 14
۵. کوبرنتیز: سیستم‌عامل ابر (Operating System of the Cloud) 14
جمع‌بندی فصل اول 15
فصل ۲: اصول بنیادین کارکرد کوبرنتیز (Principles of Operation) 16
۱. نگاه از ارتفاع ۴۰ هزار پایی: کوبرنتیز چیست؟ 16
۲. کالبدشکافی لایه کنترل پلین (The Brain of Kubernetes) 17
۳. کالبدشکافی نودهای کاری (Worker Nodes) 20
۴. چرا پادها خلق شدند؟ (تمثیل جذاب مرسولات پستی) 21
۵. دیپلویمنت‌ها (Deployments): مدیران پادها 23
۶. سرویس‌ها (Services): لنگرگاه پایدار شبکه 24
جمع‌بندی فصل دوم 25
فصل ۳: راه‌اندازی و دسترسی به کلاستر (Getting Kubernetes) 26
۱. راه‌اندازی کلاستر چندگره‌ای بر روی رایانه شخصی 26
۲. کلاسترهای ابری مدیریت‌شده (Managed Kubernetes) 28
۳. کالبدشکافی `kubectl` و فایل جادویی `kubeconfig` 28
۴. آزمودن صحت اتصال به کلاستر 30
فصل ۴: کار با پادها (Working with Pods) 31
۱. تئوری پاد (Pod Theory): چرا مستقیماً کانتینر را اجرا نمی‌کنیم؟ 31
۲. اشتراک منابع و کالبدشکافی لینوکس نیم‌اسپیس‌ها (Resource Sharing) 33
۳. زمان‌بندی پیشرفته و استقرار پادها (Pod Scheduling) 34
۴. جریان ۹ مرحله‌ای استقرار پاد در کلاستر 35
۵. فلسفه چرخه حیات: مرگ‌پذیری و تغییرناپذیری پادها 36
۶. مدل شبکه مسطح پادها (Pod Networking Model) 37
۷. الگوهای پادهای چندکانتینری: تولد سایدکارهای بومی (Native Sidecars) 38
۸. کارگاه عملیاتی جامع: لمس کار با پادها (Hands-on Lab) 40
۹. سناریوی عملی ۱: کار با کانتینر آماده‌سازی (Init Container) 44
۱۰. سناریوی عملی ۲: پیاده‌سازی سایدکار بومی مدرن (Native Sidecar with Git-Sync) 46

۱۱. پاک‌سازی ایمن منابع کلاستر (Clean up) 48
۱۲. جمع‌بندی فصل و نکات کلیدی آزمون‌های بین‌المللی (CKA & CKAD) 48
فصل ۵: کلاسترهای مجازی با نیم‌اسپیس‌ها (Namespaces) 49
۱. تئوری نیم‌اسپیس‌ها (Intro to Namespaces) 49
۲. چندمستأجری: ایزولاسیون نرم در برابر ایزولاسیون سخت 50
۳. اشیای دارای نیم‌اسپیس در برابر اشیای سراسری کلاستر (Cluster-Scoped) 51
۴. نیم‌اسپیس‌های پیش‌فرض و سیستمی کلاستر 51
۵. مدیریت سهمیه‌ها: ResourceQuota و LimitRange 52
۶. کارگاه عملیاتی: ساخت، سوئیچ هوشمند و استقرار در نیم‌اسپیس 53
۷. پاک‌سازی و بازگشت به وضعیت پیش‌فرض (Clean up) 56
۸. جمع‌بندی فصل ۵ 56
فصل ۶: دیپلویمنت‌ها و ارتقای بدون قطعی (Deployments) 57
۱. تئوری دیپلویمنت‌ها (Deployment Theory) 57
۲. کالبدشکافی مانیفست دیپلویمنت: قالب پاد و برچسب‌ها 59
۳. استقرار و بازرسی عملیاتی دیپلویمنت 60
۴. اسکیل دستی و خودکار (Scaling) 62
۵. ارتقای تدریجی بدون حتی یک ثانیه قطعی (Rolling Update) 63
۶. بازگشت آنی به نسخه قبل (Rollback) 64
۷. پاک‌سازی منابع (Clean up) 66
۸. جمع‌بندی فصل ۶ 66
فصل ۷: سرویس‌های کوبرنتیز و شبکه پایدار (Services) 67
۱. تئوری سرویس‌ها (Service Theory) 67
۲. اتصال سست (Loose Coupling) با برچسب‌ها و انتخابگرها 68
۳. پشت صحنه سرویس‌ها: تولد آبجکت‌های EndpointSlice 69
۴. کالبدشکافی kube-proxy: جادوی توزیع بار در سطح کلاستر 70
۵. انواع سرویس‌ها و معماری لایه‌ای (Service Stacking) 70
۶. سرویس‌های خاص: Headless Services و ExternalName 73
۷. کارگاه عملیاتی جامع: کار با سرویس‌ها (Hands-on Lab) 73
۸. پاک‌سازی منابع (Clean up) 75
۹. جمع‌بندی فصل ۷ 75
فصل ۸: مدیریت هوشمند ورودی با اینگرس (Ingress) 76
۱. صورت مسئله: چرا سرویس‌های لایه ۴ کافی نیستند؟ 76
۲. معماری دوگانه اینگرس: منبع اعلانی در برابر کنترلر اجرایی 77
۳. الگوهای مسیریابی: Host-Based و Path-Based 78
۴. پایان‌دهی امنیتی و مدیریت گواهینامه‌های SSL/TLS 78
۵. نگاهی به آینده: معرفی Gateway API در کوبرنتیز ۲۰۲۵ 79
۶. کارگاه عملیاتی جامع: استقرار و تست اینگرس (Hands-on Lab) 79
۷. پاک‌سازی منابع (Clean up) 82
۸. جمع‌بندی فصل ۸ 83
فصل ۹: وب‌اسمبلی در کوبرنتیز (Wasm on Kubernetes) 84

۱. وب‌اسمبلی (Wasm) چیست و چگونه به سرورها راه یافت؟ 84
۲. مزایای راهبردی اجرای Wasm در کلاستر کوبرنتیز 85
۳. کالبدشکافی معماری ران‌تایم: containerd و پروژه runwasi 85
۴. کلاس‌های اجرا (RuntimeClass) در کوبرنتیز 87
۵. کارگاه عملیاتی جامع: استقرار برنامه Wasm روی کوبرنتیز (Hands-on Lab) 88
۶. پاک‌سازی منابع (Clean up) 90
۷. جمع‌بندی فصل ۹ 90
فصل ۱۰: کالبدشکافی عمیق سرویس دیسکاوری (Service Discovery Deep Dive) 91
۱. صورت مسئله: میکروسرویس‌ها چگونه یکدیگر را پیدا می‌کنند؟ 91
۲. معماری رجیستری کلاستر و سرور داخلی CoreDNS 92
۳. جریان ثبت خودکار سرویس‌ها (Service Registration Flow) 93
۴. کشف سرویس در عمل: اسامی کوتاه در برابر FQDN 94
۵. کالبدشکافی `/etc/resolv.conf` و چالش فنی پارامتر `ndots:5` 96
۶. کارگاه عملیاتی جامع: خطایابی و کشف سرویس چندنیم‌اسپیسی (Hands-on Lab) 96
۷. پاک‌سازی منابع (Clean up) 98
۸. جمع‌بندی فصل ۱۰ 99
فصل ۱۱: سیستم ذخیره‌سازی در کوبرنتیز (Kubernetes Storage) 100
۱. معماری کلان ذخیره‌سازی و انقلاب رابط CSI 100
۲. مثلث ذخیره‌سازی: تفکیک وظایف با PV، PVC و StorageClass 101
۳. حالت‌های دسترسی به دیسک (Access Modes) 101
۴. جریان تخصیص پویای دیسک (Dynamic Provisioning) 102
۵. کارگاه عملیاتی جامع: ذخیره‌سازی پایدار در عمل (Hands-on Lab) 103
۶. پاک‌سازی منابع (Clean up) 105
۷. جمع‌بندی فصل ۱۱ 105
فصل ۱۲: مدیریت پیکربندی و داده‌های حساس (ConfigMaps & Secrets) 106
۱. فلسفه طراحی: جداسازی پیکربندی از کد برنامه 106
۲. کالبدشکافی کانفیگ‌مپ‌ها (ConfigMaps) 107
۳. کالبدشکافی سکرت‌ها (Secrets) 108
۴. کارگاه عملیاتی جامع: پیکربندی و تزریق امن (Hands-on Lab) 109
۵. پاک‌سازی منابع (Clean up) 112
۶. جمع‌بندی فصل ۱۲ 112
فصل ۱۳: برنامه‌های دارای وضعیت با استیت‌فول‌ست (StatefulSets) 113
۱. تفاوت بنیادین بارهای کاری دارای وضعیت (Stateful) با دیپلویمنت‌ها 113
۲. ارکان سه‌گانه معماری StatefulSet 114
۳. استراتژی‌های به‌روزرسانی و قابلیت جادویی Partition 115
۴. مقاومت در برابر خرابی نودها (Node Failure Resilience) 116
۵. کارگاه عملیاتی جامع: استقرار کلاستر Stateful (Hands-on Lab) 117
۶. پاک‌سازی منابع (Clean up) 120
۷. جمع‌بندی فصل ۱۳ 120
فصل ۱۴: امنیت سرور API و کنترل دسترسی نقشی (API Security & RBAC) 121

۱. مرحله اول: احراز هویت (Authentication یا AuthN) 121
۲. مرحله دوم: مجازشماری با RBAC (Authorization یا AuthZ) 122
۳. مرحله سوم: کنترل پذیرش (Admission Control) 124
۴. کارگاه عملیاتی: پیاده‌سازی کاربردی RBAC 125
۵. ده فرمان طلایی برای برقراری امنیت API در پروداکشن 127
فصل ۱۵: کالبدشکافی سرور API کوبرنتیز (The Kubernetes API) 128
۱. مقایسه مفهومی: کوبرنتیز به مثابه یک فروشگاه بزرگ مدرن 128
۲. معماری داخلی kube-apiserver: بدون‌وضعیت و اسکیل‌پذیر (Scalable) 129
۳. ساختار آدرس‌دهی REST و مفاهیم بنیادین GVK و GVR 130
۴. مکانیزم‌های Watch و پردازش رویدادمحور 131
۵. گسترش‌پذیری سرور API: کاستوم ریسورس‌ها (CRDs) و الگوی اپراتور (Operator) 132
۶. کارگاه عملیاتی: جست‌وجو در API و ساخت یک منبع سفارشی (CRD) 132
۷. جمع‌بندی نکات کلیدی 134
فصل ۱۶: مدل‌سازی تهدیدات در کوبرنتیز با چارچوب STRIDE (Threat Modeling 135
۱. چرا مدل‌سازی تهدیدات و دفاع در عمق (Defense in Depth)؟ 135
۲. معرفی مدل شش‌گانه STRIDE 135
۳. کالبدشکافی تهدیدات شش‌گانه بر روی اجزای کوبرنتیز 136
۴. ماتریس جامع STRIDE برای کوبرنتیز 138
۵. کارگاه عملیاتی: پیشگیری از بحران ارتقای دسترسی با Pod Security Standards 138
۶. جمع‌بندی 140
فصل ۱۷: امنیت کاربردی در دنیای واقعی (Real-World Kubernetes Security) 142
۱. امنیت زنجیره تأمین نرم‌افزار (Software Supply Chain Security) 142
۲. ایزولاسیون بار کاری در سطح سیستم‌عامل و ران‌تایم (Workload Isolation) 143
۳. ایزولاسیون شبکه و مدل‌های مسیریابی: Overlay در برابر BGP 144
۴. امنیت در زمان اجرا (Runtime Security) با Falco 146
۵. انطباق، حسابرسی و بنچمارک‌های CIS 146
۶. کارگاه عملیاتی: امضای تصویر و اعمال سیاست پذیرش با Kyverno 147
۷. جمع‌بندی نهایی امنیت در دنیای واقعی 148
فصل ۱۸: واژه‌نامه تخصصی اصطلاحات کوبرنتیز (Glossary & Terminology) 149
۱. مفاهیم پایه، کنترل پلین و معماری کلاستر (Core Architecture) 149
۲. بارهای کاری و منابع پردازشی (Workloads & Compute) 150
۳. شبکه، آدرس‌دهی و سرویس دیسکاوری (Networking) 151
۴. ذخیره‌سازی، پیکربندی و متادیتا (Storage, Config & Metadata) 151
۵. امنیت، احراز هویت و گسترش‌پذیری (Security & Extensibility) 152
فصل ۱۹: بخش پایانی و منابع (Outro & References) 154
۱. یادداشت پایانی نایجل پولتون (Outro) 154
۲. نقشه راه گواهینامه‌های رسمی بنیاد لینوکس و CNCF 155
۳. محیط‌های تمرینی رایگان برای کارگاه‌های عملی 155
۴. مراجع و پیوندهای تکمیلی 155

پیش‌گفتار (Preface)

کوبرنتیز با سرعتی سرسام‌آور در حال پیشرفت، دگرگونی و تکامل است؛ به همین دلیل است که من هر سال این کتاب را مورد بازبینی، بازنویسی و آزمایش مجدد قرار می‌دهم. و وقتی از «به‌روزرسانی» صحبت می‌کنم، منظورم یک به‌روزرسانی واقعی و بنیادی است: تک‌تک واژه‌ها و مفاهیم بازبینی می‌شوند و تمام سناریوها و مثال‌های عملی با آخرین نسخه‌های منتشرشده از کوبرنتیز آزموده می‌شوند. من تمام توان و تعهد خود را به کار گرفته‌ام تا این اثر همواره در جایگاه برترین و مطمئن‌ترین منبع یادگیری کوبرنتیز در دنیا باقی بماند.

به عنوان یک نویسنده، شاید آرزوی ایده‌آل این بود که کتابی می‌نوشتم و برای پنج سال بدون دست‌زدن به آن استراحت می‌کردم. اما متأسفانه در دنیای ابری، کتابی درباره کوبرنتیز که حتی دو سال از عمر آن گذشته باشد، نه تنها ناقص است، بلکه می‌تواند شما را با خطاهای بحرانی و الگوهای منسوخ، به شکلی خطرناک به بیراهه ببرد.

تحلیل مترجم و مفسر: چرا ویرایش ۲۰۲۵ اهمیت حیاتی دارد؟

در دنیای مدرن کلود، چرخه انتشار کوبرنتیز هر سال تغییرات چشمگیری در سطح پایدارسازی APIها، منسوخ شدن منابع قدیمی، و ظهور فناوری‌های جدید به خود می‌بینند. در نسخه **v1.32** که مبنای این کتاب است:

- قابلیت **سایدکار بومی (Native Sidecar Containers)** به طور کامل به بلوغ رسیده و جایگزین ترفندهای ناپایدار گذشته شده است.
- مدل‌های امنیتی سخت‌گیرانه‌تر نظیر **Pod Security Standards** جایگزین قوانین قدیمی شده‌اند.
- پشتیبانی از **WebAssembly (Wasm)** به عنوان یک ران‌تایم نوین و سریع در کنار کانتینرهای سنتی وارد زمین بازی کوبرنتیز شده است.

این اثر نه یک ترجمه کلمه‌به‌کلمه، بلکه بازخوانی تفسیری این مفاهیم همراه با نمونه‌های ملموس برای مهندسان و معماران DevOps است.

مخزن نمونه‌کدها و تمرین‌های عملی (GitHub Repository)

تمام فایل‌های مانیفست YAML، اپلیکیشن‌های آزمایشی و کدهای پیکربندی که در طول فصول مختلف این کتاب با آن‌ها کار خواهید کرد، در یک مخزن رسمی گیت‌هاب میزبانی می‌شوند.

برای همراهی با تمرین‌های عملی کتاب، کافی است ابزار `git` را بر روی سیستم خود نصب داشته باشید، مخزن کتاب را کلون کرده و به شاخه اختصاصی ویرایش ۲۰۲۵ سوییچ کنید:

۱. ابتدا مخزن رسمی را با دستور زیر کلون کنید:

```
git clone https://github.com/nigelpoulton/TKB.git
```

۲. سپس وارد پوشه ایجاد شده شوید و شاخه اختصاصی ۲۰۲۵ را دریافت و فعال نمایید:

```
cd TKB
git fetch origin
git checkout -b 2025 origin/2025
```

تبریک می‌گویم! اکنون تمام کدهای استاندارد سال ۲۰۲۵ کتاب بر روی سیستم شما آماده است.

> 💡 **ترفند و تجربه عملیاتی:**
>
> بهترین شیوه یادگیری در کوبرنتیز «دست‌به‌کد شدن» است. توصیه مؤکد ما این است که مانیفست‌های YAML را صرفاً نگاه نکنید؛ آن‌ها را بر روی کلاستر آزمایشی خود اعمال کنید، تغییر دهید و وضعیت پادها و خروجی لاگ‌ها را در لحظه مشاهده نمایید.

راهنمای ویژه کاربران سیستم‌عامل ویندوز (Windows)

بخش عمده دستورات و سناریوهای عملی این کتاب بر روی محیط‌های لینوکس، مک (macOS) و ویندوز کاملاً یکسان اجرا می‌شوند. با این حال، در موارد اندکی که تفاوت رفتاری وجود دارد، در همان بخش دقیقاً توضیح داده‌ام که چه کاری باید انجام دهید.

یکی از تفاوت‌های مرسوم در خط فرمان، شکستن دستورات طولانی در چند خط است. در محیط‌های یونیکس (لینوکس و مک) از علامت بک‌اسلش (`\`) برای اعلام ادامه دستور در خط بعد استفاده می‌شود. برای کاربران ویندوز که از PowerShell استفاده می‌کنند، دو راهکار وجود دارد:

- **حذف بک‌اسلش‌ها:** دستور را بدون علامت شکست خط، به صورت یک‌خطی در ترمینال کپی و اجرا کنید.
- **جایگزینی با بک‌تیک:** به جای کاراکتر بک‌اسلش، از علامت بک‌تیک استفاده کنید که کاراکتر استاندارد شکست خط در PowerShell است.

> ⚠️ **هشدار:**
>
> در هنگام کپی کردن دستورات چندخطی در ترمینال ویندوز، حواستان به فاصله‌های پنهان بعد از بک‌تیک یا بک‌اسلش باشد؛ چرا که یک فاصله اضافی می‌تواند باعث اجرای زودهنگام و ناقص دستور شود.

اصطلاحات استاندارد و زبان مسئولانه (Terminology)

در سرتاسر این کتاب متوجه خواهید شد که حرف اول نام اشیاء و آبجکت‌های کوبرنتیز با **حروف بزرگ انگلیسی** نوشته می‌شود؛ به عنوان مثال: `Pod` ، `Service` ، `Deployment` یا `LoadBalancer` .

دلیل این انتخاب آن است که بتوانید دقیقاً تفکیک قائل شوید که چه زمانی درباره **آبجکت رسمی کوبرنتیز (مثلاً شیء LoadBalancer)** صحبت می‌کنیم و چه زمانی مقصود ما یک مفهوم عام زیرساختی (مانند یک سرور فیزیکی یا نرم‌افزاری لودبالانسر شبکه) است.

همچنین این کتاب به طور کامل از استانداردهای **ابتکار نام‌گذاری فراگیر (Inclusive Naming Initiative)** پیروی می‌کند. در صنعت نرم‌افزار، واژگان قدیمی و نامناسب کنار گذاشته شده‌اند؛ به عنوان نمونه، به جای اصطلاح قدیمی Master Node، همیشه و با افتخار از اصطلاح دقیق **Control Plane** استفاده می‌کنیم.

📌 نکته مهم:

در متن ترجمه تفسیری فارسی، هر جا که شیء کوبرنتیز برای نخستین بار ذکر شود، تلفظ فنی روان به همراه املای دقیق انگلیسی آن قید شده است تا ذهن مهندسی شما با ادبیات واقعی تیم‌های فنی بین‌المللی کاملاً هماهنگ باشد.

فصل ۱: الفبای کوبرنتیز (Kubernetes Primer)

این فصل دروازه ورود به جهان کوبرنتیز است. هدف ما این است که بدون سردرگمی در میان کوهی از واژگان غلیظ انگلیسی و اصطلاحات گیج‌کننده، فلسفه بنیادین، ریشه‌های تاریخی و منطق مهندسی شکل‌گیری کوبرنتیز را به طور شفاف، عمیق و تفسیری واکاوی کنیم.

این فصل در دو بخش کلیدی ساختاربندی شده است: ۱. **مفاهیم و پیشینه حیاتی کوبرنتیز** (ارکستراسیون، کانتینرها، معماری ابری‌بومی و میکروسرویس‌ها) ۲. **کوبرنتیز به عنوان سیستم‌عامل ابر** (انتزاع زیرساخت و مفهوم زمان‌بندی فراگیر)

—

۱. مفاهیم و پیش‌نیازهای حیاتی کوبرنتیز

اگر منابع مختلف را ورق بزنید، بارها با تعریفی شبیه به این جمله روبه‌رو می‌شوید:

«کوبرنتیز، پلتفرم ارکستراسیون برای برنامه‌های میکروسرویس کانتینری‌شده و ابری‌بومی (Cloud-Native) است.»

این جمله در نگاه اول پر از اصطلاحات پرطمطراق فنی است. بیایید هر تکه از این پازل را باز کنیم تا معنای واقعی آن در محیط پروداکشن برایمان روشن شود:

ارکستراسیون (Orchestration) چیست؟

در دنیای موسیقی، رهبر ارکستر سازهای مختلف را هماهنگ می‌کند تا خروجی نهایی یک سمفونی بی‌نقص باشد. در دنیای نرم‌افزار، «سیستم ارکستراتور» سامانه‌ای هوشمند و خودکار است که وظیفه استقرار اپلیکیشن‌ها و واکنش پویا به نوسانات محیط را بر عهده دارد.

کوبرنتیز در جایگاه یک ارکستراتور، وظایف حیاتی زیر را بدون دخالت دستی اپراتور انجام می‌دهد:

- **استقرار خودکار (Automated Deployment):** بارگذاری و توزیع نسخه‌های نرم‌افزار روی ماشین‌های مختلف کلاستر.
- **اسکیل خودکار و پویا (Auto-scaling):** افزایش یا کاهش تعداد نمونه‌های برنامه متناسب با افزایش ترافیک یا بار پردازشی.
- **خودترمیمی و خودمراقبتی (Self-healing):** نظارت دائمی بر سلامت سرویس‌ها؛ اگر کانتینری از کار بیفتد، سریعاً متوقف، مجدداً راه‌اندازی یا روی سرور سالمی بازتولید می‌شود.
- **به‌روزرسانی بدون قطعی و رول‌بک (Rollouts & Rollbacks):** ارتقای گام‌به‌گام نسخه نرم‌افزار (Rolling Update) بدون حتی یک میلی‌ثانیه قطعی برای کاربران نهایی، و قابلیت بازگشت آنی به نسخه قبل در صورت کشف باگ.

زیبایی کار اینجاست: شما نیاز نیست ۲۴ ساعته پای سرور بنشینید؛ حالت مطلوب (Desired State) خود را تعریف می‌کنید و سپس کوبرنتیز تمام این جادو را به صورت ۲۴/۷ برای شما تضمین می‌کند.

کانتینرسازی (Containerization)

کانتینرسازی به زبان ساده یعنی فرآیند بسته‌بندی کد نرم‌افزار به همراه تمام وابستگی‌ها، کتابخانه‌ها، متغیرها و فایل‌های پیکربندی در قالب یک «ایمیج (Image)» مجزا و اجرای آن به عنوان یک «کانتینر (Container)».

می‌توان کانتینرها را نسل تکامل‌یافته ماشین‌های مجازی (VM) در نظر گرفت. هر دو وظیفه ایزولاسیون و اجرای ایزوله برنامه‌ها را انجام می‌دهند؛ اما تفاوت کلیدی اینجاست:

- ماشین مجازی نیازمند یک سیستم‌عامل مهمان (Guest OS) کامل، کرنل اختصاصی و منابع گیگابایتی سنگین است.
- کانتینرها کرنل سیستم‌عامل میزبان را به اشتراک می‌گذارند، حجم بسیار کمی دارند، در کسری از ثانیه روشن می‌شوند و در محیط‌های مختلف (توسعه، تست، پروداکشن) رفتار کاملاً یکسانی دارند.

تفسیر فنی: آیا کانتینرها ماشین‌های مجازی را منسوخ کردند؟

پاسخ کوتاه: خیر! در اکثر دیتاسنترها و کلودهای مدرن، کانتینرها دقیقاً **درون** ماشین‌های مجازی اجرا می‌شوند. ماشین‌های مجازی ایزولاسیون سخت‌افزاری و مرز امنیتی پایه را فراهم می‌کنند و کانتینرها چابکی، تراکم بالا و سرعت استقرار نرم‌افزار را به ارمغان می‌آورند.

مفهوم واقعی ابری‌بومی (Cloud-Native)

یک تصور نادرست این است که اگر برنامه سنتی و مونوپولی قدیمی خود را روی یک سرور مجازی در AWS، آژور یا سرورهای ابری ایرانی مستقر کنیم، برنامه ما «کلودنیتیو» شده است! ابداً چنین نیست.

برنامه ابری‌بومی نرم‌افزاری است که از روز اول برای بهره‌گیری از ویژگی‌های ذاتاً ابری طراحی شده باشد:

- تحمل‌پذیری خطا و قابلیت اسکیل افقی (Scale-out)
- بی‌وضعیت بودن نسبی (Statelessness) برای جایگزینی سریع کانتینرها
- ترمیم خودکار در صورت بروز خرابی در سخت‌افزار یا شبکه زیرین

معماری میکروسرویس‌ها (Microservices)

برنامه‌های میکروسرویس به جای این‌که یک تکه‌کد غول‌پیکر و یکپارچه (Monolith) باشند، از بخش‌های کوچک، مستقل، تخصصی و جداگانه تشکیل شده‌اند که از طریق شبکه (معمولاً REST API یا gRPC) با یکدیگر تبادل داده می‌کنند.

یک پلتفرم تجارت الکترونیک فرضی را تصور کنید: ۱. فرانت‌اند وب (Web Front-end)
۲. کاتالوگ و فهرست محصولات (Catalog)
۳. سبد خرید (Shopping Cart)
۴. احراز هویت و کاربران (Authentication)
۵. ثبت وقایع و لاگینگ (Logging)
۶. درگاه پرداخت و انبارداری (Payment & Store)

اگر این سامانه بر پایه میکروسرویس طراحی شود:

- هر کدام از این ۶ بخش به عنوان یک برنامه مستقل توسعه، تست و مستقر می‌شوند.
- تیم مربوط به سبد خرید می‌تواند روزی ۱۰ بار آپدیت منتشر کند بدون این‌که نیازی به بیلد مجدد کل فروشگاه باشد.
- در حراجی‌های مناسبتی (مانند جمعه سیاه)، تنها کافی است تعداد کانتینرهای سرویس «کاتالوگ» و «سبد خرید» را ۵ برابر کنید، بدون هدر دادن منابع برای سایر بخش‌ها!

> **📌 نکته مهم:**
>
> الگوی استاندارد در دنیای مدرن استقرار هر میکروسرویس به عنوان یک یا چند کانتینر مستقل در کلاستر کوبرنتیز است.

حال بیایید همان جمله ابتدای فصل را بازخوانی کنیم: **«کوبرنتیز سامانه‌ای خودکار است که وظیفه استقرار، اسکیل‌کردن (Scaling)، پایش سلامت و به‌روزرسانی اپلیکیشن‌هایی را بر عهده دارد که کارکردهای آن‌ها در قالب قطعات کوچک مستقل (میکروسرویس) توسعه یافته و به صورت کانتینرهای چابک اجرا می‌شوند.»**

—

۲. کوبرنتیز از کجا آمد؟ (ریشه‌های تاریخی)

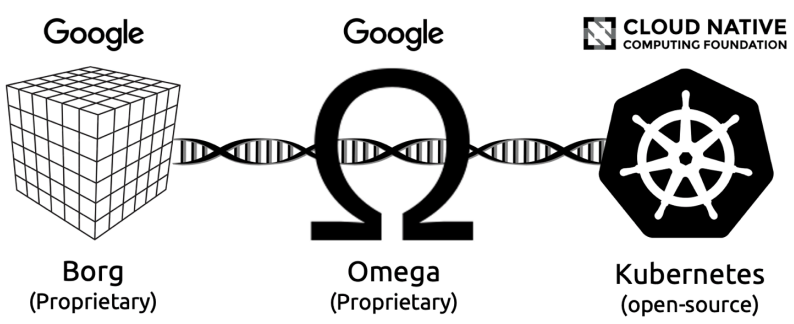
داستان کوبرنتیز بازتابی از هوشمندی مهندسی گوگل در رقابت با انقلاب پردازش ابری آمازون (AWS) و طوفان داکر (Docker) است:

۱. **جرقه رقابت ابری:** آمازون با معرفی AWS جهان IT را متحول کرد. شرکت‌های بزرگی مثل گوگل برای این‌که مشتریان سازمانی را به سوی پلتفرم ابری خود جذب کنند، نیازمند ابزاری بودند که وابستگی شدید مشتریان به زیرساخت اختصاصی آمازون را بشکند.

۲. **میراث میلیاردها کانتینر در گوگل:** گوگل بیش از یک دهه پیش از مطرح شدن نام داکر، تمامی سرویس‌های غول‌پیکر خود (از موتور جستجو گرفته تا جیمیل و یوتیوب) را بر روی میلیاردها کانتینر هفتگی مدیریت می‌کرد. گوگل برای مدیریت این بار باورنکردنی، از دو سامانه فوق‌پیشرفته داخلی به نام‌های **بورگ (Borg)** و بعدها **امگا (Omega)** استفاده می‌کرد.

۳. **پیدایش کوبرنتیز در ۲۰۱۴:** تیمی از مهندسان گوگل تجربیات ارزشمند به‌دست‌آمده از طراحی Borg را برداشتند و پروژه‌ای بازمتن و مدرن به نام کوبرنتیز خلق کردند. سپس در سال ۲۰۱۴ آن را به بنیاد نوپای رایانش ابری بومی (**CNCF**) تحت چتر بنیاد لینوکس اهدا کردند.

> **آیا کوبرنتیز همان نسخه بازمتن Borg است؟**
>
> خیر. کوبرنتیز عیناً کدهای Borg نیست؛ بلکه از نظر معماری، الگوها و دی‌ان‌ای (DNA) مستقیم‌ترین نواده Borg محسوب می‌شود که با آموختن از اشتباهات گذشته و متناسب با استانداردهای مدرن بازنویسی شد.



تصویر ۱.۳ - پیشینه و دی‌ان‌ای مشترک میان سامانه‌های Borg، Omega و کوبرنتیز

کوبرنتیز اساساً دو ارزش استراتژیک بزرگ به کل صنعت نرم‌افزار هدیه داد: ۱. **انتزاع کامل زیرساخت (Infrastructure Abstraction):** کد و معماری شما دیگر اسیر تفاوت‌های AWS، گوگل کلود، مایکروسافت آژور یا سرورهای محلی اختصاصی شما نیست.
۲. **قابلیت حمل بی‌نظیر اپلیکیشن (Portability):** کانتینری که در کلاستر محلی لپ‌تاپ شما اجرا می‌شود، با همان مانیفست دقیقاً در عظیم‌ترین کلاسترهای سازمانی ابری اجرا خواهد شد.

—

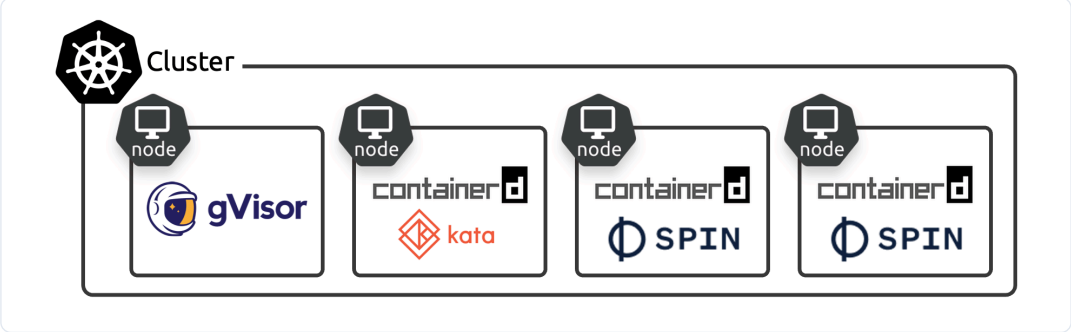
۳. رابطه کوبرنتیز و داکر: چه اتفاقی افتاد؟

در سال‌های اولیه پیدایش کوبرنتیز، داکر موتور اجرایی پیش‌فرض (Container Runtime) بود. یعنی وقتی کوبرنتیز می‌خواست کانتینری را بسازد یا خاموش کند، این دستور را به داکر می‌سپرد.

اما در گذر زمان دو چالش بزرگ رخ داد: ۱. داکر ابزاری برای انسان‌ها و توسعه‌دهندگان بود و به مرور زمان ویژگی‌های جانبی فراوانی (مانند بیلد، CLI، رابط کاربری، شبکه اختصاصی) پیدا کرد و بسیار سنگین و متورم (Bloated) شد؛ در حالی که کوبرنتیز فقط به یک موتور سبک و سریع برای اجرای کانتینر نیاز داشت.
۲. جامعه مهندسی ران‌تایم‌های تخصصی دیگری (با سطوح امنیتی بالاتر، عایق‌بندی بهتر، یا ران‌تایم‌های اختصاصی مثل Wasm) خلق کرد.

> ⚠️ **هشدار:**
>
> در نسخه **1.24** کوبرنتیز، لایه واسط داکر (Dockershim) رسماً حذف شد. این به معنای پایان کار کانتینرهای شما نبود! بلکه به این معنا بود که کوبرنتیز مستقیماً با **containerd** (هسته اجرایی سبک، کارآمد و بهینه‌سازی‌شده بر پایه استانداردهای بنیاد OCI) ارتباط برقرار می‌کند.

با معرفی استاندارد **CRI (Container Runtime Interface)**، کوبرنتیز امکان اتصال پلاگین‌محور به هر ران‌تایم استانداردی را پیدا کرد. امروزه در یک کلاستر واحد، می‌توانید نودهایی داشته باشید که در کنار `containerd`، از ران‌تایم‌های نوین نظیر **Wasm (WebAssembly)** برای کارهای سبک با مصرف رم فوق‌العاده پایین میزبانی کنند.



تصویر ۱.۲ - کلاستر چهارگره‌ای با ران‌تایم‌های چندگانه کانتینری و Wasm

برنده نبرد ارکستراتورها

در سال‌های ۲۰۱۶ و ۲۰۱۷ رقابت شدیدی میان کوبرنتیز، **Docker Swarm** و **Mesosphere DCOS** در جریان بود. کوبرنتیز به لطف اکوسیستم بازمتن بی‌نظیر، حمایت تمام غول‌های فناوری و انعطاف‌پذیری فوق‌العاده‌اش این رقابت تاریخی را با قاطعیت برد و تبدیل به استاندارد بلامنازع جهان فناوری شد.

—

۴. ریشه‌شناسی نام و رازهای لوگو

- **واژه کوبرنتیز (Kubernetes):** ریشه در واژه یونانی **Helmsman** به معنای «سکاندار کشتی» دارد. به همین دلیل نماد رسمی کوبرنتیز «فرمان یا چرخ سکان کشتی» است.
- **راز ۷ پره روی لوگو:** مهندسان اولیه گوگل قصد داشتند نام پروژه را **Seven of Nine** (شخصیت ربات معروف در سریال علمی‌تخیلی Star Trek Voyager که از نژاد Borg بود) بگذارند! قوانین کپی‌رایت اجازه نداد، اما برای حفظ این یادگار مهندسی، فرمان سکان لوگوی کوبرنتیز دقیقاً **۷ پره** طراحی شد.
- **مخفف معروف K8s:** تلفظ رایج آن «کِیتس (Kates)» است. دلیل عدد ۸ بسیار ساده است: میان حرف اول (K) و حرف آخر (s)، دقیقاً **۸ حرف** (ubernete) وجود دارد.



تصویر ۱.۴ - نشان رسمی کوبرنتیز: چرخ سکان کشتی با ۷ پره

—

۵. کوبرنتیز: سیستم‌عامل ابر (Operating System of the Cloud)

همان‌طور که سیستم‌عامل‌های سنتی نظیر لینوکس و ویندوز تفاوت‌های سخت‌افزاری سرورها را مخفی می‌کنند:

- **لینوکس/ویندوز:** منابع یک سرور فیزیکی (هسته‌های CPU، ماژول‌های رم، کنترلر هارد و کارت شبکه) را انتزاع کرده و فرآیندهای برنامه‌ها (Processes) را روی آن‌ها زمان‌بندی (Schedule) می‌کنند.
- **کوبرنتیز:** تمام سرورها، رک‌ها و دیتاسنترهای مختلف یک کلود را به عنوان یک استخر یکپارچه از منابع انتزاع کرده و برنامه‌های میکروسرویس را روی این کلاستر توزیع و زمان‌بندی می‌کند.

چرا کوبرنتیز معجزه Multi-Cloud و Hybrid Cloud است؟

به عنوان یک توسعه‌دهنده نرم‌افزار، شما هیچ‌گاه درگیر این نمی‌شوید که کدام پین حافظه رم فیزیکی یا کدام چیپ مادربرد به متغیر برنامه شما تخصیص داده شده است؛ سیستم‌عامل این کار را می‌کند.

به همین صورت در کوبرنتیز، شما نیازی ندارید بدانید کانتینر سرویس شما روی کدام سرور در زون فرانکفورت AWS اجرا می‌شود یا روی سرور اختصاصی داخل کشور؛ شما صرفاً اعلام می‌کنید: **«من ۳ نمونه از این سرویس با ۲ گیگ رم نیاز دارم»** و کوبرنتیز بهینه‌ترین گره را برای اجرای آن انتخاب می‌کند. این قابلیت مهاجرت، راه‌اندازی سناریوهای Multi-Cloud و ساخت ابرهای هیبریدی امن را بدون بازنویسی کدهای نرم‌افزار امکان‌پذیر می‌سازد.

—

جمع‌بندی فصل اول

۱. کوبرنتیز استاندارد بلامنازع ارکستراسیون میکروسرویس‌ها بر بستر زیرساخت‌های نوین ابری است.
۲. با ارائه انتزاع کامل، برنامه‌های شما را از قفل شدن در زیرساخت یک کلود خاص (Vendor Lock-in) رها می‌سازد.
۳. بر بستر استانداردهای باز OCI و واسط استاندارد CRI کار می‌کند و از ران‌تایم‌های مختلف از `containerd` تا `Wasm` پشتیبانی می‌نماید.
۴. با اتکا به مدل اعلانی، رفتاری شبیه به یک سیستم‌عامل غول‌پیکر در مقیاس چندین دیتاسنتر و سرور ابری از خود نشان می‌دهد.

فصل ۲: اصول بنیادین کارکرد کوبرنتیز (Principles of Operation)

این فصل نقشه راه و ستون فقرات درک تمام مباحث آتی کتاب است. هدف ما در این بخش این نیست که در یک شب تبدیل به متخصص ارشد کوبرنتیز شوید، بلکه پایه‌های مفهومی مستحکمی را بنا می‌کنیم تا در فصول بعدی دقیقاً بدانید هر دستور و هر مانیفست YAML در پشت صحنه چه طوفانی در کلاستر به پا می‌کند.

در این فصل مباحث زیر را به صورت گام‌به‌گام و تفسیری کالبدشکافی خواهیم کرد: ۱. **نگاه از ارتفاع ۴۰ هزار پایی به کوبرنتیز** ۲. **معماری نودهای کنترلی (Control Plane) در برابر نودهای کاری (Worker Nodes)** ۳. **بسته‌بندی برنامه‌ها و چرایی خلق مفهوم پاد (Pod)** ۴. **مدل اعلانی (Declarative) و چرخه جادویی حالت مطلوب (Desired State)** ۵. **آبجکت‌های سه‌گانه بنیادین: پادها، دیپلویمنت‌ها و سرویس‌ها**

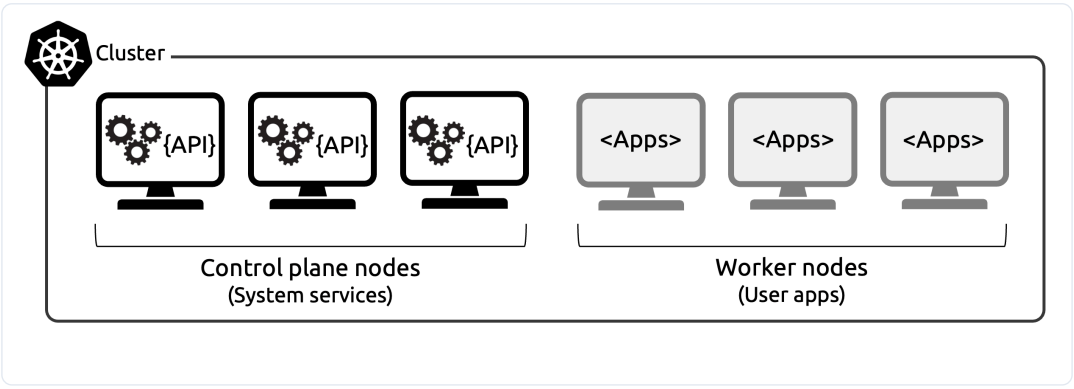
—

۱. نگاه از ارتفاع ۴۰ هزار پایی: کوبرنتیز چیست؟

برای درک ساده و جامع، کوبرنتیز همزمان دو ماهیت دارد: ۱. **یک کلاستر (Cluster):** استخری متشکل از یک یا صدها سرور که منابع پردازشی (CPU)، حافظه (RAM) و دیسک خود را تجمیع کرده‌اند. ۲. **یک ارکستراتور (Orchestrator):** سامانه‌ای فوق‌العاده هوشمند که وظیفه دارد بار کاری نرم‌افزارها را به بهینه‌ترین شکل ممکن بر روی این استخر منابع توزیع کرده و سلامت آن‌ها را تضمین نماید.

در هر کلاستر کوبرنتیز دو دسته گره (Node) یا ماشین وجود دارد:

- **گره‌های کنترل پلین (Control Plane Nodes):** مغز متفکر کلاستر که تصمیمات مدیریتی، زمان‌بندی و پایش سلامت را بر عهده دارند.
- **گره‌های کاری (Worker Nodes):** سربازان اجرایی کلاستر که کانتینرها و اپلیکیشن‌های عملیاتی واقعی شما روی آن‌ها اجرا می‌شوند.



تصویر ۲.۱ - کلاستر نمونه با ۳ نود کنترل پلین و ۳ نود ورکر

جنس فیزیکی و سیستم‌عامل نودها

هر نود می‌تواند یک سرور فیزیکی (Bare-Metal)، یک ماشین مجازی (VMware/KVM) یا یک سرور ابری (در AWS، آژور یا زیرساخت‌های محلی) بر پایه پردازنده‌های معماری x86-64 یا ARM64 باشد.

- **نودهای Control Plane** الزماً و فقط باید سیستم‌عامل لینوکس داشته باشند.
- **نودهای Worker** عمدتاً لینوکس هستند، اما اگر برنامه‌های دات‌نت سنتی و اپلیکیشن‌های ویندوزی دارید، می‌توانید نودهای کاری مبتنی بر Windows Server نیز به کلاستر متصل کنید. کوبرنتیز به اندازه کافی باهوش است که کانتینرهای لینوکسی را به نودهای لینوکسی و کارهای ویندوزی را به نودهای ویندوزی هدایت کند.

⚠️ **هشدار:**

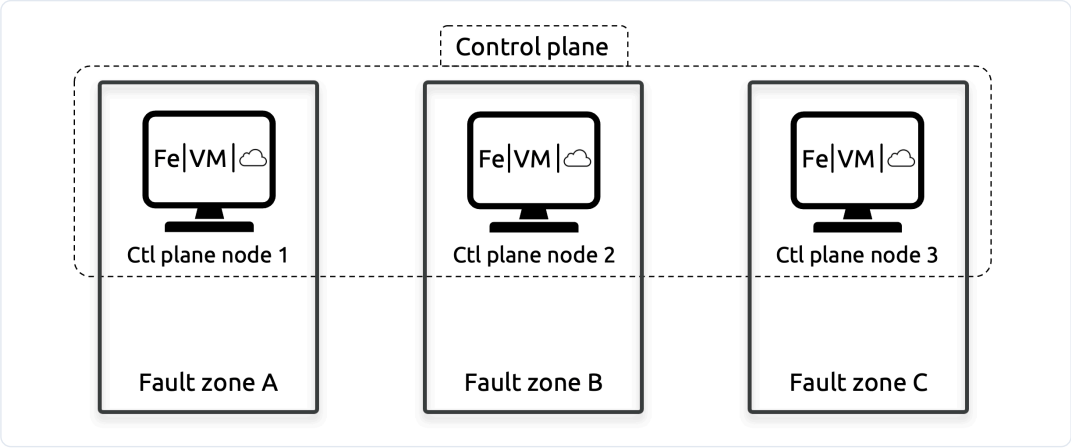
در محیط‌های تستی و آموزشی (مانند Minikube یا داکر دسکتاپ)، کنترل پلین و ورکرها روی یک ماشین ادغام می‌شوند. اما در محیط‌های واقعی پروداکشن، اجرای مستقیم اپلیکیشن‌های کاربری بر روی نودهای Control Plane یک ضدالگو (Anti-pattern) است؛ زیرا تمام منابع نودهای مدیریتی باید برای پایداری پایگاه داده کلاستر و پاسخگویی به رویدادها رزرو بماند.

—

۲. کالبدشکافی لایه کنترل پلین (The Brain of Kubernetes)

لایه کنترل پلین مجموعه‌ای از سرویس‌های سیستمی هماهنگ است که هوشمندی کوبرنتیز را تشکیل می‌دهند. این سرویس‌ها وضعیت کل کلاستر را رصد می‌کنند، به رویدادها پاسخ می‌دهند و تصمیم می‌گیرند کانتینرها کجا و چگونه مستقر شوند.

برای تضمین در دسترس‌پذیری بالا (High Availability یا HA) در سازمان‌ها، هرگز نباید به یک گره کنترلی بسنده کرد؛ استاندارد صنعتی اجرای **۳ یا ۵ نود کنترلی** توزیع‌شده در زون‌های فیزیکی مجزا است.



تصویر ۲.۲ - پایداری و High Availability در کنترل پلین با توزیع در زون‌های ابری

اجزای اصلی تشکیل‌دهنده Control Plane عبارتند از:

۱. سرور وب سرویس (kube-apiserver)

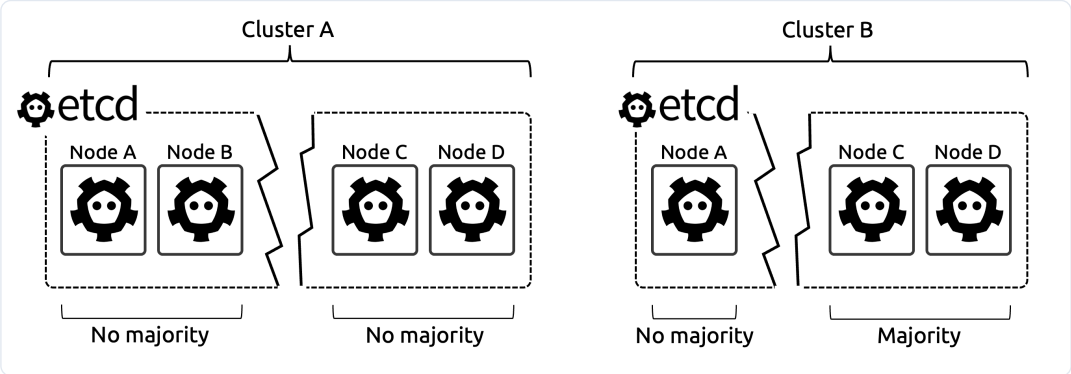
ویترین و ورودی اصلی تمام ارتباطات در کلاستر است. تمام دستورات شما با ابزار `kubectl` ، ارتباطات گره‌های ورکر، و حتی تعاملات داخلی خود سرویس‌های کنترل پلین، همگی و بدون استثنا از کانال API Server عبور می‌کنند. این سرویس یک رابط RESTful مدرن بر بستر HTTPS ارائه می‌دهد که در آن:

- هویت فرستنده دستور بررسی و احراز می‌شود (Authentication).
- مجوز دسترسی و نقش‌های کاربر سنجیده می‌شود (Authorization با مکانیزم RBAC).
- صحت داده‌ها اعتبارسنجی شده و در دیتابیس کلاستر ذخیره می‌گردد.

۲. پایگاه داده و مخزن وضعیت کلاستر (Cluster Store - etcd)

کوبرنتیز به جز پایگاه داده **etcd**، کاملاً بدون‌وضعیت (Stateless) است! مخزن etcd یک پایگاه داده توزیع‌شده کلید-مقدار (Key-Value) است که تمام اطلاعات کلاستر، مانیفست‌ها، سکرت‌ها و وضعیت لحظه‌ای پادها در آن ثبت می‌شود.

> **💡 ترفند و تجربه عملیاتی:**
>
> چرا تعداد نودهای کنترل پلین همیشه باید **فرد** (۱، ۳ یا ۵) باشد؟ پایگاه داده etcd از الگوریتم اجماع **Raft** استفاده می‌کند. برای تصمیم‌گیری در مورد ثبت اطلاعات جدید، رأی اکثریت (Quorum یعنی $\frac{N}{2}+1$) الزامی است. یک کلاستر با ۳ نود می‌تواند از دست رفتن ۱ نود را تاب بیاورد؛ اما کلاستر ۴ نودی هم دقیقاً همان ۱ نود تحمل خطا را دارد، با این تفاوت که خطر بروز انشقاق شبکه (Split-Brain) در آن بیشتر است!



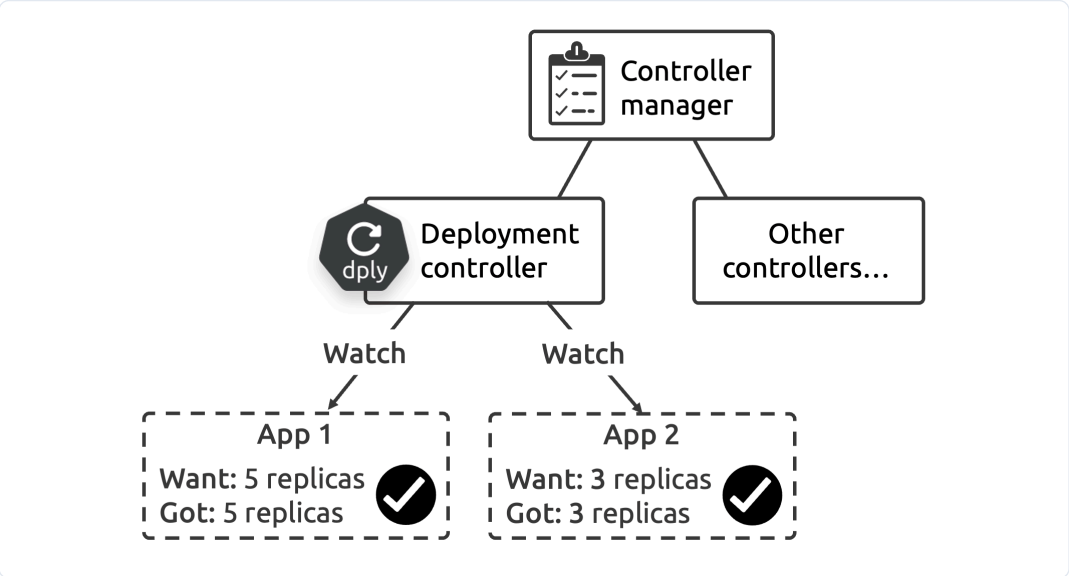
تصویر ۲.۳ - حد نصاب (Quorum) در etcd و پیشگیری از بروز شرایط Split-Brain

۳. کنترلرها و مدیر کنترلرها (kube-controller-manager)

کنترلرها ناظران خستگی‌ناپذیر کلاستر هستند. هر کنترلر در حقیقت یک «حلقه نظارت دائمی (Watch Loop)» در پس‌زمینه است که وضعیت واقعی مشاهده‌شده در سیستم (Observed State) را با وضعیتی که کاربر درخواست کرده (Desired State) مقایسه می‌کند. اگر مغایرتی پیدا کند، بلافاصله وارد عمل می‌شود تا سیستم را به حالت مطلوب بازگرداند:

- **Node Controller:** ناظر سلامت سرورها؛ اگر ارتباط نودی قطع شود، پادهای آن را به سرور دیگر منتقل می‌کند.
- **ReplicaSet Controller:** نظارت می‌کند که آیا دقیقاً همان تعداد پادی که درخواست کرده‌اید (مثلاً ۴ نمونه) فعال هستند یا خیر.
- **EndpointSlice Controller:** وظیفه به‌روزرسانی آدرس‌های شبکه سرویس‌ها را بر عهده دارد.

سرویس `kube-controller-manager` فرآیندی است که تمام این کنترلرهای مجزا را در قالب یک فایل اجرایی یکپارچه اجرا و سرپرستی می‌کند.



تصویر ۲.۴ - معماری مدیر کنترلرها (Controller Manager) و حلقه‌های کنترلی

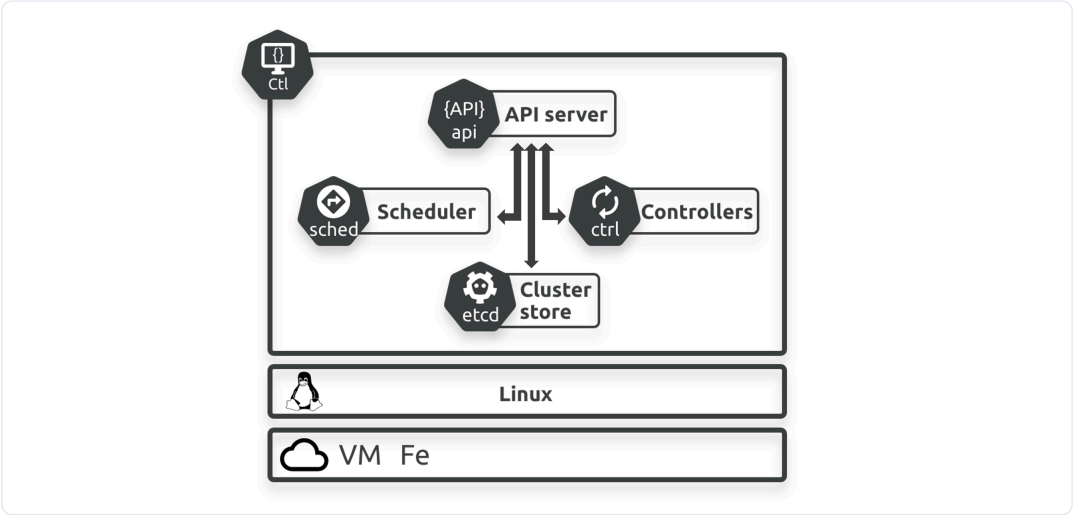
۴. زمان‌بند هوشمند (kube-scheduler)

زمان‌بند وظیفه دارد بررسی کند چه تسک‌ها و پادهای جدیدی در API Server ثبت شده‌اند و مناسب‌ترین نود کاری برای میزبانی از هر پاد کدام است. فرآیند تصمیم‌گیری زمان‌بند در دو مرحله حساس انجام می‌شود: ۱. **فیلتر کردن گره‌های ناتوان (Filtering / Predicates):** نودهایی که منابع خالی (CPU/RAM) کافی ندارند، یا با قوانین هم‌نشینی (Affinity) و طردکنندگی (Taints) پاد ناسازگارند، کنار گذاشته می‌شوند. ۲. **امتیازدهی و رتبه‌بندی (Ranking / Priorities):** از میان نودهای واجد شرایط، نودهایی که توزیع بار یکنواخت‌تری ایجاد می‌کنند، یا ایمیج برنامه از قبل روی آن‌ها کش شده، امتیاز بالاتری دریافت می‌کنند و پاد به برترین نود سپرده می‌شود.

اگر هیچ نودی منابع کافی نداشته باشد، پاد در وضعیت `Pending` باقی می‌ماند تا سروری آزاد شود یا قابلیت افزایش خودکار اندازه کلاستر (Cluster Autoscaler) سرور جدیدی به سیستم اضافه کند.

۵. مدیر کنترلر ابری (cloud-controller-manager)

اگر کلاستر شما روی کلودهایی نظیر AWS، آژور، گوگل کلود یا ارائه‌دهندگان محلی مستقر باشد، این ماژول ارتباط کلاستر با APIهای زیرساختی کلود (مثل تخصیص خودکار لودبالانسر سخت‌افزاری یا اتصال ولوم‌های دیسک ابری) را برقرار می‌سازد.



تصویر ۲.۵ - اجزای داخلی نود کنترل پلین (Control Plane Node)

—

۳. کالبدشکافی نودهای کاری (Worker Nodes)

نودهای کاری ماشین‌هایی هستند که بار کاری اصلی سازمان روی آن‌ها قرار دارد. اجزای اصلی هر ورکر عبارتند از:

۱. کیوبلت (Kubelet): فرماندار نود

مهم‌ترین ایجنت مقیم بر روی هر سرور ورکر، `kubelet` نام دارد. کیوبلت به طور دائم با `kube-apiserver` گفتگو می‌کند:

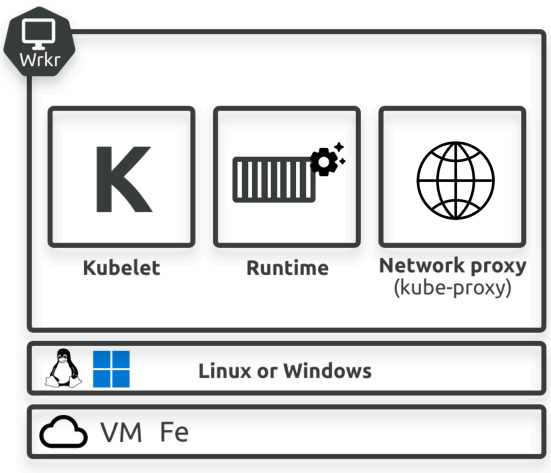
- دستورات استقرار پاد را تحویل می‌گیرد.
- با ران‌تایم کانتینری نود تماس می‌گیرد تا کانتینرها را اجرا کند.
- سلامت کانتینرها را مداوم رصد کرده و گزارش وضعیت را به API Server مخابره می‌کند.

۲. موتور اجرایی کانتینر (Container Runtime)

مسئول فیزیکی دانلود ایمیج‌ها از ریجستری، بالا آوردن کانتینرها، اختصاص حافظه و اجرای پروسه‌ها است. به لطف واسط استاندارد CRI، امروزه اکثر کلاسترها از **containerd** یا **CRI-O** استفاده می‌کنند.

۳. کیوب‌پروکسی (kube-proxy)

این کامپوننت شبکه هوشمند کوبرنتیز را بر روی نود پیاده‌سازی می‌کند. وظیفه اصلی آن ایجاد قوانین مسیریابی (با استفاده از ابزارهای لینوکسی نظیر `iptables` یا `IPVS`) است تا درخواست‌هایی که به آدرس مجازی سرویس‌ها ارسال می‌شوند، بدون مشکل به پادهای مقصد برسند.



تصویر ۲.۶ - معماری داخلی نود کاری ورکر (Worker Node)

—

۴. چرا پادها خلق شدند؟ (تمثیل جذاب مرسولات پستی)

کوبرنتیز به طور مستقیم یک کانتینر تنها، یک ماشین مجازی تنها، یا یک برنامه Wasm تنها را اجرا نمی‌کند! بلکه همه این‌ها را در قالبی به نام **پاد (Pod)** بسته‌بندی می‌کند.

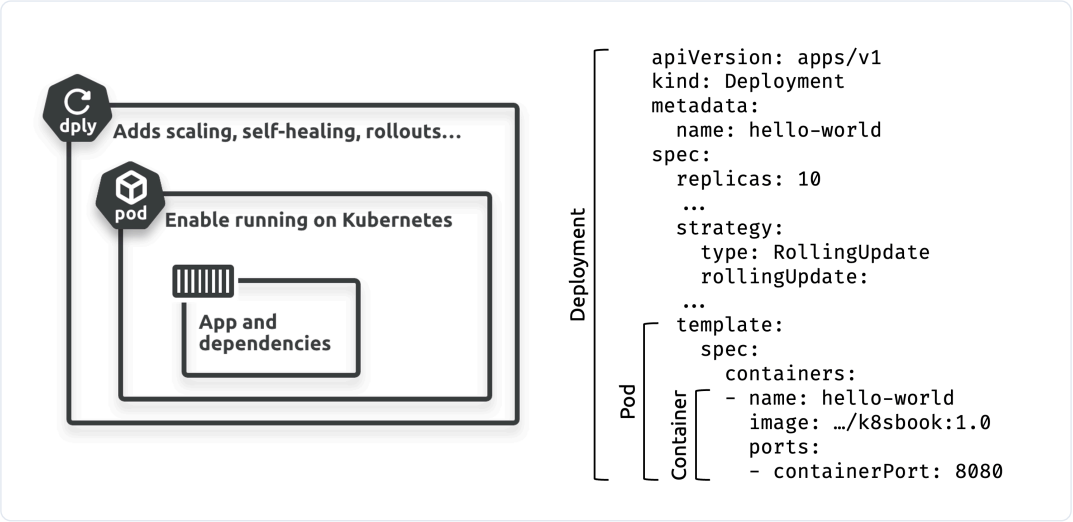
تمثیل شرکت‌های پست و بسته‌بندی استاندارد

یک شرکت باربری بین‌المللی را در نظر بگیرید. فرقی نمی‌کند شما بخواهید کتاب، پیراهن، لپ‌تاپ یا مواد غذایی ارسال کنید؛ شرکت باربری از شما می‌خواهد کالایتان را در کارتن‌های استاندارد با بارکد و آدرس مشخص قرار دهید. پس از تحویل بسته، شرکت پست دیگر درگیر محتوای داخل جعبه نیست؛ هواپیماها، راننده‌ها و سیستم‌های توزیع دقیقاً بر اساس برچسب جعبه آن را حمل و رهگیری می‌کنند.

پاد در کوبرنتیز دقیقاً همان کارتن و برچسب استاندارد است! پاد یک لایه انتزاعی دور کانتینر می‌کشد تا کوبرنتیز بدون درگیر شدن با جزئیات درونی برنامه، بتواند آن را جابه‌جا، زمان‌بندی و پایش کند.

ساختار تودرتوی اشیاء کوبرنتیز:

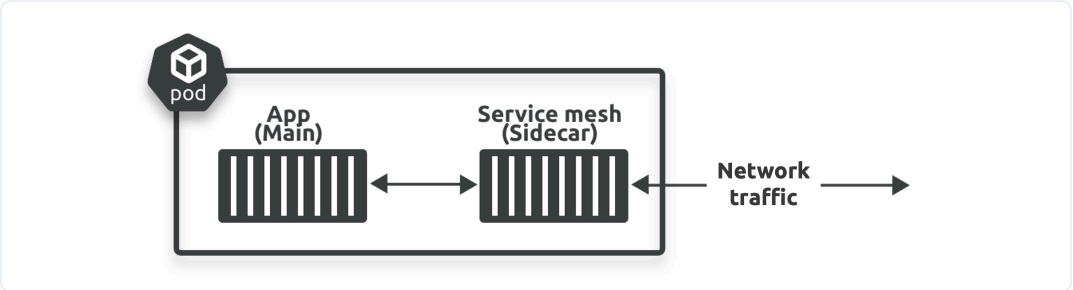
برای درک ارتباط میان کانتینر، پاد و دیپلویمنت، تصویر زیر لایه‌بندی تودرتوی آن‌ها را به زیبایی نمایش می‌دهد:



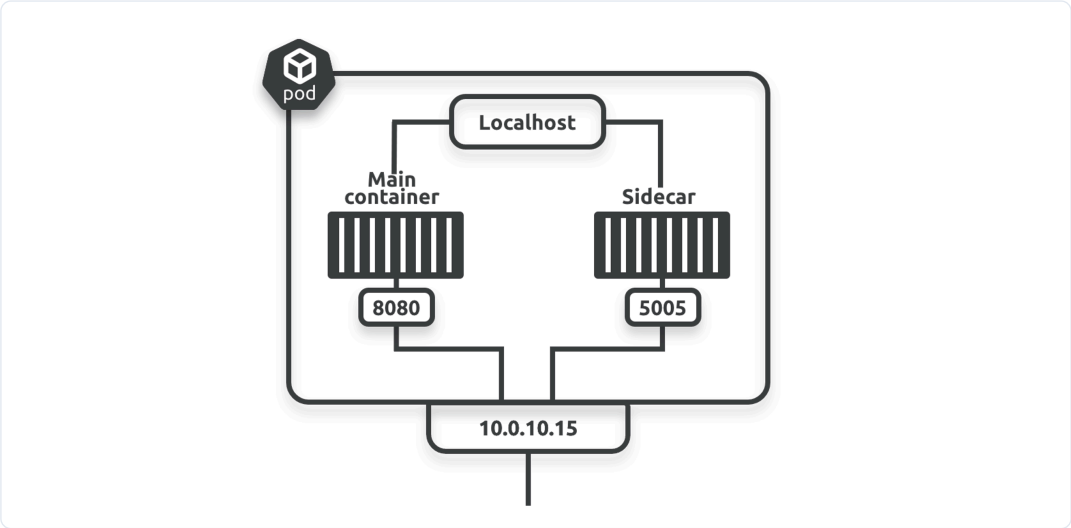
تصویر ۲.۷ - ساختار تودرتوی اشیاء: کانتینر در پاد و پاد در دیپلویمنت

ویژگی‌های کلیدی پاد:

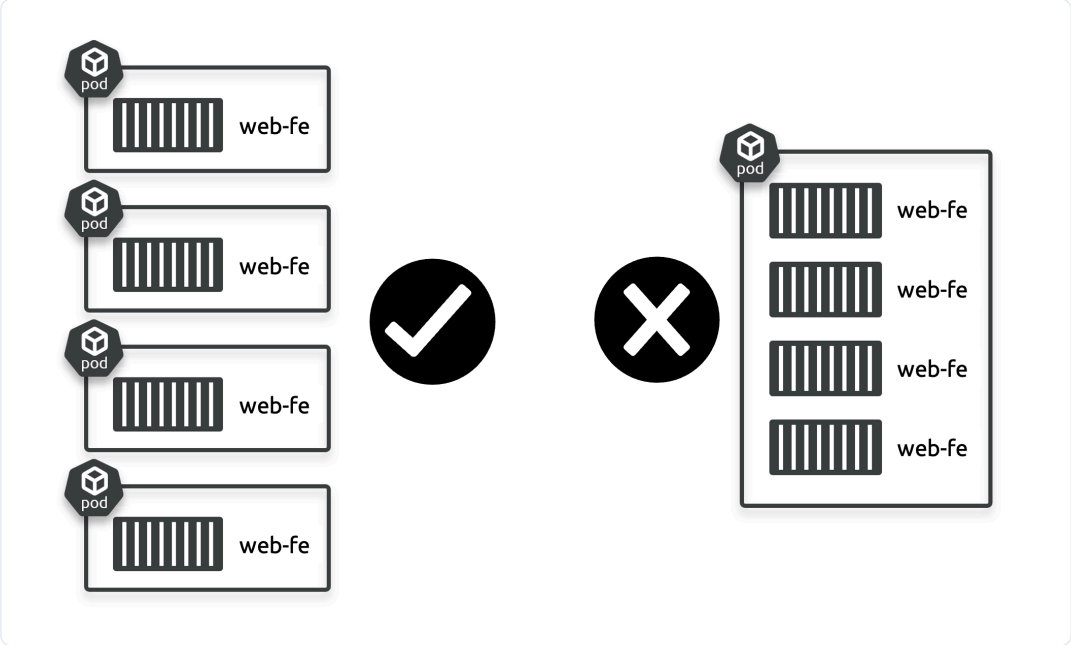
- **کوچکترین واحد استقرار و اسکیل (Scale):** شما در کوبرنتیز پادها را اسکیل می‌کنید، نه کانتینرهای درون یک پاد را.
- **محیط اجرایی اشتراکی:** تمام کانتینرهای درون یک پاد، در یک سرور واحد اجرا می‌شوند و آدرس IP، پورت‌ها و دیسک‌های مشترک دارند.
- **رفتار اتمیک (Atomic):** استقرار پاد صفر یا یک است؛ تا زمانی که تمام کانتینرهای درون پاد با موفقیت بالا نیایند، پاد در وضعیت آماده (`Ready`) قرار نخواهد گرفت.
- **تغییرناپذیری (Immutability):** پادها قابل ویرایش در حین اجرا نیستند! اگر کدی را تغییر دهید، هرگز به داخل پاد لاگین نمی‌کنید تا فایل را ویرایش کنید؛ بلکه پاد قدیمی نابود شده و پادی کاملاً تازه با تنظیمات جدید جایگزین آن می‌گردد.
- **فناپذیری (Pods are Mortal):** پادها جاودانه نیستند. متولد می‌شوند، زندگی می‌کنند و می‌میرند. هر پاد جدیدی که متولد شود، شناسه (ID) و آدرس IP متفاوتی خواهد داشت.



تصویر ۲.۸ - پاد چندکانتینری و الگوی سایدکار در مش شبکه (Service Mesh)



تصویر ۲.۹ - اشتراک‌گذاری IP و والیوم دیسک در میان کانتینرهای یک پاد



تصویر ۲.۱۰ - اسکیل کردن اپلیکیشن با افزایش پادها به عنوان واحد اسکیل (Scale)

—

۵. دیپلویمنت‌ها (Deployments): مدیران پادها

اگر پادها فناپذیرند و به سادگی از بین می‌روند، چگونه می‌توان سرویسی پایدار ارائه داد؟ پاسخ استفاده از **دیپلویمنت (Deployment)** است.

دیپلویمنت یک آبجکت کنترلی در سطح بالاتر است که مدیریت چرخه حیات پادها را به عهده می‌گیرد. دیپلویمنت قابلیت‌های زیر را اضافه می‌کند:

- **خودترمیمی (Self-healing):** اگر پادی به هر دلیلی کرش کند یا سرور آن بمیرد، دیپلویمنت فوراً پاد جایگزین می‌سازد.

- **اسکیل کردن (Scaling):** با یک تغییر ساده در مانیفست، تعداد نمونه‌ها از ۲ به ۲۰ افزایش می‌یابد.
- **به‌روزرسانی بدون قطعی (Rolling Update):** نسخه‌های جدید برنامه را دانه‌دانه جایگزین نسخه‌های قدیمی می‌کند تا ترافیک کاربران هرگز قطع نشود.
- **رول‌بک تاریخی (Rollback):** اگر نسخه جدید باگ داشت، با یک دستور به نسخه پایدار پیشین بازمی‌گردد.

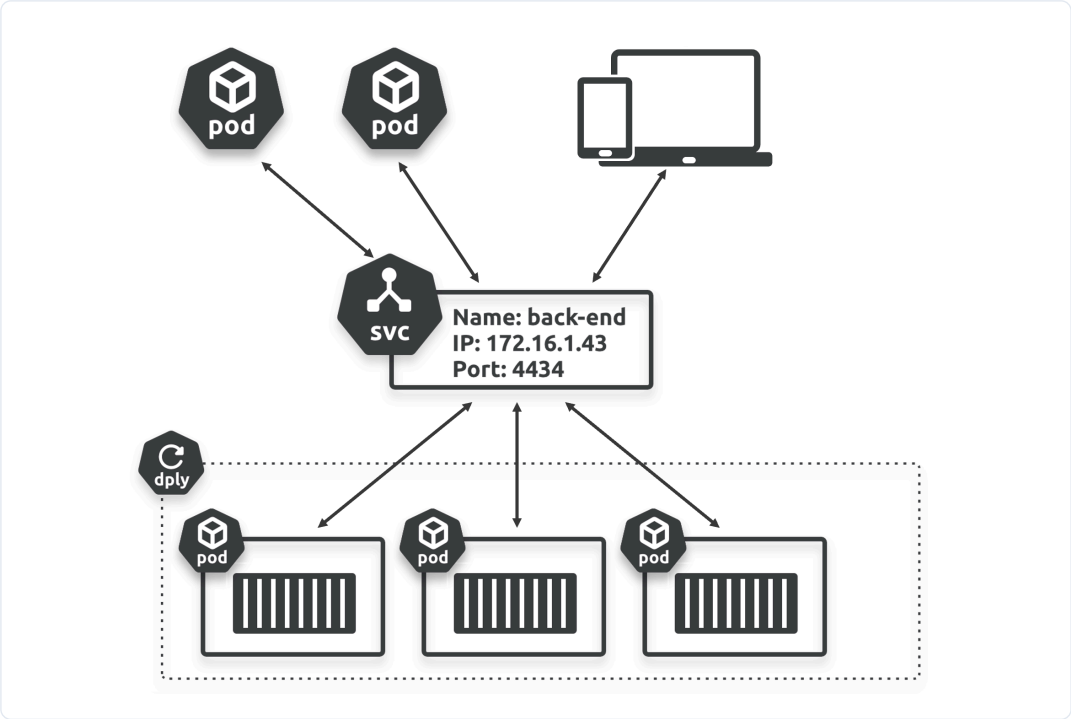
—

۶. سرویس‌ها (Services): لنگرگاه پایدار شبکه

دیدیم که پادها مدام می‌آیند و می‌روند؛ در عملیات اسکیل (Scale) پاد جدید ساخته می‌شود، در به‌روزرسانی پاد قبلی حذف می‌شود و با هر تغییر، آدرس IP پادها عوض می‌شود (IP Churn). اگر کلاینت‌ها بخواهند به پادها وصل شوند، آدرس چه کسی را صدا بزنند؟

اینجاست که شیء حیاتی **Service** وارد میدان می‌شود:

- سرویس یک آدرس IP مجازی ثابت (Virtual IP) و یک نام دامنه دائمی (DNS Name) در کلاستر در اختیارتان می‌گذارد.
- سرویس یک **فرانت‌اند پایدار** دارد که هیچ‌گاه تغییر نمی‌کند.
- در بخش **بک‌اند**، با استفاده از برچسب‌ها (Labels) پادهای فعال و سالم را شناسایی کرده و بار ترافیک کلاینت‌ها را به صورت مساوی میان آن‌ها لودبالانس می‌کند.



تصویر ۲.۱۱ - اتصال پایدار کلاینت‌ها به پادها از طریق آبجکت سرویس

کلاینت‌ها یا مرورگرها → سرویس پایدار (DNS & ClusterIP ثابت) → پادهای پویا (با IPهای متغیر)

—

جمع‌بندی فصل دوم

۱. کلاستر کوبرنتیز ترکیبی از نودهای مغز متفکر (Control Plane) و گره‌های کارگری (Worker) است.
۲. در محیط‌های پروداکشن، حداقل ۳ یا ۵ نود کنترلی برای جلوگیری از شکست quorum در دیتابیس etcd الزامی است.
۳. تمام تعاملات و امنیت کلاستر از فیلتر `kube-apiserver` می‌گذرد و `kubelet` مأمور اجرای دستورات روی نودها است.
۴. پاد کوچکترین واحد زمان‌بندی و تغییرناپذیر (Immutable) است.
۵. دیپلویمنت پادها را مدیریت و اسکیل می‌کند (Scale)، و سرویس شبکه پایدار و بدون وقفه را برای ارتباط با آن‌ها تضمین می‌نماید.

فصل ۳: راه‌اندازی و دسترسی به کلاستر (Getting Kubernetes)

تئوری بدون عمل در دنیای دواپس ارزش چندانی ندارد. برای این‌که بتوانید پادها، سرویس‌ها، والیوم‌ها و مکانیزم‌های امنیتی این کتاب را لمس کنید، نیازمند یک زمین بازی واقعی هستید: **یک کلاستر کوبرنتیز فعال**.

در این فصل، دو رویکرد مکمل را برای در اختیار داشتن کلاستر بررسی می‌کنیم: ۱. **راه‌اندازی کلاستر محلی چندگره‌ای (Multi-Node)** روی لپ‌تاپ شخصی برای آزمایش‌های سریع روزمره. ۲. **راه‌اندازی کلاستر مدیریت‌شده در کلود** (مانند LKE یا گزینه‌های ابری مشابه) برای تست سرویس‌های نیازمند دیسک واقعی و لودبالانسرهای عمومی. ۳. **کالبدشکافی عمیق ابزار `kubectl` و فایل پیکربندی `kubeconfig`** .

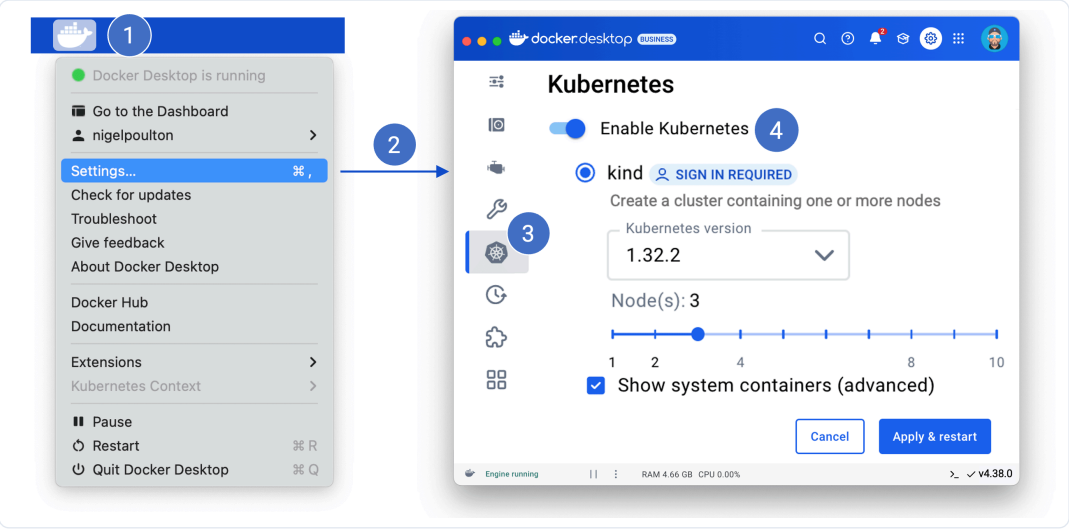
—

۱. راه‌اندازی کلاستر چندگره‌ای بر روی رایانه شخصی

تا پیش از نسخه ۲۰۲۵، اغلب کلاسترهای محلی داکر دسکتاپ تک‌گره‌ای (Single-node) بودند. اما در نسخه‌های مدرن Docker Desktop (نگارش ۴.۳۸ به بعد)، قابلیتی تعبیه شده است که به شما اجازه می‌دهد در کسری از دقیقه یک کلاستر کامل **چندگره‌ای (Multi-Node)** را بر پایه فناوری **kind (Kubernetes in Docker)** بالا بیاورید!

مراحل راه‌اندازی با Docker Desktop:

۱. نرم‌افزار Docker Desktop را متناسب با سیستم‌عامل خود (لینوکس، مک یا ویندوز با WSL2) نصب و اجرا کنید. ۲. وارد بخش تنظیمات (Settings) شوید و در تب General، گزینه استفاده از `containerd` را فعال کنید. ۳. در منوی سمت چپ به بخش **Kubernetes** بروید. ۴. تیک **Enable Kubernetes** را بزنید و نوع کلاستر را بر روی گزینه **kind (sign-in required)** قرار دهید. ۵. اسلایدر تعداد نودها را روی **۳ نود** تنظیم کنید (یک گره کنترلی و دو گره کاری). ۶. بر روی دکمه **Apply & restart** کلیک کنید.



تصویر ۳.۱ - تنظیمات کلاستر چندگره‌ای بر پایه kind در Docker Desktop

💡 ترفند و تجربه عملیاتی:

اگر از گزینه‌هایی نظیر داکر دسکتاپ استفاده نمی‌کنید، جای هیچ نگرانی نیست! ابزارهای بازمتن و رایگان زیر دقیقاً همین کلاستر چندگره‌ای را در لینوکس یا ویندوز در اختیارتان می‌گذارند:

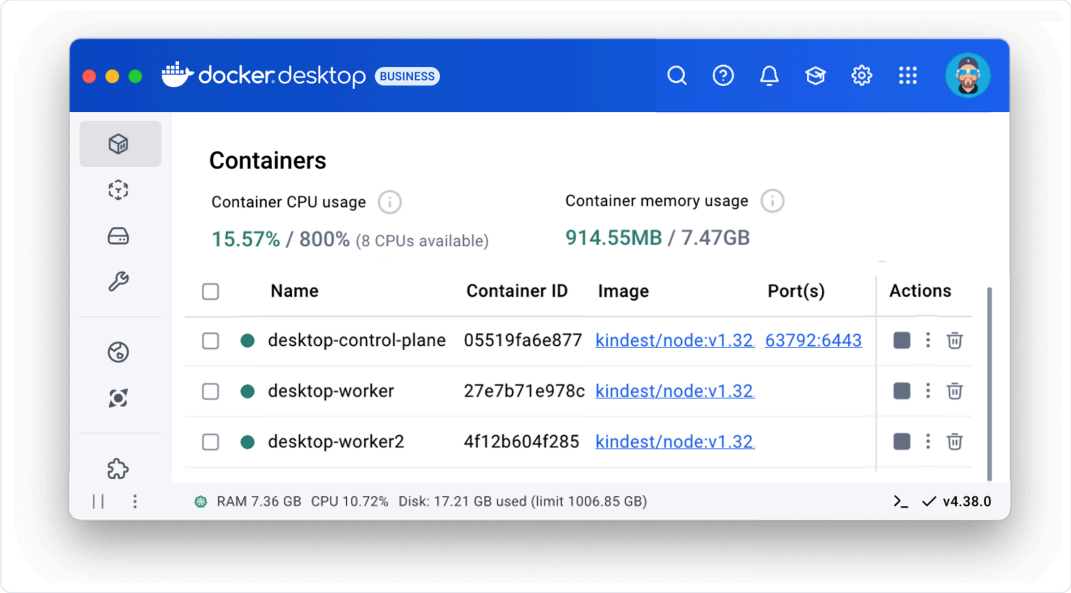
- **kind:** اجرای نودهای کلاستر به صورت کانتینرهای داکر با فرمان `kind create cluster --config ...`
- **k3d / K3s:** توزیع فوق‌العاده سبک و کم‌مصرف شرکتی Rancher برای سیستم‌های با حافظه رم پایین.
- **Minikube:** ابزار کلاسیک با دستور `minikube start --nodes 3`

پس از بالا آمدن کلاستر، ترمینال را باز کنید و فرمان زیر را اجرا کنید:

```
kubectl get nodes
```

خروجی باید مشابه زیر، ۳ گره آماده به‌کار را به شما نشان دهد:

```
NAME                    STATUS   ROLES           AGE   VERSION
desktop-control-plane   Ready    control-plane   10m   v1.32.2
desktop-worker          Ready    <none>          10m   v1.32.2
desktop-worker2         Ready    <none>          10m   v1.32.2
```



تصویر ۳.۲ - مشاهده نودهای سه‌گانه کلاستر در قالب کانتینرها در محیط Docker Desktop

تحلیل فنی: نودها درون کانتینر چگونه کار می‌کنند؟

اگر در داکر تب Containers را باز کنید، سه کانتینر با نام‌های نودهای بالا می‌بینید. در فناوری kind، هر نود کوبرنتیز در حقیقت خودش یک کانتینر داکر است که سرویس‌های کیوبلت و کانتینرد را میزبانی می‌کند. این

تکنیک به شما اجازه می‌دهد معماری شبکه و توزیع چندگره‌ای را دقیقاً مشابه سرورهای واقعی شبیه‌سازی کنید، بدون این‌که نیازمند چندین ماشین مجازی سنگین باشید.

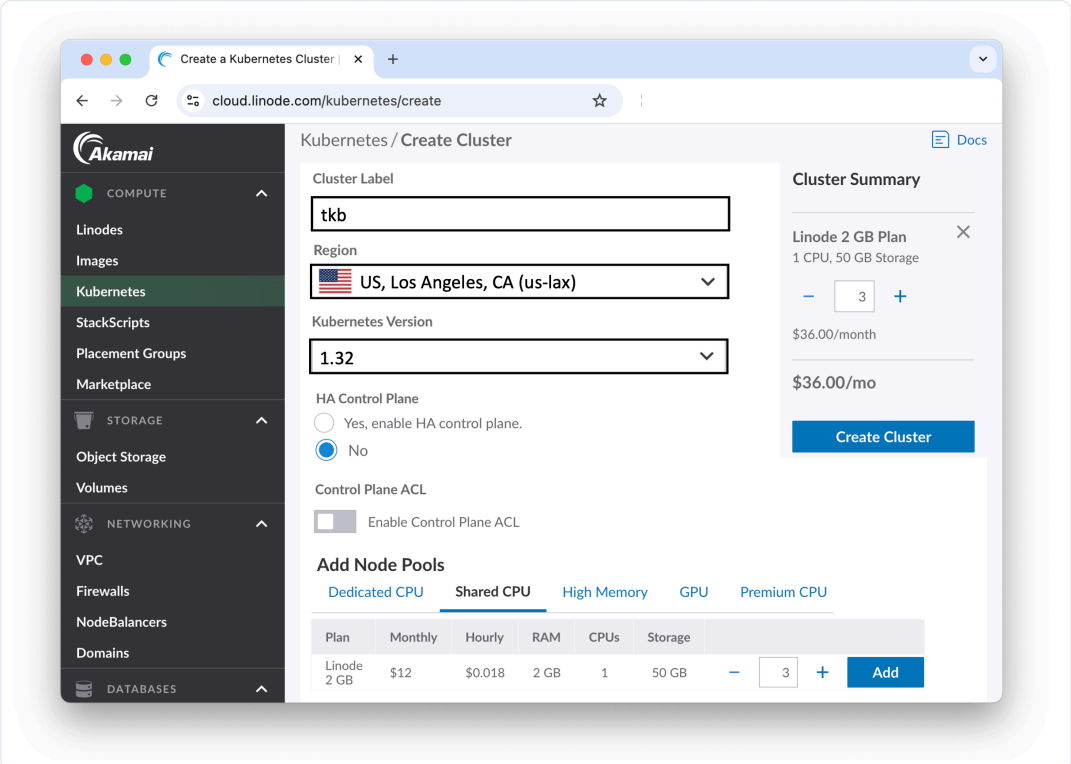
—

۲. کلاسترهای ابری مدیریت‌شده (Managed Kubernetes)

برای بخش‌هایی از کتاب (به‌ویژه فصل ۸ مربوط به اینگرس لایه ۷ و فصل ۱۱ مربوط به اختصاص خودکار دیسک‌های پایدار CSI)، شما نیاز به لودبالانسر شبکه بیرونی و استوریج ابری دارید.

در سرویس‌های مدیریت‌شده (نظیر Linode Kubernetes Engine - LKE، گوگل GKE، آمازون EKS، مایکروسافت AKS یا ارائه‌دهندگان ابری داخلی):

- وظیفه سنگین نگهداری، آپدیت و پایداری نودهای Control Plane و دیتابیس etcd بر عهده ارائه‌دهنده ابری است (معمولاً رایگان یا با هزینه حداقلی).
- شما فقط هزینه ماشین‌های ورکر و منابع مصرفی را می‌پردازید.
- پس از ساخت کلاستر در پنل ابری، یک فایل اتصال تحت عنوان `kubeconfig` در قالب YAML دانلود می‌کنید.



تصویر ۳.۳ - تنظیمات ساخت کلاستر سه‌گره‌ای در سرویس ابری مدیریت‌شده Linode LKE

—

۳. کالبدشکافی `kubectl` و فایل جادویی `kubeconfig`

ابزار خط فرمان `kubectl` چاقوی سوئیسی و ابزار دست روزمره مهندسان کوبرنتیز است.

> ⚠️ **هشدار:**
>
> **قانون عدم تطابق نسخه (Version Skew Policy):** نسخه کلاینت `kubectl` شما باید حداکثر **یک نسخه فرعی (Minor)** با نسخه سرور کلاستر فاصله داشته باشد. به عنوان مثال، اگر کلاستر شما نگارش **v1.32** است، نسخه ابزار kubectl بر روی سیستم شما نباید پایین‌تر از **v1.31** یا بالاتر از **v1.33** باشد تا از خطاهای ناسازگاری دستورات جلوگیری شود:
>
> ```
> kubectl version --client=true
> ```

فایل `kubeconfig` دقیقاً چیست؟

هر بار که فرمانی مثل `kubectl get pods` را می‌زنید، ابزار kubectl از کجا می‌داند باید به کدام سرور وصل شود؟ و با کدام کلید امنیتی وارد شود؟ پاسخ در فایلی به نام `config` در پوشه مخفی `~/.kube/` سیستم شما نهفته است.

یک فایل `kubeconfig` استاندارد از سه بخش اساسی تشکیل شده است:

```
apiVersion: v1
kind: Config
clusters:
- cluster:
    certificate-authority-data: LS0tLS...
    server: https://your-k8s-cluster.example.com:6443
  name: prod-cluster

users:
- name: devops-admin
  user:
    client-certificate-data: LS0tLS...
    client-key-data: LS0tLS...

contexts:
- context:
    cluster: prod-cluster
    user: devops-admin
    namespace: default
  name: prod-context

current-context: prod-context
```

درک سه‌گانه Context

یک **کانتکست (Context)** در حقیقت اتصال‌دهنده سه مفهوم است: **«با کاربر X به کلاستر Y متصل شو و به طور پیش‌فرض در فضای نام Z کار کن.»** شما می‌توانید ده‌ها کلاستر مختلف (توسعه، تست، پروداکشن) را درون یک فایل ذخیره کرده و تنها با یک دستور بین آن‌ها جابه‌جا شوید.

دستورات کاربردی مدیریت کانتکست‌ها:

۱. مشاهده کانتکست‌های موجود:

```
kubectl config get-contexts
```

۲. سوییچ سریع به یک کلاستر دیگر:

```
kubectl config use-context <context-name>
```

۳. ادغام چند فایل کانفیگ در یک فایل واحد:

```
export KUBECONFIG=~/.kube/config:~/.kube/tkb-kubeconfig.yaml
kubectl config view --flatten > ~/.kube/merged-config
mv ~/.kube/merged-config ~/.kube/config
```

—

۴. آزمودن صحت اتصال به کلاستر

پس از تنظیم فایل کانفیگ، دستورات ارزیابی سلامت زیر را اجرا کنید:

```
# مشاهده اطلاعات آدرس API Server و سرویس‌های سیستمی
kubectl cluster-info

# مشاهده لیست کامل نودها با جزئیات سیستم‌عامل و IP
kubectl get nodes -o wide

# مشاهده پادهای زیرساختی کنترل پلین در نیم‌اسپیس kube-system
kubectl get pods -n kube-system
```

اگر نودها در وضعیت `Ready` باشند و پادهای سیستمی (مانند kube-proxy، CoreDNS و etcd) بدون خطا در حال اجرا باشند، کلاستر شما با موفقیت متولد شده و آماده ورود به دنیای هیجان‌انگیز ساخت پادها و ورک‌لودها در فاز بعدی است!

فصل ۴: کار با پادها (Working with Pods)

در دنیای کوبرنتیز یک قانون طلایی و تخطی‌ناپذیر وجود دارد: **تمام برنامه‌ها، بدون استثنا، درون پاد (Pod) اجرا می‌شوند.**

فرقی نمی‌کند برنامه شما یک وب‌سرور سنتی باشد، یک میکروسرویس فوق‌سریع در قالب کانتینر، یک تابع بدون‌سرور (Serverless Function)، یک برنامه نوظهور مبتنی بر وب‌اسمبلی (WebAssembly/Wasm)، یا حتی یک ماشین مجازی کامل (VM)؛ اگر می‌خواهید در کلاستر کوبرنتیز اجرا شود، کوبرنتیز آن را در کپسولی به نام «پاد» بسته‌بندی می‌کند.

در این فصل، پاد را نه صرفاً به عنوان یک مفهوم تئوریک ساده، بلکه به عنوان واحد بنیادین محاسبات ابری مدرن کالبدشکافی می‌کنیم. سرفصل‌های اصلی این فصل عبارتند از: ۱. **تئوری عمیق پاد و چرایی لایه انتزاعی کوبرنتیز** ۲. **نحوه ارتقای برنامه‌ها توسط پاد و اشتراک منابع در سطح کرنل سیستم‌عامل** ۳. **سازوکار زمان‌بندی پیشرفته (Advanced Scheduling)** ۴. **چرخه حیات، مرگ‌پذیری (Mortal) و اصل تغییرناپذیری (Immutable)** ۵. **معماری شبکه مسطح و سراسری پادها (Pod Network)** ۶. **پادهای چندکانتینری: از کانتینرهای آماده‌سازی (Init Containers) تا انقلاب سایدکارهای بومی (Native Sidecars) در نسخه جدید** ۷. **کارگاه عملیاتی جامع: پیکربندی مانیفست‌ها، بازرسی عمیق وضعیت، ورود به پاد، تست تغییرناپذیری و اجرای سناریوهای زنده Init و Sidecar**

—

۱. تئوری پاد (Pod Theory): چرا مستقیماً کانتینر را اجرا نمی‌کنیم؟

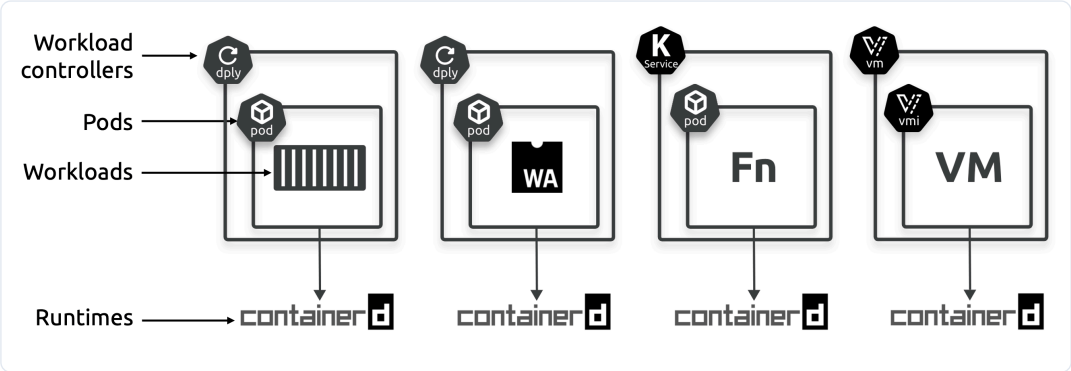
یک سوال هوشمندانه که ذهن بسیاری از مهندسان نرم‌افزار را در آغاز کار با کوبرنتیز به خود مشغول می‌کند این است: **«اگر کانتینرها تا این حد کارآمد و ایزوله هستند، چرا کوبرنتیز مستقیماً ایمیج‌های داکر یا کانتینرد را اجرا نمی‌کند و کانسپت جدیدی به نام پاد اختراع کرده است؟»**

پاسخ این پرسش در دو دلیل استراتژیک نهفته است: ۱. **ایجاد یک لایه انتزاعی همگانی (Abstraction Layer) برای بارهای کاری ناهمگون.** ۲. **ارتقا و بخشیدن امکانات فوق‌العاده به برنامه‌ها (Augmenting Workloads).**

۱. پاد به عنوان لایه انتزاعی همگانی

اکوسیستم مدرن رایانش ابری تنها به کانتینرهای سنتی OCI/Docker محدود نمی‌شود. امروزه در سازمان‌های پیشرو، گونه‌های مختلفی از بارهای کاری وجود دارند:

- **کانتینرهای استاندارد:** رایج‌ترین فرم اجرای برنامه‌ها.
- **توابع سرورلس (Serverless Functions):** توابعی که با ابزارهایی مانند Knative بر روی کلاستر مستقر می‌شوند و در زمان بی‌باری به صفر اسکیل می‌شوند (Scale-to-zero).
- **ماژول‌های وب‌اسمبلی (Wasm):** بارهای پردازشی فوق‌سبک و بدون‌سیستم‌عامل که بر پایه‌ی ران‌تایم‌هایی مانند WasmEdge یا Spin اجرا می‌شوند.
- **ماشین‌های مجازی (Virtual Machines):** به لطف پروژه‌های انقلابی نظیر **KubeVirt**، شما می‌توانید ماشین‌های مجازی سنتی را نیز مستقیماً داخل کلاستر کوبرنتیز مدیریت کنید.



تصویر ۴.۱ - پاد به عنوان لایه انتزاعی: بارهای کاری ناهمگون (کانتینرها، سرورلس، Wasm و ماشین‌های مجازی) همگی در چارچوب یکسان پاد مدیریت می‌شوند

همان‌طور که در تصویر ۴.۱ مشاهده می‌کنید، لایه کنترل پلین کوبرنتیز نیازی ندارد جزئیات فنی و ریزه‌کاری‌های متفاوت هر ران‌تایم را بشناسد. کوبرنتیز با یک موجودیت واحد و استاندارد به نام «پاد» صحبت می‌کند. هر بار کاری درون یک پاد قرار می‌گیرد، توسط کنترلرهای استاندارد مدیریت می‌شود و از طریق ران‌تایم مناسب خود به اجرا درمی‌آید.

۲. پاد چگونه بارهای کاری را ارتقا می‌دهد؟ (Pods Augment Workloads)

کانتینرها به خودی خود ابزارهایی ایزوله هستند، اما پاد امکانات فوق‌العاده‌ای به آن‌ها اضافه می‌کند که اجرای آن‌ها در مقیاس سازمانی را ممکن می‌سازد:

- **اشتراک هماهنگ منابع (Resource Sharing):** اجازه می‌دهد چند کانتینر مرتبط با هم، فضای ذخیره‌سازی، حافظه و شبکه مشترک داشته باشند.
- **زمان‌بندی هوشمند و پیشرفته (Advanced Scheduling):** هدایت بار کاری به سمت نودهای مناسب بر پایه قوانین سخت و نرم.
- **کاوشگرهای سلامت برنامه (Health Probes):** پایش مستمر وضعیت درونی برنامه برای تشخیص هنگ‌کردن یا عدم آمادگی.
- **سیاست‌های راه‌اندازی مجدد (Restart Policies):** بازیابی خودکار کانتینرهای آسیب‌دیده.
- **سیاست‌های امنیتی (Security Policies & Context):** تعیین سطوح دسترسی لینوکسی، قابلیت‌های کرنل (Linux Capabilities) و فیلترهای seccomp.
- **کنترل خروج تمیز (Termination Control):** مهلت دادن به برنامه برای بستن کانکشن‌ها و آزادسازی منابع هنگام خاموش شدن.
- **مدیریت یکپارچه حافظه پایدار (Volumes):** اتصال دیسک‌های مختلف بدون وابستگی مستقیم کانتینر به زیرساخت ذخیره‌سازی.

برای مشاهده عظمت خصیصه‌های یک پاد، کافی است دستور زیر را در ترمینال اجرا کنید:

```
kubectl explain pods --recursive | more
```

این دستور فهرستی با بیش از ۱۰۰۰ سطر از خصیصه‌ها و امکانات پیکربندی پاد را به شما نشان می‌دهد. همچنین با دستوراتی مانند زیر می‌توانید به عمق یک فیلد خاص نفوذ کنید:

```
kubectl explain pod.spec.restartPolicy
```

خروجی این دستور مقادیر مجاز برای فیلد سیاست ریستارت (`Always` ، `OnFailure` و `Never`) به همراه توضیحات فنی آن را نمایش می‌دهد. با وجود افزودن تمام این قابلیت‌های شگفت‌انگیز، پادها به شدت سبک (Lightweight) هستند و تقریباً هیچ سربار پردازشی محسوسی به سرور تحمیل نمی‌کنند.

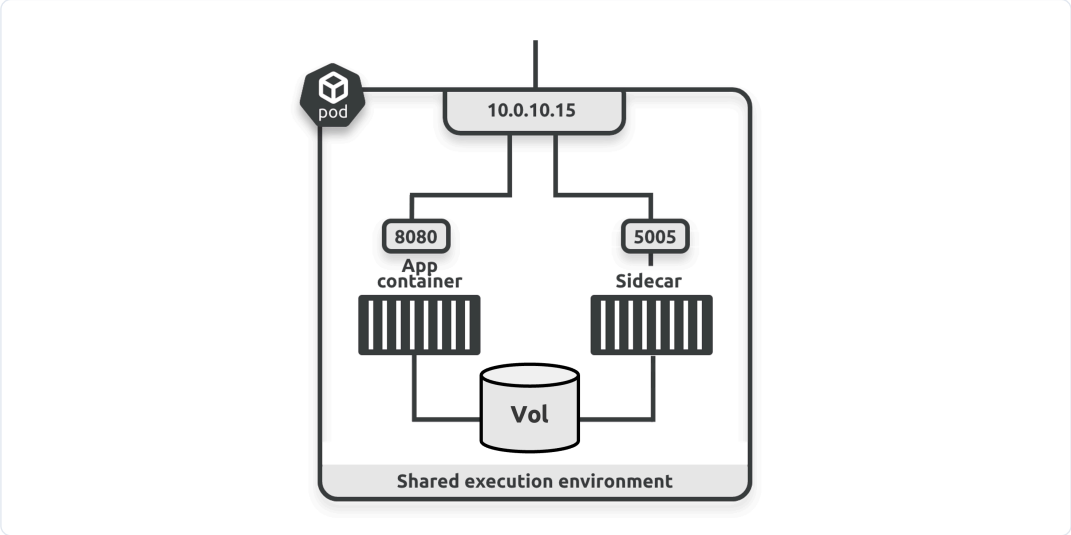
—

۲. اشتراک منابع و کالبدشکافی لینوکس نیم‌اسپیس‌ها (Resource Sharing)

یک پاد می‌تواند میزبان یک یا چند کانتینر باشد. زمانی که چندین کانتینر درون یک پاد مستقر می‌شوند، همگی در محیط اجرایی مشترک پاد سهیم خواهند بود.

در سطح هسته سیستم‌عامل لینوکس، این همزیستی از طریق به اشتراک‌گذاری چندین **Namespace** کلیدی پیاده‌سازی می‌شود:

- **سیستم فایل و دیسک‌های مشترک (`mnt` namespace):** کانتینرها می‌توانند دیسک‌های یکسانی را ماونت کرده و فایل‌ها را با سرعت بالا مبادله کنند.
- **استک شبکه مشترک (`net` namespace):** تمامی کانتینرهای یک پاد دارای **یک آدرس IP واحد** هستند.
- **ارتباطات بین‌فرآیندی مشترک (`ipc` namespace):** امکان تبادل پیام از طریق حافظه اشتراکی (Shared Memory).
- **درخت فرآیندهای مشترک (`pid` namespace):** در صورت فعال‌سازی فلگ `shareProcessNamespace` ، کانتینرهای یک پاد می‌توانند پردازه‌های یکدیگر را ببینند و سیگنال ارسال کنند.
- **نام هاست مشترک (`uts` namespace):** هر دو کانتینر یک hostname یکسان را منعکس می‌کنند.



تصویر ۴.۲ - پاد چندکانتینری: اشتراک آدرس IP واحد، لوپ‌بک و والیوم مشترک میان کانتینر اصلی و سایدکار

همان‌طور که در تصویر ۴.۲ مشاهده می‌کنید:

- کلاینت‌ها و سرویس‌های بیرونی با آدرس IP پاد (`10.0.10.15`) ارتباط برقرار می‌کنند.
- کانتینر اصلی بر روی پورت `8080` و کانتینر سایدکار بر روی پورت `5005` به ترافیک پاسخ می‌دهند. (به دلیل اشتراک استک شبکه، دو کانتینر در یک پاد هرگز نمی‌توانند روی یک پورت یکسان گوش دهند).
- دو کانتینر می‌توانند به سادگی از طریق رابط `localhost` با یکدیگر بدون سربار شبکه صحبت کنند.

• هر دو کانتینر یک والیوم مشترک را ماونت کرده‌اند؛ به عنوان مثال، سایدکار تغییرات کد را از یک مخزن گیت کلون کرده و روی والیوم می‌ریزد و کانتینر وب همان فایل‌ها را به کاربران سرو می‌کند.

—

۳. زمان‌بندی پیشرفته و استقرار پادها (Pod Scheduling)

نودهای کلاستر سرورهای میزبان (فیزیکی، مجازی یا ابری) هستند و پادها بسته‌بندی کانتینرها هستند که روی این نودها قرار می‌گیرند.

> 📌 **نکته مهم:**
>
> کوبرنتیز به طور صددرصد تضمین می‌کند که **تمامی کانتینرهای حاضر در یک پاد، دقیقاً بر روی یک نود کلاستر اجرا شوند.** کوبرنتیز هرگز کانتینرهای یک پاد واحد را بین دو ماشین مختلف پخش نمی‌کند.

با این حال، یک قاعده طلایی مهندسی را هرگز فراموش نکنید: **فقط زمانی کانتینرها را در یک پاد قرار دهید که نیازمند اشتراک محلی منابع (دیسک اشتراکی، حافظه، شبکه فوق‌سریع localhost) باشند.** اگر تنها می‌خواهید دو برنامه مستقل روی یک سرور مشترک باشند، نباید آن‌ها را در یک پاد بگذارید؛ بلکه باید آن‌ها را به صورت دو پاد مجزا بسازید و از قابلیت‌های زمان‌بندی کوبرنتیز برای کنار هم نشاندن آن‌ها استفاده کنید.

کوبرنتیز چهار ابزار قدرتمند برای هدایت و زمان‌بندی پادها ارائه می‌دهد:

۱. انتخابگرهای نود (nodeSelectors)

ساده‌ترین روش برای اختصاص پاد به ماشین‌های خاص است. شما مجموعه‌ای از برچسب‌ها (Labels) مانند `disk=ssd` یا `gpu=true` را مشخص می‌کنید و زمان‌بند کوبرنتیز (`kube-scheduler`) پاد را فقط روی سرورهایی که واجد این برچسب‌ها هستند می‌نشاند.

۲. قوانین گرایش و دافعه (Affinity & Anti-affinity)

نسخه بسیار منعطف‌تر و پیشرفته‌تر `nodeSelector` است.

• **گرایش (Affinity):** تمایل به قرارگیری کنار نودها یا سایر پادها (جذب).
• **دافعه (Anti-affinity):** پرهیز از قرارگیری روی نودهایی خاص یا نودهایی که پادهای رقیب روی آن‌هاست (دفع).
• **قوانین سخت (Hard Rules):** با عبارت `requiredDuringSchedulingIgnoredDuringExecution` تعریف می‌شوند. رعایت این شرط اجباری است؛ اگر سروری با این مشخصات پیدا نشود، پاد معلق می‌ماند و اجرا نمی‌شود.
• **قوانین نرم (Soft Rules):** با عبارت `preferredDuringSchedulingIgnoredDuringExecution` تعریف می‌شوند. این قوانین نقش اولویت و توصیه دارند؛ اگر سروری با آن شرط نبود، زمان‌بند کوتاه می‌آید و پاد را روی نودهای معمولی دیگر می‌نشاند.

> 💡 **ترفند و تجربه عملیاتی:**

یکی از رایج‌ترین کاربردهای Anti-affinity در پروداکشن، تفکیک نمونه‌های یک سرویس است: به کوبرنتیز می‌گوییم هرگز دو پاد پایگاه داده یا دو پاد سبد خرید را روی یک نود فیزیکی یکسان قرار ندهد تا اگر آن نود سوخت، تمام سرویس از دسترس خارج نشود!

۳. محدودیت‌های توزیع توپولوژی (Topology Spread Constraints)

روشی فوق‌العاده هوشمند برای پخش یکنواخت پادها در میان دامنه‌های شکست زیرساختی (Failure Domains) مانند مناطق در دسترس‌پذیری ابری (Availability Zones)، رک‌ها یا دیتاسنترهای مختلف است. این مکانیزم تضمین می‌کند توزیع ترافیک و تحمل‌پذیری خطای سرویس شما در کل زیرساخت ماکزیمم باشد.

۴. درخواست‌ها و محدودیت‌های منابع (Requests & Limits)

مشخص‌کردن حداقل و حداکثر پردازنده (CPU) و حافظه (RAM) مورد نیاز برنامه است. زمان‌بند بر پایه مقدار **Requests** تصمیم می‌گیرد کدام نود گنجایش میزبانی پاد را دارد.

—

۴. جریان ۹ مرحله‌ای استقرار پاد در کلاستر

فرآیند استقرار یک پاد از زمان نوشتن فایل مانیفست تا شروع سرویس‌دهی، یک زنجیره کاملاً اتمیک (Atomic) و هماهنگ به شرح زیر است:

۱. **تعریف اعلانی:** توسعه‌دهنده پاد را در قالب یک فایل مانیفست YAML توصیف می‌کند. ۲. **ارسال به ورودی:** مانیفست با دستور `kubectl apply` به سرور API ارسال می‌شود. ۳. **احراز هویت و امنیت:** درخواست در لایه API Server اعتبارسنجی، احراز هویت و دسترسی‌سنجی (RBAC) می‌شود. ۴. **اعتبارسنجی ساختاری:** مانیفست پاد با اسکیماهای رسمی کوبرنتیز تطبیق داده می‌شود. ۵. **فیلتر و امتیازدهی در زمان‌بند:** سرویس `kube-scheduler` وارد عمل شده و تمامی نودهای کلاستر را بر مبنای قوانین انتخابی، گرایش‌ها، توزیع توپولوژی و ظرفیت آزاد CPU/RAM فیلتر و امتیازدهی می‌کند. ۶. **انتساب به نود سالم:** پاد به بهترین نود واجد شرایط نسبت داده می‌شود (`binding`). ۷. **دریافت رویداد توسط کیوبلت:** سرویس `kubelet` مستقر روی آن نود خاص متوجه انتساب پاد جدید می‌شود. ۸. **دستور به ران‌تایم:** کیوبلت مانیفست را خوانده و به موتور ران‌تایم محلی (مانند `containerd`) دستور دانلود ایمیج‌ها و اجرای کانتینرها را می‌دهد. ۹. **گزارش پیوسته وضعیت:** کیوبلت سلامت کانتینرها را پایش کرده و وضعیت نهایی را به API Server گزارش می‌کند.

رفتار اتمیک (Atomic Operation) در اجرای پاد

دیپلوی پاد یک عملیات اتمیک (همه یا هیچ) است. یعنی یک پاد تنها و تنها زمانی وارد فاز سرویس‌دهی و پردازش ترافیک واقعی می‌شود که **تمام کانتینرهای تعریف‌شده درون آن** با موفقیت بالا آمده باشند و سلامت آن‌ها توسط پروب‌ها تایید شود.

—

۵. فلسفه چرخه حیات: مرگ‌پذیری و تغییرناپذیری پادها

پادها در فلسفه معماری ابری بر پایه دو اصل زیربنایی خلق شده‌اند: ۱. **پادها فناپذیرند (Pods are Mortal)**. ۲. **پادها تغییرناپذیرند (Pods are Immutable)**.

اصل ۱: پادها فناپذیرند (Mortal)

در دنیای سنتی، وقتی سروری دچار مشکل می‌شد، ساعت‌ها تلاش می‌کردیم آن را تعمیر و ریستارت کنیم (مدل حیوان خانگی یا Pets). اما پادها گله گوسفندان (Cattle) هستند! اگر پادی دچار خرابی سخت‌افزاری نود شود یا حذف گردد، **کوبرنتیز هرگز پاد مرده را زنده نمی‌کند!** بلکه کنترلرهای ناظر (مانند Deployment)، متوجه عدم تطابق وضعیت موجود با حالت مطلوب شده، پاد آسیب‌دیده را رها کرده و یک **پاد کاملاً جدید با هویت و آدرس IP تازه** بر روی نودی سالم متولد می‌کنند.

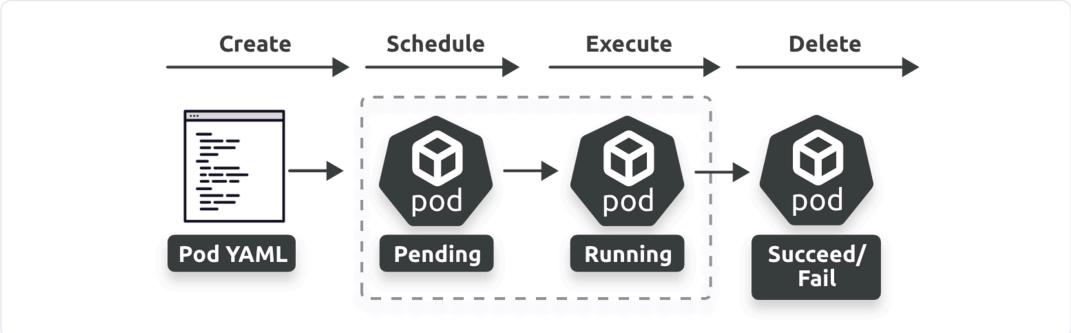
اصل ۲: پادها تغییرناپذیرند (Immutable)

هنگامی که پادی مستقر شد، شما حق تغییر پیکربندی آن (مانند تغییر پورت، تغییر ایمیج یا تغییر متغیرهای محیطی) را ندارید. ورود به کانتینر در حال اجرا و پچ کردن دستی کدها یک ضدالگو و خطای بزرگ در دواپس است. اگر تغییری نیاز است: فایل مانیفست YAML به‌روزرسانی می‌شود، پاد جدید ساخته می‌شود و جایگزین پاد پیشین می‌گردد. داده‌های ماندگار نیز هرگز در خود پاد ذخیره نمی‌شوند، بلکه درون والیوم‌های پایدار (Persistent Volumes) قرار می‌گیرند تا با نابودی پاد، هیچ داده‌ای از بین نرود.

فازهای چرخه حیات پاد (Pod Lifecycle Phases)

یک پاد در طول عمر خود از فازهای زیر عبور می‌کند:

- **در حال انتظار (Pending):** مانیفست تایید شده اما هنوز زمان‌بند نود مناسب را پیدا نکرده یا ایمیج کانتینرها در حال دانلود از رجیستری است.
- **در حال اجرا (Running):** پاد به نود مقید شده و تمام کانتینرهای اصلی آن حداقل یک بار ایجاد شده و در حال اجرا هستند.
- **موفقیت‌آمیز (Succeeded):** برای جاب‌ها و کارهای دسته‌ای (Batch Jobs)؛ زمانی که تمام کانتینرها وظیفه خود را انجام داده و با کد وضعیت ۰ خاموش شده‌اند.
- **با خطا مواجه شده (Failed):** حداقل یک کانتینر با خطای غیرصفر متوقف شده یا توسط سیستم کشته شده است.
- **نامشخص (Unknown):** به دلیل قطعی شبکه یا از کار افتادن نود، لایه کنترل پلین نمی‌تواند وضعیت سلامت پاد را از کیوبلت دریافت کند.



تصویر ۴.۳ - چرخه حیات و فازهای بنیادین یک پاد از زمان تعریف تا اجرا و خاتمه

سیاست‌های راه‌اندازی مجدد کانتینرها (Restart Policies)

اگرچه کوبرنتیز پادهای مرده را ریستارت نمی‌کند، اما **کانتینرهای درون پاد** در صورت کرش کردن می‌توانند توسط سرویس محلی کیوبلت مجدداً راه‌اندازی شوند. این رفتار با فیلد `restartPolicy` در مانیفست پاد کنترل می‌شود:

- `Always` : (مقدار پیش‌فرض) در صورت توقف کانتینر به هر دلیلی، کیوبلت فوراً آن را ریستارت می‌کند. مناسب وب‌سرورها و سرویس‌های همیشه‌روشن.
- `OnFailure` : تنها در صورتی که کانتینر با خطایی غیر از صفر کرش کند، ریستارت می‌شود. مناسب تسک‌های پردازشی.
- `Never` : اگر کانتینر متوقف شد، تحت هیچ شرایطی ریستارت نخواهد شد.

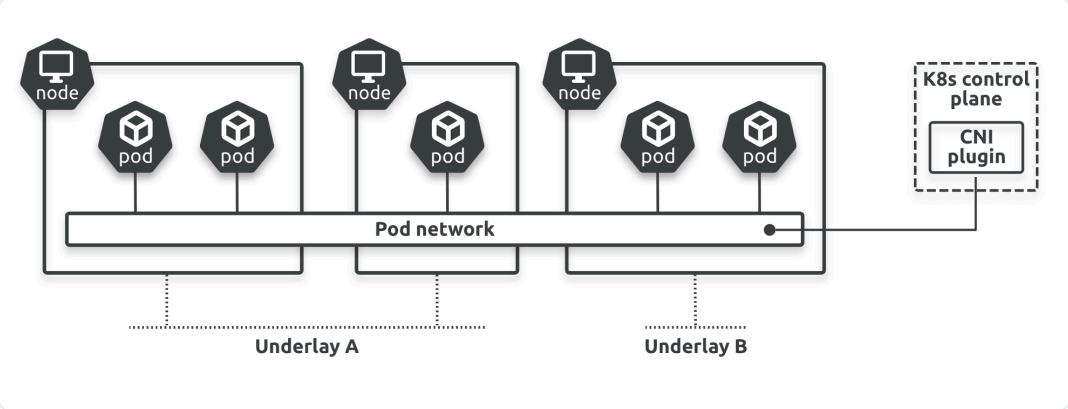
—

۶. مدل شبکه مسطح پادها (Pod Networking Model)

در کوبرنتیز مدل شبکه بر اساس یک اصل بسیار ساده و درخشان بنا شده است:

«هر پاد در کلاستر، آدرس IP منحصربه‌فرد، واقعی و قابل روت خود را در کل کلاستر دارد.»

در معماری شبکه کوبرنتیز: ۱. پادها بر روی تمام نودها می‌توانند بدون نیاز به ترجمه آدرس شبکه (NAT) مستقیماً با یکدیگر از طریق IP پاد صحبت کنند. ۲. نودها می‌توانند بدون NAT با تمام پادها تبادل داده داشته باشند. ۳. آدرسی که یک پاد برای خودش می‌بیند، دقیقاً همان آدرسی است که سایر پادها برای آن می‌بینند.



تصویر ۴.۴ - شبکه مسطح و فراگیر پادها (Pod Network): ارتباط مستقیم میان پادها در نودهای مختلف بدون NAT

همان‌طور که در تصویر ۴.۴ مشخص است، شبکه پادها (Pod Network) به صورت یک لایه مجازی (Overlay یا روت مستقیم) بر فراز تمامی ماشین‌های کلاستر گسترده شده است. این معجزه توسط **پلاگین‌های CNI (Container Network Interface)** محقق می‌شود.

انقلاب eBPF و پیشتازی Cilium در نسخه ۲۰۲۵

تا سال‌ها پلاگین‌های کلاسیک نظیر Flannel و Calico بر پایه ابزار قدیمی `iptables` لینوکس ارتباطات شبکه را مدیریت می‌کردند. اما در نسخه ۲۰۲۵، ابزار **Cilium** با بهره‌گیری از فناوری انقلابی **eBPF** در کرنل لینوکس، به محبوب‌ترین و قدرتمندترین شبکه کوبرنتیز تبدیل شده است. Cilium علاوه بر برقراری شبکه مسطح فوق‌سریع، امنیت شبکه در سطح لایه ۷، رمزنگاری خودکار ترافیک با WireGuard و پایش لحظه‌ای بدون بار محاسباتی (Observability) را به ارمغان می‌آورد.

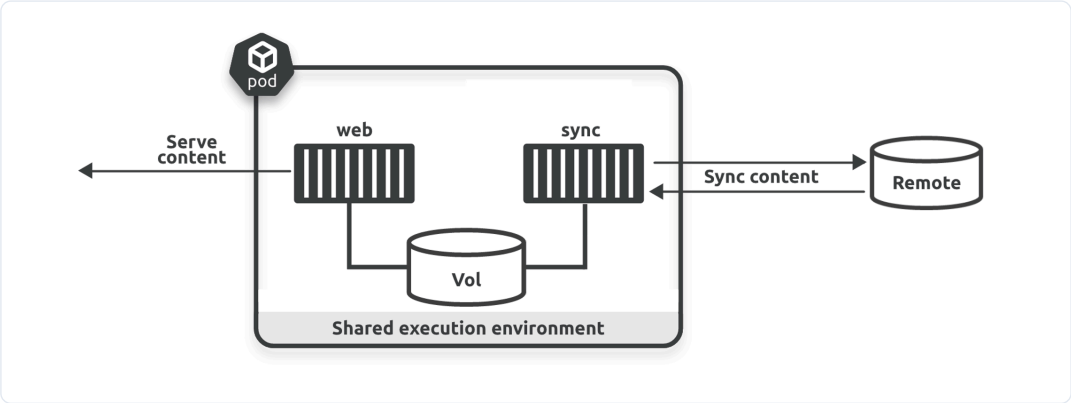
—

۷. الگوهای پادهای چندکانتینری: تولد سایدکارهای بومی (Native Sidecars)

در الگوهای طراحی میکروسرویس، یک اصل حیاتی وجود دارد: **اصل مسئولیت واحد (Single Responsibility Principle - SRP)**. هر کانتینر باید تنها یک وظیفه مشخص و مجزا داشته باشد و آن را به بهترین شکل ممکن انجام دهد.

فرض کنید برنامه‌ای دارید که دو وظیفه دارد: ۱. دریافت و همگام‌سازی فایل‌های جدید وب‌سایت از مخزن گیت. ۲. سرو کردن صفحات وب به بازدیدکنندگان از طریق وب‌سرور Nginx.

اگر هر دوی این کارها را درون یک کانتینر واحد بریزید، کانتینر شما سنگین و حجیم می‌شود، خطایابی آن سخت خواهد بود و امکان ارتقای مستقل نرم‌افزارها از بین می‌رود. راهکار صحیح: تفکیک این دو وظیفه به دو کانتینر مجزا درون یک پاد مشترک!



تصویر ۴.۵ - الگوی پاد چندکانتینری: تفکیک وظیفه همگام‌سازی گیت و وب‌سرور با استفاده از یک دیسک اشتراکی

در کوبرنتیز دو الگوی رسمی برای پادهای چندکانتینری وجود دارد: ۱. **کانتینرهای آماده‌سازی (Init Containers)** ۲. **کانتینرهای سایدکار (Sidecar Containers)**

۱. کانتینرهای آماده‌سازی (Init Containers)

کانتینرهایی ویژه هستند که در مانیفست زیر بخش `spec.initContainers` تعریف می‌شوند. کوبرنتیز تضمین‌های سفت‌وسختی برای کانتینرهای Init ارائه می‌دهد:

- این کانتینرها **همیشه پیش از شروع کانتینرهای اصلی برنامه** اجرا می‌شوند.
- اگر چند Init Container تعریف کنید، **دقیقاً به ترتیبی که در فایل آمده‌اند و یکی پس از دیگری (سریال)** اجرا خواهند شد.

- کانتینرهای اصلی تنها زمانی روشن می‌شوند که تمام کانتینرهای Init کار خود را با موفقیت (کد خروج ۰) به پایان رسانده باشند.
- این کانتینرها فقط یک‌بار اجرا می‌شوند و خاموش می‌گردند.

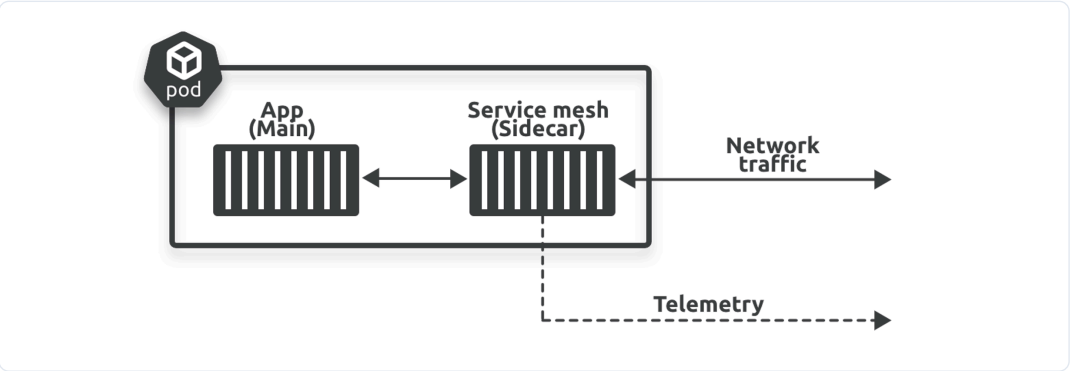
کاربردهای عملیاتی Init Containers:

- انتظار برای در دسترس قرار گرفتن پایگاه داده یا وب‌سرویس خارجی پیش از روشن شدن برنامه اصلی.
- دانلود گواهینامه‌های SSL، کلیدهای امنیتی یا بارگذاری اولیه فایل‌های کانفیگ درون دیسک اشتراکی پاد.
- رجیستر کردن پاد در سرویس‌های مانیتورینگ بیرونی.

۲. کانتینرهای سایدکار و تحول Native Sidecars در نسخه ۲۰۲۵

وظیفه سایدکار ارائه یک سرویس مکمل به کانتینر اصلی است، اما برخلاف کانتینر Init، سایدکار باید **در تمام طول عمر پاد و در کنار کانتینر اصلی زنده بماند.** نمونه‌های کلاسیک سایدکار:

- کانتینرهای جمع‌آوری و ارسال لاگ‌ها (Log Shippers مانند Fluentbit).
- کانتینرهای نظارت و همگام‌سازی کانفیگ (مانند `git-sync`).
- پروکسی‌های سرویس‌مش (مانند Envoy در ایستیو) که ترافیک شبکه برنامه را مانیتور و رمزنگاری می‌کنند.



تصویر ۴.۶ - سایدکار سرویس‌مش: رهگیری و رمزنگاری ترافیک ورودی و خروجی کانتینر اصلی برنامه

> ⚠️ **هشدار:**
>
> **معضل بزرگ سایدکارها در نسخه‌های قدیمی کوبرنتیز:** در نسخه‌های قدیمی‌تر، کوبرنتیز مفهومی مستقل برای سایدکار نداشت؛ بنابراین مهندسان سایدکار را به عنوان یک کانتینر معمولی در لیست `spec.containers` تعریف می‌کردند. این کار یک فاجعه عملیاتی به وجود می‌آورد: ۱. هیچ تضمینی وجود نداشت که سایدکار پروکسی قبل از کانتینر وب روشن شود! در نتیجه برنامه بالا می‌آمد، سعی می‌کرد به دیتابیس وصل شود اما چون پروکسی هنوز آماده نبود، برنامه کرش می‌کرد. ۲. در زمان خاموش شدن پاد، گاهی سایدکار زودتر از برنامه خاموش می‌شد و ترافیک در حال پردازش مشتریان قطع می‌گردید!

شاهکار معماری در نسخه جدید: Native Sidecars از نسخه ۱.۲۸ (آلفا) و ۱.۲۹ (بتا)، کوبرنتیز قابلیت رسمی **سایدکارهای بومی (Native Sidecar Containers)** را معرفی کرد که در نسخه فعلی (v1.32) به طور پیش‌فرض فعال است.

فرمول جادویی تعریف سایدکار بومی بسیار ساده است: کانتینر را درون بخش `spec.initContainers` می‌نویسید، اما فیلد `restartPolicy` آن را بر روی `Always` تنظیم می‌کنید!

```
spec:
  initContainers:
  - name: my-sidecar
    image: my-sidecar-image
    restartPolicy: Always     # <--- همین یک سطر، این کانتینر را به Native Sidecar تبدیل می‌کند!
  containers:
  - name: my-app
    image: my-app-image
```

با این تنظیم هوشمندانه، کوبرنتیز گارانتی‌های شگفت‌انگیز زیر را تامین می‌کند: ۱. سایدکار **حتماً قبل از کانتینر اصلی برنامه** راه‌اندازی و آماده می‌شود. ۲. با وجود قرار داشتن در بخش `initContainers` ، به دلیل سیاست `Always` خاموش نمی‌شود و **در تمام طول عمر پاد کنار کانتینر اصلی زنده می‌ماند.** ۳. در زمان نابودی پاد، کوبرنتیز تضمین می‌کند که سایدکار **حتماً بعد از کانتینر اصلی** خاموش شود تا هیچ درخواستی نصفه‌کاره قطع نشود.

—

۸. کارگاه عملیاتی جامع: لمس کار با پادها (Hands-on Lab)

برای اجرای فرامین این کارگاه، ترمینال خود را باز کنید و وارد پوشه کدهای کتاب در شاخه ۲۰۲۵ شوید:

```
git clone https://github.com/nigelpoulton/TKB.git
cd TKB
git checkout -b 2025 origin/2025
cd pods
```

۱. کالبدشکافی اولین مانیفست پاد (`pod.yml`)

فایل مانیفست `pod.yml` را بررسی می‌کنیم:

```
apiVersion: v1
kind: Pod
metadata:
  name: hello-pod
  labels:
    zone: prod
    version: v1
spec:
  containers:
  - name: hello-ctr
    image: nigelpoulton/k8sbook:1.0
    ports:
    - containerPort: 8080
```

```
    resources:
      limits:
        memory: 128Mi
        cpu: 0.5
```

هر مانیفست کوبرنتیز از چهار رکن اساسی تشکیل شده است:

- `apiVersion` : نگارش اسکیما و کاتالوگ API کوبرنتیز (برای پادها نسخه پایدار `v1`).
- `kind` : ماهیت شیء مورد نظر (اینجا `Pod`).
- `metadata` : اطلاعات هویتی شامل نام پاد (`hello-pod`) و برچسب‌های کلیدی (`labels`).
- `spec` : مشخصات فنی و وضعیت مطلوب مورد نظر (کانتینر `hello-ctr` بر پایه ایمیج نسخه ۱.۰، پورت `8080` و سقف منابع مجاز ۰.۵ هسته پردازنده و ۱۲۸ مگابایت رم).

مانیفست‌های YAML: همدلی در قالب کد (Empathy as Code)

نیرمال مهتا در کنفرانس مشهور داکرکان ۲۰۱۷ مفهوم زیبایی را مطرح کرد: مانیفست‌های YAML کوبرنتیز فراتر از دستورات اجرایی، بهترین سند مستندسازی معماری سیستم هستند. یک عضو جدید تیم توسعه یا تیم عملیات (NOC/SRE) با خواندن این مانیفست دقیقاً متوجه پورت‌های سرویس، میزان مصرف منابع، وابستگی‌ها و پیکربندی برنامه می‌شود بدون این‌که نیازی به خواندن هزاران سطر کد جاوا یا پایتون داشته باشد.

۲. استقرار پاد در کلاستر

دستور اعلانی زیر را برای ارسال مانیفست به کلاستر اجرا کنید:

```
kubectl apply -f pod.yml
```

خروجی زیر نشان‌دهنده ثبت پاد است:

```
pod/hello-pod created
```

وضعیت لحظه‌ای پاد را با دستور زیر بررسی کنید:

```
kubectl get pods
```

در لحظات اولیه، وضعیت در فاز دانلود و ایجاد کانتینر است:

```
NAME          READY    STATUS              RESTARTS    AGE
hello-pod     0/1      ContainerCreating   0           9s
```

به محض اینکه ایمیج از رجیستری Docker Hub دانلود شود، پاد وارد وضعیت عملیاتی می‌شود:

```
NAME         READY    STATUS     RESTARTS    AGE
hello-pod    1/1      Running    0           35s
```

۳. بازرسی عمیق و مهندسی معکوس وضعیت پاد (Introspecting Pods)

برای مشاهده مشخصات پیشرفته پاد، از فلگ‌های کمکی زیر استفاده کنید:

۱. **مشاهده جزئیات نود و آدرس IP با فلگ `-o wide` :**

```
kubectl get pods -o wide
```

خروجی آی‌پی اختصاصی پاد و نام سرور میزبانی‌کننده آن را نمایش می‌دهد:

```
NAME         READY    STATUS     RESTARTS    AGE    IP             NODE
NOMINATED NODE    READINESS GATES
hello-pod    1/1      Running    0           50s    10.1.0.103    desktop-worker
<none>            <none>
```

۲. **کالبدشکافی کامل با خروجی YAML و تفاوت بنیادین Spec در برابر Status:**

```
kubectl get pods hello-pod -o yaml
```

اگر خروجی این دستور را مطالعه کنید، متوجه می‌شوید بسیار طولانی‌تر از فایل ۱۷ سطری شماست. علت این است که کوبرنتیز تمام فیلدهای اختیاری را با مقادیر پیش‌فرض پر کرده است. این خروجی دارای دو بخش اساسی است:

- بخش `spec` : بیانگر **حالت مطلوب (Desired State)** که شما درخواست کرده‌اید.
- بخش `status` : بیانگر **وضعیت مشاهده‌شده لحظه‌ای (Observed State)** شامل شرایط پاد (Conditions)، زمان استارت، شناسه کانتینرد و آدرس‌های تخصیص‌یافته توسط کلاستر.

۳. **بررسی شناسنامه و لاگ رویدادها با `kubectl describe` :**

```
kubectl describe pod hello-pod
```

بخش پایانی خروجی این دستور (`Events`) باارزش‌ترین ابزار خطایابی در کل کوبرنتیز است. این بخش جدول زمانی رویدادها را نشان می‌دهد:

```
Events:
  Type     Reason      Age    From                Message
  ----     ------      ----   ----                -------
  Normal   Scheduled   1m     default-scheduler   Successfully assigned default/
hello-pod to desktop-worker
  Normal   Pulling     58s    kubelet             Pulling image "nigelpoulton/
k8sbook:1.0"
  Normal   Pulled      40s    kubelet             Successfully pulled image
```

```
"nigelpoulton/k8sbook:1.0"
  Normal   Created     39s    kubelet              Created container hello-ctr
  Normal   Started     38s    kubelet              Started container hello-ctr
```

۴. پایش لاگ‌های کانتینر

برای خواندن خروجی استاندارد برنامه:

```
kubectl logs hello-pod
```

> 💡 **ترفند و تجربه عملیاتی:**
>
> اگر پاد شما چندکانتینری باشد، دستور بالا به صورت پیش‌فرض لاگ‌های اولین کانتینر را برمی‌گرداند. برای مشاهده لاگ کانتینری خاص از فلگ `--container` استفاده کنید:
>
> ```
> kubectl logs <pod-name> --container <container-name>
> ```

۵. ورود به درون کانتینر و اجرای فرامین (`kubectl exec`)

کوبرنتیز به شما امکان می‌دهد فرامین را به دو شکل در کانتینر اجرا کنید:

روش اول: اجرای ریموت یکباره فرمان (One-off Execution)

```
kubectl exec hello-pod -- ps
```

خروجی پردازه‌های داخل کانتینر را در ترمینال محلی شما چاپ می‌کند:

```
PID   USER     TIME  COMMAND
    1 root      0:00 node ./app.js
   18 root      0:00 ps
```

روش دوم: نشست تعاملی و ورود به شل کانتینر (Interactive Exec Session) با افزودن فلگ‌های `-it` می‌توانید درست مانند یک نشست SSH وارد کانتینر شوید:

```
kubectl exec -it hello-pod -- sh
```

اکنون اعلان خط فرمان شما به `#` تبدیل شده و مستقیماً داخل کانتینر لینوکسی هستید! اگر داخل این شل سعی کنید دستور `curl localhost:8080` را بزنید با خطای نبود ابزار curl مواجه می‌شوید:

```
# curl localhost:8080
sh: curl: not found
```

چون این ایمیج سبک و مبتنی بر آلپاین است، می‌توانید با پکیج منیجر آن را نصب و تست کنید:

```
# apk add curl
# curl localhost:8080
```

خروجی کد HTML صفحه وب را چاپ می‌کند. با فشردن کلیدهای `Ctrl+D` یا تایپ `exit` از کانتینر خارج شوید.

۶. آزمون عملی تغییرناپذیری (Object Immutability Test)

بیایید با یک تست زنده ثابت کنیم که پادها غیرقابل تغییر هستند. دستور زیر را برای ویرایش زنده پاد بزنید:

```
kubectl edit pod hello-pod
```

این دستور فایل پیکربندی زنده پاد را در ادیتور سیستم شما (مانند vi یا نوت‌پد) باز می‌کند. تلاش کنید پورت کانتینر را از `8080` به `9090` تغییر دهید، یا میزان `limits` رم را عوض کنید، فایل را ذخیره کرده و ببندید.

کلاستر فوراً مچ شما را می‌گیرد و با خطای زیر تغییرات را رد می‌کند:

```
A copy of your changes has been stored to "/tmp/kubectl-edit-xxxx.yaml"
error: pods "hello-pod" was not valid:
* spec: Forbidden: pod updates may not change fields other than ...
```

کوبرنتیز صراحتاً اعلام می‌کند که ویرایش خصیصه‌های اساسی پاد در حال اجرا **ممنوع (Forbidden)** است!

۷. کالبدشکافی Requests و Limits در محیط واقعی

در مانیفست‌ها دو مفهوم کلیدی وجود دارد:

- `requests` : حداقل منابع تضمینی برای زمان‌بندی (Scheduler از این برای پیدا کردن نود استفاده می‌کند).
- `limits` : حداکثر سقف مجاز مصرف (Kubelet و هسته لینوکس از طریق مکانیزم cgroups جلوی مصرف بیشتر از این سقف را می‌گیرند).

> **اگر فقط Limits را تعیین کنیم چه رخ می‌دهد؟**
>
> در مانیفست `pod.yml` ، ما فقط `limits` تعریف کردیم و `requests` ننوشتیم. در این سناریو، کوبرنتیز به صورت خودکار مقدار `requests` را دقیقاً برابر با `limits` قرار می‌دهد تا اطمینان یابد سرور منابع کافی برای حداکثر نیاز پاد را از قبل برای آن ذخیره می‌کند.

—

۹. سناریوی عملی ۱: کار با کانتینر آماده‌سازی (Init Container)

در این سناریو، پادی ایجاد می‌کنیم که کانتینر اصلی آن تا زمان آماده‌شدن یک سرویس مستقل شبکه در کلاستر، متوقف می‌ماند.

فایل `initpod.yml` را بررسی می‌کنیم:

```
apiVersion: v1
kind: Pod
metadata:
  name: initpod
  labels:
    app: initializer
spec:
  initContainers:
  - name: init-ctr
    image: busybox:1.28.4
    command: ['sh', '-c', 'until nslookup k8sbook; do echo waiting for k8sbook;
sleep 1; done; echo Service found!']
  containers:
  - name: web-ctr
    image: nigelpoulton/web-app:1.0
    ports:
    - containerPort: 8080
```

در این تعریف:

- کانتینر `init-ctr` یک حلقه `until nslookup k8sbook` اجرا می‌کند؛ یعنی تا زمانی که سرویسی به نام `k8sbook` در شبکه DNS کلاستر رجیستر نشود، به خواب می‌رود و خارج نمی‌شود.
- کانتینر اصلی `web-ctr` منتظر می‌ماند.

مانیفست را اعمال کنید:

```
kubectl apply -f initpod.yml
```

حالا با فلگ رصد لحظه‌ای (`--watch`) وضعیت پاد را زیر نظر بگیرید:

```
kubectl get pods --watch
```

خروجی نشان می‌دهد پاد در وضعیت انتظار برای اتمام کانتینر Init است:

```
NAME       READY    STATUS      RESTARTS    AGE
initpod    0/1      Init:0/1    0           10s
```

اگر `kubectl describe pod initpod` بگیرید، می‌بینید وضعیت کلی پاد `Pending` است. اکنون برای نجات پاد، سرویس شبکه مورد انتظار را مستقر می‌کنیم:

```
kubectl apply -f initsvc.yml
```

بلافاصله در خروجی رصد لحظه‌ای خواهید دید که کانتینر Init نام سرویس را پیدا کرده، با موفقیت خارج می‌شود و کانتینر وب متولد می‌گردد:

```
NAME       READY    STATUS              RESTARTS    AGE
initpod    0/1      Init:0/1            0           35s
initpod    0/1      PodInitializing     0           42s
initpod    1/1      Running             0           45s
```

با دستور `kubectl describe pod initpod` مشاهده خواهید کرد که وضعیت کانتینر `init-ctr` به `Terminated` با کد خروج ۰ تغییر کرده و کانتینر وب در وضعیت `Running` قرار دارد.

—

۱۰. سناریوی عملی ۲: پیاده‌سازی سایدکار بومی مدرن (Native Sidecar with Git-Sync)

در این سناریوی پیشرفته، قدرت ویژگی مدرن Native Sidecars در نسخه جدید کوبرنتیز را به صورت زنده آزمایش می‌کنیم.

فایل `initsidecar.yml` در پوشه کدهای کتاب قرار دارد:

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: git-sync
  labels:
    app: sidecar
spec:
  initContainers:
  - name: ctr-sync
    restartPolicy: Always     # سایدکار بومی که همیشه زنده می‌ماند
    image: k8s.gcr.io/git-sync:v3.1.6
    volumeMounts:
    - name: html
      mountPath: /tmp/git
    env:
    - name: GIT_SYNC_REPO
      value: https://github.com/nigelpoulton/ps-sidecar
    - name: GIT_SYNC_BRANCH
      value: master
    - name: GIT_SYNC_DEPTH
      value: "1"
    - name: GIT_SYNC_DEST
      value: "html"
  containers:
  - name: ctr-web
    image: nginx
    volumeMounts:
    - name: html
```

```
      mountPath: /usr/share/nginx/html
  volumes:
  - name: html
    emptyDir: {}
```

کالبدشکافی معماری این سناریو: ۱. سایدکار `ctr-sync` با ایمیج `git-sync` به عنوان یک `initContainer` با تنظیم `restartPolicy: Always` تعریف شده است. ۲. یک دیسک موقت اشتراکی با تکنولوژی `emptyDir` تعریف شده و هر دو کانتینر آن را ماونت کرده‌اند. ۳. سایدکار به طور مداوم مخزن گیت را زیر نظر می‌گیرد و تغییرات را در پوشه اشتراکی می‌نویسد. ۴. وب‌سرور `nginx` همان پوشه را خوانده و صفحات وب را بدون نیاز به بازسازی ایمیج یا ریستارت سرور، بلافاصله به‌روزرسانی می‌کند.

پاد را مستقر کنید:

```
kubectl apply -f initsidecar.yml
```

وضعیت پاد را بررسی کنید:

```
kubectl get pods
```

خروجی نشان می‌دهد که هر دو کانتینر به طور همزمان در وضعیت آماده هستند:

```
NAME        READY   STATUS    RESTARTS   AGE
git-sync    2/2     Running   0          15s
```

برای اثبات این‌که کانتینر `ctr-sync` از نوع Init است، دستور کاستوم زیر را اجرا کنید:

```
kubectl get pod git-sync -o "custom-
columns=NAME:.metadata.name,INIT:.spec.initContainers[*].name,CONTAINERS:.spec.contai
```

خروجی تفکیک دقیق کانتینرها را نشان می‌دهد:

```
NAME        INIT        CONTAINERS
git-sync    ctr-sync    ctr-web
```

همچنین اگر رویدادهای `kubectl describe pod git-sync` را بررسی کنید، متوجه می‌شوید که کیوبلت ابتدا کانتینر `ctr-sync` را ساخته و روشن کرده و چند ثانیه بعد به سراغ کشیدن ایمیج `nginx` و روشن کردن کانتینر وب رفته است.

> **💡 ترفند و تجربه عملیاتی:**

اگر مخزن فوق را در حساب گیت‌هاب شخصی خود فورک کنید و آدرس آن را در فیلد `GIT_SYNC_REPO` مانیفست قرار دهید، با ویرایش فایل `index.html` در گیت‌هاب و ذخیره کامیت، ظرف کمتر از ۱۰ ثانیه سایدکار تغییرات را پول کرده و با رفرش مرورگر، صفحه وب جدید را مشاهده خواهید کرد!

—

۱۱. پاک‌سازی ایمن منابع کلاستر (Clean up)

در پایان تمرینات، برای آزادسازی منابع رم و پردازنده کلاستر خود، پادها و سرویس‌های ایجادشده را حذف کنید:

```
# حذف پادها بر اساس نام
kubectl delete pod hello-pod initpod git-sync

# حذف سرویسهای تستی
kubectl delete svc k8sbook svc-sidecar
```

همچنین می‌توانید تمام اشیاء ساخته‌شده را مستقیماً با ارجاع به فایل‌های مانیفست آن‌ها حذف نمایید:

```
kubectl delete -f initsidecar.yml -f initpod.yml -f pod.yml
```

—

۱۲. جمع‌بندی فصل و نکات کلیدی آزمون‌های بین‌المللی (CKA & CKAD)

در این فصل مبانی عمیق کار با پادها را بررسی کردیم. مهم‌ترین گزاره‌های کلیدی این فصل عبارتند از:

- **پاد اتم بنیادین است:** تمام بارهای کاری (کانتینر، سرورلس، Wasm و VM) درون پاد اجرا می‌شوند.
- **اشتراک منابع:** کانتینرهای یک پاد در استک شبکه (آدرس IP مشترک)، لوپ‌بک `localhost`، والیوم‌ها و فضای نام IPC/PID سهیم هستند.
- **زمان‌بندی یکپارچه:** تمام کانتینرهای یک پاد الزاماً روی یک ماشین واحد اجرا می‌شوند.
- **فناپذیری و تغییرناپذیری:** پادهای مرده هرگز ریستارت نمی‌شوند بلکه کنترلرها پاد جدیدی جایگزین آن‌ها می‌کنند. تغییر پیکربندی پاد زنده در API Server قدغن است.
- **شبکه سراسری مسطح:** در کوبرنتیز هر پاد یک IP یکتا دارد و پادها بدون نیاز به پورت‌مپینگ یا NAT با یکدیگر تبادل داده دارند.
- **الگوی Init Containers:** کانتینرهای سریالی موقت که حتماً قبل از کانتینر برنامه با موفقیت تمام می‌شوند.
- **الگوی Native Sidecars:** تحول مدرن نسخه ۲۰۲۵ که با `restartPolicy: Always` در بخش `initContainers` تعریف می‌شود؛ پیش از برنامه روشن شده و تا پایان عمر پاد پابرجا می‌ماند.
- **مدیریت منابع:** مقادیر `requests` تضمین زمان‌بندی را فراهم می‌کنند و `limits` سقف مصرف پردازشی را کنترل می‌نمایند.

فصل ۵: کلاسترهای مجازی با نیم‌اسپیس‌ها (Namespaces)

وقتی کلاستر کوبرنتیز را راه‌اندازی می‌کنید، با یک استخر بزرگ و یکپارچه از منابع پردازشی روبه‌رو هستید. اما در دنیای واقعی، شرکت‌ها و تیم‌های نرم‌افزاری نمی‌توانند همه برنامه‌ها، تست‌ها، دیتابیس‌ها و سرویس‌های خود را بدون تفکیک درون یک فضای مشترک رها کنند.

اینجاست که کوبرنتیز مفهومی بنیادین و فوق‌العاده کاربردی به نام **نیم‌اسپیس (Namespace)** را معرفی می‌کند. نیم‌اسپیس‌ها راهکاری استاندارد برای تقسیم یک کلاستر فیزیکی بزرگ به چندین **کلاستر مجازی (Virtual Clusters)** مستقل، منظم و قابل مدیریت هستند.

در این فصل مباحث زیر را به صورت گام‌به‌گام و تفسیری واکاوی خواهیم کرد: ۱. **تئوری نیم‌اسپیس‌ها و تفاوت بنیادین با نیم‌اسپیس‌های کرنل لینوکس** ۲. **تحلیل معماری چندمستأجری: ایزولاسیون نرم (Soft) در برابر ایزولاسیون سخت (Hard)** ۳. **کالبدشکافی اشیای دارای نیم‌اسپیس در برابر اشیای سراسری کلاستر (Cluster-scoped)** ۴. **نیم‌اسپیس‌های پیش‌فرض و سیستمی کلاستر** ۵. **مدیریت سهمیه‌ها و محدودیت منابع با ResourceQuota و LimitRange** ۶. **کارگاه عملیاتی: ساخت اعلانی و دستوری، سوئیچ هوشمند کانتکست و دیپلوی برنامه‌ها در نیم‌اسپیس‌ها**

—

۱. تئوری نیم‌اسپیس‌ها (Intro to Namespaces)

نخستین نکته حیاتی که باید بدانید این است: **نیم‌اسپیس‌های کوبرنتیز هیچ ارتباطی با نیم‌اسپیس‌های کرنل لینوکس ندارند!**

- **نیم‌اسپیس‌های کرنل لینوکس (Linux Kernel Namespaces):** در سطح سیستم‌عامل عمل می‌کنند و منابع سیستم‌عامل (پردازه‌ها، شبکه، دیسک) را ایزوله کرده تا مفهوم «کانتینر» شکل بگیرد.
- **نیم‌اسپیس‌های کوبرنتیز (Kubernetes Namespaces):** در لایه ارکستراسیون و API عمل می‌کنند و یک کلاستر کوبرنتیز را به چند کلاستر منطقی و مجازی تقسیم می‌کنند.

> 📌 **نکته مهم:**
>
> در متون استاندارد کوبرنتیز، کلمه Namespace با حرف بزرگ انگلیسی نوشته می‌شود تا مشخص گردد یک شیء رسمی در API کوبرنتیز است و با نیم‌اسپیس‌های کرنل لینوکس اشتباه گرفته نشود.

فلسفه جداسازی منابع: چرا به نیم‌اسپیس نیاز داریم؟

فرض کنید شرکتی دارای چند تیم توسعه، چندین محیط اجرایی و ده‌ها میکروسرویس است. نیم‌اسپیس‌ها چهار ارزش استراتژیک خلق می‌کنند: ۱. **تفکیک نام‌ها (Name Scoping):** در کوبرنتیز، نام اشیا (مانند پاد یا سرویس) در یک نیم‌اسپیس باید یکتا باشد، اما دو تیم مختلف می‌توانند سرویسی با نام یکسان `frontend` یا `database` را در دو نیم‌اسپیس مجزا بدون هیچ‌گونه تداخلی بسازند! ۲. **کنترل دسترسی نقشی (RBAC):** می‌توانید به تیم توسعه‌دهندگان

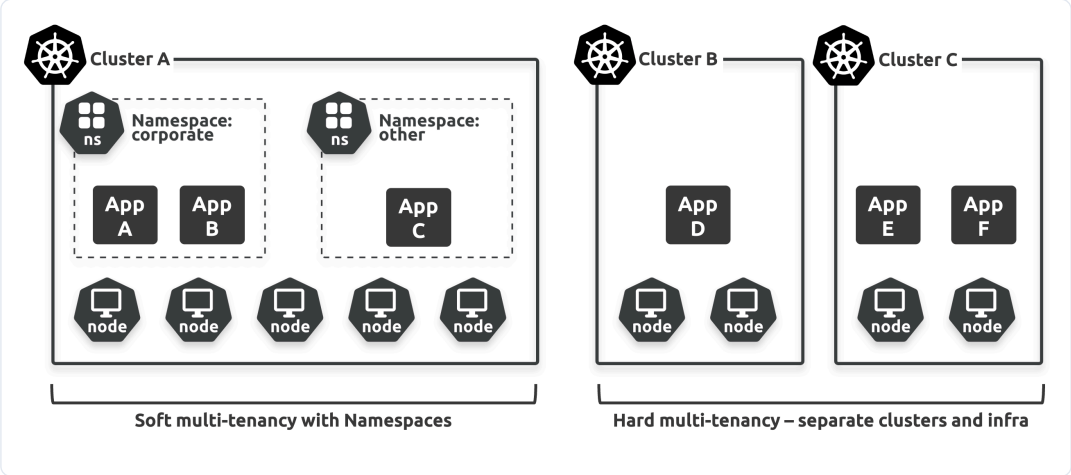
فقط دسترسی کامل به نیم‌اسپیس `development` بدهید، در حالی که دسترسی به نیم‌اسپیس `production` کاملاً مسدود یا صرفاً خواندنی باشد. ۳. **سهمیه‌بندی منابع (Resource Quotas):** جلوگیری از این‌که یک برنامه با مصرف بی‌رویه رم یا پردازنده، کل سرورهای کلاستر را ببلعد و برنامه‌های سایر تیم‌ها را از کار بیندازد. ۴. **مدیریت چرخه حیات:** حذف آسان و یکپارچه کل منابع یک محیط (مثلاً حذف یک محیط تستی موقت فقط با حذف نیم‌اسپیس آن).

—

۲. چندمستأجری: ایزولاسیون نرم در برابر ایزولاسیون سخت

اصطلاح «مستأجر (Tenant)» در دنیای ابری مفهومی منعطف است؛ می‌تواند نشان‌دهنده یک میکروسرویس خاص، یک تیم مستقل، یک بخش سازمانی (مثل حسابداری و منابع انسانی)، یا مشتریان بیرونی شرکت باشد.

نکته‌ای که هر معمار سیستم و مهندس دواپس باید عمیقاً درک کند، تفاوت میان دو مدل ایزولاسیون است: ۱. **ایزولاسیون نرم (Soft Multi-tenancy):** به اشتراک‌گذاری یک کلاستر میان چند تیم قابل‌اعتماد با استفاده از نیم‌اسپیس‌ها. ۲. **ایزولاسیون سخت (Hard Multi-tenancy):** جداسازی مطلق برنامه‌ها با استقرار کلاسترهای کاملاً مجزا بر روی سخت‌افزارها یا ماشین‌های اختصاصی.



تصویر ۵.۱ - مقایسه ایزولاسیون نرم بر پایه نیم‌اسپیس‌ها در یک کلاستر در برابر ایزولاسیون سخت با کلاسترهای مستقل

همان‌طور که در تصویر ۵.۱ به وضوح نمایش داده شده است:

- در کلاستر سمت چپ، از نیم‌اسپیس‌ها برای ایزولاسیون نرم استفاده شده است. همه برنامه‌ها از نودهای مشترک، کرنل سیستم‌عامل یکسان و کنترل پلین مشترک استفاده می‌کنند. اگر کاربری دسترسی ادمین پیدا کند یا کانتینری از طریق باگ‌های امنیتی فرار از کانتینر (Container Escape) کرنل نود را آلوده سازد، می‌تواند به داده‌های سایر نیم‌اسپیس‌ها آسیب بزند.
- در سمت راست، دو کلاستر کاملاً مستقل با کنترل پلین و سرورهای فیزیکی جداگانه راه‌اندازی شده‌اند. این سناریو ایزولاسیون سخت را فراهم می‌کند و برای مشتریان برون‌سازمانی یا بارهای کاری فوق‌امنیتی ضروری است.

نتیجه‌گیری مدیریتی درباره معماری کلاسترها

نیم‌اسپیس‌ها بسیار سبک، سریع و ارزان هستند و برای تفکیک تیم‌های داخلی یک سازمان عالی عمل می‌کنند. اما اگر قرار است بارهای کاری ناشناس، رقبای تجاری یا کدهای غیرقابل اعتماد کلاینت‌ها را میزبانی کنید، ایجاد کلاسترهای جداگانه بهترین و امن‌ترین تصمیم مهندسی است.

—

۳. اشیای دارای نیم‌اسپیس در برابر اشیای سراسری کلاستر (Cluster-Scoped)

اکثر آبجکت‌های کوبرنتیز وابسته به نیم‌اسپیس هستند (Namespaced Resources)، اما منابعی در کلاستر وجود دارند که ماهیتی زیرساختی دارند و به هیچ نیم‌اسپیس خاصی تعلق ندارند (Cluster-scoped Resources).

برای مشاهده فهرست کامل و تفکیک آن‌ها، دستور زیر را در ترمینال اجرا کنید:

```
kubectl api-resources
```

در ستون `NAMESPACED` :

- اگر مقدار `true` باشد، یعنی آن شیء باید در یک نیم‌اسپیس مستقر شود. نمونه‌ها: `Pods` ، `Deployments` ، `Services` ، `ConfigMaps` ، `Secrets` ، `PersistentVolumeClaims` .
- اگر مقدار `false` باشد، یعنی شیء در سطح کل کلاستر تعریف می‌شود و مستقل از نیم‌اسپیس‌هاست. نمونه‌ها: `Nodes` ، `PersistentVolumes` ، `StorageClasses` ، `Namespaces` ، `ClusterRoles` .

> ⚠️ **هشدار:**
>
> شما نمی‌توانید یک نود فیزیکی یا یک کلاس ذخیره‌سازی (StorageClass) را درون یک نیم‌اسپیس بسازید. این منابع زیرساخت مشترک کل کلاستر هستند.

—

۴. نیم‌اسپیس‌های پیش‌فرض و سیستمی کلاستر

هر کلاستر کوبرنتیز بلافاصله پس از تولد، دارای چندین نیم‌اسپیس پیش‌ساخته سیستمی است. با دستور زیر می‌توانید آن‌ها را ببینید:

```
kubectl get namespaces
```

خروجی استاندارد معمولاً چهار نیم‌اسپیس زیر را نمایش می‌دهد:

```
NAME                 STATUS   AGE
default              Active   2d
kube-node-lease   Active   2d
```

```
kube-public       Active   2d
kube-system       Active   2d
```

بیایید نقش حیاتی هر یک را مرور کنیم:

- `default` : اگر در زمان اجرای دستورات یا نوشتن فایل‌های YAML نام نیم‌اسپیسی را مشخص نکنید، کوبرنتیز به طور خودکار شیء شما را درون این نیم‌اسپیس می‌نشاند.
- `kube-system` : خانه مغز متفکر و سرویس‌های زیرساختی کلاستر است. پادهای شبکه CoreDNS، پادهای `kube-proxy` ، کنترلرهای CNI و سرور متریک در این نیم‌اسپیس فعال هستند. هرگز نباید برنامه‌های شخصی یا سازمانی خود را در این بخش مستقر کنید!
- `kube-public` : فضایی برای داده‌هایی است که باید حتی برای کاربران احراز هویت‌نشده نیز قابل خواندن باشند (مانند اطلاعات اولیه‌ای که کلاینت‌ها برای اتصال به کلاستر به آن نیاز دارند).
- `kube-node-lease` : این نیم‌اسپیس میزبان آبجکت‌های Lease مربوط به ضربان قلب (Heartbeat) نودها است. در نسخه‌های مدرن، کیوبلت به جای به‌روزرسانی مداوم شیء سنگین Node، یک شیء بسیار سبک Lease را هر چند ثانیه تمدید می‌کند تا کنترل پلین از زنده بودن نود مطلع شود. این نوآوری بار پردازشی etcd را به شدت کاهش داده است.

برای مشاهده جزئیات یک نیم‌اسپیس از دستور `describe` استفاده کنید (می‌توانید به جای `namespace` از کلمه مخفف `ns` استفاده نمایید):

```
kubectl describe ns default
```

برای فیلتر کردن خروجی دستورات بر اساس یک نیم‌اسپیس خاص، از فلگ `-n` یا `--namespace` استفاده کنید:

```
kubectl get svc -n kube-system
```

همچنین اگر می‌خواهید تمام اشیای یک نوع خاص را در سرتاسر کلاستر بدون در نظر گرفتن نیم‌اسپیس مشاهده کنید، فلگ `--all-namespaces` یا فرم کوتاه `-A` را به کار ببرید:

```
kubectl get pods -A
```

—

۵. مدیریت سهمیه‌ها: ResourceQuota و LimitRange

یکی از بزرگ‌ترین مزایای نیم‌اسپیس‌ها امکان اعمال سیاست‌های سخت‌گیرانه مصرف منابع است.

۱. **سهمیه منابع (ResourceQuota):** تعیین سقف کل منابعی که تمام پادهای یک نیم‌اسپیس در مجموع می‌توانند مصرف کنند.

```
apiVersion: v1
kind: ResourceQuota
```

```
metadata:
  name: compute-quota
  namespace: development
spec:
  hard:
    requests.cpu: "4"
    requests.memory: 8Gi
    limits.cpu: "8"
    limits.memory: 16Gi
    pods: "20"
```

در این مثال، تیم توسعه در نیم‌اسپیس `development` هرگز نمی‌تواند بیش از ۲۰ پاد بسازد یا در مجموع بیش از ۱۶ گیگابایت حافظه مصرف کند.

۲. **محدوده‌های پیش‌فرض (LimitRange):** اگر توسعه‌دهنده‌ای برای پاد خود مقادیر `requests` و `limits` را تعریف نکند، شیء `LimitRange` به طور خودکار مقادیر پیش‌فرضی را به کانتینرهای آن نیم‌اسپیس تزریق می‌کند تا منابع سرور هدر نرود.

—

۶. کارگاه عملیاتی: ساخت، سوئیچ هوشمند و استقرار در نیم‌اسپیس

وارد پوشه کدهای کتاب شوید:

```
cd TKB/namespaces
```

۱. ساخت دستوری (Imperative) در برابر ساخت اعلانی (Declarative)

برای ساخت سریع یک نیم‌اسپیس در خط فرمان، دستور زیر را اجرا کنید:

```
kubectl create ns hydra
```

اکنون یک نیم‌اسپیس دیگر را با رویکرد حرفه‌ای اعلانی از روی فایل `shield-ns.yml` بسازید:

```
apiVersion: v1
kind: Namespace
metadata:
  name: shield
  labels:
    env: marvel
```

اعمال مانیفست:

```
kubectl apply -f shield-ns.yml
```

لیست نیم‌اسپیس‌ها را بررسی کنید:

```
kubectl get ns
```

همان‌طور که علاقه‌مندان به دنیای مارول می‌دانند، دو گروه SHIELD و HYDRA دشمنان خونی یکدیگر هستند و قرار دادن آن‌ها در یک کلاستر با ایزولاسیون نرم کار خطرناکی است! بیایید با یک دستور نیم‌اسپیس هایدرا را حذف کنیم:

```
kubectl delete ns hydra
```

۲. سوئیچ هوشمند کانتکست: پایان کابوس تایپ مداوم `-n` !

وقتی روی یک پروژه کار می‌کنید، تایپ مداوم فلگ `-n shield` خسته‌کننده است. راهکار حرفه‌ای، تغییر نیم‌اسپیس فعال در فایل `kubeconfig` جاری است:

```
kubectl config set-context --current --namespace shield
```

از این لحظه به بعد، هر دستوری مثل `kubectl get pods` یا `kubectl get svc` بدون نیاز به هیچ فلگی، مستقیماً روی نیم‌اسپیس `shield` اجرا می‌شود!

> **💡 ترفند و تجربه عملیاتی:**
>
> در محیط‌های کاری دواپس، مهندسان معمولاً از ابزار متن‌باز و بسیار محبوب **kubens** (بخشی از بسته kubectx) استفاده می‌کنند. با این ابزار تنها با زدن فرمان `kubens shield` کانتکست شما فوراً تغییر می‌کند.

۳. استقرار برنامه چندجزئی درون نیم‌اسپیس

فایل `app.yml` را بررسی می‌کنیم. این فایل سه شیء شامل `ServiceAccount` ، سرویس لودبالانسر `the-bus` و پاد `triskelion` را به صورت اعلانی هدف‌گذاری کرده است:

```yaml
apiVersion: v1
kind: ServiceAccount
metadata:
  namespace: shield
  name: default
---
apiVersion: v1
kind: Service
metadata:
  namespace: shield
```

```
  name: the-bus
spec:
  type: LoadBalancer
  ports:
  - port: 8080
    targetPort: 8080
  selector:
    env: marvel
---
apiVersion: v1
kind: Pod
metadata:
  namespace: shield
  name: triskelion
  labels:
    env: marvel
spec:
  containers:
  - name: web
    image: nigelpoulton/k8sbook:1.0
    ports:
    - containerPort: 8080
```

مانیفست را در کلاستر اعمال کنید:

```
kubectl apply -f app.yml
```

بررسی صحت استقرار:

```
kubectl get pods
kubectl get svc
```

هر دو شیء در نیم‌اسپیس `shield` ساخته شده و به درستی کار می‌کنند. برای تست دسترسی:

```
curl localhost:8080
```

صفحه وب برنامه با موفقیت پاسخ می‌دهد.

۴. ارتباطات میان‌نیم‌اسپیسی با آدرس‌های FQDN در DNS

یک نکته کلیدی: اگر برنامه‌ای در نیم‌اسپیس `default` بخواهد به سرویس `the-bus` در نیم‌اسپیس `shield` متصل شود، چگونه آن را صدا می‌زند؟ پاسخ: با نام کامل دامنه (FQDN):

```
the-bus.shield.svc.cluster.local
```

کوبرنتیز به طور پیش‌فرض ترافیک شبکه میان نیم‌اسپیس‌ها را مسدود نمی‌کند. اگر مایلید ارتباط شبکه بین نیم‌اسپیس‌ها قطع شود، باید از اشیای **NetworkPolicy** استفاده کنید.

—

۷. پاک‌سازی و بازگشت به وضعیت پیش‌فرض (Clean up)

حذف یک نیم‌اسپیس رفتار آبشاری (Cascading) دارد؛ یعنی با حذف نیم‌اسپیس، تمام پادها، سرویس‌ها، سکرت‌ها و دیسک‌های درون آن به طور کامل پاک می‌شوند:

```
kubectl delete ns shield
```

فراموش نکنید کانتکست کلاینت خود را مجدداً به نیم‌اسپیس `default` برگردانید:

```
kubectl config set-context --current --namespace default
```

—

۸. جمع‌بندی فصل ۵

- نیم‌اسپیس‌ها کلاسترهای مجازی برای تقسیم منطقی یک کلاستر هستند.
- ایزولاسیون نیم‌اسپیس‌ها از نوع «نرم» است؛ برای امنیت مطلق و مشتریان ناشناس، کلاسترهای فیزیکی جداگانه ضروری است.
- منابعی مانند Node و StorageClass در سطح کل کلاستر هستند و نیم‌اسپیس ندارند.
- کلاستر دارای نیم‌اسپیس‌های پیش‌فرض `default`، `kube-system`، `kube-public` و `kube-node-lease` است.
- با `ResourceQuota` می‌توان مصرف منابع هر تیم را به دقت محدود کرد.
- برای ارجاع به سرویسی در نیم‌اسپیس دیگر از ساختار `<service>.<namespace>.svc.cluster.local` استفاده می‌شود.

فصل ۶: دیپلویمنت‌ها و ارتقای بدون قطعی (Deployments)

در فصل ۴ آموختیم که تمام برنامه‌ها در کوبرنتیز درون پادها زندگی می‌کنند. اما یک حقیقت بزرگ وجود دارد: **در محیط‌های واقعی و پروداکشن، شما تقریباً هرگز یک پاد خام و مجزا را مستقیماً مستقر نمی‌کنید!**

چرا؟ زیرا پادها فناپذیرند (Mortal)؛ اگر سروری خاموش شود یا کانتینری کرش کند، پاد خام به حال خود رها می‌شود و کسی آن را جایگزین نمی‌کند. پادها توانایی اسکیل خودکار ندارند، نمی‌توانند به‌روزرسانی بدون قطعی انجام دهند و تاریخچه‌ای از نسخه‌های قبلی خود نگه نمی‌دارند.

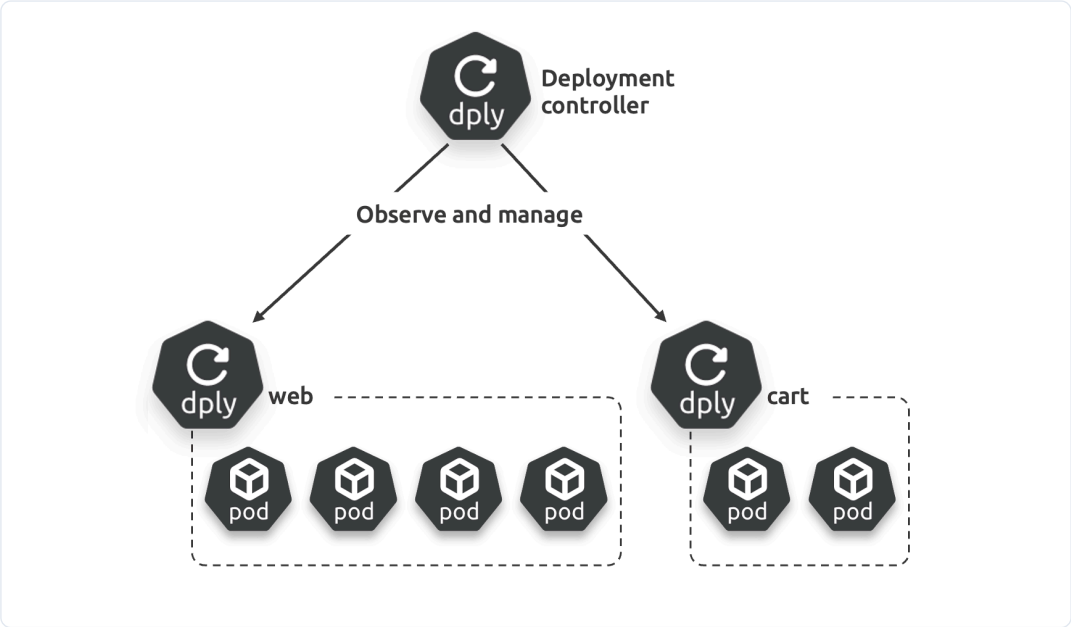
اینجاست که شیء قدرتمند و پادشاه مدیریت بارهای کاری، یعنی **دیپلویمنت (Deployment)** وارد صحنه می‌شود. دیپلویمنت‌ها قابلیت‌های خارق‌العاده‌ای مانند **خودترمیمی (Self-healing)**، **اسکیل دستی و خودکار (Scaling)**، **ارتقای تدریجی بدون ثانیه‌ای قطعی (Rolling Updates)** و **قابلیت بازگشت آنی به نسخه قبل (Rollback)** را به پادها هدیه می‌دهند.

در این فصل مباحث زیر را به صورت کامل و تفسیری فرا خواهیم گرفت: ۱. **تئوری دیپلویمنت و معماری سه‌لایه (Deployment -> ReplicaSet -> Pod)** ۲. **کالبدشکافی قالب پاد (Pod Template) و هش تغییرات** ۳. **استقرار عملیاتی و ارتباط دیپلویمنت با سرویس** ۴. **اسکیل دستی و خودکار (Scaling / HPA)** ۵. **معماری رولینگ‌آپدیت (Rolling Update)، تحلیل پارامترهای maxSurge و maxUnavailable**. ۶ **کالبدشکافی رول‌بک (Rollback)، تاریخچه نسخه‌ها و استراتژی‌های Canary / Blue-Green**

—

۱. تئوری دیپلویمنت‌ها (Deployment Theory)

دیپلویمنت یک آبجکت سطح‌بالا در گروه API به نام `apps/v1` است. کار اصلی دیپلویمنت این است که شما فقط به صورت اعلانی بگویید: **«من ۱۰ نسخه از این برنامه وب با نسخه ۲.۰ می‌خواهم.»** کنترلر دیپلویمنت به طور ۲۴ ساعته این وضعیت مطلوب را تضمین می‌کند.



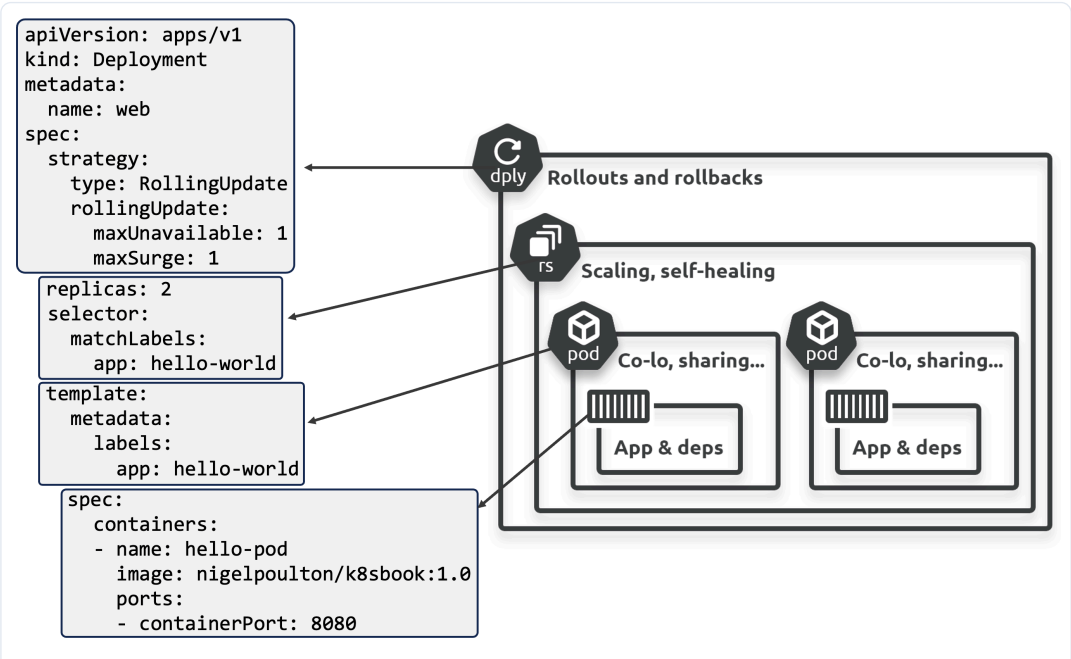
تصویر ۶.۱ - کنترلر دیپلویمنت وضعیت مطلوب پادها را به طور مداوم رصد و تضمین می‌کند

همان‌طور که در تصویر ۶.۱ مشاهده می‌کنید، اگر برنامه‌ای متشکل از فرانت‌اند وب و سبد خرید باشد، دو دیپلویمنت مجزا تعریف می‌شوند: یکی ۴ نمونه از پادهای وب را مدیریت می‌کند و دیگری ۲ نمونه از پادهای سبد خرید را سرپا نگه می‌دارد.

معماری سه‌لایه دیپلویمنت

یک تصور اشتباه رایج این است که دیپلویمنت مستقیماً پادها را می‌سازد. در واقعیت، کوبرنتیز از یک معماری سلسله‌مراتبی فوق‌العاده هوشمندانه استفاده می‌کند:

دیپلویمنت (Deployment) → رپلیکاست (ReplicaSet) → پادها (Pods) → کانتینرها (Containers)



تصویر ۶.۳ - معماری سلسله‌مراتبی دیپلویمنت، رپلیکاست و پادها و نگاشت آن‌ها در مانیفست YAML

بیایید وظیفه هر لایه را واکاوی کنیم: ۱. **دیپلویمنت (Deployment):** مغز استراتژیک است. وظیفه مدیریت نسخه‌ها، اجرای رولینگ‌آپدیت، متوقف کردن یا از سرگیری استقرار و مدیریت رول‌بک را بر عهده دارد. دیپلویمنت این کار را از طریق ایجاد و مدیریت ReplicaSetها انجام می‌دهد. ۲. **رپلیکاست (ReplicaSet - rs):** سرباز تضمین تعداد است! تنها یک وظیفه دارد: تضمین کند که در هر لحظه دقیقاً تعداد مشخصی (مثلاً ۵ کپی) از پادهای یک نسخه خاص روی کلاستر زنده باشند. ۳. **پاد (Pod):** کپسول اجرایی کانتینرها است.

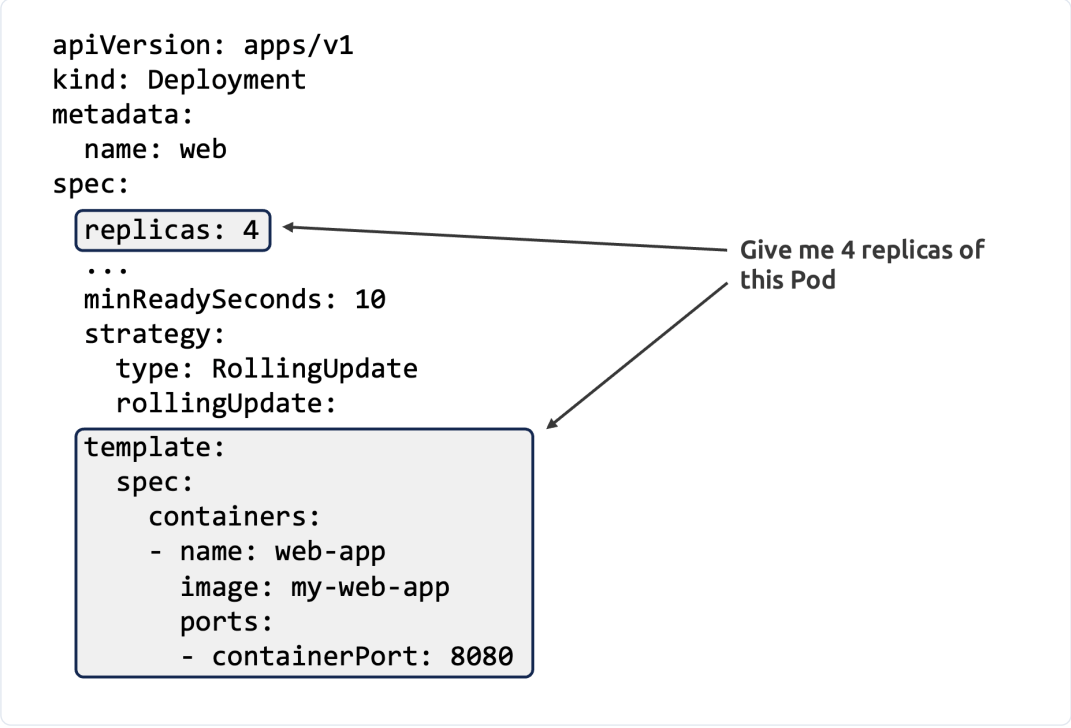
چرا این تفکیک وظایف این‌قدر حیاتی است؟

اگر دیپلویمنت مستقیماً پادها را مدیریت می‌کرد، در زمان به‌روزرسانی نرم‌افزار نمی‌توانست نسخه قدیمی را در حالت آماده‌باش نگه دارد! به لطف این معماری، در زمان آپدیت، دیپلویمنت یک ReplicaSet جدید برای نسخه ۲ می‌سازد و به آرامی پادهای ReplicaSet قدیمی را خاموش می‌کند. اما ReplicaSet قدیمی با تنظیم تعداد ۰ در پایگاه داده کلاستر باقی می‌ماند تا در صورت بروز هرگونه باگ، رول‌بک به نسخه قبل در کسری از ثانیه انجام شود!

—

۲. کالبدشکافی مانیفست دیپلویمنت: قالب پاد و برچسب‌ها

تصویر ۶.۲ بخش‌های مختلف یک فایل مانیفست دیپلویمنت را به تصویر کشیده است:



تصویر ۶.۲ - تعریف تعداد نسخه‌ها (replicas)، انتخابگر برچسب و قالب پاد در مانیفست Deployment

فایل `deploy.yml` از پوشه کدهای کتاب را بررسی می‌کنیم:

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: m-deploy
  labels:
    app: sample-app
spec:
  replicas: 10
  selector:
    matchLabels:
      app: sample-app
  template:
    metadata:
      labels:
        app: sample-app
    spec:
      containers:
      - name: web-ctr
        image: nigelpoulton/k8sbook:1.0
        ports:
        - containerPort: 8080
```

در این فایل سه بخش حیاتی وجود دارد که هماهنگی آن‌ها بسیار مهم است: ۱. `spec.replicas` : تعداد پادهای مطلوبی که باید همواره در حال اجرا باشند (اینجا ۱۰ کپی). ۲. `spec.selector.matchLabels` : انتخابگر برچسب است. دیپلویمنت با این برچسب (`app: sample-app`) می‌فهمد کدام پادهای کلاستر متعلق به او هستند. ۳. `spec.template` : **قالب پاد (Pod Template)** است. هر زمان که دیپلویمنت نیاز به ساخت پاد جدیدی داشته باشد (مثلاً هنگام اسکیل آپ یا جایگزینی پاد خراب)، عیناً از این قالب برای تولد پاد استفاده می‌کند.

> ⚠️ **هشدار:**
>
> برچسب‌های تعریف‌شده در `template.metadata.labels` باید حتماً و دقیقاً با `selector.matchLabels` همخوانی داشته باشند؛ در غیر این صورت سرور API درخواست شما را رد خواهد کرد!

—

۳. استقرار و بازرسی عملیاتی دیپلویمنت

وارد پوشه کدهای مربوط به دیپلویمنت شوید:

```
cd TKB/deployments
```

مانیفست را بر روی کلاستر اعمال کنید:

```
kubectl apply -f deploy.yml
```

خروجی ثبت دیپلویمنت را اعلام می‌کند:

```
deployment.apps/m-deploy created
```

اکنون با دستورات زیر وضعیت لایه‌های مختلف را بررسی کنید:

```
# مشاهده وضعیت دیپلویمنت
kubectl get deploy m-deploy

# مشاهده رپلیکاست ایجادشده
kubectl get rs

# مشاهده پادهای دەگانه تولیدشده
kubectl get pods --show-labels
```

خروجی `kubectl get rs` را بررسی کنید:

```
NAME                   DESIRED   CURRENT   READY   AGE
m-deploy-74b7897c8d    10        10        10      30s
```

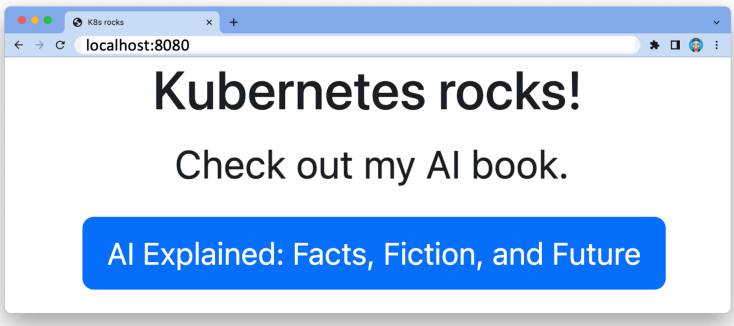
به پسوند هش نام رپلیکاست دقت کنید (`74b7897c8d`). کوبرنتیز از روی محتویات بخش `spec.template` یک مقدار هش محاسباتی می‌سازد و نام ReplicaSet را به آن مزین می‌کند. به این هش `pod-template-hash` می‌گویند. تمام پادهای این نسخه نیز این هش را در برچسب‌های خود دارند.

اتصال دیپلویمنت به سرویس و مشاهده برنامه در مرورگر

برای این‌که بتوانیم به این ۱۰ پاد دسترسی پیدا کنیم، سرویس لودبالانسری که در فایل `svc.yml` قرار دارد را مستقر می‌کنیم:

```
kubectl apply -f svc.yml
```

وضعیت سرویس را با `kubectl get svc` مشاهده کنید و سپس آدرس `localhost:8080` (یا IP عمومی کلود) را در مرورگر باز کنید.



تصویر ۶.۶ - دسترسی به نسخه ۱.۰ برنامه از طریق پورت ۸۰۸۰ در مرورگر

همان‌طور که در تصویر ۶.۶ دیده می‌شود، نسخه ۱.۰ برنامه با موفقیت بارگذاری شده است.

—

۴. اسکیل دستی و خودکار (Scaling)

یکی از لذت‌بخش‌ترین بخش‌های کار با کوبرنتیز، افزایش و کاهش آنی ظرفیت پردازشی برنامه است.

۱. اسکیل دستی و دستوری (Imperative Scaling)

فرض کنید ترافیک ناگهانی به سمت سایت شما سرازیر شده و می‌خواهید ظرفیت را از ۱۰ پاد به ۵ پاد کاهش یا به ۱۵ پاد افزایش دهید:

```
kubectl scale --replicas=5 deploy/m-deploy
```

تنها ظرف چند ثانیه، کنترلر ReplicaSet اقدام به خاموش کردن ۵ پاد اضافی می‌کند تا تعداد به ۵ برسد.

۲. اسکیل اعلانی (Declarative Scaling)

روش استاندارد در خطوط لوله CI/CD، ویرایش مستقیم فیلد `replicas` در فایل `deploy.yml` و اعمال مجدد آن با دستور `kubectl apply -f deploy.yml` است.

۳. اسکیل خودکار افقی پادها (Horizontal Pod Autoscaler - HPA)

در پروژه‌های پروداکشن، شما پای مانیتور نمی‌نشینید تا دستور `scale` بزنید! بلکه با تعریف شیء **HPA** به کوبرنتیز دستور می‌دهید: **«اگر میانگین مصرف پردازنده (CPU) پادها از ۷۰٪ بالاتر رفت، تعداد پادها را به صورت پویا تا سقف ۵۰ پاد افزایش بده و در زمان کاهش ترافیک مجدداً آن را کم کن.»**

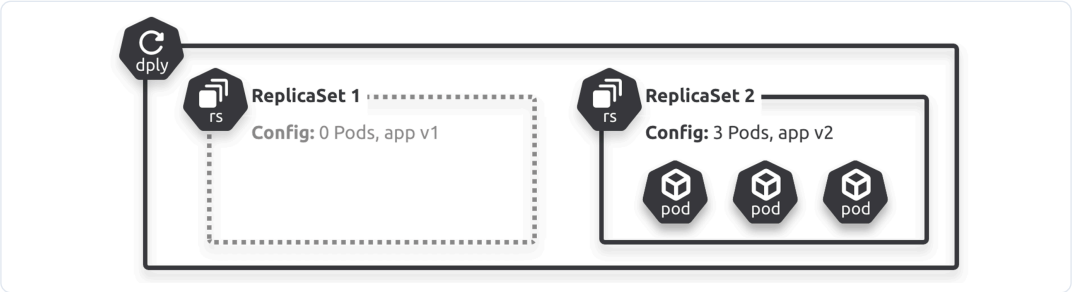
—

۵. ارتقای تدریجی بدون حتی یک ثانیه قطعی (Rolling Update)

بزرگ‌ترین کابوس مهندسان نرم‌افزار در روش‌های سنتی، قطعی سرویس در زمان انتشار نسخه جدید (Downtime) بود. در کوبرنتیز، الگوریتم **Rolling Update** این کابوس را برای همیشه به خاطره‌ها سپرده است!

الگوریتم Rolling Update چگونه کار می‌کند؟

هنگامی که تغییری در بخش `spec.template` دیپلویمنت (مانند تغییر نسخه ایمیج کانتینر، تغییر متغیر محیطی یا تغییر پورت) رخ می‌دهد: ۱. دیپلویمنت یک ReplicaSet کاملاً جدید برای نسخه جدید می‌سازد. ۲. دیپلویمنت با سرعتی کنترل‌شده شروع به افزایش پادها در ReplicaSet جدید و کاهش همزمان پادها در ReplicaSet قدیمی می‌کند. ۳. سرویس لودبالانسر شبکه به محض آماده شدن هر پاد جدید، ترافیک را به سمت آن هدایت کرده و پادهای قدیمی را به آرامی از چرخه ترافیک خارج می‌سازد.



تصویر ۶.۴ - ارتقای تدریجی: رپلیکاست سمت راست (نسخه جدید) فعال شده و رپلیکاست سمت چپ (نسخه قدیم) به صفر کاهش یافته است

کنترل نرخ انتشار با `maxSurge` و `maxUnavailable`

دیپلویمنت دارای دو پارامتر کنترلی حیاتی در بخش `spec.strategy.rollingUpdate` است:

- `maxSurge` : حداکثر تعداد پادهایی که می‌توانند در طول فرآیند ارتقا بیش از تعداد مطلوب اصلی (`replicas`) ساخته شوند (مقدار پیش‌فرض: ۲۵٪).
- `maxUnavailable` : حداکثر تعداد پادهایی که در طول آپدیت اجازه دارند به طور موقت از دسترس خارج باشند (مقدار پیش‌فرض: ۲۵٪).

این مقادیر می‌توانند به صورت درصد (مثلاً `25%`) یا عدد صحیح (مثلاً `2`) تنظیم شوند و تضمین می‌کنند هیچ‌گاه ظرفیت پاسخگویی سیستم در حین آپدیت سقوط نکند.

اجرای عملی ارتقا به نسخه ۲.۰

فایل `deploy.yml` را ویرایش کرده و تگ ایمیج را از `1.0` به `2.0` تغییر دهید:

```
spec:
  containers:
  - name: web-ctr
    image: nigelpoulton/k8sbook:2.0     # نسخه جدید
```

اکنون نسخه جدید را اعمال کنید:

```
kubectl apply -f deploy.yml
```

برای مشاهده زنده و هیجان‌انگیز فرآیند تعویض پادها، دستور زیر را اجرا کنید:

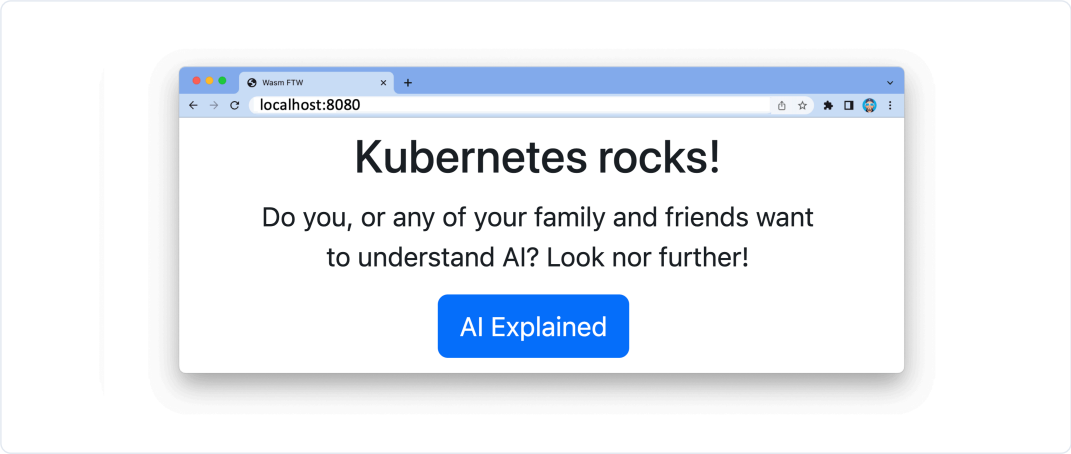
```
kubectl rollout status deploy/m-deploy
```

خروجی مرحله به مرحله پیشرفت را به شما گزارش می‌دهد:

```
Waiting for deployment "m-deploy" rollout to finish: 3 out of 10 new replicas
have been updated...
Waiting for deployment "m-deploy" rollout to finish: 6 of 10 updated replicas
are available...
deployment "m-deploy" successfully rolled out
```

دستور `kubectl get rs` را بزنید: خواهید دید که دو ReplicaSet وجود دارد: رپلیکاست قدیمی تعداد پادهایش صفر شده و رپلیکاست جدید دارای ۱۰ پاد است!

مرورگر خود را رفرش کنید؛ صفحه نسخه ۲.۰ بدون حتی یک ثانیه قطعی نمایان می‌شود:

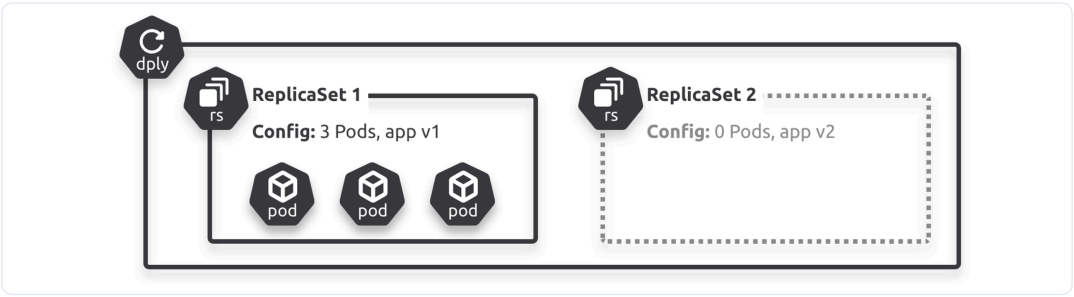


تصویر ۶.۷ - نسخه ۲.۰ برنامه بدون کوچک‌ترین قطعی در خدمت کاربران قرار گرفته است

—

۶. بازگشت آنی به نسخه قبل (Rollback)

فرض کنید نسخه ۲.۰ دارای یک باگ پیش‌بینی‌نشده در پایگاه داده است و باید فوراً برنامه را به نسخه ۱.۰ برگردانید. در روش‌های سنتی این کار ساعت‌ها بحران ایجاد می‌کرد. در کوبرنتیز این کار تنها با یک دستور در چند ثانیه انجام می‌شود!



تصویر ۶.۵ - فرآیند رول‌بک: احیای مجدد رپلیکاست قبلی و خاموش‌سازی رپلیکاست دارای باگ

بررسی تاریخچه نسخه‌ها (Rollout History)

دستور زیر تاریخچه تمام استقرارهای این دیپلویمنت را به شما نشان می‌دهد:

```
kubectl rollout history deploy/m-deploy
```

خروجی:

```
REVISION  CHANGE-CAUSE
1         <none>
2         <none>
```

> **💡 ترفند و تجربه عملیاتی:**
>
> اگر در زمان اعمال مانیفست‌ها از ابزارهای استاندارد GitOps (مثل ArgoCD) یا حاشیه‌نویسی اعلانی `kubernetes.io/change-cause` استفاده کنید، ستون `CHANGE-CAUSE` پیام کامیت یا دلیل تغییر را نیز مستندسازی می‌کند.

دستور اجرای رول‌بک

برای بازگشت به نگارش قبلی (Revision 1)، دستور زیر را تایپ کنید:

```
kubectl rollout undo deploy/m-deploy
```

همچنین می‌توانید مستقیماً به یک نسخه مشخص بازگردید:

```
kubectl rollout undo deploy/m-deploy --to-revision=1
```

کنترلر دیپلویمنت بلافاصله رپلیکاست شماره ۱ را بالا آورده و رپلیکاست شماره ۲ را به آرامی به صفر می‌رساند. مرورگر را بازخوانی کنید؛ نسخه ۱.۰ بازگشته است!

استراتژی‌های Canary و Blue-Green با توقف و ازسرگیری (Pause & Resume)

کوبرنتیز به شما اجازه می‌دهد فرآیند انتشار را متوقف کنید:

```
# متوقف کردن رول‌اوت در میانه راه (مناسب تست Canary از کاربران روی بخش کوچکی)
kubectl rollout pause deploy/m-deploy

# پس از اطمینان از سلامت نسخه در ترافیک واقعی
kubectl rollout resume deploy/m-deploy

# اگر کلاً می‌خواهید فرآیند ارتقا ریستارت شود
kubectl rollout restart deploy/m-deploy
```

—

۷. پاک‌سازی منابع (Clean up)

با حذف مانیفست دیپلویمنت و سرویس، تمامی رپلیکاست‌ها و پادهای مرتبط به طور کامل و تمیز پاک‌سازی می‌شوند:

```
kubectl delete -f deploy.yml -f svc.yml
```

—

۸. جمع‌بندی فصل ۶

- پادها به تنهایی فناپذیرند؛ در محیط عملیاتی همیشه باید پادها را درون دیپلویمنت بسته‌بندی کرد.
- معماری دیپلویمنت سه‌لایه است: `Deployment -> ReplicaSet -> Pod` .
- رپلیکاست ضامن حفظ تعداد دقیق کپی‌هاست، در حالی که دیپلویمنت مدیر ارتقا و مدیریت نسخه‌هاست.
- با تغییر بخش `spec.template` ، فرآیند Rolling Update آغاز شده و یک ReplicaSet جدید ساخته می‌شود.
- دو متغیر `maxSurge` و `maxUnavailable` نرخ ایجاد پادهای جدید و خروج پادهای قدیم را تنظیم می‌کنند.
- دیپلویمنت تاریخچه رپلیکاست‌های قدیمی را حفظ می‌کند تا با دستور `kubectl rollout undo` بازگشت آنی (Rollback) به نگارش‌های پیشین محقق شود.

فصل ۷: سرویس‌های کوبرنتیز و شبکه پایدار (Services)

یک اصل بنیادین در معماری سیستم‌های توزیع‌شده وجود دارد: **پادها بی‌ثبات و غیرقابل‌اعتمادند و شما هرگز نباید مستقیماً به آدرس IP پادها متصل شوید!**

پادها موجوداتی زودگذر (Ephemeral) هستند؛ با عملیات کاهش اسکیل (Scale-down)، ارتقای تدریجی (Rolling Update)، رول‌بک، خاموشی نودها یا کرش نرم‌افزاری، پادها نابود می‌شوند و پادهای جدیدی با آدرس‌های IP کاملاً متفاوت متولد می‌گردند. اگر فرانت‌اند وب مستقیماً با IP پاد دیتابیس یا سبد خرید صحبت کند، با اولین آپدیت کل سیستم فرو خواهد ریخت!

راهکار درخشان کوبرنتیز برای حل این چالش، شیء قدرتمندی به نام **سرویس (Service)** است. یک سرویس به عنوان یک لایه پایدار، نقطه‌اتصال شبکه (IP ثابت، پورت ثابت و نام DNS ماندگار) را در اختیار کلاینت‌ها قرار می‌دهد و ترافیک را به صورت هوشمند و توزیع‌شده میان پادهای سالم هدایت می‌کند.

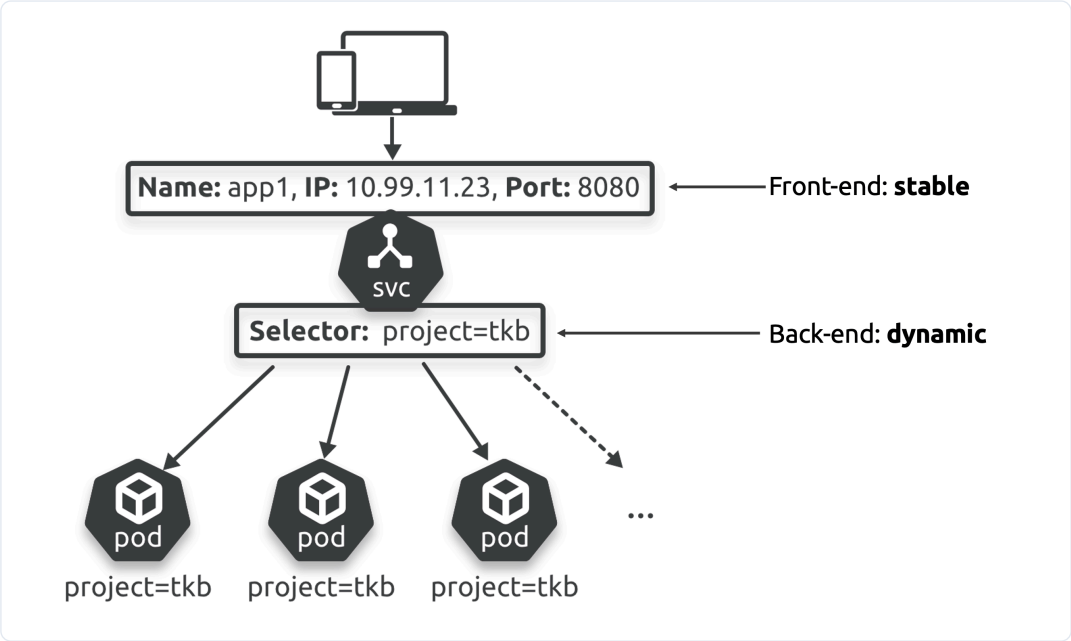
در این فصل مباحث زیر را به صورت کامل و تفسیری کالبدشکافی خواهیم کرد: ۱. **تئوری سرویس‌ها: تبیین ساختار Front-end پایدار در برابر Back-end پویا** ۲. **اتصال سست (Loose Coupling) بر پایه برچسب‌ها و انتخابگرها (Labels & Selectors)** ۳. **کالبدشکافی لایه زیرین: اشیای EndpointSlice و کنترلر ناظر سلامت** ۴. **معماری kube-proxy و مکانیزم توزیع بار کلاینت‌محور با iptables و** ۵ **eBPF**. **انواع سرویس‌ها (ClusterIP، NodePort، LoadBalancer) و معماری سلسله‌مراتبی (Service Stacking)** ۶. **سرویس‌های اختصاصی: Headless Services و** ۷ **ExternalName**. **کارگاه عملیاتی جامع: استقرار دیپلویمنت، اکسپوز کردن دستوری و اعلانی و بررسی ترافیک شبکه**

—

۱. تئوری سرویس‌ها (Service Theory)

در ادبیات کوبرنتیز، کلمه Service با حرف بزرگ انگلیسی نوشته می‌شود تا مشخص شود اشاره به یک منبع رسمی در API دارد.

هر سرویس دارای دو بخش کاملاً مجزا است: ۱. **بخش جلویی (Front-end):** شامل یک نام DNS یکتا، یک آدرس IP مجازی اختصاصی (Virtual IP یا VIP) و یک پورت شبکه است. **کوبرنتیز رسماً ضمانت می‌کند که این مشخصات در تمام طول عمر سرویس هرگز تغییر نخواهند کرد!** ۲. **بخش عقبی (Back-end):** یک انتخابگر برچسب (Label Selector) است که ترافیک ورودی را به پادهای سالمی که واجد آن برچسب‌ها هستند هدایت می‌کند.



تصویر ۷.۱ - کلاینت‌ها با آدرس IP و نام ثابت سرویس صحبت می‌کنند و سرویس ترافیک را به پادهای متغیر پشت سر خود هدایت می‌کند

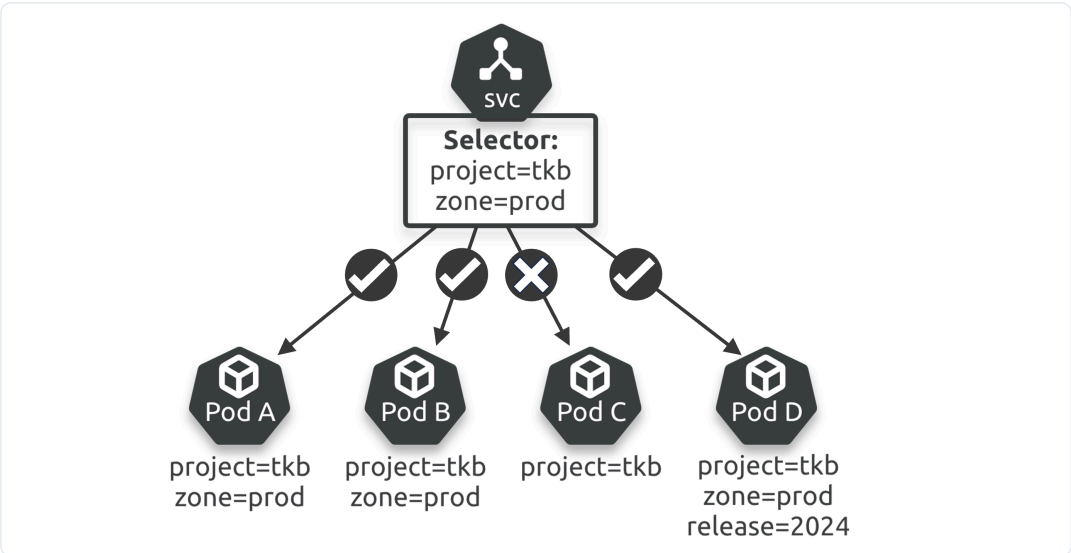
همان‌طور که در تصویر ۷.۱ مشاهده می‌کنید:

- کلاینت‌ها فقط آدرس سرویس (`app1:8080` یا `10.99.11.23:8080`) را می‌شناسند.
- پادهای پشت صحنه می‌توانند آزادانه کم یا زیاد شوند، نسخه‌شان ارتقا یابد یا حتی کرش کنند؛ سرویس همواره فهرستی به‌روز از پادهای سالم را در اختیار دارد و ترافیک را بدون سردرگمی به سمت نمونه‌های زنده می‌فرستد.

—

۲. اتصال سست (Loose Coupling) با برچسب‌ها و انتخابگرها

ارتباط میان سرویس و پادها از نوع اتصال سست (Loosely Coupled) است. سرویس هیچ اطلاعی از این‌که پادها چگونه ساخته شده‌اند یا چند پاد وجود دارد ندارد؛ سرویس فقط به دنبال برچسب‌های خاص (Labels) می‌گردد.



تصویر ۷.۲ - سازوکار هدایت ترافیک سرویس بر پایه تطبیق دقیق برچسب‌ها (Labels & Selectors)

تصویر ۷.۲ این مکانیزم را به زیبایی شرح داده است:

- سرویس به دنبال پادهایی است که دارای دو برچسب `project=tkb` و `zone=prod` باشند.
- پاد A، پاد B و پاد D هر دو برچسب را دارند؛ بنابراین ترافیک را دریافت می‌کنند (حتی اگر پاد D برچسب‌های اضافه دیگری نیز داشته باشد).
- اما پاد C چون فاقد برچسب `zone=prod` است، از لیست دریافت‌کنندگان ترافیک این سرویس حذف می‌شود.

مانیفست زیر هماهنگی اعلانی میان یک دیپلویمنت و یک سرویس را نشان می‌دهد:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: tkb-deployment
spec:
  replicas: 5
  template:
    metadata:
      labels:
        project: tkb        # پادها با این برچسب‌ها متولد می‌شوند
        zone: prod
    spec:
      containers:
      - name: web
        image: nginx
---
apiVersion: v1
kind: Service
metadata:
  name: tkb-svc
spec:
  ports:
  - port: 8080
    targetPort: 80
  selector:
    project: tkb            # سرویس ترافیک را به پادهای دارای این برچسب‌ها
می‌فرستد
    zone: prod
```

—

۳. پشت صحنه سرویس‌ها: تولد آبجکت‌های EndpointSlice

شاید این سوال برایتان پیش بیاید: **«سرویس در پشت صحنه چگونه آدرس IP پادهای سالم را پیدا می‌کند؟»**

پاسخ در یک آبجکت هوشمند به نام **EndpointSlice** نهفته است: ۱. به محض ساخت یک سرویس، کنترلر EndpointSlice به طور خودکار یک شیء متناظر برای آن ایجاد می‌کند. ۲. کوبرنتیز به طور مداوم کلاستر را رصد می‌کند؛ به محض آماده شدن یک پاد جدید با برچسب‌های منطبق، آدرس IP و پورت پاد به لیست EndpointSlice اضافه می‌شود. ۳. اگر پادی متوقف شود یا پروب آمادگی (Readiness Probe) آن شکست بخورد، بلافاصله از لیست حذف می‌گردد. ۴. کلاینت درخواست را به IP سرویس می‌فرستد و سیستم ترافیک را به یکی از IPهای ثبت‌شده در EndpointSlice فوروارد می‌کند.

تحول EndpointSlice در برابر Endpoints سنتی

در نسخه‌های قدیمی کوبرنتیز، تمام آدرس‌های یک سرویس در یک شیء واحد به نام `Endpoints` ذخیره می‌شد. در کلاسترهای بزرگ سازمانی که یک سرویس دارای هزاران پاد بود، با تغییر وضعیت حتی یک پاد، کل این شیء بزرگ باید در etcd بازنویسی و در کل نودها همگام می‌شد که بار پردازشی عظیمی ایجاد می‌کرد. شیء مدرن **EndpointSlice** این داده‌ها را به قطعات کوچک حداکثر ۱۰۰ تایی تقسیم می‌کند که باعث اسکیل‌پذیری فوق‌العاده (Scalability) و عملکرد بی‌نقص شبکه در کلاسترهای غول‌پیکر می‌شود.

—

۴. کالبدشکافی kube-proxy: جادوی توزیع بار در سطح کلاستر

سرویس در کوبرنتیز یک سرور فیزیکی یا یک فرآیند نرم‌افزاری فعال نیست! پس ترافیک ورودی به IP مجازی سرویس چگونه مسیریابی می‌شود؟

پاسخ: توسط سرویس سیستمی **kube-proxy** که روی تک‌تک نودهای کلاستر به صورت DaemonSet در حال اجراست.

- `kube-proxy` سرور API را رصد می‌کند و از ایجاد سرویس‌ها و تغییرات EndpointSlice مطلع می‌شود.
- سپس قواعد مسیریابی را در هسته لینوکس هر نود (از طریق `iptables` یا `IPVS`) تزریق می‌کند.
- در کلاسترهای مدرن مبتنی بر **Cilium**، این کار مستقیماً با کدهای فوق‌سریع **eBPF** در سوکت‌های شبکه کرنل انجام می‌شود بدون این‌که نیازی به جدول‌های تودرتوی iptables باشد.
- نتیجه: توزیع بار کاملاً نامتمرکز و توزیع‌شده (Client-side Load Balancing) بدون وجود هیچ نقطه گلودگاه مرکزی!

—

۵. انواع سرویس‌ها و معماری لایه‌ای (Service Stacking)

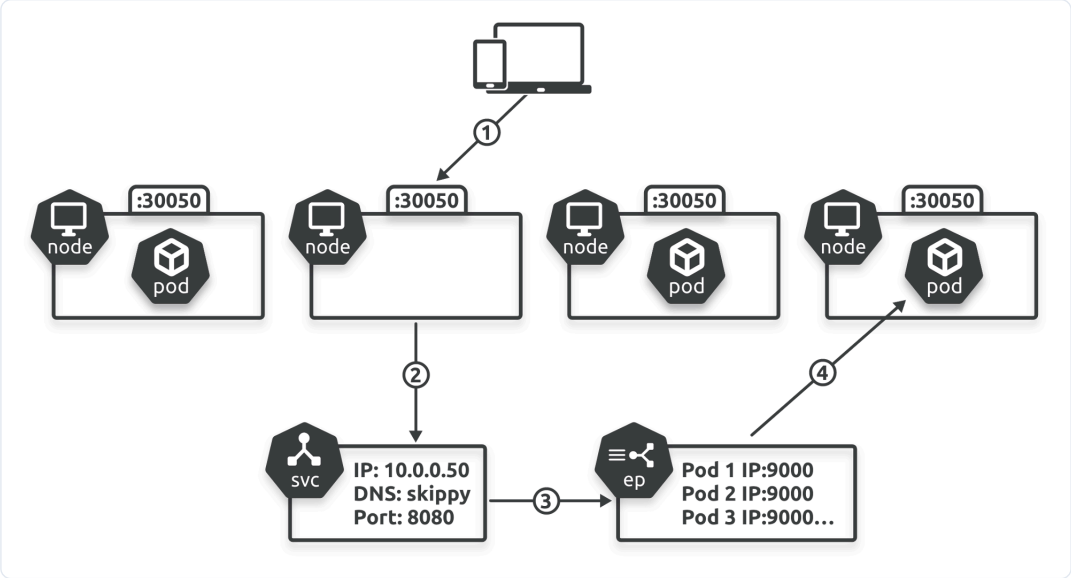
کوبرنتیز سه نوع اصلی سرویس برای نیازمندی‌های مختلف ارائه می‌دهد:

۱. سرویس نوع ClusterIP (حالت پیش‌فرض)

ساده‌ترین نوع سرویس است. یک آدرس IP مجازی درون‌کلاستری به سرویس اختصاص داده می‌شود که فقط و فقط از درون خود کلاستر (توسط سایر پادها) قابل دسترسی است. برای ارتباطات میکروسرویس‌های داخلی (مثل اتصال فرانت‌اند به بک‌اند یا بک‌اند به دیتابیس) از این نوع استفاده می‌شود.

۲. سرویس نوع NodePort (دسترسی مستقیم از بیرون کلاستر)

سرویس NodePort بر روی سرویس ClusterIP سوار می‌شود و یک پورت اختصاصی در رنج بالا (**۳۰۰۰۰ تا ۳۲۷۶۷**) را بر روی **تک‌تک نودهای کلاستر** باز می‌کند.



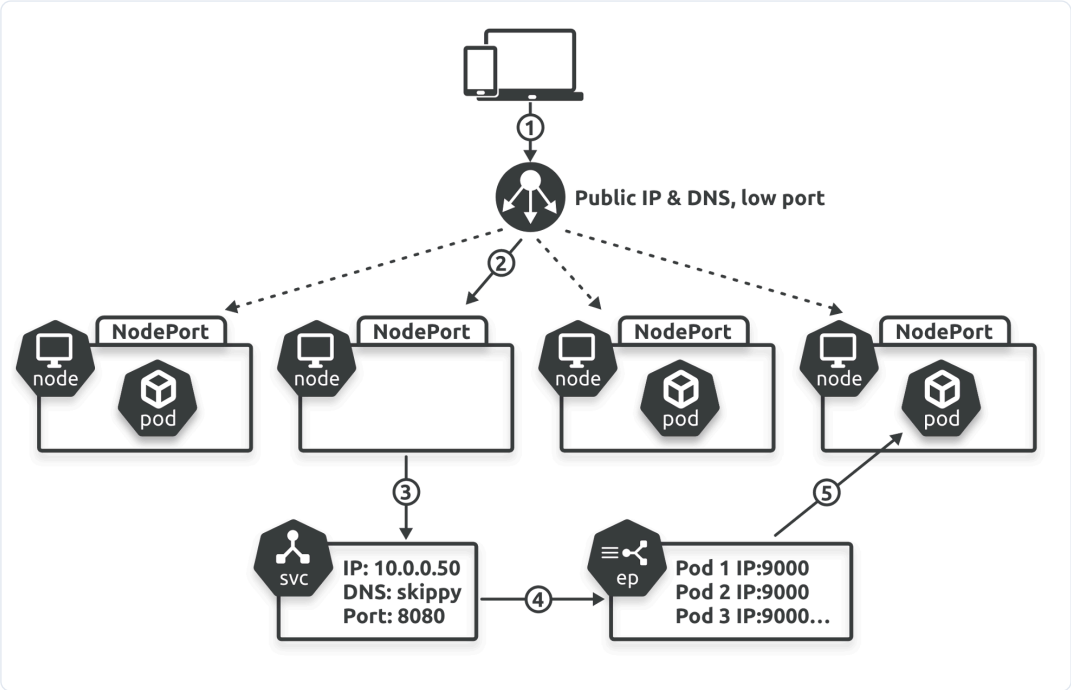
تصویر ۷.۳ - سرویس NodePort: باز شدن پورت اختصاصی روی تمامی نودها و هدایت ترافیک به سمت ClusterIP و پادها

همان‌طور که در تصویر ۷.۳ نمایش داده شده است: ۱. کلاینت بیرونی به آدرس هر یک از نودهای کلاستر روی پورت NodePort (مثلاً `30050`) متصل می‌شود. ۲. آن نود ترافیک را به ClusterIP سرویس فوروارد می‌کند. ۳. سرویس پادی سالم را از لیست EndpointSlice انتخاب کرده و بسته را به مقصد می‌رساند.

محدودیت‌های NodePort: استفاده از پورت‌های غیررسمی بالای ۳۰۰۰۰ و الزام کلاینت به دانستن آدرس IP نودها. به همین دلیل در محیط پروداکشن معمولاً از نوع بعدی استفاده می‌شود.

۳. سرویس نوع LoadBalancer (استاندارد اتصال اینترنتی ابری)

سرویس LoadBalancer شاهکار راحتی اتصال است. این نوع سرویس بر روی دوش NodePort و ClusterIP سوار می‌شود، اما یک لودبالانسر شبکه ابری واقعی (مانند AWS NLB، گوگل لودبالانسر یا آژور LB) را در جلوی نودها مستقر می‌کند.

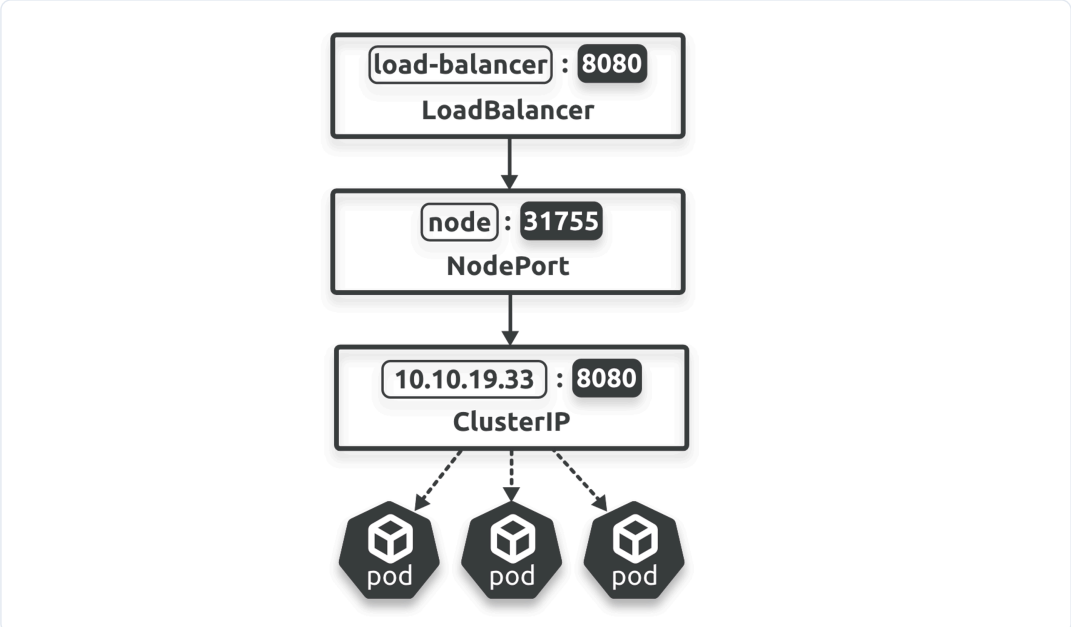


تصویر ۷.۴ – سرویس LoadBalancer: لودبالانسر ابری در ورودی کلاستر ترافیک پورت‌های استاندارد (۸۰/۴۴۳) را به نودها توزیع می‌کند

کلاینت به سادگی به یک نام دامنه‌ای معتبر یا آدرس IP عمومی روی پورت‌های استاندارد `80` یا `443` متصل می‌شود، و لودبالانسر ابری ترافیک را میان نودها پخش می‌کند.

معماری پشته‌ای سرویس‌ها (Service Stacking)

تصویر ۷.۵ معماری لایه‌ای سرویس‌ها را خلاصه می‌کند:



تصویر ۷.۵ – معماری پشته‌ای سرویس‌ها: سرویس LoadBalancer بر پایه NodePort و NodePort بر پایه ClusterIP بنا شده است

هر نوع پیشرفته‌تر، تمامی قابلیت‌ها و ساختارهای نوع قبلی را در بطن خود دارد:

```
LoadBalancer  --->  شامل NodePort  --->  شامل ClusterIP
```

—

۶. سرویس‌های خاص: Headless Services و ExternalName

علاوه بر سه حالت اصلی، دو سناریوی تخصصی نیز وجود دارد:

۱. **سرویس‌های بدون‌سر (Headless Services):** با تنظیم `clusterIP: None` تعریف می‌شوند. این سرویس‌ها هیچ VIP اختصاصی نمی‌گیرند! در عوض، سیستم CoreDNS کلاستر به جای برگرداندن یک IP مجازی، مستقیماً آدرس‌های A Record تک‌تک پادها را برمی‌گرداند. این الگو ستون فقرات اجرای پایگاه‌های داده کلاستری نظیر Cassandra، Kafka و MongoDB در کوبرنتیز است. ۲. **سرویس ExternalName:** هیچ انتخابی روی پادها انجام نمی‌دهد، بلکه یک نگاشت DNS CNAME به یک دامین خارجی (مثلاً `database.aws.com`) ایجاد می‌کند تا برنامه‌های داخلی بتوانند بدون هاردکد کردن آدرس‌های بیرونی، با یک نام محلی به دیتابیس ابری وصل شوند.

—

۷. کارگاه عملیاتی جامع: کار با سرویس‌ها (Hands-on Lab)

وارد پوشه کدهای مربوط به سرویس‌ها شوید:

```
cd TKB/services
```

۱. استقرار برنامه با دیپلویمنت اولیه

فایل `deploy.yml` یک دیپلویمنت با ۱۰ پاد وب مستقر می‌کند:

```
kubectl apply -f deploy.yml
```

بررسی صحت اجرای ۱۰ پاد:

```
kubectl get deploy svc-test
```

خروجی نشان‌دهنده آماده بودن تمامی پادهاست:

```
NAME        READY   UP-TO-DATE   AVAILABLE   AGE
svc-test    10/10   10           10          20s
```

۲. ایجاد دستوری سرویس لودبالانسر با `kubectl expose`

دستور `kubectl expose` یک ابزار هوشمند در خط فرمان است که دیپلویمنت فعال را بررسی کرده و بر اساس پورت و برچسب‌های آن، یک سرویس مناسب می‌سازد:

```
kubectl expose deployment svc-test --type=LoadBalancer
```

خروجی:

```
service/svc-test exposed
```

بررسی مشخصات سرویس ایجادشده:

```
kubectl get svc -o wide
```

خروجی مشخصات کامل را نمایش می‌دهد:

```
NAME           TYPE           CLUSTER-IP      EXTERNAL-IP     PORT(S)
AGE    SELECTOR
kubernetes     ClusterIP      10.96.0.1       <none>          443/TCP
2d     <none>
svc-test       LoadBalancer   10.10.19.33     localhost       8080:31755/TCP    30s
chapter=services
```

بیایید این خط را موشکافی کنیم:

- نام سرویس برابر با نام دیپلویمنت (`svc-test`) است.
- یک آی‌پی داخلی `CLUSTER-IP` (`10.10.19.33`) به آن داده شده است.
- یک آدرس عمومی `EXTERNAL-IP` به آن اختصاص یافته است (در داکر دسکتاپ `localhost`).
- پورت دسترسی کلاینت `8080` است و یک NodePort تصادفی (`31755`) نیز روی نودها باز شده است.
- انتخابگر برچسب (`chapter=services`) به طور خودکار منطبق بر پادها تنظیم شده است.

۳. بررسی مانیفست اعلانی معادل

در دنیای حرفه‌ای، سرویس‌ها به صورت فایل اعلانی تعریف می‌شوند:

```
apiVersion: v1
kind: Service
metadata:
  name: svc-test
spec:
  type: LoadBalancer
  ports:
  - port: 8080            # پورتی که کلاینت‌ها با آن صدا میزنند
    targetPort: 8080      # پورت واقعی کانتینر در پاد
```

```
  selector:
    chapter: services
```

۴. مشاهده وضعیت پادهای سالم در EndpointSlice

دستور زیر را برای بررسی سلامت اعضای سرویس بزنید:

```
kubectl get endpointslices -l kubernetes.io/service-name=svc-test
```

خواهید دید که آی‌پی هر ۱۰ پاد فعال به دقت در این لیست رجیستر شده است.

مرورگر خود را باز کنید و آدرس `http://localhost:8080` را مشاهده کنید. با چندین بار فشردن کلید رفرش، متوجه می‌شوید که درخواست‌های شما به صورت چرخشی (Round-Robin) میان ۱۰ پاد مختلف توزیع می‌شود.

—

۸. پاک‌سازی منابع (Clean up)

با فرامین زیر پادها و سرویس را حذف کنید:

```
kubectl delete svc svc-test
kubectl delete -f deploy.yml
```

—

۹. جمع‌بندی فصل ۷

- اتصال مستقیم به آدرس پادها یک ضدالگو است؛ همیشه باید از سرویس‌ها به عنوان درگاه مطمئن شبکه استفاده کرد.
- هر سرویس یک Front-end تغییرناپذیر (IP، DNS، پورت) و یک Back-end داینامیک دارد.
- اتصال سرویس به پادها به وسیله برچسب‌ها (Labels & Selectors) انجام می‌شود.
- پایش لحظه‌ای پادهای زنده توسط آبجکت‌های مدرن `EndpointSlice` مدیریت می‌گردد.
- سرویس `kube-proxy` و ابزارهای CNI نظیر Cilium/eBPF توزیع بار را به صورت نامتمرکز در سطح نودها هدایت می‌کنند.
- سه نوع رایج سرویس عبارتند از: `ClusterIP` (مخصوص شبکه داخلی)، `NodePort` (باز کردن پورت روی نودها) و `LoadBalancer` (یکپارچگی مستقیم با لودبالانسر شبکه ابری).

فصل ۸: مدیریت هوشمند ورودی با اینگرس (Ingress)

در فصل ۷ با سرویس‌های نوع `LoadBalancer` آشنا شدیم و دیدیم که چگونه می‌توانند ترافیک بیرونی را مستقیماً از اینترنت به پادهای داخل کلاستر برسانند. اما در دنیای واقعی و سامانه‌های بزرگ سازمانی که از ده‌ها یا صدها میکروسرویس تشکیل شده‌اند، این روش با دو بن‌بست جدی مواجه می‌شود: ۱. **هزینه‌های سرسام‌آور زیرساختی:** در ارائه‌دهندگان ابری (مانند AWS، گوگل، آژور یا کلودهای محلی)، ایجاد هر سرویس `LoadBalancer` به معنای سفارش یک دستگاه لودبالانسر سخت‌افزاری/نرم‌افزاری اختصاصی با آدرس IP عمومی مجزاست. داشتن ۵۰ میکروسرویس یعنی پرداخت هزینه اجاره ۵۰ لودبالانسر شبکه! ۲. **ناتوانی در مسیریابی هوشمند لایه ۷ (Application Layer):** سرویس‌های استاندارد کوبرنتیز در لایه ۴ شبکه (TCP/UDP) کار می‌کنند. آن‌ها پکت‌های خام شبکه را می‌بینند و نمی‌توانند محتوای درخواست‌های HTTP/HTTPS، نام دامنه وب‌سایت (Host Header) یا مسیر URL (مانند `/api` یا `/checkout`) را بررسی و تفکیک کنند.

برای حل این چالش‌های بنیادین، کوبرنتیز ابزار قدرتمند **اینگرس (Ingress)** را معرفی کرده است: یک دروازه ورودی هوشمند لایه ۷ که با استفاده از **تنها یک لودبالانسر و یک آدرس IP عمومی واحد**، ترافیک ورودی را بر اساس نام دامنه و مسیر به ده‌ها میکروسرویس مختلف هدایت می‌کند.

در این فصل مباحث زیر را به صورت کامل و تفسیری فرا خواهیم گرفت: ۱. **چرایی پیدایش اینگرس و مقایسه لایه ۴ در برابر لایه ۷** ۲. **معماری دوگانه: شیء اینگرس (Ingress Resource) در برابر کنترلر اینگرس (Ingress Controller)** ۳. **مسیریابی مبتنی بر هاست (Host-based) و مسیر (Path-based)** ۴. **پایان‌دهی امنیتی گواهینامه‌های SSL/TLS در لایه ورودی** ۵. **معرفی معماری نسل بعدی: استاندارد Gateway API در کوبرنتیز مدرن** ۶. **کارگاه عملیاتی گام‌به‌گام: استقرار اینگرس با سناریوی چنددامنه‌ای و تست با curl و مرورگر**

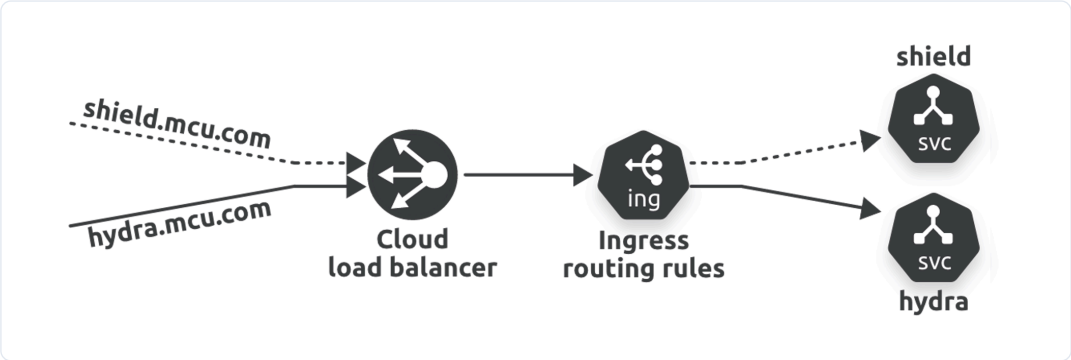
—

۱. صورت مسئله: چرا سرویس‌های لایه ۴ کافی نیستند؟

بیایید یک سناریوی تجاری واقعی را بررسی کنیم. فرض کنید سامانه‌ای با دامنه‌های زیر دارید:

- `mcu.com` : وب‌سایت اصلی
- `shield.mcu.com` : پرتال امنیتی
- `hydra.mcu.com` : سرویس اعتبارسنجی
- `mcu.com/api/v1` : مسیر درخواست‌های برنامه‌نویسی

اگر بخواهید با سرویس‌های کلاسیک `LoadBalancer` این سامانه را بالا بیاورید، برای هر دامنه به یک سرویس و یک لودبالانسر بیرونی نیاز دارید. علاوه بر این، مسیریابی بر اساس آدرس URL (`/api/v1`) روی لایه ۴ اصلاً امکان‌پذیر نیست!



تصویر ۸.۱ - مسیریابی مبتنی بر هاست با یک لودبالانسر واحد: ترافیک دامنه‌های مختلف به یک IP عمومی برخورد کرده و توسط اینگرس تفکیک می‌شود

همان‌طور که در تصویر ۸.۱ مشاهده می‌کنید، راه‌حل اینگرس فوق‌العاده هوشمندانه است: ۱. هر دو دامنه در سیستم DNS عمومی اینترنت به **یک آدرس IP عمومی مشترک** اشاره می‌کنند. ۲. این IP متعلق به یک لودبالانسر ورودی واحد است. ۳. در پشت لودبالانسر، **کنترلر اینگرس** نشسته است؛ درخواستی که برای `shield.mcu.com` می‌آید را به پادهای شیلد، و درخواستی که برای `hydra.mcu.com` می‌آید را به پادهای هایدرا هدایت می‌کند.

—

۲. معماری دوگانه اینگرس: منبع اعلانی در برابر کنترلر اجرایی

برای فهم عمیق اینگرس، باید بدانید که اینگرس بر خلاف پاد یا دیپلویمنت، از دو بخش کاملاً مجزا تشکیل شده است:

شیء اینگرس (قوانین روی کاغذ) + کنترلر اینگرس (موتور اجرایی واقعی)

۱. شیء اینگرس (Ingress Resource)

یک مانیفست YAML استاندارد است که به سرور API ارسال می‌کنید. در این فایل فقط قوانین مسیریابی را می‌نویسید: **«اگر درخواستی با هاست x آمد به سرویس y بفرست؛ اگر مسیر z بود به سرویس w هدایت کن.»** این شیء به خودی خود هیچ کدی اجرا نمی‌کند و ترافیکی را جابه‌جا نمی‌سازد!

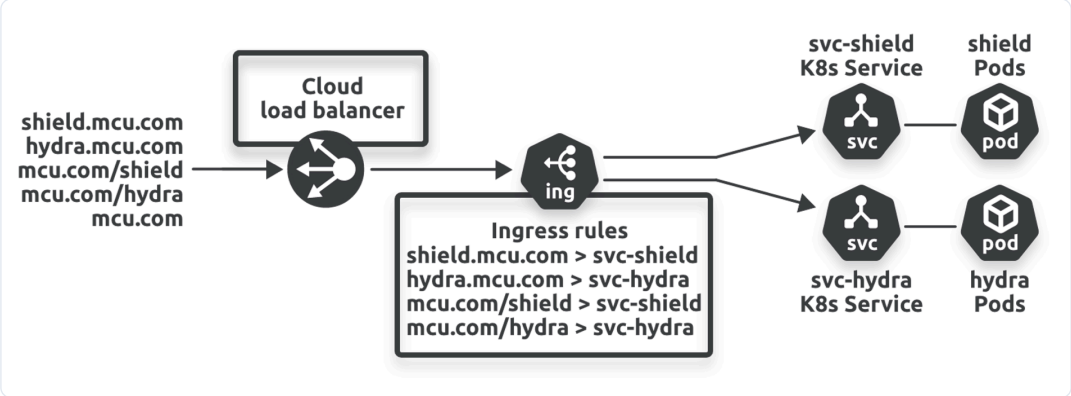
۲. کنترلر اینگرس (Ingress Controller)

نرم‌افزار معکوس‌پروکسی (Reverse Proxy) قدرتمندی است که باید به صورت جداگانه روی کلاستر نصب شود (مانند HAProxy، Traefik، Ingress-NGINX یا Envoy). کنترلر اینگرس به طور مداوم سرور API را رصد می‌کند؛ به محض این‌که شما یک شیء Ingress جدید می‌سازید یا ویرایش می‌کنید، کنترلر فایل کانفیگ داخلی پروکسی خود را بازنویسی کرده و بدون ریستارت ترافیک را طبق قوانین شما مسیریابی می‌کند.

چرا کنترلر اینگرس به صورت پیش‌فرض روی کلاستر نصب نیست؟

تیم توسعه هسته کوبرنتیز تصمیم گرفتند معماری را باز و ماژولار نگه دارند تا هر سازمانی متناسب با نیاز خود از کنترلر مورد نظرش استفاده کند. شرکت‌هایی که با زیرساخت NGINX آشنا هستند از NGINX Ingress،

کسانی که میکروسرویس‌های مدرن دارند از Traefik و شرکت‌های نیازمند سرویس‌مش از Envoy/Contour استفاده می‌کنند.



تصویر ۸.۲ - معماری جامع اینگرس: ترکیب لودبالانسر ورودی، کنترلر اینگرس و تفکیک مسیرها میان سرویس‌های مختلف

—

۳. الگوهای مسیریابی: Host-Based و Path-Based

اینگرس دو الگوی اصلی مسیریابی را در اختیار شما قرار می‌دهد:

۱. مسیریابی مبتنی بر نام هاست (Host-based Routing)

هدایت ترافیک بر اساس مقدار هدر `Host` در پروتکل HTTP:

- ترافیک `shield.mcu.com` به سمت سرویس `svc-shield`
- ترافیک `hydra.mcu.com` به سمت سرویس `svc-hydra`

۲. مسیریابی مبتنی بر مسیر URL (Path-based Routing)

هدایت ترافیک دامنه‌ای واحد به سمت میکروسرویس‌های تخصصی بر اساس URI:

- ترافیک `mcu.com/video` به سمت سرویس پردازش تصویر
- ترافیک `mcu.com/cart` به سمت سرویس سبد خرید

۳. سرویس پیش‌فرض (Default Backend)

اگر کاربری آدرس یا مسیری را وارد کند که با هیچ‌کدام از قوانین مانیفست شما همخوانی نداشته باشد، اینگرس درخواست را به یک سرویس پیش‌فرض (که معمولاً صفحه خطای زیبای 404 یا صفحه فرود اصلی است) هدایت می‌کند.

—

۴. پایان‌دهی امنیتی و مدیریت گواهینامه‌های SSL/TLS

یکی از درخشان‌ترین ویژگی‌های اینگرس، **پایان‌دهی لایه انتقال (TLS Termination)** است. در معماری سنتی، باید گواهینامه‌های SSL را درون تک‌تک کانتینرهای وب نصب و تمدید می‌کردید که یک کابوس مدیریتی بود. با اینگرس: ۱. گواهینامه امنیتی (`tls.crt` و `tls.key`) درون یک شیء محرمانه **Secret** در کوبرنتیز ذخیره می‌شود. ۲. کنترلر اینگرس گواهینامه را می‌خواند و ارتباط امن HTTPS را در همان ورودی کلاستر پایان می‌دهد (رمزگشایی می‌کند). ۳. ترافیک درون شبکه امن داخلی کلاستر میان اینگرس و پادها با سرعت بالا به صورت HTTP جریان می‌یابد.

```
spec:
  tls:
  - hosts:
    - shield.mcu.com
    secretName: shield-tls-cert     # کلید و سرتیفیکیت در این سکرت قرار دارد
```

> 💡 **ترفند و تجربه عملیاتی:**
>
> با استفاده از ابزار متن‌باز و استاندارد **cert-manager**، می‌توانید فرآیند دریافت، اعتبارسنجی و تمدید خودکار گواهینامه‌های رایگان Let’s Encrypt را به طور صددرصد اتوماتیک به اینگرس کلاستر خود متصل کنید!

—

۵. نگاهی به آینده: معرفی Gateway API در کوبرنتیز ۲۰۲۵

در حالی که Ingress ابزاری بسیار موفق بوده است، اما در سناریوهای بزرگ سازمانی محدودیت‌هایی داشت (وابستگی شدید به Annotationهای غلیظ و اختصاصی هر کنترلر). به همین دلیل در نسخه ۲۰۲۵، پروژه انقلابی **Gateway API** به عنوان استاندارد رسمی تکامل‌یافته جایگزین معرفی شده است. Gateway API مسئولیت‌ها را میان نقش‌های سازمانی تفکیک می‌کند:

- **مهندس زیرساخت (Infra Provider):** سخت‌افزار و کلاس گیت‌وی (`GatewayClass`) را تعریف می‌کند.
- **مدیر کلاستر (Cluster Operator):** دروازه ورودی و پورت‌ها (`Gateway`) را پیکربندی می‌کند.
- **توسعه‌دهنده نرم‌افزار (App Developer):** مسیرها و قوانین روتینگ (`HTTPRoute` یا `GRPCRoute`) را بدون نیاز به دسترسی به تنظیمات شبکه کلود مستقر می‌کند.

—

۶. کارگاه عملیاتی جامع: استقرار و تست اینگرس (Hands-on Lab)

وارد پوشه کدهای کتاب در بخش اینگرس شوید:

```
cd TKB/ingress
```

۱. استقرار سرویس‌های پشت‌صحنه

ابتدا پادها و سرویس‌های داخلی دو گروه SHIELD و HYDRA را با مانیفست‌های آماده دیپلوی می‌کنیم:

```
kubectl apply -f app.yml
```

بررسی صحت اجرا:

```
kubectl get deploy,svc
```

مشاهده خواهید کرد که دو دیپلویمنت و دو سرویس داخلی از نوع `ClusterIP` به نام‌های `svc-shield` و `svc-hydra` ایجاد شده‌اند.

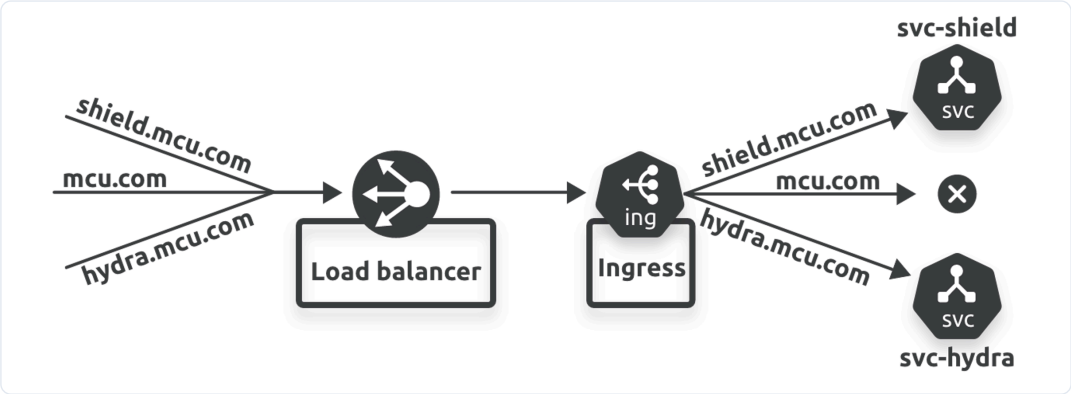
۲. کالبدشکافی مانیفست اینگرس (`ingress.yml`)

فایل مانیفست قوانین ورودی را بررسی می‌کنیم:

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: mcu-ingress
spec:
  ingressClassName: traefik     # یا nginx متناسب با کنترلر نصب‌شده
  rules:
  - host: shield.mcu.com
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: svc-shield
            port:
              number: 8080
  - host: hydra.mcu.com
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: svc-hydra
            port:
              number: 8080
```

در این فایل:

- فیلد `ingressClassName` : مشخص می‌کند کدام کنترلر باید این قوانین را اجرا کند.
- بخش `rules` : قوانین مبتنی بر هاست را تعریف کرده است؛ هاست `shield.mcu.com` به سمت `svc-shield` و هاست `hydra.mcu.com` به سمت `svc-hydra` روت می‌شود.



تصویر ۸.۴ - تفکیک جریان ترافیک ورودی به پادهای shield و hydra بر اساس هدر Host درخواست

مانیفست اینگرس را اعمال کنید:

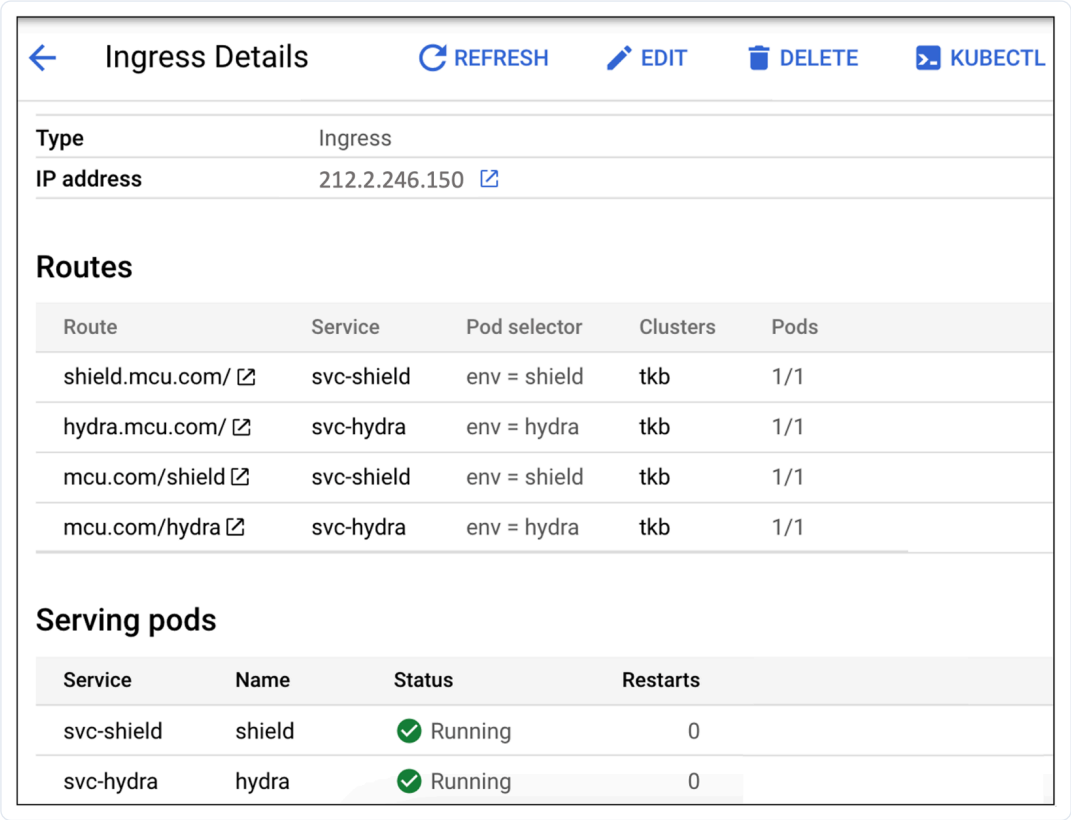
```
kubectl apply -f ingress.yml
```

مشخصات اینگرس ایجادشده را بررسی کنید:

```
kubectl get ingress mcu-ingress
```

خروجی آدرس IP یا نام هاست اختصاص‌یافته توسط لودبالانسر را در ستون `ADDRESS` نمایش می‌دهد:

```
NAME            CLASS      HOSTS                                ADDRESS
PORTS    AGE
mcu-ingress     traefik    shield.mcu.com,hydra.mcu.com         192.168.65.3      80
45s
```



تصویر ۸.۳ - تشکیل خودکار ساختار لودبالانسر شبکه و سلامت آدرس‌دهی در پشت صحنه اینگرس

۳. تست عملی مسیریابی بدون نیاز به خرید دامنه اینترنتی

از آنجا که دامنه‌های `shield.mcu.com` دامنه‌های تستی هستند و در اینترنت واقعی ثبت نشده‌اند، برای تست عملی نیازی به دستکاری فایل `hosts` ندارید! کافی است از قابلیت تزریق هدر با ابزار `curl` استفاده کنید:

تست درخواست به پورتال شیلد:

```
curl -H "Host: shield.mcu.com" localhost:80
```

خروجی صفحه خوش‌آمدگویی و مشخصات اختصاصی سرویس SHIELD را با موفقیت چاپ می‌کند.

تست درخواست به پورتال هایدرا:

```
curl -H "Host: hydra.mcu.com" localhost:80
```

خروجی دقیقاً محتوای برنامه HYDRA را برمی‌گرداند. تنها با **یک پورت ۸۰ و یک لودبالانسر ورودی**، کوبرنتیز ترافیک دو سرویس کاملاً مستقل را با بررسی هدر لایه ۷ تفکیک کرد!

—

۷. پاک‌سازی منابع (Clean up)

در پایان تمرینات، منابع ایجادشده را پاک کنید:

```
kubectl delete -f ingress.yml -f app.yml
```

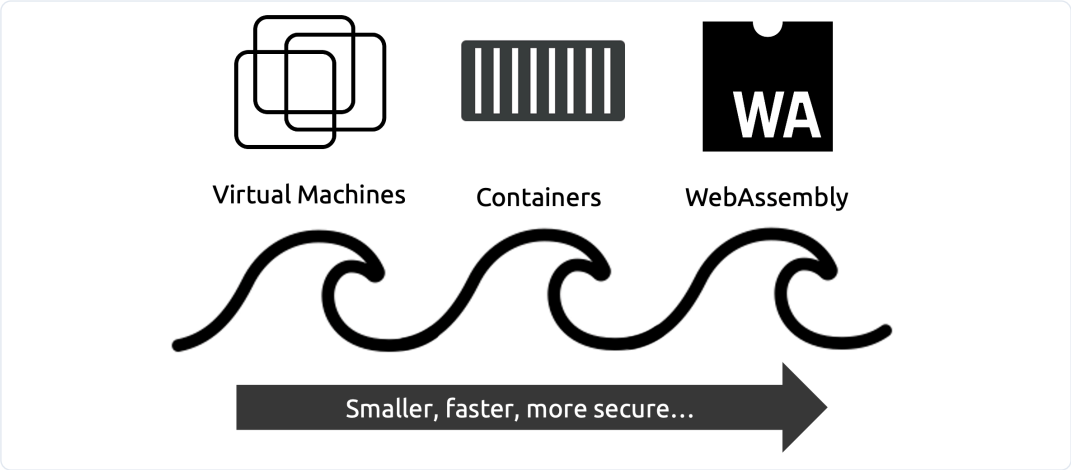
—

۸. جمع‌بندی فصل ۸

- سرویس‌های `LoadBalancer` در لایه ۴ کار می‌کنند و برای سامانه‌های چندمیکروسرویسی پرهزینه و ناکارآمد هستند.
- شیء Ingress یک دروازه ورودی لایه ۷ (Application Gateway) است که با یک IP واحد، امکان مسیریابی هوشمند بر پایه هاست (`Host-based`) و مسیر (`Path-based`) را فراهم می‌آورد.
- اینگرس از دو رکن تشکیل شده است: شیء اعلانی در API و کنترلر اجرایی فعال (نظیر NGINX، Traefik، Envoy).
- پایان‌دهی گواهینامه‌های SSL/TLS در ورودی اینگرس به کمک اشیای Secret انجام می‌شود و بار امنیتی را از دوش کانتینرها برمی‌دارد.
- استاندارد Gateway API نسل مدرن و آینده‌نگر اینگرس با تفکیک نقش‌های مهندسی زیرساخت، اپراتور و توسعه‌دهنده است.

فصل ۹: وب‌اسمبلی در کوبرنتیز (Wasm on Kubernetes)

دنیای زیرساخت و پردازش ابری همواره در حال تکامل است. طی دو دهه گذشته، صنعت نرم‌افزار سه موج عظیم از تحول در شیوه اجرای برنامه‌ها را تجربه کرده است: ۱. **موج اول - ماشین‌های مجازی (Virtual Machines):** مجازی‌سازی سخت‌افزار با هایپروایزرها؛ سنگین، نیازمند گیگابایت‌ها فضا و سیستم‌عامل مهمان مستقل با زمان راه‌اندازی در مقیاس دقیقه. ۲. **موج دوم - کانتینرها (Containers):** مجازی‌سازی در سطح سیستم‌عامل با اشتراک هسته لینوکس؛ چابک، در حجم مگابایت با زمان راه‌اندازی در مقیاس چند ثانیه. ۳. **موج سوم - وب‌اسمبلی (WebAssembly / Wasm):** اجرای مستقیم بایت‌کد در یک محیط ایزوله شنی (Sandbox) کاملاً بدون‌سیستم‌عامل؛ فوق‌العاده سبک در ابعاد کیلوبایت و با سرعت راه‌اندازی باورنکردنی در مقیاس **میکروثانیه**!



تصویر ۹.۱ - امواج سه‌گانه تکامل پردازش ابری: از ماشین‌های مجازی تا کانتینرها و انقلاب وب‌اسمبلی

در نسخه ۲۰۲۵ کتاب، کوبرنتیز فراتر از یک ارکستراتور صرف برای کانتینرها ظاهر شده و بستری همه‌منظوره برای اجرای همزمان کانتینرها و برنامه‌های مدرن وب‌اسمبلی در کنار یکدیگر فراهم ساخته است.

در این فصل مباحث زیر را به صورت کامل و تفسیری واکاوی خواهیم کرد: ۱. **مبانی وب‌اسمبلی و واسط سیستمی WASI** ۲. **چرا اجرای Wasm روی کوبرنتیز یک انقلاب محاسباتی است؟** ۳. **کالبدشکافی معماری ران‌تایم: containerd، شیم‌های runwasi و موتورهای WasmEdge و Spin** ۴. **مفهوم RuntimeClass در کوبرنتیز** ۵. **بسته‌بندی ماژول‌های Wasm در قالب آرتیفکت‌های استاندارد OCI** ۶. **کارگاه عملیاتی: پیکربندی کلاس اجرا، دیپلوی برنامه وب‌اسمبلی بدون‌کانتینر و تست کارایی**

—

۱. وب‌اسمبلی (Wasm) چیست و چگونه به سرورها راه یافت؟

وب‌اسمبلی در ابتدا به عنوان یک استاندارد جهانی توسط کنسرسیوم وب (W3C) معرفی شد تا امکان اجرای کدهای با کارایی بالا (نوشته‌شده با زبان‌هایی نظیر C، Rust++ یا Go) را با سرعت نزدیک به زبان ماشین (Near-Native Speed) درون مرورگرهای اینترنتی فراهم سازد.

اما طولی نکشید که جامعه مهندسی نرم‌افزار متوجه پتانسیل شگفت‌انگیز این فناوری در سمت سرور (Server-side) شد. با معرفی رابط سیستمی **WASI (WebAssembly System Interface)**، ماژول‌های وب‌اسمبلی توانستند بدون وابستگی به مرورگر، مستقیماً بر روی سیستم‌عامل‌های سرور اجرا شده و با فایل‌ها، شبکه و متغیرها با امنیت فوق‌العاده بالا تعامل کنند.

جمله تاریخی سالامون هایکز، خالق داکر

سالامون هایکز (Solomon Hykes) هم‌بنیان‌گذار داکر در سال ۲۰۱۹ توییتی منتشر کرد که کل صنعت آی‌تی را تکان داد: **«اگر فناوری‌های WASM و WASI در سال ۲۰۰۸ وجود داشتند، ما اصلاً نیازی به اختراع داکر پیدا نمی‌کردیم! وب‌اسمبلی در سمت سرور، آینده قطعی پردازش ابری است.»**

—

۲. مزایای راهبردی اجرای Wasm در کلاستر کوبرنتیز

اجرای کدهای Wasm در کنار کانتینرهای سنتی چهار برتری خارق‌العاده به همراه دارد:

۱. **استارت آنی در کسری از میلی‌ثانیه (Sub-millisecond Cold Start):** کانتینرها برای روشن شدن باید Namespaceها را بسازند، cgroup را مقداردهی کنند و پردازه سیستم‌عامل را لود نمایند که حداقل چند ثانیه طول می‌کشد. اما Wasm هیچ سیستم‌عاملی ندارد؛ در کمتر از ۱ میلی‌ثانیه بیدار می‌شود! این ویژگی رویای دیرینه سامانه‌های **سرورلس (Serverless)** و اسکیل از صفر (Scale-to-Zero) را با کارایی بی‌نظیر محقق می‌سازد.

۲. **حجم فوق‌العاده کوچک و تراکم حداکثری:** ایمیج‌های کانتینری حتی در مینیمال‌ترین حالت‌ها ده‌ها تا صدها مگابایت حجم دارند. اما یک برنامه کامل Wasm معمولاً کمتر از ۲ مگابایت (و گاهی چندصد کیلوبایت) است! شما می‌توانید هزاران نمونه Wasm را روی یک سرور بدون افت رم اجرا کنید.

۳. **قابلیت حمل مطلق میان پردازنده‌ها (True Portability):** کانتینرها وابسته به معماری پردازنده هستند؛ ایمیج ساخته‌شده برای پردازنده x86-64 اینتل روی تراشه‌های ARM64 (مانند پردازنده‌های Graviton آمازون یا تراشه‌های اپل) اجرا نمی‌شود مگر اینکه Multi-arch ساخته شود. اما بایت‌کد Wasm کاملاً مستقل از معماری است؛ یک‌بار کامپایل می‌کنید و بدون حتی یک خط تغییر روی هر سخت‌افزاری اجرا می‌شود!

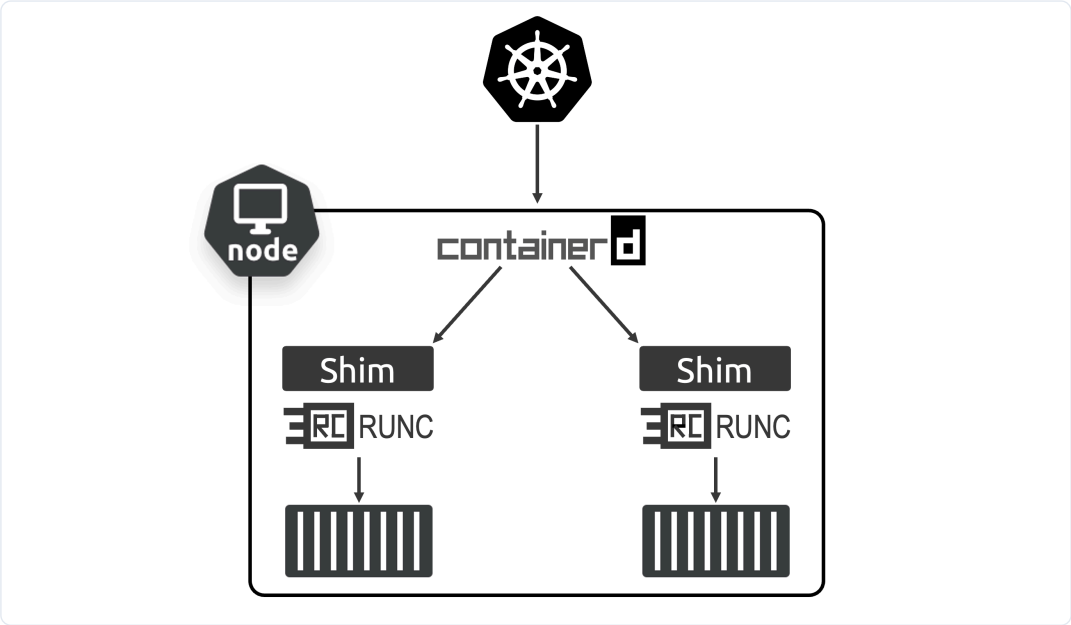
۴. **امنیت ایزوله بر پایه قابلیت‌ها (Capability-based Security):** ماژول‌های Wasm در یک محفظه حافظه کاملاً بسته اجرا می‌شوند. به صورت پیش‌فرض برنامه به هیچ بخشی از دیسک، اینترنت یا ساعت سیستم دسترسی ندارد مگر این‌که در زمان اجرا مجوز صریح آن صادر شود.

—

۳. کالبدشکافی معماری ران‌تایم: containerd و پروژه runwasi

شاید بپرسید: **«اگر Wasm کانتینر نیست، کوبرنتیز چگونه آن را اجرا می‌کند؟»**

برای درک این موضوع، ابتدا به معماری اجرای کانتینرها در تصویر ۹.۲ نگاه کنیم:

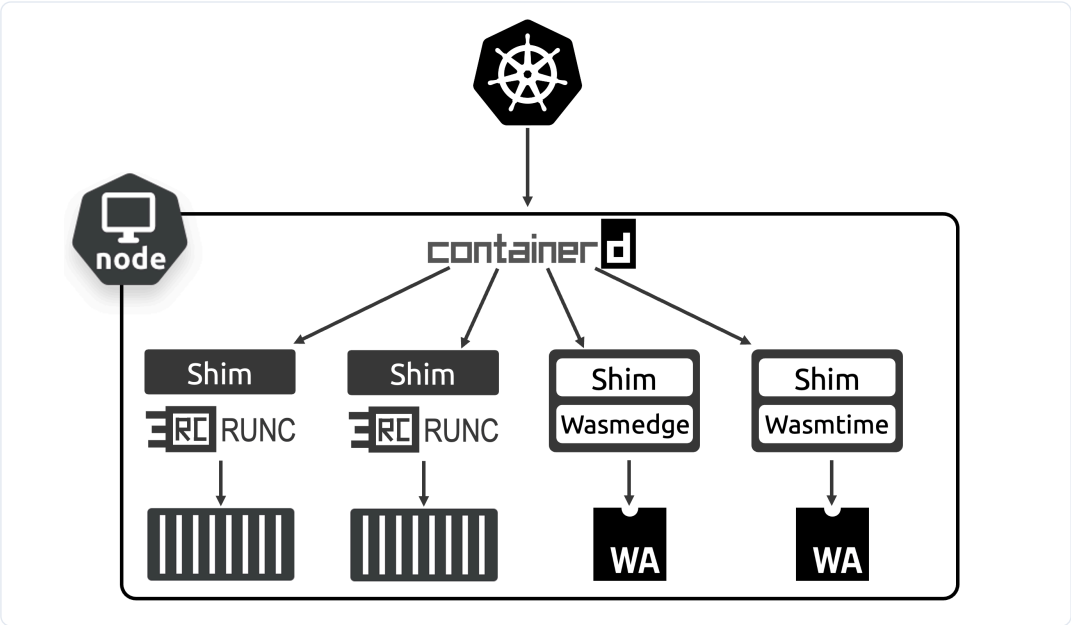


تصویر ۹.۲ - معماری استاندارد ران‌تایم کانتینرها: کیوبلت با containerd صحبت کرده و containerd از طریق runc کانتینرها را می‌سازد

در حالت عادی:

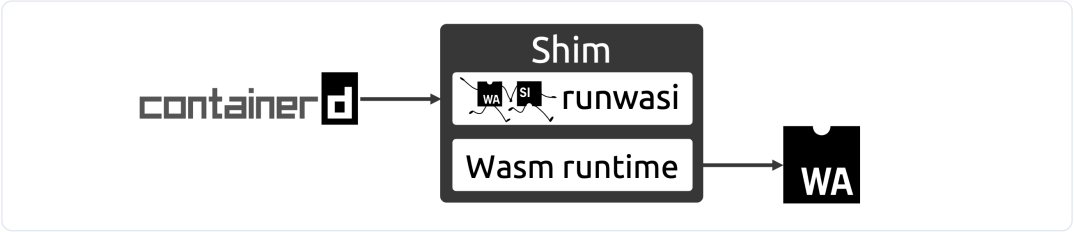
- کیوبلت دستور ساخت پاد را به موتور `containerd` می‌دهد.
- موتور containerd از ران‌تایم استاندارد لینوکسی به نام **runc** برای ساخت کانتینر استفاده می‌کند.

حالا به معجزه پروژه متن‌باز **runwasi** تحت بنیاد رایانش ابری (CNCF) در تصویر ۹.۳ نگاه کنید:



تصویر ۹.۳ - ادغام ران‌تایم وب‌اسمبلی با containerd: ابزار runwasi ترافیک را به جای runc به سمت موتورهای Wasm هدایت می‌کند

به کمک واسط شیم (Shim)، یک افزونه به containerd اضافه می‌شود که کارهای مربوط به Wasm را به جای runc، به یک موتور وب‌اسمبلی مانند **Spin**، **WasmEdge** یا **Wasmtime** تحویل می‌دهد!



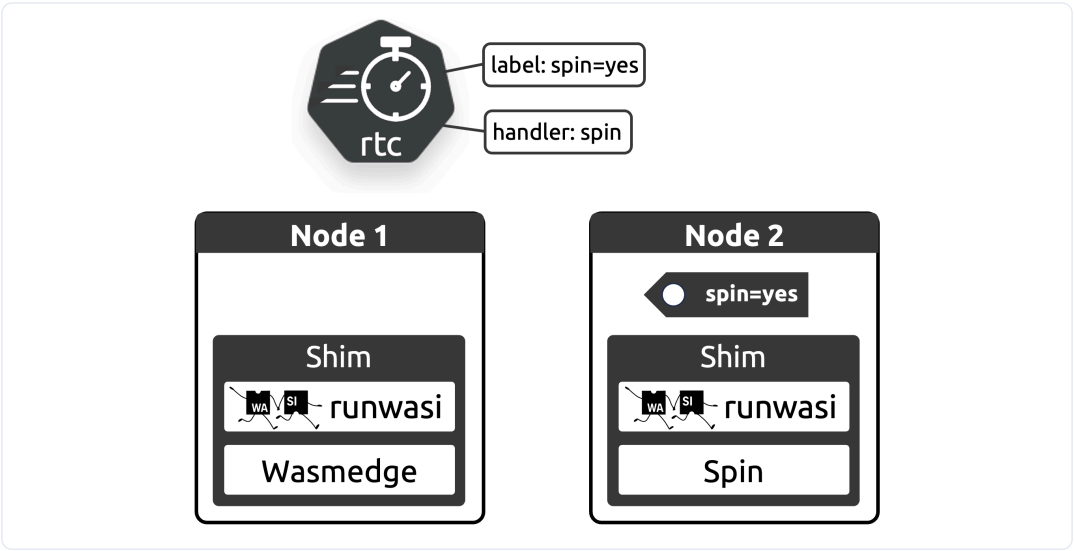
تصویر ۹.۴ - معماری شیم Wasm: یک باینری یکپارچه شامل کدهای واسط شیم و موتور ران‌تایم وب‌اسمبلی

تصویر ۹.۴ نشان می‌دهد که شیم وب‌اسمبلی یک باینری سبک واحد است که کدهای تعامل با containerd و هسته پردازشی وب‌اسمبلی را در خود جای داده است.

—

۴. کلاس‌های اجرا (RuntimeClass) در کوبرنتیز

کوبرنتیز برای این‌که بفهمد کدام پاد باید با کانتینر معمولی و کدام پاد با موتور Wasm اجرا شود، از آبجکت رسمی **RuntimeClass** استفاده می‌کند.



تصویر ۹.۵ - کلاستر چندموتوره: نودها مجهز به شیم‌های مختلف WasmEdge و Spin هستند و کوبرنتیز پادها را به نود متناظر هدایت می‌کند

شما یک شیء `RuntimeClass` در کلاستر تعریف می‌کنید:

```yaml
apiVersion: node.k8s.io/v1
kind: RuntimeClass
metadata:
  name: wasmtime-spin
handler: spin           # مستقر روی نود containerd-shim-spin-v2 اشاره به شیم
```

سپس در مانیفست پاد، تنها کافی است فیلد `runtimeClassName` را قرار دهید:

```
spec:
  runtimeClassName: wasmtime-spin    # به کوبرنتیز می‌گوید این پاد را به موتور Wasm بسپار!
  containers:
  - name: wasm-app
    image: nigelpoulton/wasm-spin:v1
```

> **📌 نکته مهم:**
>
> ماژول‌های Wasm دقیقاً مانند ایمیج‌های داکر درون رجیستری‌های استاندارد OCI (مانند Docker Hub یا Harbor) ذخیره و توزیع می‌شوند. بنابراین خط لوله CI/CD شما هیچ تغییری نخواهد کرد!

—

۵. کارگاه عملیاتی جامع: استقرار برنامه Wasm روی کوبرنتیز (Hands-on Lab)

وارد پوشه کدهای مربوط به Wasm شوید:

```
cd TKB/wasm
```

۱. ثبت کلاس اجرا (RuntimeClass)

ابتدا مانیفست `runtime.yml` را بررسی و اعمال می‌کنیم:

```
kubectl apply -f runtime.yml
```

بررسی صحت ایجاد کلاس اجرا:

```
kubectl get runtimeclass
```

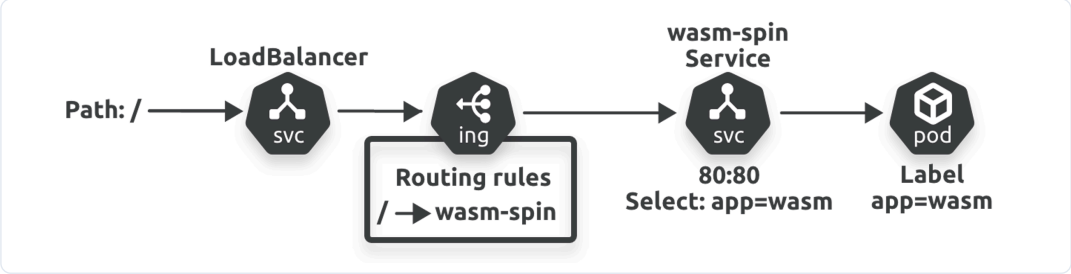
خروجی کلاس `wasmtime-spin` را با هندلر متناظر نمایش می‌دهد.

۲. استقرار برنامه بدون‌کانتینر Wasm

فایل `app.yml` یک دیپلویمنت و یک سرویس شبکه را مستقر می‌سازد. به فیلد `runtimeClassName` در مانیفست دقت کنید:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: wasm-deployment
```

```
spec:
  replicas: 3
  selector:
    matchLabels:
      app: wasm
  template:
    metadata:
      labels:
        app: wasm
    spec:
      runtimeClassName: wasmtime-spin
      containers:
      - name: wasm-ctr
        image: nigelpoulton/wasm-spin:v1
        ports:
        - containerPort: 80
---
apiVersion: v1
kind: Service
metadata:
  name: wasm-svc
spec:
  type: LoadBalancer
  ports:
  - port: 8080
    targetPort: 80
  selector:
    app: wasm
```



تصویر ۹.۶ - جریان ترافیک شبکه به سمت پادهای فوق‌سریع وب‌اسمبلی

مانیفست را در کلاستر اعمال کنید:

```
kubectl apply -f app.yml
```

پادها را بررسی کنید:

```
kubectl get pods -o wide
```

پادها در کسری از ثانیه بالا آمده و به وضعیت `Running` می‌رسند.

۳. تست دسترسی و ارزیابی عملکرد

با ابزار curl یا مرورگر، سرویس را آزمایش کنید:

```
curl localhost:8080
```

پاسخ وب‌سرور Wasm با سرعت و کارایی فوق‌العاده نمایش داده می‌شود:

```
Hello from WebAssembly running on Kubernetes!
```

شما با موفقیت یک برنامه کاملاً بدون‌سیستم‌عامل و مبتنی بر بایت‌کد Wasm را در دل یک کلاستر کوبرنتیز اجرا کردید!

—

۶. پاک‌سازی منابع (Clean up)

با دستور زیر منابع کارگاه را پاک کنید:

```
kubectl delete -f app.yml -f runtime.yml
```

—

۷. جمع‌بندی فصل ۹

- وب‌اسمبلی (Wasm) موج سوم پردازش ابری پس از ماشین‌های مجازی و کانتینرها است.
- به لطف رابط WASI، برنامه‌های Wasm در سمت سرور مستقل از سیستم‌عامل و مرورگر اجرا می‌شوند.
- مزایای Wasm در کوبرنتیز: راه‌اندازی زیر ۱ میلی‌ثانیه، مصرف رم در مقیاس کیلوبایت، قابلیت حمل کامل میان معماری‌های x86 و ARM، و امنیت مبتنی بر سندباکس.
- کوبرنتیز از طریق پروژه `runwasi` و واسط شیم موتور `containerd` ، بارهای کاری Wasm را از طریق ران‌تایم‌هایی مانند WasmEdge و Spin اجرا می‌کند.
- شیء `RuntimeClass` در کوبرنتیز مشخص می‌کند که آیا یک پاد باید توسط ران‌تایم سنتی `runc` اجرا شود یا شیم وب‌اسمبلی.

فصل ۱۰: کالبدشکافی عمیق سرویس دیسکاوری (Service Discovery Deep Dive)

در یک کلاستر کوبرنتیز پویا و ابری، پادها مدام ایجاد، حذف، جابه‌جا و اسکیل (Scale) می‌شوند و آدرس‌های IP آن‌ها لحظه‌به‌لحظه تغییر می‌کند. در این اقیانوس متلاطم، میکروسرویس‌ها چگونه بدون سردرگمی یکدیگر را در کسری از ثانیه پیدا می‌کنند؟

پاسخ این معمای بزرگ، در سیستم هوشمند و شگفت‌انگیز **کشف سرویس (Service Discovery)** و ستون فقرات آن یعنی سرور داخلی **CoreDNS** نهفته است.

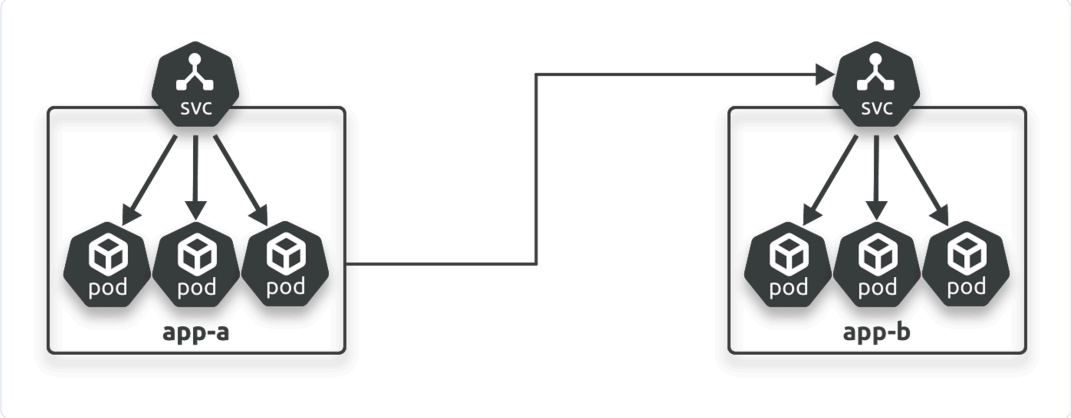
در این فصل، سازوکار داخلی کشف سرویس را با نگاهی عمیق و مهندسی کالبدشکافی خواهیم کرد: ۱. **نقشه راه کشف سرویس و چرخه چهارمرحله‌ای تفکیک نام** ۲. **معماری داخلی CoreDNS و رجیستری سرویس کلاستر** ۳. **جریان ثبت خودکار سرویس‌ها (Service Registration)** ۴. **کالبدشکافی حل نام در همان نیم‌اسپیس در برابر عبور از مرز نیم‌اسپیس‌ها با FQDN** ۵. **تحلیل فایل `/etc/resolv.conf` درون کانتینرها و چالش پارامتر `ndots:5`** ۶. **روش‌های پیشرفته خطایابی و عیب‌یابی شبکه DNS کلاستر** ۷. **کارگاه عملیاتی: برقراری ارتباط میان میکروسرویس‌های دو محیط dev و prod و تست با nslookup و curl**

—

۱. صورت مسئله: میکروسرویس‌ها چگونه یکدیگر را پیدا می‌کنند؟

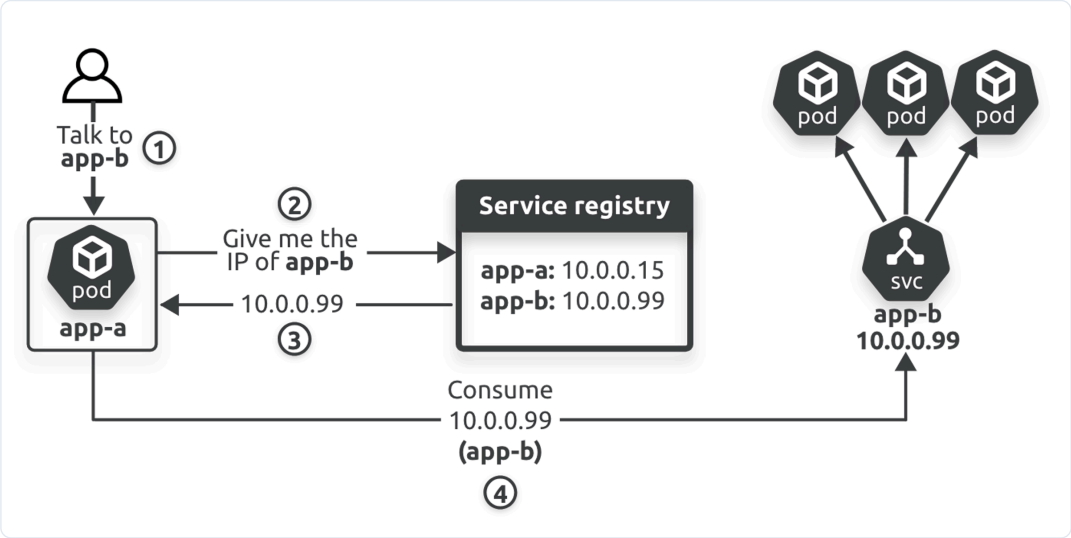
فرض کنید دو برنامه سازمانی به نام‌های `enterprise` و `cerritos` روی کلاستر دارید. برنامه `enterprise` باید برای پردازش صورت‌حساب‌ها درخواستی به برنامه `cerritos` بفرستد.

همان‌طور که در تصویر ۱۰.۱ مشاهده می‌کنید، هیچ برنامه‌ای نباید مستقیماً با IP ناپایدار پادها صحبت کند؛ بلکه برنامه مقصد پشت یک شیء پایدار Service قرار می‌گیرد.



تصویر ۱۰.۱ – ارتباط کلاینت‌ها با اپلیکیشن‌ها از طریق شیء پایدار Service به عنوان نقطه مطمئن شبکه

اما توسعه‌دهنده چگونه برنامه `enterprise` را تنظیم می‌کند تا سرویس `cerritos` را پیدا کند؟ تصویر ۱۰.۲ چرخه چهارمرحله‌ای تفکیک نام را از ابتدا تا انتها نشان می‌دهد:



تصویر ۱۰.۲ - مراحل چهارگانه تفکیک نام: از تنظیم نام سرویس در کد تا تبدیل نام به IP و مسیریابی نهایی به پاد

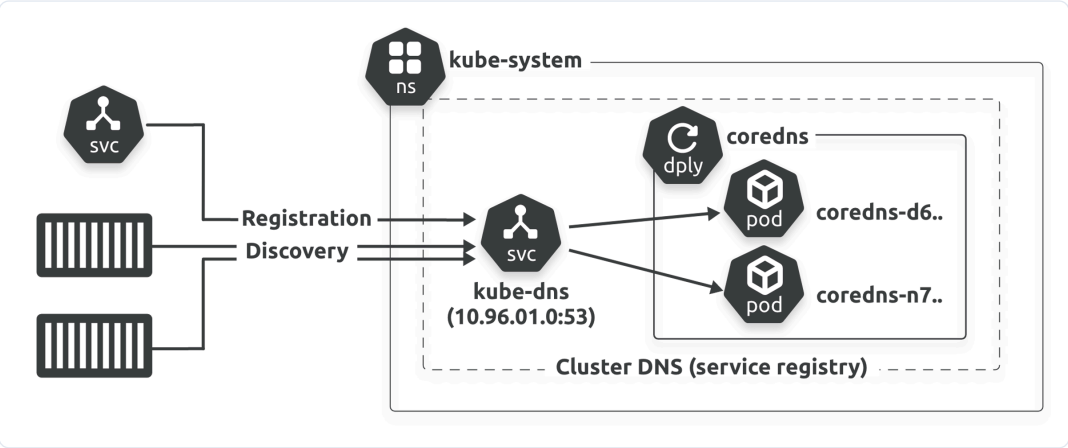
بیایید این چهار مرحله را موشکافی کنیم:

- **مرحله ۱ (تنها گام انسانی):** توسعه‌دهنده درون تنظیمات یا کد برنامه، نام سرویس مقصد (مثلاً `cer`) را می‌نویسد.
- **مرحله ۲ (تبدیل نام به IP):** هنگامی که برنامه سعی می‌کند به آدرس `cer` متصل شود، درخواست DNS به سرور CoreDNS کلاستر فرستاده شده و نام به آدرس مجازی سرویس (`ClusterIP`) ترجمه می‌شود.
- **مرحله ۳ (برقراری ارتباط):** برنامه کانتینری بسته شبکه را به سمت IP دریافت‌شده ارسال می‌کند.
- **مرحله ۴ (تحویل به پاد نهایی):** مکانیزم‌های شبکه (سرویس و EndpointSlice) ترافیک را به یکی از پادهای سالم پشت سرویس تحویل می‌دهند.

—

۲. معماری رجیستری کلاستر و سرور داخلی CoreDNS

رجیستری مرکزی سرویس‌های کلاستر توسط سامانه **CoreDNS** مدیریت می‌شود؛ یک سرور DNS ماژولار، انعطاف‌پذیر و برگزیده بنیاد CNCF که با زبان Go توسعه یافته است.



تصویر ۱۰.۳ - معماری سرویس رجیستری: دیپلویمنت CoreDNS در نیم‌اسپیس سیستمی و پادهایی که از آن برای ترجمه نام استفاده می‌کنند

معماری CoreDNS در کلاستر متشکل از سه رکن است: ۱. **دیپلویمنت** `coredns` (یا `kube-dns`): معمولاً شامل ۲ کپی از پادهای CoreDNS در نیم‌اسپیس `kube-system` برای تضمین در دسترس‌پذیری بالا (HA). ۲. **سرویس** `kube-dns`: یک سرویس داخلی پایدار با آدرس IP ثابت (معمولاً دهمین آدرس رنج سرویس‌ها، مانند `10.96.0.10` یا `10.43.0.10`) که به پادهای CoreDNS اشاره می‌کند. ۳. **پادهای کاربر:** تمام کانتینرهایی که در کلاستر متولد می‌شوند، این IP ثابت را به عنوان DNS سرور پیش‌فرض خود می‌شناسند.

دستورات زیر وضعیت CoreDNS را در کلاستر شما نشان می‌دهند:

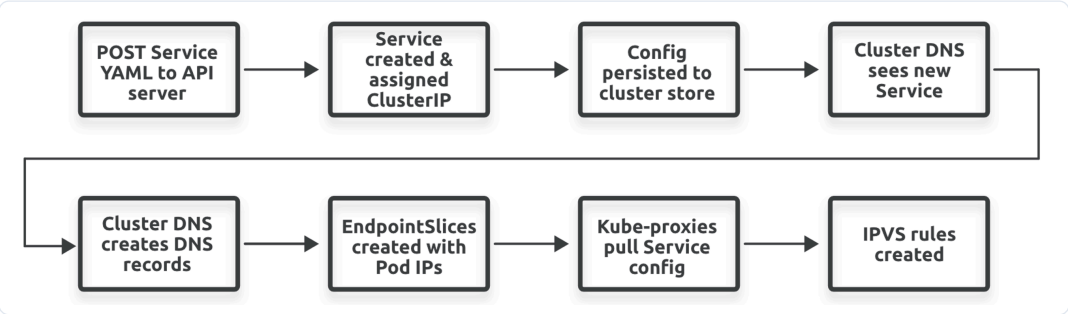
```
# مشاهده پادهای CoreDNS در حال اجرا
kubectl get pods -n kube-system -l k8s-app=kube-dns

# مشاهده سرویس داخلی شبکه DNS
kubectl get svc -n kube-system -l k8s-app=kube-dns
```

—

۳. جریان ثبت خودکار سرویس‌ها (Service Registration Flow)

یکی از جادوهای کوبرنتیز این است که هیچ نیازی به ثبت دستی رکوردهای DNS ندارید! فرآیند کاملاً خودکار است:



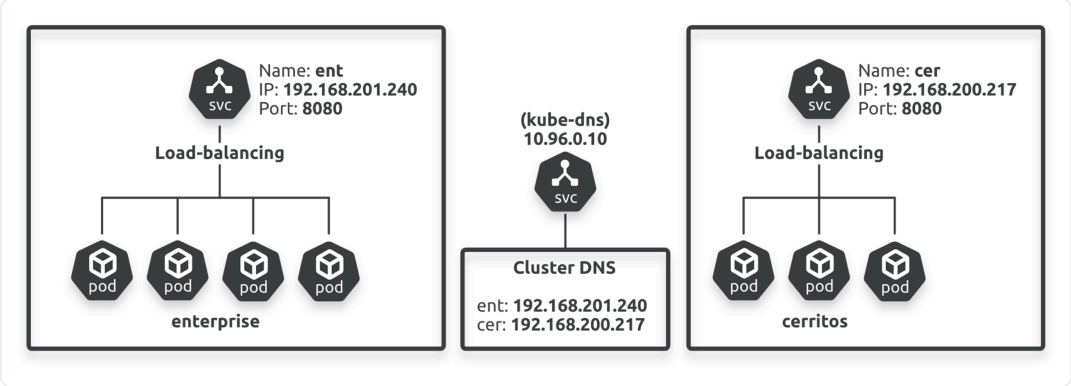
تصویر ۱۰.۴ - جریان ثبت خودکار سرویس در DNS کلاستر: از اعمال مانیفست تا به‌روزرسانی آنی حافظه CoreDNS

مراحل ثبت مطابق تصویر ۱۰.۴: ۱. شما مانیفست سرویس جدیدی (مثلاً به نام `cer`) را با دستور `kubectl apply` به API Server ارسال می‌کنید. ۲. کنترل پلین یک آدرس مجازی `ClusterIP` به این سرویس تخصیص می‌دهد و آن را در پایگاه داده etcd ذخیره می‌کند. ۳. پادهای CoreDNS که از طریق رابط API به رویدادهای کلاستر گوش می‌دهند، بلافاصله متوجه تولد سرویس جدید می‌شوند. ۴. CoreDNS بدون نیاز به هیچ‌گونه ریستارت، فوراً رکوردهای DNS (شامل رکوردهای A برای IPv4، رکوردهای AAAA برای IPv6 و رکوردهای SRV برای پورت‌ها) را به طور بلادرنگ در حافظه رم خود بارگذاری می‌کند.

—

۴. کشف سرویس در عمل: اسامی کوتاه در برابر FQDN

دو حالت ارتباطی در کلاستر وجود دارد:



تصویر ۱۰.۵ - دو برنامه سازمانی enterprise و cerritos که نیازمند تبادل داده با یکدیگر هستند

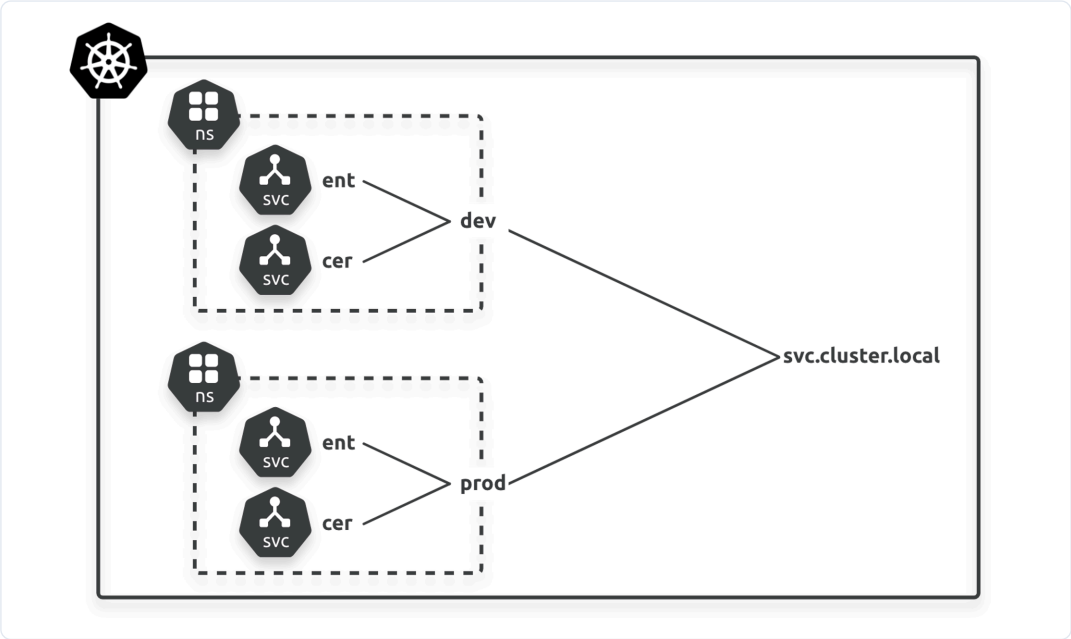
حالت اول: هر دو برنامه در یک نیم‌اسپیس مشترک هستند

اگر پاد `enterprise` و سرویس `cer` هر دو در نیم‌اسپیس `default` باشند، پاد می‌تواند به سادگی از نام کوتاه سرویس استفاده کند:

```
curl http://cer:8080
```

حالت دوم: برنامه‌ها در نیم‌اسپیس‌های متفاوتی قرار دارند

نام اشیا در کوبرنتیز در سطح نیم‌اسپیس یکتاست، اما در سراسر کلاستر می‌تواند تکرار شود! به عنوان مثال ممکن است در نیم‌اسپیس `dev` یک سرویس `cer` و در نیم‌اسپیس `prod` نیز یک سرویس دیگر با همان نام `cer` داشته باشید (تصویر ۱۰.۷).

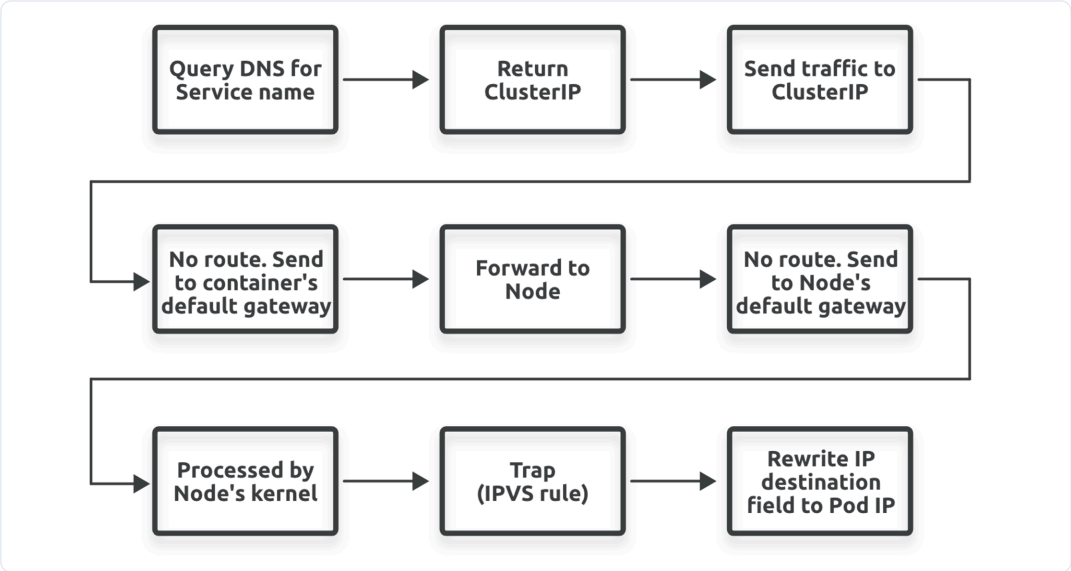


تصویر ۱۰.۷ - وجود پیکربندی‌ها و سرویس‌های هم‌نام در دو نیم‌اسپیس مجزای dev و prod

در این شرایط، برای جلوگیری از اشتباه، برنامه‌ها از **نام کامل دامنه (Fully Qualified Domain Name یا FQDN)** استفاده می‌کنند:

`<service-name>.<namespace>.svc.cluster.local`

- پاد اگر بخواهد به نسخه توسعه وصل شود: `cer.dev.svc.cluster.local`
- پاد اگر بخواهد به نسخه عملیاتی وصل شود: `cer.prod.svc.cluster.local`



تصویر ۱۰.۶ - فلوچارت حل نام و کشف سرویس میان پاد و سرور CoreDNS

—

۵. کالبدشکافی `/etc/resolv.conf` و چالش فنی پارامتر `ndots:5`

چگونه کانتینر لینوکسی متوجه می‌شود که با تایپ نام کوتاه `cer` ، باید به کدام دامنه متصل شود؟ پاسخ درون فایل `/etc/resolv.conf` هر پاد قرار دارد که در زمان ساخت توسط کیوبلت تزریق می‌شود:

```
nameserver 10.96.0.10
search default.svc.cluster.local svc.cluster.local cluster.local
options ndots:5
```

- سطر `nameserver` : آدرس سرویس CoreDNS کلاستر را مشخص می‌کند.
- سطر `search` : فهرست دامنه‌های جستجو را تعیین می‌کند. وقتی پاد کلمه `cer` را جستجو می‌کند، سیستم‌عامل به نوبت این پسوندها را به انتهای آن می‌چسباند تا به جواب برسد: ۱. ابتدا `cer.default.svc.cluster.local` را تست می‌کند (که با موفقیت پیدا می‌شود!).

معضل عملکردی ndots:5 و راهکار بهینه‌سازی

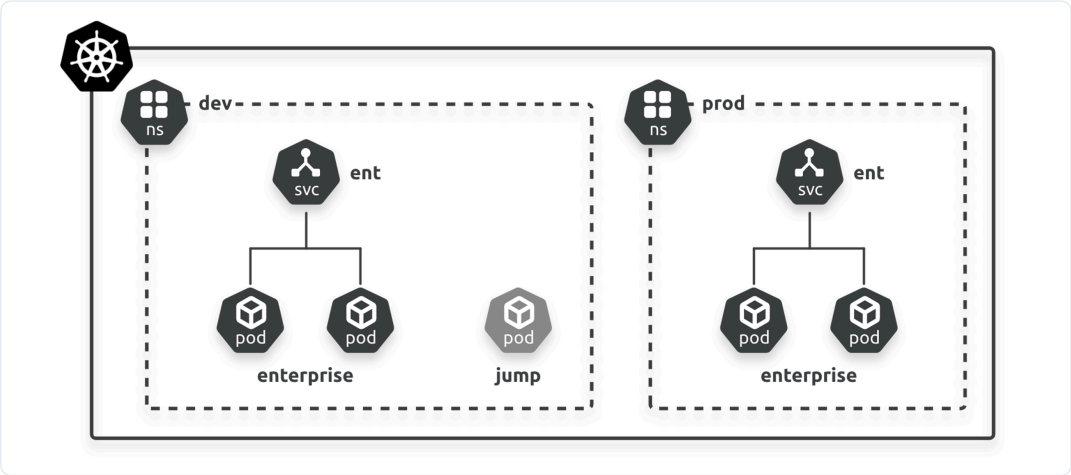
گزینه `ndots:5` به سیستم‌عامل می‌گوید: **«اگر نام دامنه‌ای کمتر از ۵ نقطه داشت، ابتدا پسوندهای محلی کلاستر را تست کن و اگر پیدا نشد به سراغ اینترنت عمومی برو!»** این تنظیم برای سرویس‌های داخلی عالی است؛ اما اگر برنامه شما بخواهد به `google.com` (که فقط ۱ نقطه دارد) وصل شود، لینوکس ابتدا دامنه‌های مضحک زیر را از CoreDNS می‌پرسد: ۱. `google.com.default.svc.cluster.local` ۲. `google.com.svc.cluster.local` ۳. `google.com.cluster.local` و پس از شکست هر سه، تازه به سراغ اینترنت می‌رود! این رفتار می‌تواند بار ترافیک غیرضروری سنگینی به CoreDNS تحمیل کند. راهکار مهندسی: در پادهایی که ترافیک خارجی زیادی دارند، یک نقطه به انتهای آدرس اضافه کنید (`google.com.`) تا به عنوان FQDN مطلق شناخته شود و مستقیماً به اینترنت ارسال گردد.

—

۶. کارگاه عملیاتی جامع: خطایابی و کشف سرویس چندنیم‌اسپیسی (Hands-on Lab)

وارد پوشه کدهای کتاب شوید:

```
cd TKB/service-discovery
```



تصویر ۱۰.۸ - معماری سناریوی کارگاه: دو نیم‌اسپیس dev و prod، دو سرویس هم‌نام و یک پاد پرش (Jump Pod)

فایل `sd-example.yml` یک معماری کامل شامل دو نیم‌اسپیس `dev` و `prod`، دو برنامه `enterprise` و `cerritos` در هر دو محیط، و یک پاد تستی برای عیب‌یابی به نام `jump` را مستقر می‌کند (تصویر ۱۰.۸).

مانیفست را اعمال کنید:

```
kubectl apply -f sd-example.yml
```

صحت استقرار را در هر دو نیم‌اسپیس چک کنید:

```
kubectl get pods -n dev
kubectl get pods -n prod
```

۱. ورود به پاد تستی و بررسی `/etc/resolv.conf`

با نشست تعاملی وارد پاد `jump` شوید:

```
kubectl exec -it jump -n dev -- sh
```

محتوای فایل DNS را چاپ کنید:

```
cat /etc/resolv.conf
```

خواهید دید که دامین جستجوی اول `dev.svc.cluster.local` تنظیم شده است.

۲. تست کشف سرویس در همان نیم‌اسپیس با نام کوتاه

در همان خط فرمان پاد، نام کوتاه سرویس `cer` را صدا بزنید:

```
curl cer:8080
```

پاسخ می‌دهد:

```
Hello from cerritos in the DEV namespace!
```

سرویس داخلی به سرعت توسط CoreDNS پیدا شد و ارتباط برقرار گردید.

۳. تست کشف سرویس در نیم‌اسپیس دیگر با FQDN

حالا از همان پاد `jump` که در نیم‌اسپیس `dev` است، می‌خواهیم به سرویس `cerritos` مستقر در نیم‌اسپیس `prod` وصل شویم. اگر فقط تایپ کنیم `curl cer:8080` ، طبیعتاً به نسخه dev وصل می‌شود! اما با استفاده از نام کامل FQDN:

```
curl cer.prod.svc.cluster.local:8080
```

پاسخ شگفت‌انگیز است:

```
Hello from cerritos in the PROD namespace!
```

CoreDNS به زیبایی متوجه شد که مقصد نیم‌اسپیس `prod` است و درخواست را از مرز نیم‌اسپیس عبور داد.

۴. استعلام رکوردهای DNS با `nslookup`

برای مشاهده مستقیم آی‌پی‌های حل‌شده:

```
nslookup cer.prod.svc.cluster.local
```

خروجی آدرس سرور CoreDNS و آدرس `ClusterIP` سرویس مقصد را دقیقاً به شما نشان می‌دهد:

```
Server:     10.96.0.10
Address 1: 10.96.0.10 kube-dns.kube-system.svc.cluster.local

Name:       cer.prod.svc.cluster.local
Address 1: 10.10.85.142 cer.prod.svc.cluster.local
```

با دستور `exit` از پاد خارج شوید.

—

۷. پاک‌سازی منابع (Clean up)

با حذف مانیفست سناریو، تمامی پادها، سرویس‌ها و هر دو نیم‌اسپیس به صورت کامل پاک خواهند شد:

```
kubectl delete -f sd-example.yml
```

—

۸. جمع‌بندی فصل ۱۰

- کشف سرویس فرآیند پویای تبدیل نام منطقی سرویس‌ها به آدرس‌های IP و اتصال به پادهای فعال است.
- ابزار CoreDNS به عنوان سامانه رجیستری معتبر (Authoritative DNS) کلاستر عمل می‌کند.
- پادهای CoreDNS به طور پیوسته سرور API را رصد کرده و با تولد هر سرویس جدید، رکوردهای DNS آن را در لحظه ثبت می‌کنند.
- ارتباطات درون یک نیم‌اسپیس با نام کوتاه سرویس (Short Name) انجام می‌شود.
- ارتباطات میان نیم‌اسپیس‌های مختلف نیازمند نام کامل دامنه یعنی `<service>.<namespace>.svc.cluster.local` است.
- فایل `/etc/resolv.conf` پادها توسط کیوبلت با آدرس سرویس `kube-dns` و الگوهای جستجوی نیم‌اسپیسی تنظیم می‌شود.

فصل ۱۱: سیستم ذخیره‌سازی در کوبرنتیز (Kubernetes Storage)

کانتینرها و پادها موجوداتی ذاتا زودگذر (Ephemeral) هستند؛ اگر پادی کرش کند یا جابه‌جا شود، دیسک داخلی و لایه نوشتن ران‌تایم آن نیست و نابود می‌شود. اما در دنیای واقعی، دارایی اصلی هر کسب‌وکار داده‌های آن است: پایگاه‌های داده، اسناد بارگذاری‌شده مشتریان، تصاویر و فایل‌های لاگ باید حتی پس از مرگ صدها پاد و نود، دست‌نخورده و پایدار باقی بمانند.

کوبرنتیز برای حل این مسئله، یکی از پیشرفته‌ترین و بالغ‌ترین زیرسیستم‌های ذخیره‌سازی در صنعت ابر را طراحی کرده است: زیرسیستم **Persistent Volume** بر پایه استاندارد صنعتی **CSI (Container Storage Interface)**.

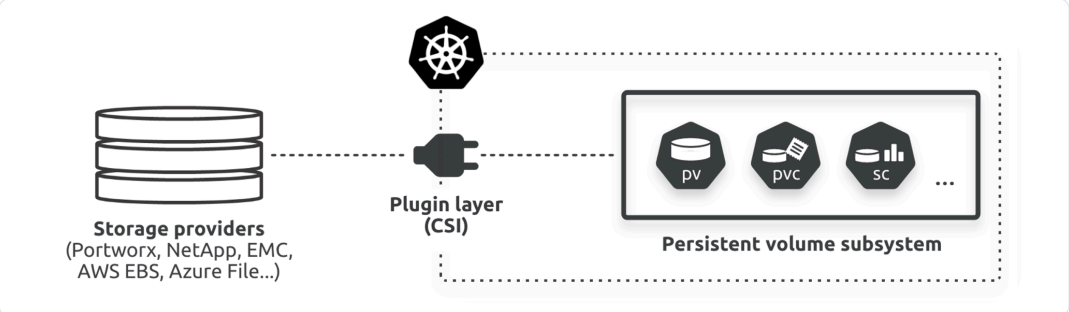
در این فصل مباحث زیر را به صورت کامل و تفسیری واکاوی خواهیم کرد: ۱. **معماری کلان ذخیره‌سازی و انقلاب رابط استاندارد CSI**. ۲. **مثلث ذخیره‌سازی کوبرنتیز: PV، PVC و StorageClass**. ۳. **حالت‌های دسترسی به دیسک (Access Modes: RWO، ROX، RWX، RWOP)**. ۴. **فرآیند تخصیص پویای دیسک (Dynamic Volume Provisioning)**. ۵. **سیاست‌های بازپس‌گیری دیسک (Reclaim Policies: Delete در برابر Retain)**. ۶. **کارگاه عملیاتی گام‌به‌گام: ایجاد ادعای دیسک، اتصال خودکار به کلود، ذخیره داده و اثبات بقای داده‌ها پس از نابودی پاد**

—

۱. معماری کلان ذخیره‌سازی و انقلاب رابط CSI

در نسخه‌های اولیه کوبرنتیز، کدهای مربوط به اتصال به استوریج‌های مختلف (مانند دیسک‌های AWS، گوگل کلود، Ceph و NFS) مستقیماً درون سورس‌کد اصلی کوبرنتیز نوشته می‌شد (به این رویکرد In-Tree می‌گفتند). این کار دو فاجعه به همراه داشت: ۱. اگر شرکتی می‌خواست استوریج جدیدی به کلاستر متصل کند، باید کد خود را به مخزن رسمی کوبرنتیز ارسال می‌کرد و منتظر ریلیز رسمی کوبرنتیز می‌ماند! ۲. هرگونه باگ در درایور یک استوریج، پایداری کل سرورهای کنترل پلین کوبرنتیز را به خطر می‌انداخت.

برای حل این معضل، استاندارد جهانی **CSI (Container Storage Interface)** به وجود آمد (تصویر ۱۱.۱).



تصویر ۱۱.۱ - معماری کلان سیستم ذخیره‌سازی کوبرنتیز: استوریج‌ها از طریق پلاگین‌های ماژولار CSI به زیرسیستم پایدار کوبرنتیز متصل می‌شوند

به لطف CSI:

- کدهای ذخیره‌سازی کاملاً از هسته کوبرنتیز خارج شدند (Out-of-Tree).

• تولیدکنندگان تجهیزات ذخیره‌سازی (مانند NetApp، EMC، Portworx یا کلودهای ابری) یک پلاگین استاندارد CSI می‌نویسند و در کلاستر نصب می‌کنند بدون این‌که نیازی به تغییر حتی یک خط از کدهای کوبرنتیز باشد.

—

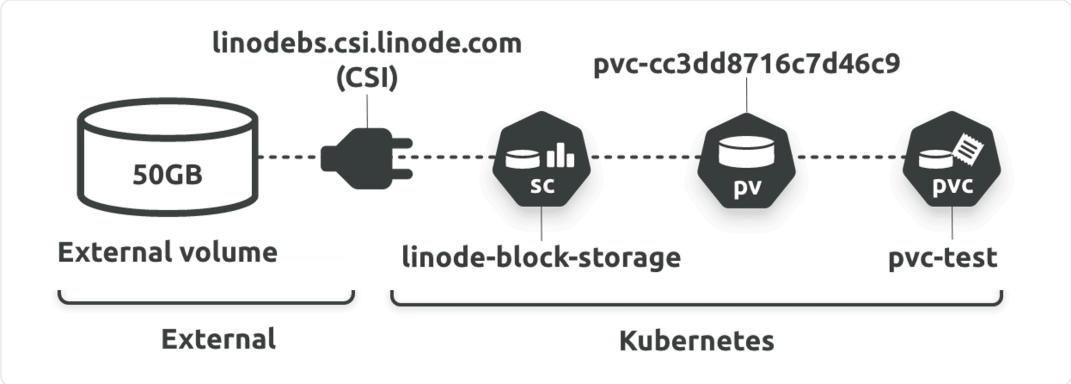
۲. مثلث ذخیره‌سازی: تفکیک وظایف با PV، PVC و StorageClass

کوبرنتیز به زیبایی مرز میان نیازمندی‌های توسعه‌دهنده نرم‌افزار و اختیارات مدیر زیرساخت را با سه شیء تفکیک کرده است:

۱. **دیسک پایدار (PersistentVolume - PV):** نمایانگر یک قطعه حافظه یا دیسک فیزیکی/ابری واقعی در کلاستر است (مثلاً یک دیسک ۱۰۰ گیگابایتی SSD در AWS EBS یا سان‌استوریج دیتاسنتر). شیء PV یک منبع در سطح کل کلاستر (Cluster-scoped) است و معمولاً توسط ادمین زیرساخت یا به صورت خودکار ایجاد می‌شود.

۲. **ادعای دیسک (PersistentVolumeClaim - PVC):** درخواست یک توسعه‌دهنده برای دریافت دیسک است! توسعه‌دهنده بدون این‌که بداند دیسک فیزیکی در کدام رک یا کدام سرویس ابری قرار دارد، یک فیش درخواست (Claim) پر می‌کند: **«من به یک دیسک ۲۰ گیگابایتی با دسترسی خواندن/نوشتن سریع نیاز دارم.»** شیء PVC یک منبع وابسته به نیم‌اسپیس (Namespaced) است.

۳. **کلاس ذخیره‌سازی (StorageClass - SC):** موتور خودکارسازی و کارخانه تولید دیسک است! به جای این‌که ادمین بنشیند و دستی دیسک بسازد، کلاس ذخیره‌سازی نوع پرووِیژنر (پلاگین CSI)، نوع دیسک (SSD فوق‌سریع یا HDD ارزان)، فایل‌سیستم (`ext4` یا `xfs`) و سیاست بازپس‌گیری را تعریف می‌کند تا به محض ارسال یک PVC، دیسک واقعی به طور خودکار متولد شود.



تصویر ۱۱.۳ - چگونگی پیوند اجزای سیستم ذخیره‌سازی: پاد به PVC متصل می‌شود، PVC از طریق StorageClass به PV مقید شده و PV بر روی دیسک فیزیکی سوار است

—

۳. حالت‌های دسترسی به دیسک (Access Modes)

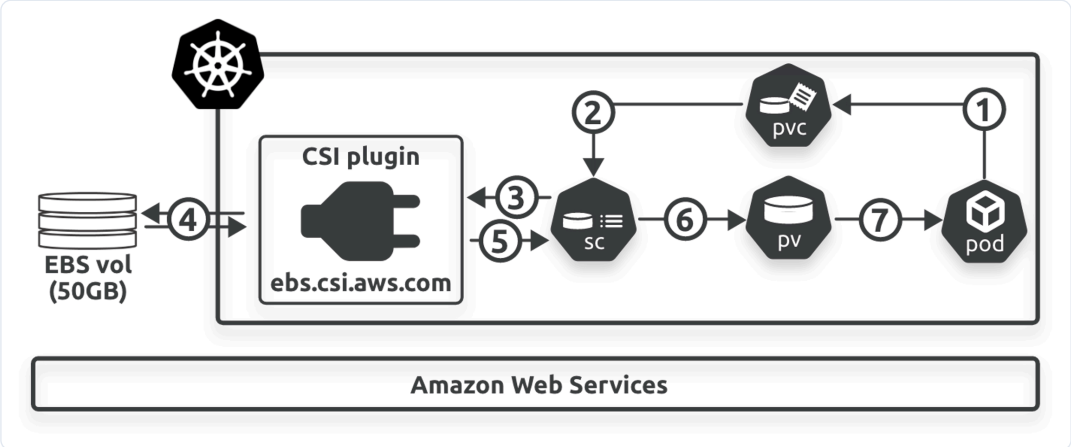
هنگام درخواست دیسک، باید مشخص کنید که پادها چگونه قرار است به این دیسک متصل شوند:

- `ReadWriteOnce (RWO)` : دیسک می‌تواند توسط **تنها یک نود کلاستر** با قابلیت خواندن و نوشتن ماونت شود. دیسک‌های بلوکی ابری (Block Storage) مانند دیسک‌های ابری AWS EBS، Linode و Azure Disk) به دلیل ماهیت سخت‌افزاری خود تنها از حالت RWO پشتیبانی می‌کنند.
- `ReadOnlyMany (ROX)` : دیسک می‌تواند توسط **چندین نود به طور همزمان** ماونت شود، اما فقط به صورت خواندنی (Read-Only). مناسب توزیع داده‌های کاتالوگ یا مدل‌های هوش مصنوعی.
- `ReadWriteMany (RWX)` : دیسک می‌تواند توسط **چندین نود به طور همزمان** با قابلیت خواندن و نوشتن متصل شود. نیازمند سیستم فایل‌های شبکه‌ای نظیر CephFS، NFS یا GlusterFS است.
- `ReadWriteOncePod (RWOP)` : (ویژگی مدرن) تضمین می‌کند که دیسک در کل کلاستر فقط و فقط توسط **یک پاد منفرد** باز شود (حتی پاد دیگری روی همان نود هم حق دسترسی ندارد).

—

۴. جریان تخصیص پویای دیسک (Dynamic Provisioning)

در روزگاران قدیم، تخصیص دیسک ایستا (Static) بود؛ ادمین باید از قبل دیسک می‌ساخت و اگر دیسک خالی نبود، پاد معلق می‌ماند. امروزه در معماری ابری‌بومی، تخصیص دیسک صددرصد پویا (Dynamic) است.



تصویر ۱۱.۲ - جریان خودکار تخصیص پویا: از ارسال PVC توسط کاربر تا دستور به درایور CSI، ساخت دیسک ابری و باید شدن نهایی

مراحل تخصیص پویا مطابق تصویر ۱۱.۲: ۱. توسعه‌دهنده مانیفست PVC را ارسال می‌کند و در آن نام `StorageClass` مورد نظر خود را ذکر می‌کند. ۲. کنترلر کوبرنتیز متوجه درخواست شده و درایور CSI مربوط به آن کلاس ذخیره‌سازی را صدا می‌زند. ۳. درایور CSI با API ارائه‌دهنده ابری صحبت کرده و یک ولوم جدید (مثلاً یک دیسک ۲۰ گیگابایتی) ایجاد می‌کند. ۴. کوبرنتیز به طور خودکار یک شیء PV متناظر در کلاستر ساخته و آن را به PVC متصل (Bound) می‌کند. ۵. پاد بالا آمده و دیسک به عنوان یک پوشه محلی درون کانتینر ماونت می‌شود.

سیاست‌های بازپس‌گیری (Reclaim Policies)

تعیین می‌کند که وقتی کاربر PVC را حذف کرد، چه بر سر دیسک فیزیکی و داده‌ها بیاید:

- `Delete` : (حالت معمول) با حذف PVC، دیسک فیزیکی در کلود نیز فوراً پاک شده و هزینه‌اش قطع می‌شود.
- `Retain` : با حذف PVC، دیسک فیزیکی و داده‌های حیاتی آن حفظ می‌شوند تا ادمین به صورت دستی داده‌ها را استخراج یا آرشیو کند.

—

۵. کارگاه عملیاتی جامع: ذخیره‌سازی پایدار در عمل (Hands-on Lab)

وارد پوشه کدهای کتاب در بخش استوریج شوید:

```
cd TKB/storage
```

۱. بررسی کلاس‌های ذخیره‌سازی کلاستر

دستور زیر را برای مشاهده کلاس‌های ذخیره‌سازی فعال اجرا کنید:

```
kubectl get sc
```

کلاستر شما حداقل یک کلاس ذخیره‌سازی با علامت `(default)` دارد که به عنوان پیش‌فرض عمل می‌کند.

۲. کالبدشکافی مانیفست ادعای دیسک (`pvc.yml`)

فایل مانیفست را باز کنید:

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: pvc-test
spec:
  accessModes:
  - ReadWriteOnce
  resources:
    requests:
      storage: 10Gi
```

در این فایل درخواست ۱۰ گیگابایت دیسک با دسترسی `ReadWriteOnce` شده است. مانیفست را اعمال کنید:

```
kubectl apply -f pvc.yml
```

وضعیت PVC را بررسی کنید:

```
kubectl get pvc pvc-test
```

خواهید دید که در کمتر از چند ثانیه، وضعیت به `Bound` تغییر کرده و در ستون `VOLUME` نام دیسک خودکار تولیدشده (`pvc-xxxx`) درج شده است. اگر دستور `kubectl get pv` را نیز بزنید، دیسک واقعی تولیدشده را در سطح کلاستر مشاهده خواهید کرد!

۳. اتصال دیسک به پاد و اثبات بقای داده‌ها

فایل `pod.yml` را بررسی می‌کنیم. این پاد دیسک درخواست‌شده در PVC را در مسیر `/data` کانتینر ماونت می‌کند:

```
apiVersion: v1
kind: Pod
metadata:
  name: volpod
spec:
  volumes:
  - name: data
    persistentVolumeClaim:
      claimName: pvc-test
  containers:
  - name: app
    image: nigelpoulton/k8sbook:1.0
    volumeMounts:
    - name: data
      mountPath: /data
```

پاد را مستقر کنید:

```
kubectl apply -f pod.yml
```

پس از رسیدن به وضعیت `Running` ، وارد شل کانتینر شوید و فایلی در مسیر دیسک پایدار بنویسید:

```
kubectl exec -it volpod -- sh
# اکنون درون کانتینر هستید
echo "Kubernetes Storage Works!" > /data/test.txt
cat /data/test.txt
exit
```

اکنون زمان آزمون است؛ پاد را به طور کامل نابود کنید:

```
kubectl delete pod volpod
```

پاد مرده است و کانتینر آن نابود شد. اکنون همان مانیفست پاد را دوباره ایجاد کنید:

```
kubectl apply -f pod.yml
```

وارد پاد تازه متولدشده شوید و محتوای فایل را بخوانید:

```
kubectl exec -it volpod -- cat /data/test.txt
```

خروجی شگفت‌انگیز است:

```
Kubernetes Storage Works!
```

داده‌های شما زنده مانده‌اند! دیسک با همان محتویات به پاد جدید متصل شد.

—

۶. پاک‌سازی منابع (Clean up)

با دستور زیر پاد و PVC را پاک کنید:

```
kubectl delete -f pod.yml -f pvc.yml
```

—

۷. جمع‌بندی فصل ۱۱

- پادها و کانتینرها زودگذرند و برای ذخیره اطلاعات ماندگار باید از زیرسیستم Persistent Volume استفاده کرد.
- واسط CSI استاندارد مدرن و برون‌کلاستری اتصال انواع ارائه‌دهندگان استوریج به کوبرنتیز است.
- مثلث ذخیره‌سازی از PV (دیسک فیزیکی در سطح کلاستر)، PVC (ادعای دیسک در سطح نیم‌اسپیس) و StorageClass (موتور تخصیص خودکار) تشکیل شده است.
- چهار حالت دسترسی اصلی به دیسک عبارتند از RWO (تک‌نود)، ROX (چندنود فقط خواندنی)، RWX (چندنود خواندن/نوشتن) و RWOP (تک‌پاد اختصاصی).
- با قابلیت Dynamic Provisioning، دیگر نیازی به ساخت دستی دیسک نیست؛ با ارسال هر PVC، دیسک ابری به طور آنی ساخته و متصل می‌شود.

فصل ۱۲: مدیریت پیکربندی و داده‌های حساس (ConfigMaps & Secrets)

یکی از اصول بنیادین معماری ابری‌بومی در متدولوژی مشهور **Factor App-۱۲**، اصل تفکیک کامل کد از پیکربندی (Config) است: **ایمیج‌های کانتینری باید تغییرناپذیر، عمومی و کاملاً عاری از هرگونه تنظیمات محیطی یا اطلاعات هویتی باشند.**

شما باید بتوانید دقیقاً همان یک ایمیج ساخته‌شده در گیت‌هاب را بدون هیچ‌گونه کامپایل مجدد، در محیط توسعه (Dev)، محیط آزمون (Staging) و محیط عملیاتی پروداکشن (Production) مستقر کنید و تنها با تزریق مقادیر پیکربندی متفاوت، رفتار برنامه را تغییر دهید.

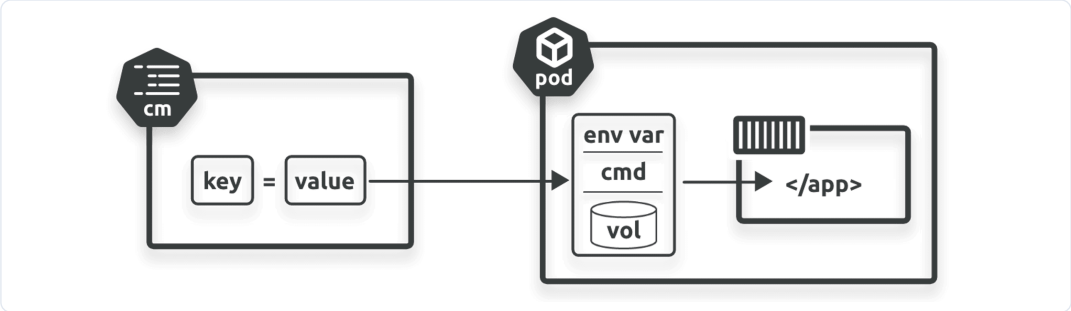
کوبرنتیز برای مدیریت این نیازمندی استراتژیک، دو آبجکت تخصصی طراحی کرده است: ۱. **کانفیگ‌مپ (ConfigMap):** برای ذخیره داده‌های متنی غیرمحرمانه (آدرس پایگاه داده، پورت‌ها، فیچرفلگ‌ها و فایل‌های تنظیمات). ۲. **سکرت (Secret):** برای نگهداری داده‌های حساس و محرمانه (رمزهای عبور، توکن‌های API، کلیدهای خصوصی و گواهینامه‌های SSL/TLS).

در این فصل مباحث زیر را به صورت کامل و تفسیری فرا خواهیم گرفت: ۱. **فلسفه جداسازی پیکربندی از کد ایمیج و استانداردهای دوازده‌عاملی** ۲. **کالبدشکافی کامل ConfigMaps و روش‌های سه‌گانه تزریق به کانتینر** ۳. **به‌روزرسانی زنده (Live Reload) فایل‌های کانفیگ درون پاد بدون ریستارت** ۴. **کالبدشکافی عمیق Secrets، انواع سکرت‌ها و واقعیت رمزنگاری در کوبرنتیز** ۵. **الگوهای امنیتی سازمانی: رمزنگاری در پایگاه داده (KMS v2) و اپراتورهای External Secrets** ۶. **کارگاه عملیاتی جامع: ایجاد دستوری و اعلانی، تزریق همزمان متغیر و دیسک موقت رم (tmpfs) و تست پاد**

—

۱. فلسفه طراحی: جداسازی پیکربندی از کد برنامه

تصویر ۱۲.۱ ایده اصلی را به زیبایی نشان می‌دهد:



تصویر ۱۲.۱ - جداسازی پیکربندی از ایمیج: ایمیج ثابت برنامه از بیرون کلاستر با داده‌های کانفیگ‌مپ پیوند می‌خورد

اگر آدرس پایگاه داده را درون کدهای جاوا، پایتون یا داکرفایل هاردکد کنید:

- با هر تغییر کوچک در آدرس سرور دیتابیس، باید کل برنامه را از ابتدا بیلد، تست و منتشر کنید.
- ایمیج شما خطرات امنیتی جدی پیدا می‌کند چون ممکن است در مخازن عمومی بارگذاری شود.
- امکان تست برنامه در محیط‌های محلی از بین می‌رود.

با استفاده از ConfigMap و Secret، ایمیج کانتینر کاملاً بدون وضعیت و تمیز باقی می‌ماند و تمامی جزئیات محیط در لحظه استارت پاد توسط کوبرنتیز به آن تزریق می‌شود.

—

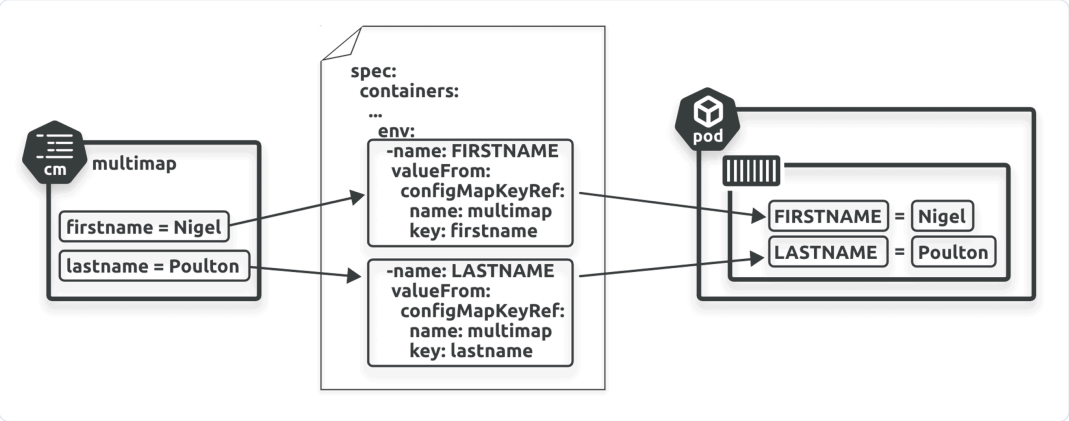
۲. کالبدشکافی کانفیگ‌مپ‌ها (ConfigMaps)

یک ConfigMap شیئی در API کوبرنتیز است که داده‌ها را به صورت جفت‌های **کلید-مقدار (Key-Value)** یا در قالب محتوای کامل یک فایل متنی (مانند `nginx.conf` یا `redis.conf`) در دیتابیس کلاستر ذخیره می‌کند.

کوبرنتیز سه روش برای تزریق اطلاعات ConfigMap به کانتینر در اختیار شما قرار می‌دهد:

۱. تزریق در قالب متغیرهای محیطی (Environment Variables)

شایع‌ترین روش برای مقادیر ساده مانند پورت یا نام دیتابیس است (تصویر ۱۲.۲).



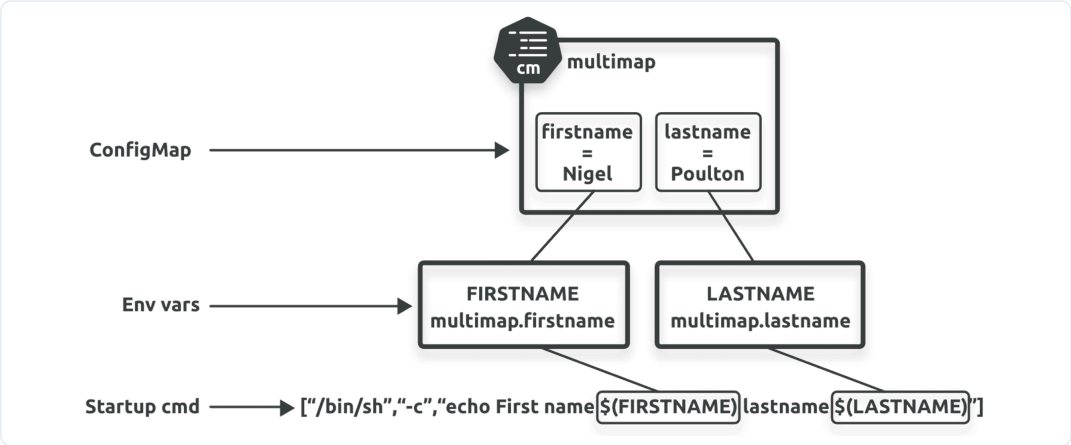
تصویر ۱۲.۲ - تزریق داده‌های کلید-مقدار کانفیگ‌مپ به عنوان متغیرهای محیطی درون کانتینر

شما می‌توانید یک کلید خاص را با `valueFrom.configMapKeyRef` یا تمام کلیدهای یک کانفیگ‌مپ را به صورت یکجا با فیلد `envFrom` به کانتینر تزریق کنید:

```
envFrom:
- configMapRef:
    name: app-config
```

۲. تزریق در قالب آرگومان‌های خط فرمان (Command-line Arguments)

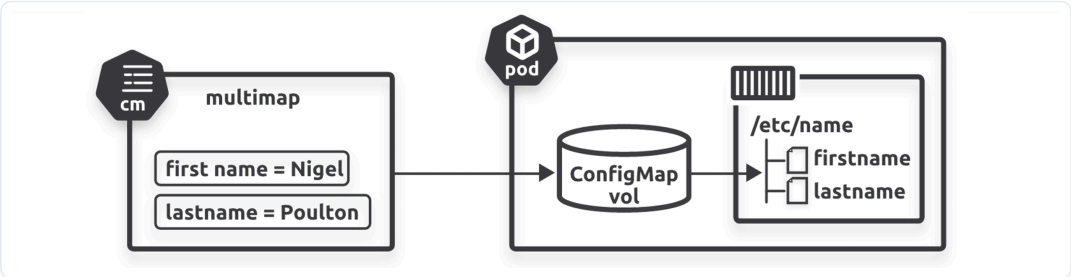
اگر برنامه شما پارامترها را در زمان استارت از طریق فلگ‌های خط فرمان دریافت می‌کند، می‌توانید متغیرهای محیطی ناشی از ConfigMap را درون دستور `command` یا `args` فراخوانی کنید (تصویر ۱۲.۳).



تصویر ۱۲.۳ - نگاشت ورودی‌های کانفیگ‌مپ به عنوان آرگومان‌های اجرایی دستور استارت برنامه

۳. ماونت کردن به عنوان فایل درون یک والیوم (Volume Mounts)

بهترین روش برای فایل‌های پیکربندی چندسطری و حجیم (مانند فایل‌های کانفیگ وب‌سرورها). هر کلید در ConfigMap به یک فایل مجزا در پوشه ماونت‌شده تبدیل می‌شود (تصویر ۱۲.۴).



تصویر ۱۲.۴ - ماونت کردن کانفیگ‌مپ به عنوان فایل‌های متنی درون یک دیسک مجازی در فایل‌سیستم کانتینر

معجزه به‌روزرسانی زنده (Live Reload) با Volume Mounts

اگر مقادیر ConfigMap را به عنوان متغیر محیطی تزریق کنید، در صورت تغییر ConfigMap مقادیر درون کانتینر تغییر نمی‌کند مگر اینکه پاد ریستارت شود. اما اگر ConfigMap را در قالب یک Volume ماونت کرده باشید، سرویس کیوبلت تغییرات را به صورت خودکار ظرف چند ثانیه درون فایل‌های کانتینر همگام می‌سازد! اگر برنامه شما قابلیت Hot-reload داشته باشد، بدون حتی یک میلی‌ثانیه قطعی یا ریستارت پاد، کانفیگ جدید را اعمال می‌کند.

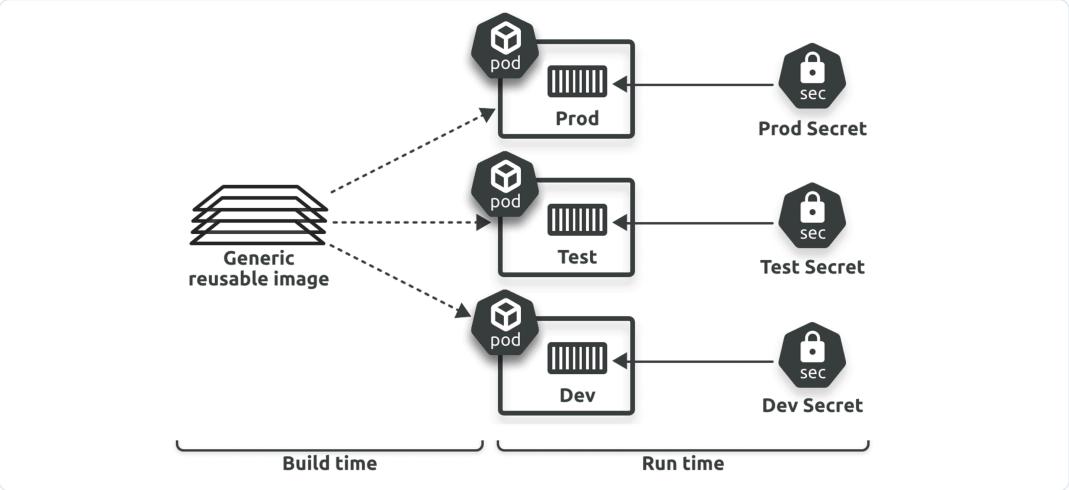
—

۳. کالبدشکافی سکرت‌ها (Secrets)

سکرت‌ها برای مدیریت اطلاعات حساس طراحی شده‌اند:

- کلمات عبور دیتابیس

- کلیدهای خصوصی SSH و توکن‌های OAuth
- گواهینامه‌های SSL/TLS (`tls.key` و `tls.crt`)
- مشخصات لاگین به رجیستری‌های خصوصی داکر (`.dockerconfigjson`)



تصویر ۱۲.۵ - تزریق اسرار و کلیدهای امنیتی به محیط اجرای برنامه در لحظه راه‌اندازی پاد

واقعیت تکان‌دهنده درباره سکرت‌های کوبرنتیز:

> ⚠️ **هشدار:**
>
> کوبرنتیز به صورت پیش‌فرض سکرت‌ها را رمزنگاری (Encrypt) نمی‌کند! مقادیر سکرت تنها با الگوریتم **Base64** کدگذاری شده‌اند که هر کسی در کسری از ثانیه با دستور `base64 -d` می‌تواند آن‌ها را بخواند!

برای تبدیل کلاستر به یک محیط امن سازمانی، باید استانداردهای زیر را رعایت کنید:

۱. **رمزنگاری در حالت سکون (Encryption at Rest با KMS v2):** در نسخه‌های مدرن کوبرنتیز (v1.28 تا v1.32)، لایه کنترل پلین از ارائه‌دهنده **KMS v2** پشتیبانی می‌کند. در این پیکربندی، داده‌ها پیش از ذخیره در etcd با کلیدهای قدرتمند سخت‌افزاری (AWS KMS، Azure Key Vault یا HashiCorp Vault) رمزنگاری می‌شوند.

۲. **استفاده از حافظه موقت رم (`tmpfs`):** هنگامی که یک سکرت را به عنوان Volume ماونت می‌کنید، کوبرنتیز آن را روی حافظه موقت RAM (فایل‌سیستم `tmpfs`) قرار می‌دهد نه روی هارددیسک نود؛ بنابراین با ریستارت سرور یا بررسی فیزیکی دیسک، هیچ داده حساسی فاش نمی‌شود.

۳. **اپراتورهای مدیریت کلید بیرونی (External Secrets Operator - ESO):** استاندارد روز دنیا این است که سکرت‌ها در گاوصندوق‌های متمرکز شرکتی مانند HashiCorp Vault نگهداری شوند و ابزار ESO آن‌ها را به صورت خودکار و امن به کلاستر سینک کند.

—

۴. کارگاه عملیاتی جامع: پیکربندی و تزریق امن (Hands-on Lab)

وارد پوشه کدهای کتاب شوید:

```
cd TKB/configmaps-secrets
```

۱. ایجاد دستوری و اعلانی ConfigMap

برای ایجاد یک کانفیگ‌مپ دستوری در خط فرمان:

```
kubectl create configmap app-settings --from-literal=ENVIRONMENT=production --
from-literal=LOG_LEVEL=debug
```

اکنون یک فایل اعلانی `cm.yml` بسازید:

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: test-cm
data:
  app.properties: |
    theme=dark
    max_connections=100
    cache_enabled=true
```

مانیفست را اعمال کنید:

```
kubectl apply -f cm.yml
```

۲. ایجاد امن یک Secret

دستور زیر یک سکرت عمومی (نوع Opaque) ایجاد می‌کند:

```
kubectl create secret generic db-credentials --from-literal=db-
password='SuperSecretPassword123'
```

اگر سکرت را با دستور `kubectl get secret db-credentials -o yaml` مشاهده کنید، خواهید دید که رمز عبور به رشته Base64 تبدیل شده است:

```
data:
  db-password: U3VwZXJTZWNyZXRQYXNzd29yZDEyMw==
```

۳. استقرار پاد و مصرف همزمان کانفیگ و سکرت

فایل `pod.yml` را بررسی می‌کنیم. این پاد داده‌های کانفیگ‌مپ را در قالب متغیر محیطی دریافت کرده و سکرت را به عنوان یک فایل امن درون مسیر `/etc/secrets` ماونت می‌کند:

```
apiVersion: v1
kind: Pod
metadata:
  name: secure-app
spec:
  containers:
  - name: app
    image: nigelpoulton/k8sbook:1.0
    env:
    - name: ENV_MODE
      valueFrom:
        configMapKeyRef:
          name: app-settings
          key: ENVIRONMENT
    volumeMounts:
    - name: secret-volume
      mountPath: /etc/secrets
      readOnly: true
  volumes:
  - name: secret-volume
    secret:
      secretName: db-credentials
```

پاد را اعمال کنید:

```
kubectl apply -f pod.yml
```

۴. راستی‌آزمایی تزریق درون کانتینر

وارد شل پاد شوید و متغیر محیطی و فایل سکرت را چک کنید:

```
# چاپ متغیر محیطی تزریق‌شده از ConfigMap
kubectl exec secure-app -- printenv ENV_MODE

# خواندن رمز عبور از درون فایل امن ماونت‌شده
kubectl exec secure-app -- cat /etc/secrets/db-password
```

خروجی دقیقاً مقادیر تزریق‌شده را به شما نشان می‌دهد:

```
production
SuperSecretPassword123
```

برنامه شما با موفقیت پیکربندی‌ها و داده‌های حساس را بدون ذخیره در ایمیج، از کلاستر تحویل گرفت!

—

۵. پاک‌سازی منابع (Clean up)

با دستورات زیر منابع تمرینی را حذف کنید:

```
kubectl delete pod secure-app
kubectl delete configmap app-settings test-cm
kubectl delete secret db-credentials
```

—

۶. جمع‌بندی فصل ۱۲

- بر طبق اصل ۱۲-Factor App، هرگز نباید تنظیمات و پسوردها درون ایمیج‌های کانتینری ذخیره شوند.
- شیء ConfigMap برای داده‌های متنی غیرمحرمانه است و از سه طریق متغیرهای محیطی، آرگومان‌های استارت و والیوم‌ها به پاد تزریق می‌شود.
- کانفیگ‌هایی که در قالب Volume ماونت می‌شوند از قابلیت به‌روزرسانی زنده (Live Reload) بدون ریستارت پاد بهره‌مندند.
- شیء Secret برای اطلاعات حساس است و به صورت پیش‌فرض در قالب Base64 نگهداری می‌شود نه رمزنگاری واقعی!
- در کلاسترهای سازمانی، فعال‌سازی رمزنگاری در دیتابیس با KMS v2 و استفاده از ابزارهایی مانند HashiCorp Vault و External Secrets الزامی است.
- ماونت کردن سکرت‌ها روی دیسک‌های رم موقت (`tmpfs`) امن‌ترین روش مصرف داده‌های حساس است.

فصل ۱۳: برنامه‌های دارای وضعیت با استیت‌فول‌ست (StatefulSets)

تا این بخش از کتاب، اکثر بارهای کاری که بررسی کردیم از نوع **بدون‌وضعیت (Stateless)** بودند؛ برنامه‌هایی نظیر وب‌سرورهای Nginx یا میکروسرویس‌های API که هیچ داده ماندگاری روی دیسک محلی خود ذخیره نمی‌کنند. در دیپلویمنت‌ها، پادها موجوداتی کاملاً یکسان، قابل تعویض و فاقد هویت هستند (Cattle)؛ پادها با نام‌های تصادفی مانند `web-74b7897c8d-x9z2p` متولد می‌شوند و اگر پادی بمیرد، پاد دیگری با اسمی متفاوت جایش را می‌گیرد.

اما آیا می‌توان پایگاه‌های داده کلاستری نظیر PostgreSQL (با معماری Primary/Replica)، MongoDB، Cassandra، Elasticsearch، Kafka یا ZooKeeper را با Deployment مدیریت کرد؟ **پاسخ یک «خیر» قاطع است!**

پایگاه‌های داده نیازمند سه ویژگی حیاتی هستند: ۱. **هویت شبکه پایدار و پیش‌بینی‌پذیر (نه نام‌ها و هدرهای تصادفی).** ۲. **دیسک اختصاصی و ماندگار منحصربه‌فرد برای هر عضو کلاستر (پاد شماره ۱ همیشه باید به دیسک شماره ۱ وصل شود، حتی اگر روی سرور دیگری متولد گردد).** ۳. **ترتیب دقیق در راه‌اندازی و خاموشی (ابتدا گره اصلی بالا بیاید، سپس نودهای کپی؛ و در زمان خاموشی برعکس).**

کوبرنتیز برای مدیریت این دسته از بارهای کاری حیاتی، کنترلر قدرتمند **StatefulSet** را خلق کرده است.

در این فصل مباحث زیر را به صورت کامل و تفسیری فرا خواهیم گرفت: ۱. **تفاوت بنیادین بارهای کاری Stateful در برابر Stateless**. ۲. **معماری سه‌گانه StatefulSet: هویت ترتیبی، سرویس بدون‌سر (Headless Service) و قالب ادعای دیسک (volumeClaimTemplates)** ۳. **ترتیب در اسکیل (Scaling) و گارانتی‌های راه‌اندازی و خاموش‌سازی** ۴. **الگوریتم‌های به‌روزرسانی RollingUpdate و قابلیت انتشار قطعه‌ای (Partitioned Canary)** ۵. **تحلیل رفتار خودمراقبتی و مدیریت خطای نودها در سرورهای ابری و محلی** ۶. **کارگاه عملیاتی جامع: استقرار کلاستر Stateful، بررسی نام‌گذاری ترتیبی، تست تفکیک دیسک‌ها و آزمون از دست رفتن نود**

—

۱. تفاوت بنیادین بارهای کاری دارای وضعیت (Stateful) با دیپلویمنت‌ها

جدول زیر مقایسه کلیدی این دو کنترلر را نشان می‌دهد:

ویژگی	دیپلویمنت (Deployment)	استیت‌فول‌ست (StatefulSet)
نوع بارهای کاری	برنامه‌های بدون‌وضعیت (وب‌سرورها، APIها)	پایگاه‌های داده و سیستم‌های کلاستری
نام‌گذاری پادها	نام تصادفی بر پایه هش (مثلاً `web-5d8f-k2x`)	ترتیبی و مشخص (`db-0`، `db-1`، `db-2`)
هویت شبکه	آی‌پی‌های متغیر بدون نام ماندگار فردی	نام DNS پایدار برای تک‌تک پادها
مدیریت دیسک	دیسک مشترک یا بدون دیسک	یک دیسک اختصاصی مجزا برای هر پاد
ترتیب راه‌اندازی	همزمان و موازی	گام‌به‌گام و ترتیبی (از ۰ تا N-1)
ترتیب خاموش‌سازی	تصادفی و همزمان	ترتیبی معکوس (از N-1 تا ۰)

—

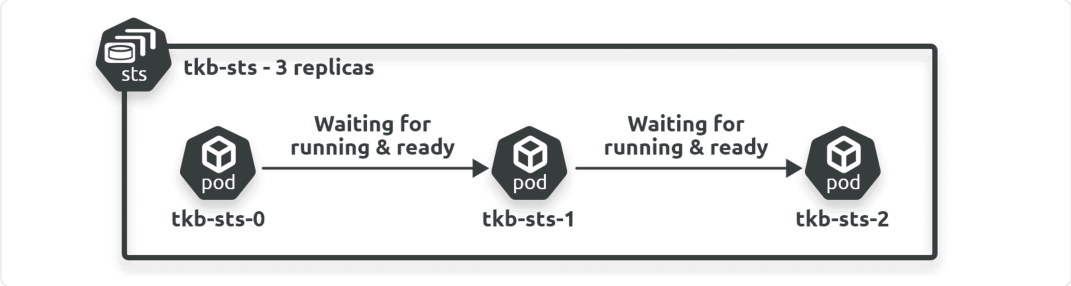
۲. ارکان سه‌گانه معماری StatefulSet

برای درک استیت‌فول‌ست، باید سه مکانیزم مهندسی زیر را بشناسید:

۱. ایندکس‌های ترتیبی (Ordinal Indexing)

اگر تعداد کپی‌ها (`replicas`) را برابر با ۳ قرار دهید، پادها با اسامی ثابت و ترتیبی زیر متولد می‌شوند (تصویر ۱۳.۱):

```
tkb-sts-0
tkb-sts-1
tkb-sts-2
```



تصویر ۱۳.۱ - هویت ترتیبی و گارانتی راه‌اندازی گام‌به‌گام: پادها به نوبت و پس از تایید سلامت پاد قبلی متولد می‌شوند

کوبرنتیز تضمین می‌کند:

- پاد `tkb-sts-0` ابتدا ساخته می‌شود؛ تا زمانی که این پاد کاملاً بالا نیامده و پروب‌های سلامت آن تایید نشوند، پاد `tkb-sts-1` ساخته نخواهد شد!
- در زمان کاهش ظرفیت یا حذف، پاد با بالاترین شماره (`tkb-sts-2`) ابتدا خاموش می‌شود و تا خاموشی کامل آن، دست به پادهای قبلی زده نمی‌شود.

۲. سرویس بدون‌سر (Headless Service)

در فصل ۷ آموختیم که سرویس‌های معمولی یک IP مجازی واحد (ClusterIP) دارند که ترافیک را به طور تصادفی پخش می‌کند. اما در یک کلاستر پایگاه داده، برنامه باید بداند کدام نود **Primary (اصلی)** و کدام نودها **Replica (کپی)** هستند تا درخواست‌های نوشتن را فقط به نود اصلی بفرستد!

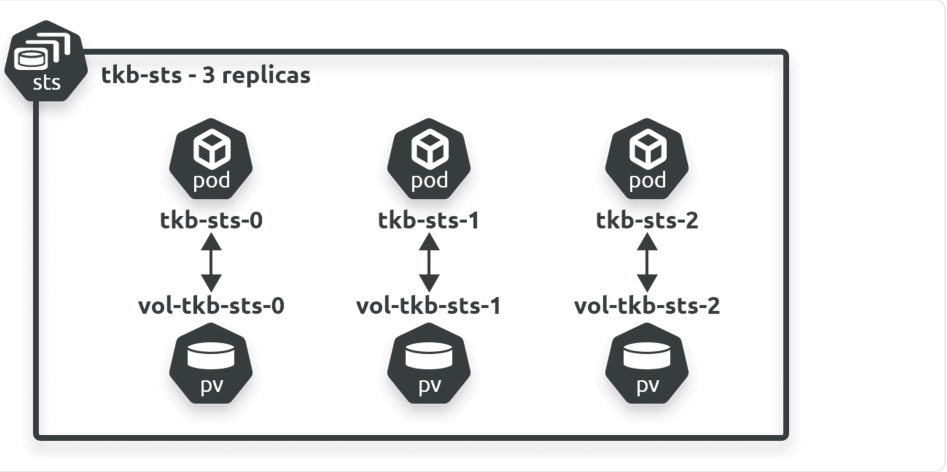
برای این منظور، StatefulSet به همراه یک **Headless Service** (`clusterIP: None`) مستقر می‌شود. این سرویس رکوردهای DNS مستقیم به ازای تک‌تک پادها در CoreDNS می‌سازد:

```
tkb-sts-0.tkb-headless.default.svc.cluster.local
tkb-sts-1.tkb-headless.default.svc.cluster.local
tkb-sts-2.tkb-headless.default.svc.cluster.local
```

این نام‌های DNS در طول عمر کلاستر **کاملاً جاودانه و ثابت** هستند!

۳. قالب‌های ادعای دیسک (volumeClaimTemplates)

در دیپلویمنت اگر ۱۰ پاد بسازید و به یک PVC وصل کنید، هر ۱۰ پاد به همان یک دیسک وصل می‌شوند. اما در استیت‌فول‌ست، هر پاد نیازمند دیسک مستقل خود است!



تصویر ۱۳.۲ - انتساب دیسک اختصاصی به هر نمونه پاد: پاد شماره ۰ به دیسک شماره ۰ و پاد شماره ۱ به دیسک شماره ۱ مقید می‌شود

بخش `volumeClaimTemplates` در مانیفست استیت‌فول‌ست مانند یک کارخانه اختصاصی عمل می‌کند؛ به ازای هر نمونه پاد، یک PVC مجزا تولید می‌کند (تصویر ۱۳.۲):

- پاد `tkb-sts-0` به ادعای دیسک `data-tkb-sts-0`
- پاد `tkb-sts-1` به ادعای دیسک `data-tkb-sts-1`

> 📌 **نکته مهم:**
>
> اگر پاد `tkb-sts-1` به دلیل سوختن سرور بمیرد و کوبرنتیز آن را روی یک نود کاملاً دیگر زنده کند، کوبرنتیز تضمین می‌کند که دقیقاً همان دیسک `data-tkb-sts-1` به پاد جدید متصل شود؛ بنابراین هیچ داده‌ای گم نمی‌شود!

> ⚠️ **هشدار:**
>
> برای جلوگیری از فاجعه نابودی سهوی داده‌ها، کوبرنتیز در زمان حذف یا اسکیل‌داون StatefulSet، **هرگز PVCها و دیسک‌های فیزیکی را پاک نمی‌کند!** دیسک‌ها در کلاستر باقی می‌مانند و ادمین در صورت اطمینان باید آن‌ها را به صورت دستی حذف کند.

—

۳. استراتژی‌های به‌روزرسانی و قابلیت جادویی Partition

استیت‌فول‌ست برای ارتقای نسخه از الگوریتم RollingUpdate ترتیبی معکوس استفاده می‌کند (ابتدا پاد ۲ آپدیت می‌شود، سپس ۱، و در نهایت گره اصلی ۰).

علاوه بر این، قابلیتی به نام **Partitioning** وجود دارد:

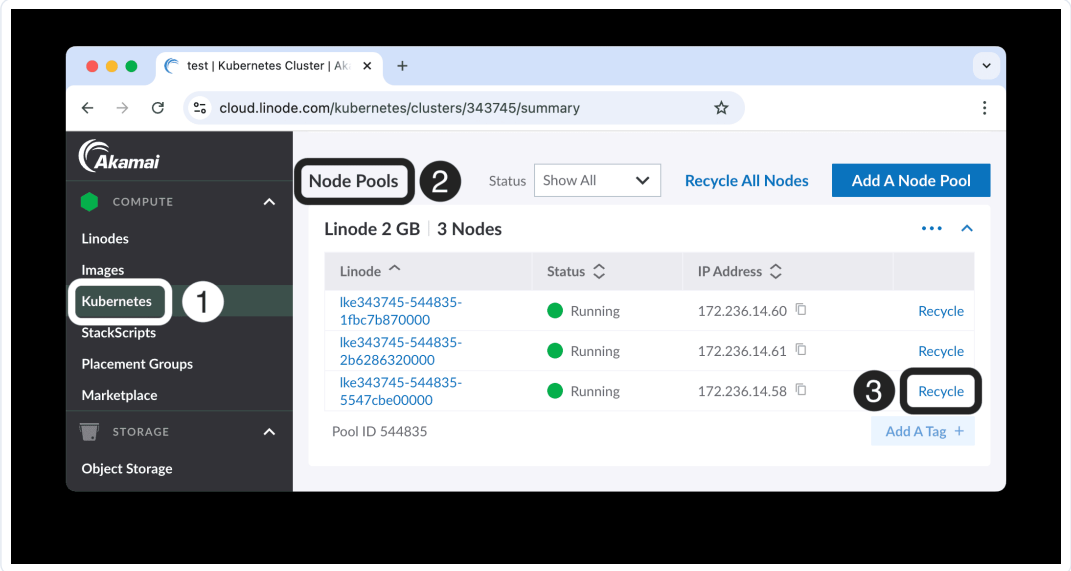
```
updateStrategy:
  rollingUpdate:
    partition: 2
```

این دستور به کوبرنتیز می‌گوید: **«فقط پادهایی را به‌روزرسانی کن که شماره ایندکس آن‌ها بزرگ‌تر یا مساوی ۲ است!»** این قابلیت استثنایی به مهندسان اجازه می‌دهد نسخه‌های جدید پایگاه داده را به صورت تدریجی (Canary Deployment) روی تنها یکی از نودهای کپی آزمایش کنند، پیش از آن‌که کل کلاستر دیتابیس آپدیت شود.

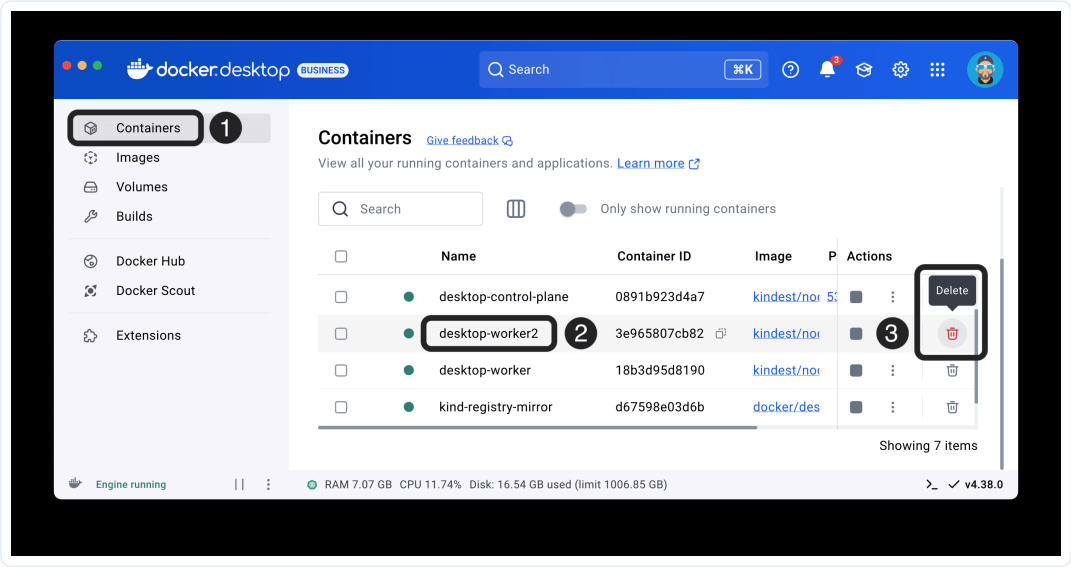
—

۴. مقاومت در برابر خرابی نودها (Node Failure Resilience)

تصاویر ۱۳.۳ و ۱۳.۴ آزمایش سناریوی خرابی سرور را در دو محیط ابری (LKE) و محلی (Docker Desktop) نشان می‌دهند:



تصویر ۱۳.۳ - آزمایش از دست رفتن یکی از نودهای کلاستر در محیط ابری Linode Kubernetes Engine



تصویر ۱۳.۴ - رفتار خودمراقبتی StatefulSet در صورت حذف نود در کلاستر محلی Docker Desktop

هنگامی که نودی از کار می‌افتد: ۱. کنترل پلین کوبرنتیز نود را در وضعیت `NotReady` قرار می‌دهد. ۲. کنترلر StatefulSet پاد آسیب‌دیده را شناسایی کرده و آن را روی نودی سالم مجدداً متولد می‌کند. ۳. درایور CSI دیسک ابری قبلی را از نود سوخته جدا (Detach) کرده و به نود جدید متصل (Attach) می‌سازد. ۴. پاد جدید دقیقاً با همان نام و همان آدرس DNS و متصل به همان دیسک به سرویس‌دهی ادامه می‌دهد!

—

۵. کارگاه عملیاتی جامع: استقرار کلاستر Stateful (Hands-on Lab)

وارد پوشه کدهای کتاب در بخش StatefulSets شوید:

```
cd TKB/statefulsets
```

۱. کالبدشکافی مانیفست (`sts.yml`)

مانیفست شامل یک سرویس Headless و یک StatefulSet با ۳ نسخه است:

```yaml
apiVersion: v1
kind: Service
metadata:
  name: tkb-sts
spec:
  clusterIP: None        # این خط سرویس را به Headless Service تبدیل می‌کند!
  selector:
    app: sts
  ports:
  - port: 8080
```

```
---
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: tkb-sts
spec:
  serviceName: "tkb-sts"
  replicas: 3
  selector:
    matchLabels:
      app: sts
  template:
    metadata:
      labels:
        app: sts
    spec:
      containers:
      - name: ctr-web
        image: nigelpoulton/k8sbook:1.0
        ports:
        - containerPort: 8080
        volumeMounts:
        - name: web-vol
          mountPath: /data
  volumeClaimTemplates:
  - metadata:
      name: web-vol
    spec:
      accessModes: [ "ReadWriteOnce" ]
      resources:
        requests:
          storage: 1Gi
```

۲. استقرار و مشاهده تولد گام‌به‌گام پادها

دستور زیر را اعمال کنید و فوراً با دستور رصد لحظه‌ای وضعیت را تماشا کنید:

```
kubectl apply -f sts.yml
kubectl get pods -w
```

خواهید دید که پادها یکی‌یکی متولد می‌شوند:

```
NAME         READY    STATUS              RESTARTS    AGE
tkb-sts-0    0/1      ContainerCreating    0          5s
tkb-sts-0    1/1      Running               0           15s
```

```
tkb-sts-1    0/1     ContainerCreating    0           17s
tkb-sts-1    1/1     Running              0           30s
tkb-sts-2    0/1     ContainerCreating    0           32s
tkb-sts-2    1/1     Running              0           45s
```

۳. بررسی دیسک‌های اختصاصی تولیدشده

دستور زیر را برای مشاهده ادعاهای دیسک اجرا کنید:

```
kubectl get pvc
```

خروجی نشان می‌دهد که به تعداد پادها، PVCهای مستقل ساخته شده است:

```
NAME                  STATUS    VOLUME
CAPACITY
web-vol-tkb-sts-0    Bound     pvc-878593be-1123-4c91-a185-1fa32e5e1a10    1Gi
web-vol-tkb-sts-1    Bound     pvc-878593be-1123-4c91-a185-1fa32e5e1a11    1Gi
web-vol-tkb-sts-2    Bound     pvc-878593be-1123-4c91-a185-1fa32e5e1a12    1Gi
```

۴. تست بقای داده‌ها و هویت پاد در صورت حذف

وارد پاد شماره ۱ شوید و دیتایی بنویسید:

```
kubectl exec -it tkb-sts-1 -- sh
echo "Stateful Data from Node 1" > /data/state.txt
exit
```

اکنون پاد شماره ۱ را با دستور دستی حذف کنید:

```
kubectl delete pod tkb-sts-1
```

کنترلر StatefulSet بلافاصله متوجه حذف پاد شده و پادی با **دقیقاً همان نام** (`tkb-sts-1`) می‌سازد. پس از آماده شدن پاد، محتوای فایل را بخوانید:

```
kubectl exec -it tkb-sts-1 -- cat /data/state.txt
```

خروجی:

```
Stateful Data from Node 1
```

پاد جدید بی‌درنگ به دیسک اختصاصی قبلی متصل شده و تمامی داده‌ها حفظ شدند!

—

۶. پاک‌سازی منابع (Clean up)

برای پاک‌سازی کامل:

```
# حذف StatefulSet و سرویس
kubectl delete -f sts.yml

# حذف دستی PVCهای ابری برای آزادسازی دیسک‌ها
kubectl delete pvc -l app=sts
```

—

۷. جمع‌بندی فصل ۱۳

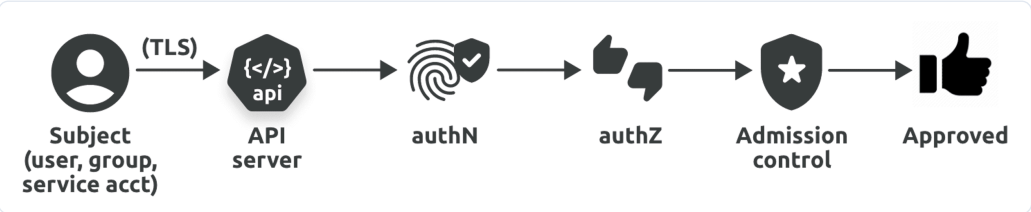
- پایگاه‌های داده و برنامه‌های دارای وضعیت نیازمند هویت شبکه پایدار، دیسک‌های اختصاصی و راه‌اندازی ترتیبی هستند.
- شیء StatefulSet این نیازمندی‌ها را با ایندکس‌های ترتیبی (از ۰ تا N-1) برآورده می‌سازد.
- سرویس بدون‌سر (Headless Service) با `clusterIP: None` نام‌های دامنه DNS یکتا و ثابت برای تک‌تک پادها می‌سازد.
- بخش `volumeClaimTemplates` به ازای هر نمونه پاد یک PVC و PV اختصاصی به صورت پویا تولید می‌کند.
- پادها به صورت گام‌به‌گام و پس از تایید سلامت پاد قبلی روشن می‌شوند و در زمان خاموشی با ترتیب معکوس متوقف می‌گردند.
- حذف StatefulSet هرگز دیسک‌های PVC را پاک نمی‌کند تا خطر نابودی تصادفی داده‌ها به صفر برسد.

فصل ۱۴: امنیت سرور API و کنترل دسترسی نقشی (API Security & RBAC)

سرور API کوبرنتیز (`kube-apiserver`) قلب تپنده، مغز متفکر و تنها درگاه ورودی (Single Front Door) تمام تعاملات در کلاستر است. تک‌تک دستوراتی که با `kubectl` صادر می‌کنید، ارتباطات میان کنترلرها و نودها، استقرارهای خودکار در پایپ‌لاین‌های CI/CD و حتی کوئری‌های مانیتورینگ، همگی در قالب درخواست‌های HTTPS به سمت API Server ارسال می‌شوند. اگر این درگاه نفوذپذیر باشد، تمام کلاستر و داده‌های درون آن در معرض سقوط قرار می‌گیرند.

کوبرنتیز برای ایمن‌سازی این نقطه حیاتی، از یک **قیف امنیتی سه‌مرحله‌ای (Three-Stage Security Pipeline)** بسیار پیشرفته و سخت‌گیرانه استفاده می‌کند: ۱. **احراز هویت (Authentication - AuthN):** شما چه کسی هستید؟ ۲. **مجازشماری و کنترل دسترسی نقشی (Authorization - AuthZ / RBAC):** آیا اجازه انجام این کار مشخص را دارید؟ ۳. **کنترل پذیرش (Admission Control):** آیا درخواست شما با سیاست‌های کلان امنیتی، قانونی و بهینه‌سازی کلاستر همخوانی دارد؟

در این فصل، این زنجیره امنیتی را به صورت مفهومی و عمیق کالبدشکافی خواهیم کرد.



تصویر ۱۴.۱: قیف سه‌مرحله‌ای پردازش درخواست‌ها در سرور API کوبرنتیز (احراز هویت، مجازشماری و کنترل پذیرش)

—

۱. مرحله اول: احراز هویت (Authentication یا AuthN)

نخستین پرسشی که سرور API مطرح می‌کند این است: **«شما کیستید؟»** درخواست ارسالی باید هویت فرستنده را به اثبات برساند. در صورت شکست این مرحله، سرور بلافاصله پاسخ `HTTP 401 Unauthorized` برمی‌گرداند و پردازش متوقف می‌شود.

تفکیک حیاتی: کاربران انسانی در برابر حساب‌های سرویس (ServiceAccounts)

کوبرنتیز کاربران را به دو دسته بنیادین تقسیم می‌کند:

۱. **کاربران عادی و انسانی (Human / Normal Users):**

- توسعه‌دهندگان، مهندسان DevOps و مدیران سیستم.
- **نکته شگفت‌انگیز معماری:** کوبرنتیز شیء یا دیتابیسی به نام «User» درون `etcd` ذخیره نمی‌کند! کوبرنتیز مدیریت هویت انسان‌ها را عمداً برون‌سپاری (Outsource) کرده است.

- هویت انسان‌ها از طریق گواهی‌های دیجیتال سمت کلاینت (X.509 Client Certificates)، شناسه‌های ابری، یا پروتکل‌های OIDC (نظیر Google Workspace، Okta، Keycloak یا Microsoft Entra ID) تأیید می‌شود.

۲. **حساب‌های سرویس (ServiceAccounts):**

- برای برنامه‌ها، پادها، کانتینرها، کنترلرها و ابزارهای اتوماسیون درون کلاستر.
- حساب‌های سرویس اشیای بومی و رسمی کوبرنتیز هستند (`kind: ServiceAccount`) که درون etcd ذخیره شده و به یک نیم‌اسپیس خاص تعلق دارند.

> **امنیت مدرن توکن‌های ServiceAccount**
>
> در نگارش‌های قدیمی کوبرنتیز، برای هر ServiceAccount یک توکن استاتیک و نامحدود در Secret ایجاد می‌شد که تا ابد معتبر بود! از نسخه ۱.۲۴ به بعد، کوبرنتیز مکانیزم **Bound Service Account Token Volume** را استاندارد کرد. توکن‌ها به صورت توکن‌های JWT کوتاه‌مدت تولید می‌شوند، تاریخ انقضای محدود دارند، به پاد مشخصی مقید (Bound) شده‌اند و از طریق Projection در مسیر `/var/run/secrets/kubernetes.io/serviceaccount/token` پاد مانت می‌گردند.

—

۲. مرحله دوم: مجازشماری با RBAC (Authorization یا AuthZ)

پس از آنکه هویت فرستنده با موفقیت اثبات شد، نوبت به پاسخ به پرسش دوم می‌رسد: **«آیا شما مجاز به انجام عملیات درخواستی بر روی این منبع مشخص هستید؟»**

اگر مجاز نباشید، سرور خطای `HTTP 403 Forbidden` برمی‌گرداند.

مدل استاندارد و حاکم در کوبرنتیز **کنترل دسترسی مبتنی بر نقش (Role-Based Access Control یا RBAC)** است.

> 📌 **نکته مهم: اصل حداقل اختیارات (Least Privilege) و پیش‌فرض مسدود (Deny by Default)**
>
> سیستم RBAC در کوبرنتیز ماهیتی کاملاً **Deny-by-Default** دارد. در لحظه راه‌اندازی، هیچ هویتی هیچ اجازه‌ای برای هیچ کاری ندارد مگر آنکه به صراحت و مکتوب، دسترسی انجام آن فعل (Verb) روی آن منبع (Resource) به او اعطا شده باشد.

واژگان سه‌گانه در تعریف دسترسی‌ها

هر قانون دسترسی در کوبرنتیز از ترکیب سه رکن زیر ساخته می‌شود: ۱. **گروه API (API Group):** دسته ساختاری منبع (مثلاً هسته `""` برای پادها، `apps` برای دیپلویمنت‌ها). ۲. **منبع (Resource):** موجودیتی که می‌خواهید روی آن کار کنید (`pods` ، `services` ، `deployments`). ۳. **افعال (Verbs):** عملیاتی که مجاز به اجرای آن هستید (`get` ، `list` ، `watch` ، `create` ، `update` ، `patch` ، `delete`).

چهار شیء بنیادین در معماری RBAC

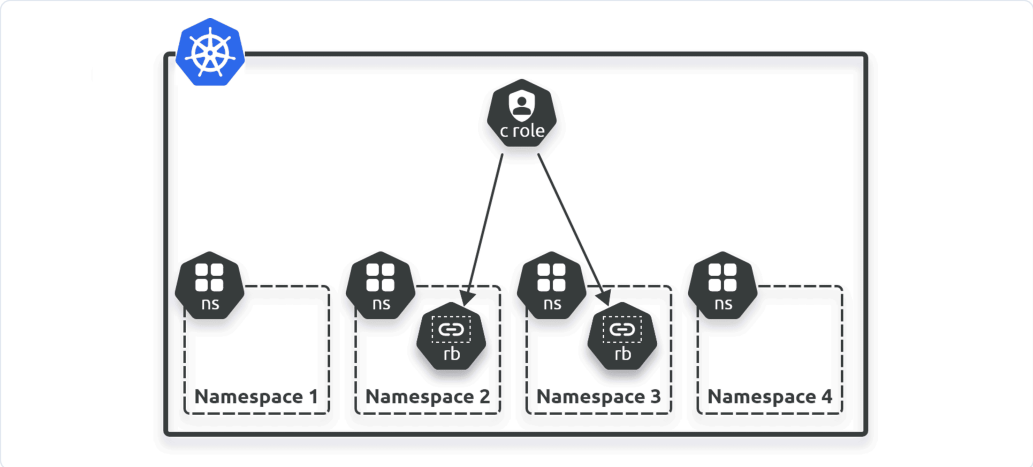
برای پیاده‌سازی RBAC در کلاستر، چهار شیء زیر وجود دارند که به دو دستهٔ **محدود به نیم‌اسپیس (Namespaced)** و **سراسری در کلاستر (Cluster-Scoped)** تقسیم می‌شوند:

دامنه اثر	تعریف نقش (مجوزها چیست؟)	پیوند نقش (مجوزها به چه کسی تعلق گیرد؟)
محدود به یک Namespace	**Role**	**RoleBinding**
کل کلاستر (Cluster-wide)	**ClusterRole**	**ClusterRoleBinding**

۱. **نقش محلی (Role):** مجموعه‌ای از مجوزها را درون **یک نیم‌اسپیس معین** تعریف می‌کند. ۲. **پیوند محلی (RoleBinding):** یک Role را به یک کاربر، گروه یا ServiceAccount در همان نیم‌اسپیس پیوند می‌زند. ۳. **نقش سراسری (ClusterRole):** نقش‌هایی که می‌توانند منابع غیروابسته به نیم‌اسپیس (مانند گره‌ها `nodes`، فضاهای نام `namespaces` یا دیسک‌های ماندگار `persistentvolumes`)، آدرس‌های غیرمنبعی مثل `/healthz` و `/metrics`، یا منابع استاندارد در سراسر کلاستر را مدیریت کنند. ۴. **پیوند سراسری (ClusterRoleBinding):** مجوزهای یک ClusterRole را در تمام نیم‌اسپیس‌های کلاستر به سوژه اعطا می‌کند.

الگوی جادویی و بهینه: پیوند ClusterRole با RoleBinding!

یکی از تکنیک‌های هوشمندانه و پرکاربرد مهندسی این است: یک نقش عمومی را تنها **یک‌بار** به صورت `ClusterRole` تعریف کنید (برای نمونه نقشی برای مشاهده پادها و لاگ‌ها)، اما به جای اعطای دسترسی به کل کلاستر با `ClusterRoleBinding`، آن را در هر نیم‌اسپیس دلخواه با یک `RoleBinding` محلی متصل نمایید!

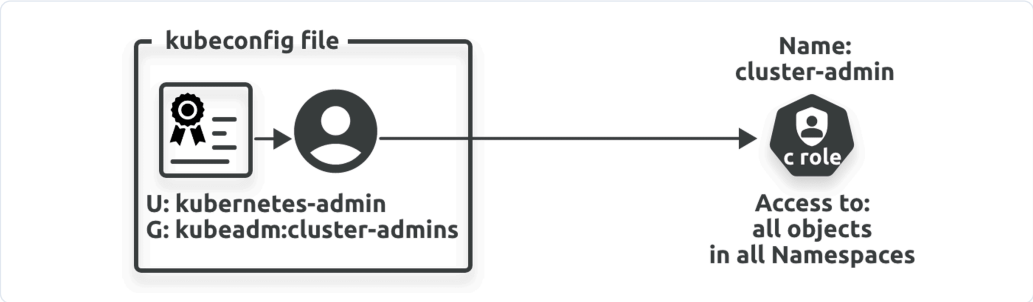


تصویر ۱۴.۲: ترکیب یک ClusterRole مشترک با RoleBindingهای محلی در نیم‌اسپیس‌های مجزا جهت جلوگیری از تکرار کد

این الگو باعث می‌شود تا بدون تکرار ده‌ها فایل Role یکسان در پروژه‌ها، به سرعت قوانین سازمانی را اعمال نمایید.

تحلیل نقش کلیدی `cluster-admin` و ساختار اعتبارسنجی پیش‌فرض

در تمامی کلاسترهای استاندارد کوبرنتیز، نقشی توکار و بسیار قدرتمند به نام `cluster-admin` تعبیه شده است:



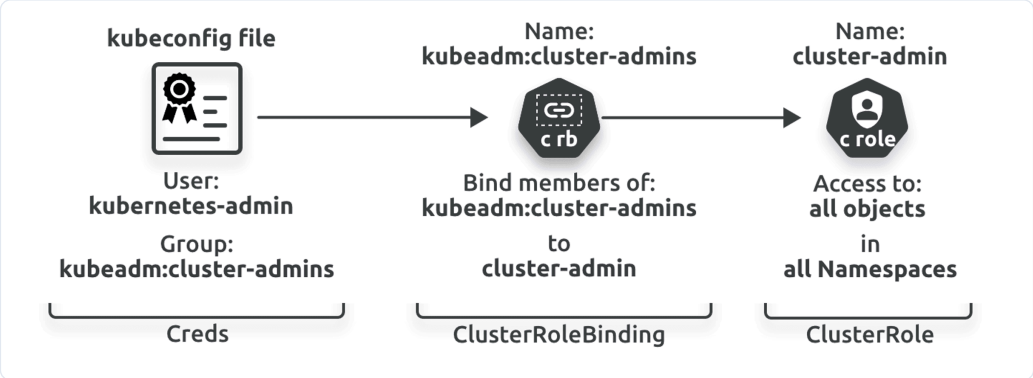
تصویر ۱۴.۳: نقش حیاتی cluster-admin و اعطای اختیارات نامحدود در سطح کلاستر

این نقش دارای قانون زیر است:

```
rules:
- apiGroups: ["*"]
  resources: ["*"]
  verbs: ["*"]
```

یعنی دسترسی به تمام افعال روی تمامی منابع در کل کلاستر!

زمانی که ابزارهایی مانند Docker Desktop، K3s یا Kubeadm کلاستر را برپا می‌کنند، یک کاربر به نام `kubernetes-admin` (عضو گروه `kubeadm:cluster-admins`) در فایل `kubeconfig` شما قرار می‌گیرد و با یک `ClusterRoleBinding` به این نقش متصل شده است (تصویر ۱۴.۴):



تصویر ۱۴.۴: اتصال کاربر kubectl به نقش cluster-admin از طریق گواهی دیجیتال و پیوند سراسری

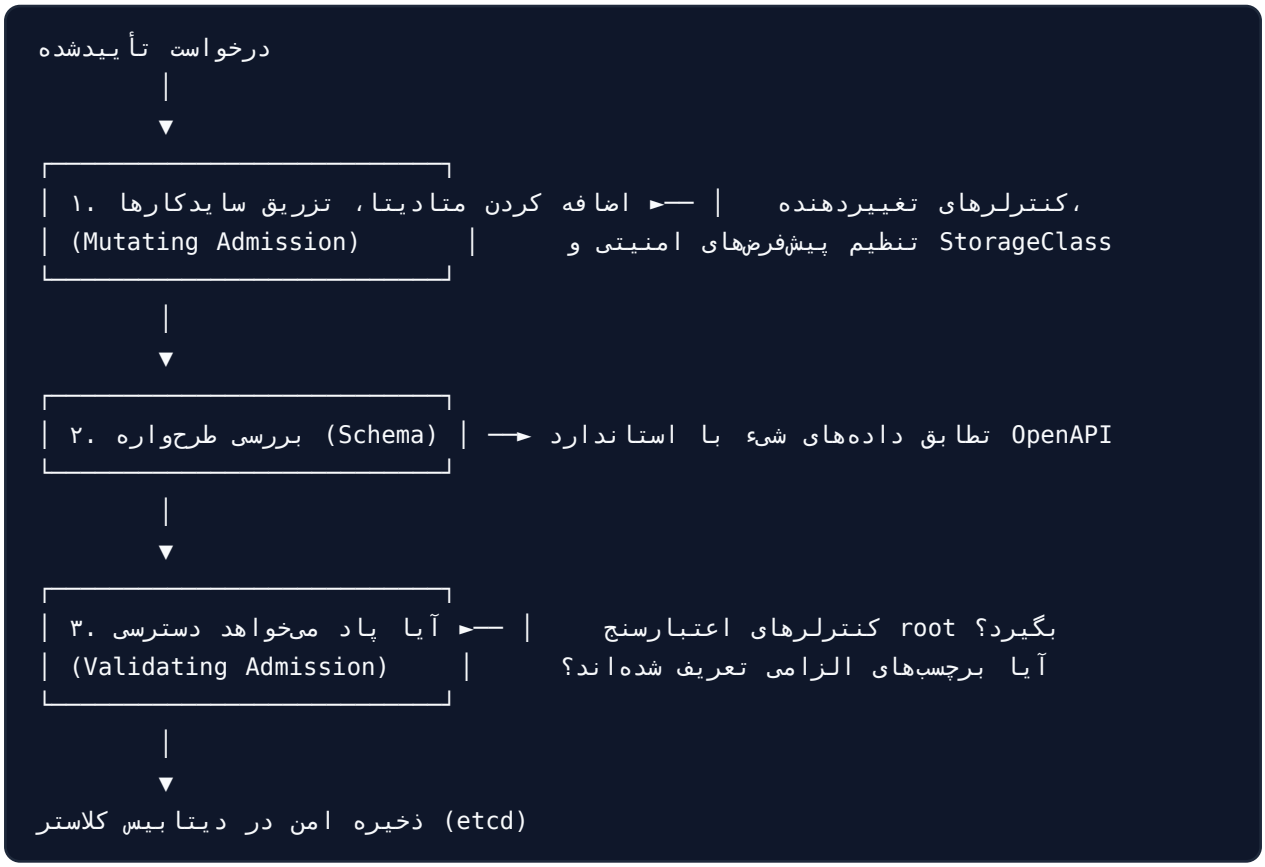
—

۳. مرحله سوم: کنترل پذیرش (Admission Control)

اگر درخواست شما از دو گذرگاه احراز هویت و مجازشماری با موفقیت عبور کرد، آیا بلافاصله در etcd ذخیره می‌شود؟ **خیر! مرحله سوم کنترل پذیرش (Admission Control) است.**

کنترلرهای پذیرش نرم‌افزارهای پلاگین‌محوری هستند که می‌توانند درخواست را قبل از ذخیره نهایی بررسی، دستکاری (Mutate) و یا رد (Validate) کنند.

کنترل پذیرش در دو فاز پیاپی اجرا می‌شود:



۱. **کنترلرهای تغییردهنده (Mutating Controllers):** می‌توانند شیء را ویرایش کنند. نمونه مشهور آن تزریق خودکار پراکسی Envoy در کلاسترهای دارای Istio و Linkerd است. ۲. **کنترلرهای اعتبارسنج (Validating Controllers):** شیء نهایی را بازرسی کرده و در صورت نقض خط‌مشی‌ها (مثلاً استفاده از رجیستری‌های غیرمجاز یا درخواست کانتینر Privileged) درخواست را پس می‌زنند. ۳. **کنترلرهای پذیرش پویا (Webhook Admission Controllers):** کوبرنتیز به شما اجازه می‌دهد تا وب‌هوک‌های دلخواه بنویسید یا از موتورهای مدرن Policy-as-Code نظیر **Kyverno** و **OPA Gatekeeper** برای وضع قوانین سفارشی سازمانی بهره ببرید.

—

۴. کارگاه عملیاتی: پیاده‌سازی کاربردی RBAC

بیایید گام‌به‌گام یک محیط امن ایزوله بسازیم و اختیارات را بسنجیم.

گام ۱: ساخت نیم‌اسپیس و حساب سرویس

ابتدا یک نیم‌اسپیس به نام `rbac-lab` و یک ServiceAccount به نام `dev-deployer` ایجاد می‌کنیم:

```
kubectl create namespace rbac-lab
kubectl create serviceaccount dev-deployer -n rbac-lab
```

گام ۲: تعریف یک نقش محلی (Role)

می‌خواهیم این هویت تنها قادر باشد پادها و دیپلویمنت‌ها را مشاهده کند و فقط دیپلویمنت بسازد یا تغییر دهد، اما حق حذف نداشته باشد:

```
# role-dev.yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  namespace: rbac-lab
  name: deployment-manager
rules:
- apiGroups: [""]
  resources: ["pods", "pods/log"]
  verbs: ["get", "list", "watch"]
- apiGroups: ["apps"]
  resources: ["deployments"]
  verbs: ["get", "list", "watch", "create", "update", "patch"]
```

اعمال فایل:

```
kubectl apply -f role-dev.yaml
```

گام ۳: اتصال نقش به ServiceAccount با RoleBinding

```
# rolebinding-dev.yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: bind-dev-deployer
  namespace: rbac-lab
subjects:
- kind: ServiceAccount
  name: dev-deployer
  namespace: rbac-lab
roleRef:
  kind: Role
  name: deployment-manager
  apiGroup: rbac.authorization.k8s.io
```

اعمال پیوند:

```
kubectl apply -f rolebinding-dev.yaml
```

گام ۴: آزمون اختیارات با دستور جادویی `kubectl auth can-i`

دستور فوق‌العاده کاربردی `kubectl auth can-i` به مدیران امکان می‌دهد تا بدون لاگین کردن با کاربری دیگر، سناریوهای دسترسی را شبیه‌سازی و بررسی کنند:

```
# آیا اکانت dev-deployer می‌تواند در نیم‌اسپیس rbac-lab دیپلویمنت بسازد؟
kubectl auth can-i create deployment \
  --as=system:serviceaccount:rbac-lab:dev-deployer \
  -n rbac-lab
# پاسخ: yes

# آیا می‌تواند پادی را در این نیم‌اسپیس حذف کند؟
kubectl auth can-i delete pods \
  --as=system:serviceaccount:rbac-lab:dev-deployer \
  -n rbac-lab
# پاسخ: no

# آیا می‌تواند در نیم‌اسپیس پیش‌فرض (default) پادها را ببیند؟
kubectl auth can-i get pods \
  --as=system:serviceaccount:rbac-lab:dev-deployer \
  -n default
# پاسخ: no
```

همان‌طور که مشاهده می‌کنید، اصل دسترسی حداقلی با کمال دقت اعمال شده است.

—

۵. ده فرمان طلایی برای برقراری امنیت API در پروداکشن

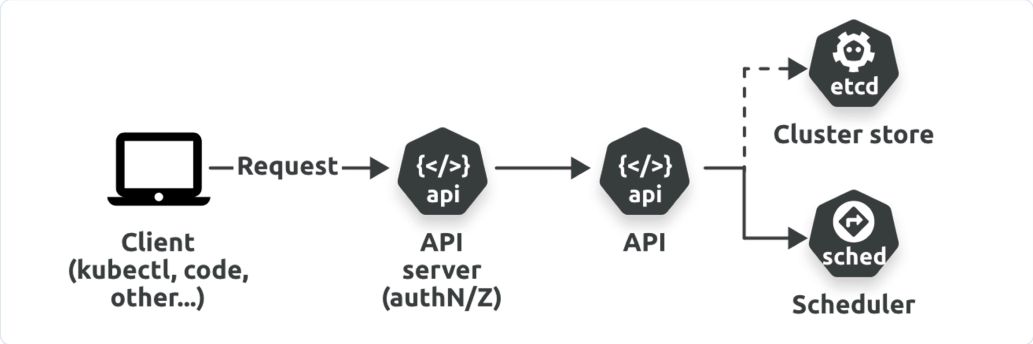
۱. هیچ‌گاه از کاراکتر ستاره (`*`) در قواعد RBAC پروداکشن استفاده نکنید. ۲. دستورات موقت با نقش‌های دارای تاریخ انقضا صادر شوند. ۳. دسترسی به پورت ناامن سرور API کاملاً حذف شده و باید فقط ارتباطات TLS نسخه ۱.۳ برقرار باشد. ۴. گزینه `automountServiceAccountToken: false` را روی پادهایی که نیازی به صحبت با سرور API ندارند فعال کنید. ۵. از اعطای دسترسی‌های `escalate` و `bind` در RBAC خودداری نمایید؛ زیرا مهاجم می‌تواند با آن‌ها سطح اختیارات خود را ارتقا دهد. ۶. دسترسی به رازها (`secrets`) را فقط به کامپوننت‌های به شدت تاییدشده محدود کنید، چرا که خواندن سکرت‌ها مساوی با دستیابی به کلیدهای کلاستر است. ۷. کنترلر پذیرش NodeRestriction را فعال نگه دارید تا نودهای آلوده نتوانند برچسب‌ها و تنظیمات خارج از قلمرو خود را تغییر دهند. ۸. از ابزارهای متن‌باز مانند `rback` و `rakkess` برای ممیزی بصری ماتریس دسترسی‌ها در کلاستر استفاده کنید. ۹. سیاست‌های امنیتی پاد را با ابزار استاندارد Pod Security Standards (PSS) در سه پروفایل Baseline، Privileged و Restricted پیکربندی نمایید. ۱۰. تمامی لاگ‌های دسترسی سرور API (Audit Logs) را بی‌درنگ به یک مخزن مرکزی و غیرقابل‌تغییر (مانند Elasticsearch یا SIEM) ارسال نمایید.

فصل ۱۵: کالبدشکافی سرور API کوبرنتیز (The Kubernetes API)

اگر کوبرنتیز را به یک ارکستر سمفونیک بزرگ تشبیه کنیم، سرور API (`kube-apiserver`) هم رهبر ارکستر است و هم سالن ورودی آن. این سرور تنها کامپوننتی است که با پایگاه داده کلاستر (`etcd`) به صورت مستقیم گفتگو می‌کند؛ هیچ جزئی—نه Kubelet، نه Scheduler، نه Controller Manager و نه کاربران—حق و توانایی لمس مستقیم etcd را ندارند.

سرور API کوبرنتیز یک واسط برنامه‌نویسی نرم‌افزارِ مبتنی بر استاندارد **REST**، اعلانی (Declarative) و به شدت ماژولار است که کلاستر کوبرنتیز را به یکی از انعطاف‌پذیرترین پلتفرم‌های ابری تاریخ مهندسی کامپیوتر بدل ساخته است.

در این فصل، معماری درونی سرور API، ساختار آدرس‌دهی REST، مفاهیم کلیدی GVK و GVR و مکانیزم‌های قدرتمند توسعه‌پذیری مانند CRD و الگوی اپراتور (Operator Pattern) را بررسی خواهیم کرد.

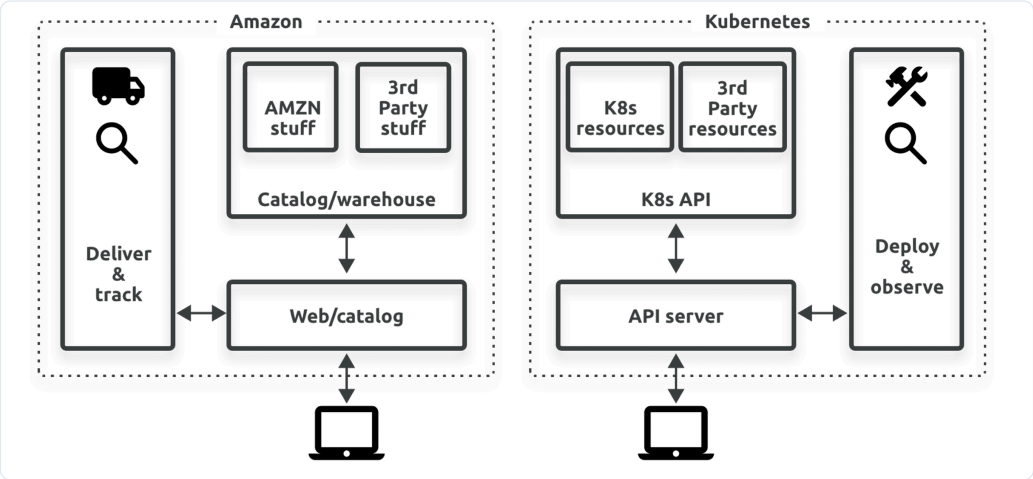


تصویر ۱۵.۱: جریان کلی و موقعیت محوری سرور API در دریافت، اعتبارسنجی، ذخیره‌سازی و زمان‌بندی اشیا در کلاستر

—

۱. مقایسه مفهومی: کوبرنتیز به مثابه یک فروشگاه بزرگ مدرن

برای درک بهتر چرخه حیات درخواست‌ها، نایجل پولتون این فرآیند را به سیستم یک فروشگاه خرید آنلاین (نظیر آمازون یا دیجی‌کالا) تشبیه می‌کند (تصویر ۱۵.۲):



تصویر ۱۵.۲: تشبیه معماری سرور API کوبرنتیز به سیستم سفارش‌گذاری و انبارداری یک فروشگاه آنلاین

بخش فروشگاه	معادل در کوبرنتیز	وظیفه و ماهیت
کاتالوگ محصولات	طرح‌واره API (API Schema)	تعریف اشیای مجاز و مشخصات فیلدهای آن‌ها
سبد خرید و ثبت سفارش	درخواست اعلامی (Declarative) (YAML	اعلام وضعیت مطلوب توسط کاربر به سرور API
پایگاه داده سفارشات	مخزن کلاستر (etcd)	ذخیره نسخه سریالی‌شده (JSON/ Protobuf) وضعیت اشیا
سیستم تخصیص انبار	زمان‌بند (Kube-Scheduler)	انتخاب مناسب‌ترین سرور فیزیکی/مجازی برای استقرار
کارگران انبار و ارسال	عامل نود (Kubelet و Container Runtime)	اجرای فیزیکی کانتینرها روی سرور بر اساس سفارش
تیم بررسی و جبران کسری	کنترلرها (Controllers)	تطبیق دائم وضعیت واقعی با وضعیت اعلام‌شده در سفارش

—

۲. معماری داخلی kube-apiserver: بدون‌وضعیت و اسکیل‌پذیر (Scalable)

برخلاف etcd که به شدت حساس به تاخیر دیسک و نیازمند سازوکار اجماع رافت (Raft) است، خود سرور API **کاملاً بدون‌وضعیت (Stateless)** طراحی شده است.

این ویژگی حیاتی به مهندسان زیرساخت امکان می‌دهد تا در کلاسترهای پروداکشن با ترافیک میلیونی، چندین نمونه همزمان از `kube-apiserver` را در کنار هم برپا کنند و یک لودبالانسر شبکه (NLB یا HAProxy) جلوی آن‌ها قرار دهند. تمامی نودها و کلاینت‌ها بدون نگرانی از وضعیت، به هر کدام از نسخه‌های API Server متصل می‌شوند.

پروتکل‌ها و قالب‌های داده

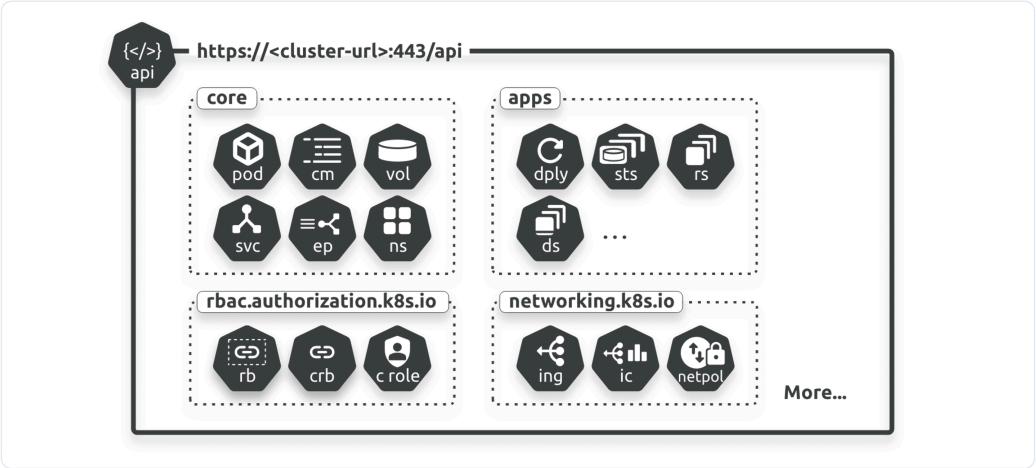
- **HTTP/REST:** کلیه درخواست‌ها از طریق HTTPS استاندارد منتقل می‌گردند.

• **JSON و Protocol Buffers:** در حالی که کاربران با فایل‌های خوانای YAML با `kubectl` تعامل دارند، کلاینت داخلی `kubectl` فایل را به JSON تبدیل می‌کند. همچنین ارتباطات پرسرعت داخلی میان کامپوننت‌های کنترل پلین بر بستر فرمت باینری فوق‌العاده سریع **Protocol Buffers (protobuf)** صورت می‌پذیرد.

—

۳. ساختار آدرس‌دهی REST و مفاهیم بنیادین GVK و GVR

یکی از زیبایی‌های مهندسی کوبرنتیز، دسته‌بندی منظم و سلسله‌مراتبی میلیاردها نوع منبع مختلف در ساختاری پیش‌بینی‌پذیر به نام گروه‌های API است.



تصویر ۱۵.۳: دسته‌بندی منظم منابع کوبرنتیز در گروه‌های مجزای API

برای کار با سرور API، باید دو اصطلاح فنی زیر را بشناسید:

• **GVK (Group, Version, Kind):** قالبی که در فایل‌های مانیفست YAML و کدهای Go برای تشخیص نوع شیء به کار می‌رود (مثلاً `apps/v1` با نوع `Deployment`).

• **GVR (Group, Version, Resource):** مسیر آدرس‌دهی فیزیکی HTTP بر روی وب‌سرور API (مثلاً `/apis/apps/v1/namespaces/default/deployments`).

تفاوت میان Kind و Resource

• **نوع (Kind):** نام مفرد، پاسکال‌کیس (PascalCase) و ساختار معنایی شیء در مانیفست است؛ مثل `Pod` یا `StorageClass` .

• **منبع (Resource):** نام جمع، حروف کوچک و آدرس واقعی REST در URL است؛ مثل `pods` یا `storageclasses` .

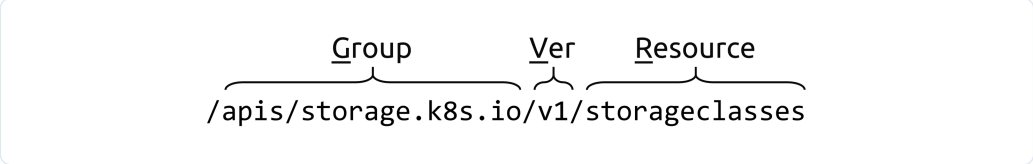
مسیردهی گروه هسته (Core) در برابر گروه‌های نام‌گذاری‌شده (Named Groups)

۱. **گروه هسته یا میراثی (Core / Legacy Group):** منابع اولیه کوبرنتیز مانند پادها، سرویس‌ها، نیم‌اسپیس‌ها و کانفیگ‌مپ‌ها در مسیر ساده زیر آدرس‌دهی می‌شوند (فاقد نام گروه در URL هستند):

```
/api/v1/namespaces/{namespace}/pods
```

۲. **گروه‌های نام‌گذاری‌شده (Named Groups):** تمامی منابع مدرن‌تر و تکمیلی کوبرنتیز در زیرمسیر `/apis/` قرار دارند:

```
/apis/{api-group}/{version}/namespaces/{namespace}/{resources}
```



تصویر ۱۵.۴: تجزیه مسیر REST برای منبع StorageClass در گروه storage.k8s.io با نسخه v1

تصویر ۱۵.۴ مسیر مستقیم به منبع `storageclasses` را نشان می‌دهد:

```
/apis/storage.k8s.io/v1/storageclasses
```

چرخه حیات نسخه‌ها (Stability Levels)

کوبرنتیز برای جلوگیری از بروز ناپایداری در نسخه‌های پروداکشن، یک خط‌مشی سفت‌وسخت سه‌مرحله‌ای برای نسخه‌ها دارد:

- `v1alpha1` , `v1alpha2` : ویژگی‌های آزمایشی، ممکن است در نسخه‌های بعدی بدون اعلام قبلی تغییر کنند یا حذف شوند.
- `v1beta1` , `v1beta2` : ویژگی‌های پایدارشده با آزمون‌های گسترده؛ رفتارهای عمومی حفظ می‌شوند و در مسیر ارتقا به نسخه اصلی هستند.
- `v1` : نسخه نهایی و پایدار (Production Ready) با تعهد ماندگاری طولانی‌مدت سازگاری معکوس.

—

۴. مکانیزم‌های Watch و پردازش رویدادمحور

آیا کنترلرها و Kubelet برای اطلاع از تغییرات مدام سرور API را پول (Poll) می‌کنند و پهنای باند را می‌سوزانند؟ **قطعاً خیر!**

کوبرنتیز از مکانیزم هوشمند و رویدادمحور **Watch** بهره می‌برد: ۱. کلاینت یک درخواست HTTP GET به سرور API ارسال می‌کند و پارامتر `?watch=true` را همراه با آخرین نسخه تغییرات (`resourceVersion`) می‌فرستد. ۲. سرور API اتصال HTTP را باز نگه می‌دارد (با استاندارد Chunked Transfer Encoding). ۳. به محض اضافه شدن، تغییر یا حذف یک شیء در etcd، سرور API یک استریم با فرمت JSON حاوی نوع رویداد (`ADDED` , `MODIFIED` , `DELETED`) را مستقیماً روی سوکت باز کلاینت ارسال می‌کند.

این ویژگی کلاستر را قادر می‌سازد تا در کسری از ثانیه به مرگ کانتینرها یا اضافه شدن درخواست‌های جدید اسکیل (Scale) واکنش نشان دهد.

—

۵. گسترش‌پذیری سرور API: کاستوم ریسورس‌ها (CRDs) و الگوی اپراتور (Operator)

قدرت اصلی کوبرنتیز در دهه اخیر، تبدیل شدن آن از یک ابزار صرف اجرای کانتینر به یک **پلتفرم مدیریت هر نوع زیرساخت نرم‌افزاری** بوده است. چگونه می‌توان به کوبرنتیز شیء جدیدی آموخت که تا کنون نمی‌شناخته است؟

۱. تعاریف منابع سفارشی (Custom Resource Definitions یا CRD)

یک CRD به زبان ساده به سرور API می‌گوید: «از این لحظه به بعد یک شیء جدید به نام `DatabaseCluster` وجود دارد که فیلدهای آن شامل نسخه، سایز رم و تعداد رپلیکاهاست.»

سرور API به صورت آنی و خودکار مسیرهای REST جدید، ولیدیشن‌های طرح‌واره با OpenAPI v3، ذخیره‌سازی در etcd و پشتیبانی از `kubectl` را برای این منبع جدید مهیا می‌کند.

۲. کنترلر سفارشی و الگوی اپراتور (Operator Pattern)

داشتن یک CRD به تنهایی مانند نوشتن یک قانون روی کاغذ است. شیء درون دیتابیس ذخیره می‌شود، اما تا زمانی که یک فرآیند هوشمند ناظر آن نباشد اتفاقی در دنیای واقعی رخ نمی‌دهد.

الگوی اپراتور از ترکیب دو مفهوم ساخته می‌شود:

```
الگوی اپراتور (Operator) = منبع سفارشی (CRD) + کنترلر اختصاصی (Custom Controller)
```

یک کنترلر سفارشی کدی است که به زبان Go یا Python نوشته شده، روی کلاستر در قالب پاد اجرا می‌شود و با مکانیزم Watch رویدادهای CRD مربوطه را مانیتور می‌کند:

- اگر کاربری نوشت: `kind: PostgresCluster, replicas: 3`، اپراتور به طور خودکار دیسک‌های ابری را می‌سازد، فایل‌های پیکربندی رپلیکیشن را ایجاد می‌کند، پشتیبان‌گیری دوره‌ای راه می‌اندازد و در صورت مرگ مستر، به صورت خودکار یکی از رپلیکاها را پروموت می‌نماید!

—

۶. کارگاه عملیاتی: جست‌وجو در API و ساخت یک منبع سفارشی (CRD)

گام ۱: بازرسی مستقیم مسیرهای خام API

با دستور `kubectl get --raw` می‌توانید دقیقاً مانند ابزار curl مسیرهای سرور API را کاوش کنید:

```
# فهرست کلیه گروه‌های API موجود
kubectl get --raw /apis | jq .groups[].name

# فهرست منابع موجود در گروه storage.k8s.io نسخه v1
kubectl get --raw /apis/storage.k8s.io/v1 | jq .resources[].name
```

گام ۲: ایجاد یک CRD واقعی به نام `AppRelease`

بیایید یک موجودیت جدید برای مدیریت نسخه‌های نرم‌افزاری سازمان در کلاستر ثبت کنیم:

```yaml
# crd-apprelease.yaml
apiVersion: apiextensions.k8s.io/v1
kind: CustomResourceDefinition
metadata:
  name: appreleases.devops.company.ir
spec:
  group: devops.company.ir
  versions:
    - name: v1
      served: true
      storage: true
      schema:
        openAPIV3Schema:
          type: object
          properties:
            spec:
              type: object
              required: ["appName", "version", "environment"]
              properties:
                appName:
                  type: string
                version:
                  type: string
                environment:
                  type: string
                  enum: ["staging", "production"]
  scope: Namespaced
  names:
    plural: appreleases
    singular: apprelease
    kind: AppRelease
    shortNames:
    - ar
```

اعمال CRD در کلاستر:

```
kubectl apply -f crd-apprelease.yaml
```

بلافاصله می‌توانید ببینید که کوبرنتیز نوع جدید را شناخت:

```
kubectl api-resources | grep appreleases
```

گام ۳: ساخت نمونه‌ای از این شیء جدید

اکنون هر توسعه‌دهنده‌ای می‌تواند یک شیء اعلانی با ساختار بالا در کلاستر بسازد:

```
# my-release.yaml
apiVersion: devops.company.ir/v1
kind: AppRelease
metadata:
  name: payment-service-v2
spec:
  appName: payment-api
  version: "2.4.1"
  environment: production
```

اعمال و مشاهده با ابزارهای استاندارد:

```
kubectl apply -f my-release.yaml
kubectl get ar
kubectl describe ar payment-service-v2
```

بدین ترتیب، کوبرنتیز به شکل پویا و زنده گسترش یافت، بدون آنکه حتی یک ثانیه سرور API ری‌استارت گردد!

—

۷. جمع‌بندی نکات کلیدی

- سرور API یک معماری اعلانی، بدون‌وضعیت و لایه‌بندی‌شده دارد.
- تمامی ارتباطات خارجی و داخلی از درگاه امن HTTPS و REST عبور می‌کنند.
- اشیا با دو ساختار GVK (در کد و فایل) و GVR (در مسیرهای HTTP) مدیریت می‌شوند.
- مکانیزم Watch اتصالات استریمی بلادرنگ را برای واکنش به تغییرات فراهم می‌آورد.
- با استفاده از CRD و اپراتورها، کوبرنتیز به بستری بی‌همتا برای خودکارسازی عمیق عملیات دوام‌پذیر و پیشرفته تبدیل می‌شود.

فصل ۱۶: مدل‌سازی تهدیدات در کوبرنتیز با چارچوب STRIDE (Threat Modeling)

امنیت در سیستم‌های ابری و کانتینری هرگز یک راه‌حل مقطعی، ابزار جانبی یا چک‌لیست نهایی قبل از انتشار نیست؛ بلکه یک **فرآیند مهندسی مداوم** است. سیستم‌های توزیع‌شده پیچیده‌ای مانند کوبرنتیز به دلیل تعدد اجزا (گره‌ها، شبکه، سرور API، کانتینرها و پایگاه داده etcd) سطح حمله (Attack Surface) گسترده‌ای دارند.

اگر منتظر بمانید تا پس از وقوع حادثه امنیتی و نفوذ هکرها آسیب‌پذیری‌ها را کشف کنید، هزینه بازیابی غیرقابل جبران خواهد بود. راه‌حل پیشگیرانه و علمی برای این چالش، **مدل‌سازی سیستماتیک تهدیدات (Threat Modeling)** است.

این فصل که با مشارکت پوشکار جوگلکار (Pushkar Joglekar، نویسنده و متخصص سرشناس امنیت کوبرنتیز) تدوین شده است، چارچوب جهانی **STRIDE** را به عنوان یک استاندارد تحلیل ریسک بر روی تک‌تک اجزای کلاستر پیاده‌سازی می‌کند.

—

۱. چرا مدل‌سازی تهدیدات و دفاع در عمق (Defense in Depth)؟

مدل‌سازی تهدیدات فرآیندی ساختاریافته است که در آن مهندسان قبل از استقرار سیستم، این سه پرسش حیاتی را مطرح می‌کنند: ۱. **چه چیزی ممکن است در این معماری دچار نقص یا خرابکاری شود؟** ۲. **اگر یک مهاجم فرضی بتواند به فلان بخش نفوذ کند، چه خساراتی وارد خواهد شد؟** ۳. **برای کاهش، دفع یا بی‌اثر ساختن این تهدید چه تمهیداتی باید پیش‌بینی کنیم؟**

رویکرد استاندارد در مهندسی مدرن، پایبندی به اصل **دفاع در عمق (Defense in Depth)** است؛ یعنی فرض بر این است که یک لایه امنیتی قطعاً روزی شکسته خواهد شد و لایه‌های بعدی باید بتوانند مانع از سقوط کل سیستم شوند (لایه‌های کد، کانتینر، کلاستر و زیرساخت ابری).

—

۲. معرفی مدل شش‌گانه STRIDE

چارچوب **STRIDE** که در ابتدا توسط مایکروسافت تدوین شد، تمام تهدیدات امنیتی دنیای نرم‌افزار را به شش دسته بنیادین طبقه‌بندی می‌کند:

حرف	عنوان تهدید (Threat)	هدف امنیتی در برابر تهدید (Security Property)
S	جعل هویت (Spoofing)	احراز هویت قوی (Authentication)
T	دستکاری و تخریب داده (Tampering)	حفظ تمامیت داده‌ها (Integrity)
R	انکارناپذیری (Repudiation)	پاسخگویی و ثبت ردپا (Non-Repudiation / Auditing)

I	افشای اطلاعات (Information Disclosure)	محرمانگی (Confidentiality / Encryption)
D	اختلال در سرویس‌دهی (Denial of Service - DoS)	دردسترس‌بودن پایدار (Availability)
E	ارتقای غیرمجاز دسترسی (Elevation of Privilege)	مجازشماری و کنترل حداقل اختیارات (Authorization)

بیایید هر یک از این تهدیدات را در دنیای کوبرنتیز کالبدشکافی کنیم.

—

۳. کالبدشکافی تهدیدات شش‌گانه بر روی اجزای کوبرنتیز

۱. جعل هویت (Spoofing)

تهدید در کوبرنتیز: مهاجم تظاهر می‌کند که یک کاربر مجاز، یک نود معتبر یا یک کامپوننت داخلی کلاستر است. برای نمونه، ثبت یک نود قلابی در کلاستر توسط مهاجم جهت دریافت بارهای کاری حاوی اطلاعات حساس، یا جعل آدرس‌های DNS داخلی.

راهکارهای مقابله:

- پیاده‌سازی گواهی‌های دیجیتال دوطرفه (Mutual TLS) برای تمام ارتباطات کنترل پلین با کارگرها.
- استفاده از پلاگین مجازشماری نود (`--authorization-mode=Node,RBAC`)؛ در این حالت هر Kubelet تنها حق دارد اطلاعات مربوط به پادهای مستقر روی خود را از API Server دریافت کند.
- کنترلر پذیرش `NodeRestriction` برای جلوگیری از دستکاری برچسب‌های نودها توسط Kubeletهای نفوذیافته.
- احراز هویت انسان‌ها از طریق پروتکل‌های OIDC سازمانی با احراز هویت چندعاملی (MFA).

۲. دستکاری (Tampering)

تهدید در کوبرنتیز: تغییر دادن مخرب تصاویر کانتینری در مخازن، دستکاری مستقیم پایگاه داده `etcd`، تزریق اسکریپت‌های آلوده به ConfigMapها یا دستکاری وب‌هوک‌های پذیرش.

راهکارهای مقابله:

- ایزوله‌سازی فیزیکی و شبکه‌ای `etcd`؛ هیچ کلاینتی جز خود سرور API نباید دسترسی شبکه به پورت ۲۳۷۹ داشته باشد.
- امضای دیجیتال و رمزنگاری تصاویر کانتینری با ابزار استاندارد **Cosign** (مجموعه ابزارهای Sigstore) و اعتبارسنجی خودکار امضاها قبل از استقرار.
- قفل کردن سیستم فایل ریشه کانتینرها با فلگ `readOnlyRootFilesystem: true`.

۳. انکارناپذیری (Repudiation)

تهدید در کوبرنتیز: کاربری با دسترسی معتبر یک دیتابیس ماندگار (PVC) را حذف می‌کند یا یک آسیب‌پذیری در مانیفست‌ها می‌کارد و سپس ادعا می‌کند: «من این کار را نکرده‌ام و مدرکی علیه من وجود ندارد!»

راهکارهای مقابله:

- فعال‌سازی سامانه حسابرسی جامع سرور API (`Kubernetes Audit Logging`).
- ارسال بلادرنگ لاگ‌های آدیت به مقاصد غیرقابل‌دستکاری (مانند Object Storage با قابلیت Write-Once-Read-Many یا سیستم‌های SIEM).
- تفکیک کامل دسترسی‌های فردی از حساب‌های کاربری اشتراکی.

۴. افشای اطلاعات (Information Disclosure)

تهدید در کوبرنتیز: نشت سکرت‌ها، خوانده شدن داده‌های محرمانه از حافظه پادهای مشترک روی یک گره، یا شنود ترافیک رمزنشده درون شبکه کلاستر؛ همچنین حملات SSRF به سرویس متادیتای ارائه‌دهندگان ابری در آدرس معروف `169.254.169.254` .

راهکارهای مقابله:

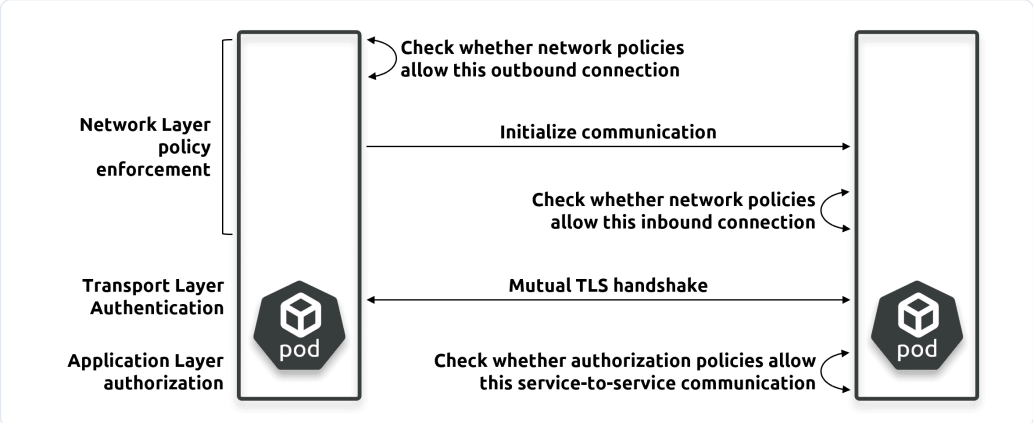
- رمزنگاری سکرت‌ها درون etcd با استفاده از قابلیت **KMS Encryption Provider** (کلیدهای رمزنگاری در HashiCorp Vault یا AWS KMS نگهداری شوند).
- مسدودسازی ترافیک دسترسی پادها به آدرس متادیتای ابری با استفاده از NetworkPolicy.
- رمزنگاری ترافیک شرق-به-غرب (Pod-to-Pod) با استانداردهای WireGuard و IPsec در Calico/Cilium یا به کارگیری Service Mesh با mTLS خودکار.

۵. ممانعت از سرویس‌دهی (Denial of Service - DoS)

تهدید در کوبرنتیز: یک کانتینر حریص یا نفوذیافته با مصرف بی‌رویه CPU و RAM، مکانیزم OOM-Killer هسته لینوکس را فعال کرده و باعث کشته شدن پادهای حیاتی همسایه روی نود می‌شود؛ یا ارسال طوفانی از درخواست‌های بی‌پایان به سرور API برای از کار انداختن آن.

راهکارهای مقابله:

- تعیین اجباری `resources.requests` و `resources.limits` روی تمام کانتینرها.
- فعال‌سازی قوانین `LimitRange` و `ResourceQuota` در نیم‌اسپیس‌ها.
- مکانیزم اولویت‌بندی و انصاف سرور API با نام **API Priority and Fairness (APF)** برای تضمین رسیدن درخواست‌های حیاتی سیستمی.



تصویر ۱۶.۱: دفاع چندلایه‌ای و عمیق در برابر حملات منع سرویس (DoS) از لایه شبکه تا کنترل پلین و برنامه‌ها

همان‌طور که در تصویر ۱۶.۱ مشاهده می‌شود، ترکیب لایه‌های حفاظت شبکه، سهمیه‌بندی منابع در سرور API و ایزولاسیون اپلیکیشن، مانع از فلج شدن کل کلاستر در زمان هجوم بارهای کاری مخرب می‌گردد.

۶. ارتقای غیرمجاز سطح دسترسی (Elevation of Privilege)

تهدید در کوبرنتیز: مهاجم از درون یک کانتینر محدود، با بهره‌گیری از آسیب‌پذیری‌های هسته لینوکس یا پیکربندی‌های فاجعه‌باری نظیر `privileged: true` یا مانت سوکت داکر (`/var/run/docker.sock`) به سرور میزبان (Host Breakout) نفوذ کرده و کنترل کامل نود و کلاستر را تصاحب می‌کند.

راهکارهای مقابله:

- مسدودسازی کامل اجرای کانتینرها با کاربر ریشه (`runAsNonRoot: true`).
- حذف تمامی قابلیت‌های اختیاری هسته لینوکس با قانون `capabilities.drop: ["ALL"]` .
- استقرار استانداردهای Pod Security Standards (PSS) با سطح سخت‌گیرانه `restricted` .
- ممنوعیت مانت فضاهای نام نود (`hostPID` , `hostIPC` , `hostNetwork`).

—

۴. ماتریس جامع STRIDE برای کوبرنتیز

جدول کاربردی زیر خلاصه راهبردی مقابله با تهدیدات در چرخه حیات کلاستر را ارائه می‌دهد:

تهدید	نقطه تمرکز خطر	پیامد نفوذ	اقدام مهندسی پیشگیرانه
Spoofing	سرور API و کنترلرها	ثبت گره یا کلاینت آلوده	mTLS و NodeRestriction
Tampering	ایمیج‌ها و پایگاه etcd	اجرای باج‌افزار و تغییر داده	امضای Cosign و دیسک ReadOnly
Repudiation	لاگ‌های سیستمی	فرار از مسئولیت خطاهای عمدی	Audit Logs غیرقابل‌دستکاری
Info Leak	سکرت‌ها و شبکه داخلی	سرقت کلیدهای خصوصی دیتابیس	رمزنگاری KMS و NetworkPolicy
DoS	حافظه و پردازنده گره‌ها	کرش کردن پادهای پروداکشن	LimitRange و APF
Privilege	کانتینر روت و مانت ریشه	خروج از کانتینر و تصاحب هاست	Pod Security Standards و Non-Root

—

۵. کارگاه عملیاتی: پیشگیری از بحران ارتقای دسترسی با Pod Security Standards

در نسخه‌های اخیر کوبرنتیز، استاندارد **Pod Security Standards (PSS)** جایگزین سیستم قدیمی و پیچیده PodSecurityPolicy (PSP) شده است. این استاندارد دارای سه پروفایل مشخص است: ۱. **Privileged:** کاملاً باز و بدون محدودیت (تنها برای مقاصد سیستمی). ۲. **Baseline:** تعادل میان راحتی توسعه و حداقل‌های امنیتی شناخته‌شده. ۳. **Restricted:** بالاترین درجه سخت‌گیری و رعایت کامل تمامی اصول Hardening.

گام ۱: فعال‌سازی برچسب‌های امنیتی روی نیم‌اسپیس

بیایید یک نیم‌اسپیس امن به نام `secure-zone` بسازیم و سطح `restricted` را به شکل اجباری (`enforce`) روی آن حاکم کنیم:

```
kubectl create namespace secure-zone

kubectl label namespace secure-zone \
  pod-security.kubernetes.io/enforce=restricted \
  pod-security.kubernetes.io/enforce-version=latest
```

گام ۲: تست نفوذ با ارسال یک پاد غیراستاندارد و ناامن

اکنون تستی انجام می‌دهیم و سعی می‌کنیم یک پاد با تنظیمات ناامن (اجرا با کاربر Root یا تلاش برای اخذ دسترسی‌های فراتر) در این نیم‌اسپیس مستقر کنیم:

```
# bad-pod.yaml
apiVersion: v1
kind: Pod
metadata:
  name: insecure-workload
  namespace: secure-zone
spec:
  containers:
  - name: hacker-box
    image: alpine
    command: ["sleep", "3600"]
    securityContext:
      privileged: true # درخواست دسترسی ویرانگر به کل کرنل!
```

ارسال به سرور API:

```
kubectl apply -f bad-pod.yaml
```

نتیجه اعتبارسنجی سرور API: سرور API بلافاصله این درخواست را در مرحله Admission Controller متوقف کرده و خطای زیر را صادر می‌کند:

```
Error from server (Forbidden): pods "insecure-workload" is forbidden:
violates PodSecurity "restricted:latest":
- privileged (container "hacker-box" must not set
securityContext.privileged=true)
- allowPrivilegeEscalation != false
- unrestricted capabilities
- runAsNonRoot != true
```

همان‌طور که ملاحظه می‌کنید، حتی اگر کاربری مجوز ساخت پاد در این نیم‌اسپیس را از طریق RBAC داشته باشد، کنترل پذیرش مانع از استقرار پاد مخاطره‌آمیز خواهد شد.

گام ۳: اصلاح پاد مطابق با استانداردهای امنیتی

اکنون پاد را به شکلی بازنویسی می‌کنیم که شایسته محیط پروداکشن باشد:

```
# secure-pod.yaml
apiVersion: v1
kind: Pod
metadata:
  name: hardened-workload
  namespace: secure-zone
spec:
  securityContext:
    runAsNonRoot: true
    runAsUser: 10001
    runAsGroup: 10001
    seccompProfile:
      type: RuntimeDefault
  containers:
  - name: safe-app
    image: alpine
    command: ["sleep", "3600"]
    securityContext:
      allowPrivilegeEscalation: false
      readOnlyRootFilesystem: true
      capabilities:
        drop:
        - ALL
```

اعمال پاد اصلاح‌شده:

```
kubectl apply -f secure-pod.yaml
```

خروجی:

```
pod/hardened-workload created
```

پاد بدون هیچ اخطاری با موفقیت در بالاترین سطح امنیتی به پرواز درآمد!

—

۶. جمع‌بندی

چارچوب STRIDE چشم‌اندازی جامع، علمی و قابل اندازه‌گیری برای سنجش سطح امنیت کلاسترها به دست می‌دهد. ترکیب مدل‌سازی تهدیدات در فاز معماری، همراه با پیکربندی دقیق سیاست‌های RBAC و اعمال استانداردهای PSS، کلاستر کوبرنتیز را به دژی مستحکم در برابر حملات سایبری بدل می‌سازد.

فصل ۱۷: امنیت کاربردی در دنیای واقعی (Real-World Kubernetes Security)

در فصل‌های گذشته مبانی احراز هویت، کنترل دسترسی RBAC و چارچوب مدل‌سازی تهدیدات STRIDE را آموختیم. اکنون در این فصل، تئوری را به میدان عمل پروداکشن پیوند می‌زنیم: **در سازمان‌های بزرگ و محیط‌های عملیاتی مدرن، امنیت کوبرنتیز چگونه در عمل پیاده‌سازی و تضمین می‌شود؟**

امنیت در دنیای واقعی یک رویکرد جامع، یکپارچه و زنجیره‌ای است که از لحظه نوشتن نخستین خط کد توسط برنامه‌نویس تا زمان اجرای باینری‌ها در حافظه نودها (از زنجیره تأمین تا مانیتورینگ بلادرنگ هسته) امتداد دارد.

در این فصل مباحث حیاتی زیر را بررسی خواهیم کرد: ۱. **امنیت زنجیره تأمین نرم‌افزار (Software Supply Chain): لایه‌بندی ایمیج‌ها، تصاویر پایه مینیمال و امضای دیجیتال با Cosign**. ۲. **ایزولاسیون بار کاری (Workload Isolation): تنظیمات AppArmor، Seccomp و استفاده از ران‌تایم‌های شنی نظیر gVisor و Kata Containers**. ۳. **ایزولاسیون شبکه در مقیاس سازمانی: تفاوت شبکه‌های Overlay با BGP و سیاست‌های میکروزگمنتیشن**. ۴. **تشخیص تهدیدات در زمان اجرا (Runtime Security) با موتور قدرتمند Falco**. ۵. **حسابرسی، انطباق و بنچمارک‌های امنیتی CIS**

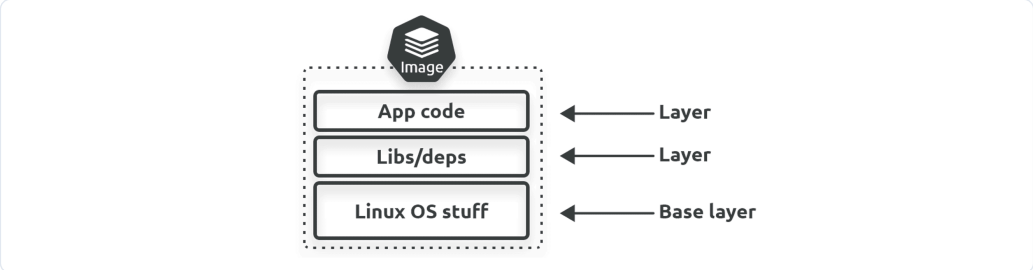
—

۱. امنیت زنجیره تأمین نرم‌افزار (Software Supply Chain Security)

بیش از ۸۰ درصد حوادث امنیتی در بستر کانتینرها ناشی از کد خود اپلیکیشن نیست، بلکه از پکیج‌های آلوده شخص ثالث، ایمیج‌های پایه حجیم با هزاران باگ شناخته‌شده (CVE) و دستکاری در پایپ‌لاین‌های CI/CD نشأت می‌گیرد.

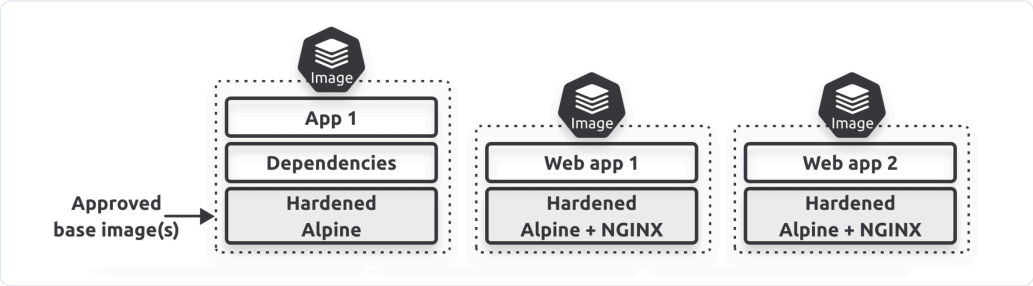
لایه‌بندی تصاویر کانتینری و کاهش سطح حمله

یک تصویر کانتینری از لایه‌های متعددی تشکیل شده است که روی یکدیگر چیده شده‌اند (تصویر ۱۷.۱):



تصویر ۱۷.۱: لایه‌بندی ساختاری یک ایمیج کانتینری از سیستم‌عامل پایه تا کتابخانه‌ها و کد اصلی برنامه

اگر به عنوان لایه پایه از توزیع‌های کامل مانند Ubuntu یا Debian با حجم صدها مگابایت استفاده کنید، پکیج‌هایی مانند `curl`، مفسر پایتون، مدیر پکیج `apt` و صدها ابزار کمکی دیگر را ناخواسته به محیط پروداکشن می‌برید؛ ابزارهایی که بهترین سلاح‌ها برای یک مهاجم به شمار می‌روند!



تصویر ۱۷.۲: بهره‌گیری از تصاویر پایه تاییدشده و امن (Approved Base Images) برای ساخت برنامه‌ها

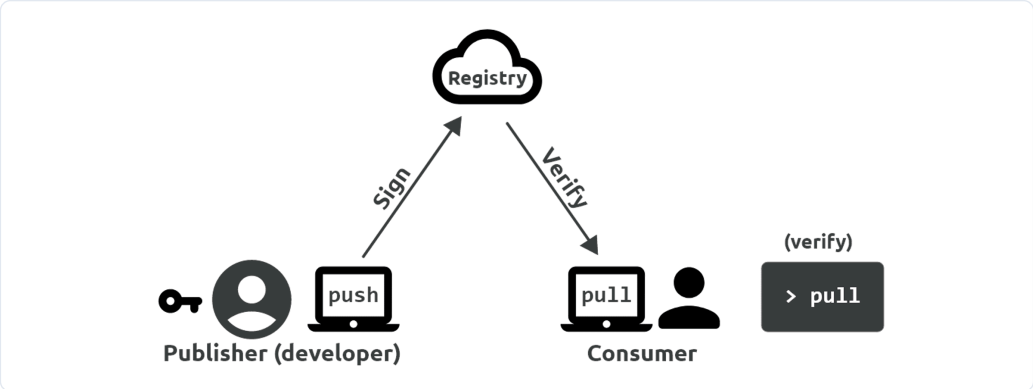
همان‌طور که در تصویر ۱۷.۲ نشان داده شده است، رویکرد بهینه سازمانی استفاده از ایمیج‌های پایه مشترک، مینیمال و پاکسازی‌شده است:

- **ایمیج‌های بدون توزیع (Distroless Images):** ساخته تیم گوگل؛ فاقد شل (Bash/Sh)، فاقد پکیج منیجر و شامل تنها برنامه شما و کتابخانه‌های وابسته به زمان اجرای آن.
- **Chainguard Images:** تصاویری کاملاً سخت‌سازی‌شده با نرخ صفر آسیب‌پذیری CVE.
- **ایمیج پایه `scratch` :** خالی‌ترین ایمیج ممکن برای برنامه‌های کامپایل‌شده استاتیک (مانند باینری‌های Go یا Rust).

امضا و اعتبارسنجی دیجیتال تصاویر با Cosign

چگونه مطمئن شویم ایمیجی که در کلاستر در حال اجراست، دقیقاً همان کدی است که توسط پایپ‌لاین رسمی سازمان ساخته شده و در مسیر دستکاری نشده است؟

پاسخ در **امضای دیجیتال رمزنگاری‌شده (Cryptographic Image Signing)** نهفته است (تصویر ۱۷.۳):



تصویر ۱۷.۳: فرآیند امضای ایمیج در پایپ‌لاین CI/CD و اعتبارسنجی خودکار آن توسط وب‌هوک پذیرش کلاستر

۱. ابزار CI (مثلاً GitHub Actions یا GitLab CI) تصویر را می‌سازد و اسکن امنیتی (Trivy/Grype) انجام می‌دهد. ۲. با استفاده از ابزار متن‌باز **Cosign** (از پروژه Sigstore)، ایمیج امضا می‌شود و امضای دیجیتال در کنار ایمیج در رجیستری ذخیره می‌گردد. ۳. در کلاستر کوبرنتیز، یک کنترلر پذیرش (مانند **Kyverno** یا **Ratify**) مستقر است. اگر کسی بخواهد ایمیج امضانشده یا ایمیجی با کلید نامعتبر را دیپلوی کند، درخواست در همان ثانیه اول رد می‌شود.

—

۲. ایزولاسیون بار کاری در سطح سیستم‌عامل و ران‌تایم (Workload Isolation)

کانتینرها ماشین‌های مجازی نیستند؛ آن‌ها فرآیندهایی روی هسته مشترک هاست هستند. بنابراین اگر هسته لینوکس مورد حمله قرار گیرد، تمام کلاستر به خطر می‌افتد. برای ایزولاسیون کامل، سه سنگر اصلی وجود دارد:

۱. فیلتر کردن فراخوان‌های سیستمی با Seccomp

یک کرنل لینوکس مدرن بیش از ۴۰۰ فراخوان سیستمی (Syscall) دارد، اما یک برنامه عادی شاید تنها به ۵۰ مورد از آن‌ها نیاز داشته باشد. قابلیت **Seccomp (Secure Computing Mode)** فراخوان‌های پرخطر یا غیرضروری را در سطح کرنل مسدود می‌کند.

از نسخه ۱.۲۷ کوبرنتیز، فعال‌سازی پروفایل پیش‌فرض `RuntimeDefault` به شدت توصیه می‌شود:

```
securityContext:
  seccompProfile:
    type: RuntimeDefault
```

۲. کنترل دسترسی اجباری با AppArmor و SELinux

ماژول‌های امنیتی لینوکس (LSM) نظیر AppArmor به شما امکان می‌دهند تا دسترسی کانتینرها را حتی در صورت روت بودن به فایل‌ها و عملیات شبکه خاص محدود کنید.

۳. محیط‌های اجرای شنی (Sandboxed Container Runtimes)

اگر کلاستر شما میزبان بارهای کاری چندمستاجره (Multi-Tenant) یا کدهای ناشناس و غیرقابل‌اعتماد کاربران است، ایزولاسیون اشتراکی کرنل لینوکس کافی نیست! در این موارد از استاندارد `RuntimeClass` و ران‌تایم‌های ایزوله استفاده می‌شود:

- **gVisor (`runsc`):** توسعه‌یافته توسط گوگل؛ کرنلی شبیه‌سازی‌شده در فضای کاربر (User-space) که تمامی فراخوان‌های سیستمی کانتینر را قبل از رسیدن به کرنل واقعی رهگیری و تصفیه می‌کند.
- **Kata Containers:** اجرای هر پاد درون یک میکرو ماشین مجازی فوق‌العاده سبک و پرسرعت (بر بستر QEMU یا Cloud-Hypervisor) با گارانتی سخت‌افزاری ایزولاسیون.

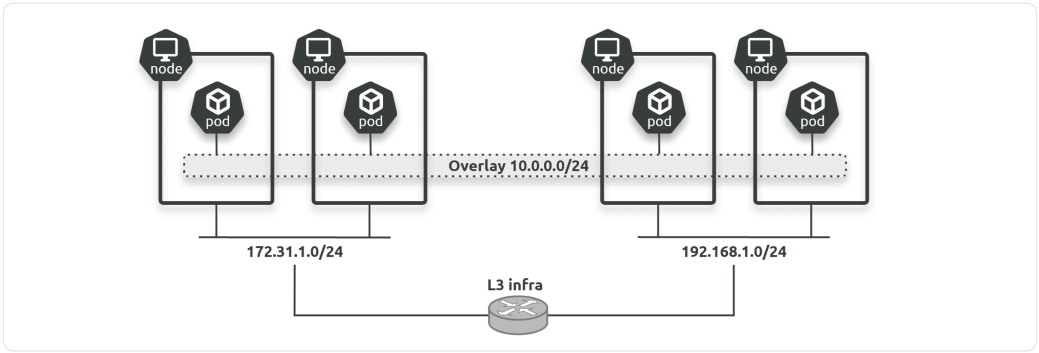
—

۳. ایزولاسیون شبکه و مدل‌های مسیریابی: Overlay در برابر BGP

شبکه کوبرنتیز ستون فقرات ارتباطات است. برای طراحی معماری امن و اسکیل‌پذیر (Scalable)، درک تفاوت دو مدل ارتباطی اصلی الزامی است:

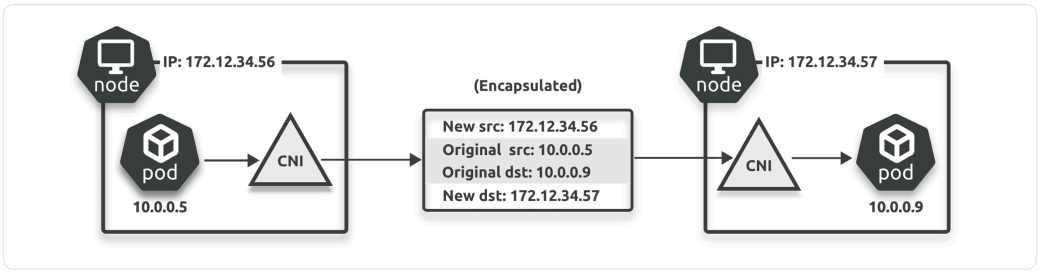
الف) شبکه‌های اورلی (Overlay Networks) با کپسوله‌سازی VXLAN

تصویر ۱۷.۴ چهار گره را نشان می‌دهد که روی دو شبکه فیزیکی مجزا (Subnet 1 و Subnet 2) قرار دارند:



تصویر ۱۷.۴: چهار نود کلاستر مستقر بر روی دو زیرشبکه فیزیکی مجزا

در شبکه‌های اورلی (نظیر Flannel پیش‌فرض یا تنظیمات پایه Calico)، بسته‌های اطلاعاتی پادها درون بسته‌های شبکه فیزیکی بسته‌بندی یا کپسوله (Encapsulate) می‌شوند (تصویر ۱۷.۵):

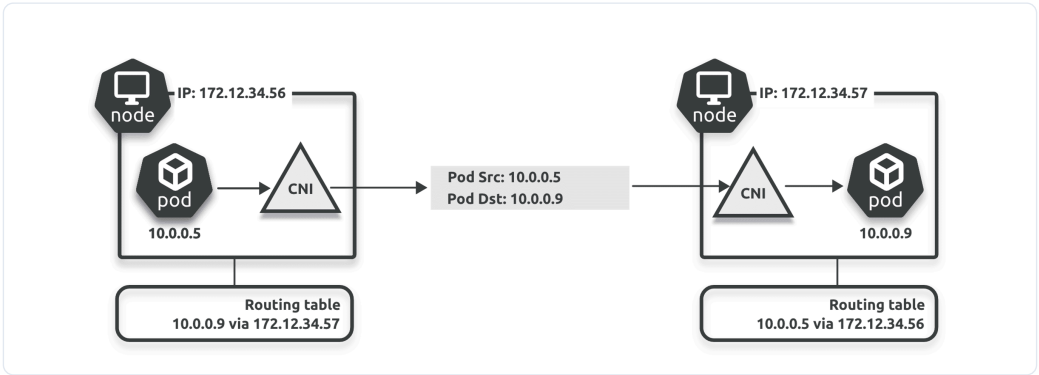


تصویر ۱۷.۵: کپسوله‌سازی بسته‌های پاد درون پروتکل VXLAN در شبکه‌های اورلی

مزیت: راه‌اندازی بسیار آسان و کارکرد روی هر زیرساخت و کلود بدون نیاز به هماهنگی با تیم زیرساخت شبکه. **عیب:** بار پردازشی کپسوله‌سازی (افت جزئی کارایی) و سخت‌تر شدن مانیتورینگ ترافیک در فایروال‌های فیزیکی دیتاسنتر.

ب) مسیریابی مستقیم با پروتکل BGP (Border Gateway Protocol)

در راهکارهای پیشرفته نظیر Calico یا Cilium، نودهای کوبرنتیز خود به مسیریاب‌های شبکه تبدیل می‌شوند و مسیر پادها را مستقیماً با سوئیچ‌ها و روترهای دیتاسنتر با BGP به اشتراک می‌گذارند (تصویر ۱۷.۶):



تصویر ۱۷.۶: مسیریابی مستقیم و بدون کپسوله‌سازی بسته‌ها بر بستر پروتکل اینترنتی BGP

مزیت: کارایی خیره‌کننده (Zero Overhead)، توان انتقال حداکثری داده‌ها و قابلیت ردیابی دقیق IP پادها در سیستم‌های مانیتورینگ سازمانی.

میکروزگمنتیشن با NetworkPolicies

صرف نظر از مدل شبکه، کلیه نیم‌اسپیس‌ها و پادها باید با اعمال قوانین `NetworkPolicy` ایزوله شوند تا از حرکت عرضی (Lateral Movement) نفوذگران جلوگیری به عمل آید.

—

۴. امنیت در زمان اجرا (Runtime Security) با Falco

حتی با سخت‌ترین تنظیمات، ممکن است آسیب‌پذیری روز-صفرم (Zero-Day) رخ دهد و کانتینری نفوذ یابد. در این لحظه، **تشخیص ناهنجاری رفتاری در زمان اجرا (Runtime Security)** آخرین خط دفاعی است.

محبوب‌ترین و استانداردترین ابزار در این حوزه، پروژه متن‌باز **Falco** (پذیرفته‌شده در بنیاد CNCF) است.

```
هسته لینوکس (Linux Kernel)
        |
        ▼ [فراخوان‌های سیستمی و رویدادهای eBPF]
┌──────────────────────────────────────────┐
| موتور قوانین پالایشی فالکو (Falco Engine)  |
| - آیا شلی درون کانتینر باز شد؟ (bash/sh)  |
| - آیا فایلی در /etc/shadow خوانده شد؟     |
| - آیا اتصال مشکوک به اینترنت برقرار گردید؟ |
└──────────────────────────────────────────┘
        |
        ▼
ارسال فوری هشدار (Slack, Webhook, SIEM) یا مسدودسازی خودکار
```

فالکو با نشستن بر روی درایور هسته یا برنامه‌های فوق‌سریع **eBPF**، تک‌تک فراخوان‌های سیستمی را به صورت بلادرنگ بازرسی کرده و در صورت مشاهده رفتارهای مشکوک، هشدارهای امنیتی حیاتی صادر می‌کند.

—

۵. انطباق، حسابرسی و بنچمارک‌های CIS

برای ارزیابی و سنجش میزان استحکام پیکربندی کلاستر در برابر هک، بنیاد امنیت اینترنت **Center for Internet Security (CIS)** استاندارد دقیقی به نام **CIS Kubernetes Benchmark** منتشر کرده است.

ابزار متن‌باز `kube-bench` به صورت خودکار تمامی تنظیمات نودها، فایل‌های پیکربندی etcd، پارامترهای سرور API و دسترسی‌های Kubelet را با این استاندارد مقایسه کرده و گزارش قبولی یا مردودی (Pass/Fail) ارائه می‌دهد:

```
# در قالب یک جاب کوبرنتیز kube-bench اجرای خودکار ارزیابی با
kubectl apply -f https://raw.githubusercontent.com/aquasecurity/kube-bench/main/
job.yaml
```

—

۶. کارگاه عملیاتی: امضای تصویر و اعمال سیاست پذیرش با Kyverno

بیایید یک سناریوی پروداکشن واقعی را شبیه‌سازی کنیم: **هیچ ایمیجی حق اجرا در کلاستر را ندارد مگر اینکه امضای تاییدشده سازمان را دارا باشد!**

گام ۱: تولید کلیدهای رمزنگاری با Cosign

```
# نصب و تولید جفت کلید خصوصی و عمومی
cosign generate-key-pair
# کلیدها تولید می‌شوند: cosign.key (محرمانه در CI) و cosign.pub (عمومی در کلاستر)
```

گام ۲: امضای ایمیج در رجیستری

```
cosign sign --key cosign.key docker.io/mycompany/secure-api:v1.0.0
```

گام ۳: اعمال سیاست پذیرش در کلاستر با Kyverno

سیاست زیر به سرور API اعلام می‌کند که تنها ایمیج‌های امضاشده با کلید عمومی سازمان پذیرفته شوند:

```
# policy-verify-image.yaml
apiVersion: kyverno.io/v1
kind: ClusterPolicy
metadata:
  name: verify-image-signature
spec:
  validationFailureAction: Enforce
  webhookTimeoutSeconds: 30
  rules:
    - name: verify-signature
      match:
        any:
        - resources:
            kinds:
              - Pod
      verifyImages:
```

```
    - imageReferences:
      - "docker.io/mycompany/*"
      attestors:
      - entries:
        - keys:
            publicKeys: |-
              -----BEGIN PUBLIC KEY-----
              MFkwEwYHKoZIzj0CAQYIKoZIzj0DAQcDQgAE...
              -----END PUBLIC KEY-----
```

اعمال سیاست:

```
kubectl apply -f policy-verify-image.yaml
```

از این لحظه به بعد، هرگونه تلاش برای اجرای تصاویر دستکاری‌شده، حتی با دستور مستقیم مدیران، به طور خودکار مسدود و خنثی خواهد شد.

—

۷. جمع‌بندی نهایی امنیت در دنیای واقعی

- امنیت یک چرخه پیوسته است: از ایمیج‌های مینیمال در فاز کد، تا امضای دیجیتال در فاز ساخت و اعتبارسنجی در فاز استقرار.
- بهره‌گیری از Seccomp و ران‌تایم‌های مدرن، لایه‌های حفاظتی عمیقی میان بارهای کاری و کرنل لینوکس پدید می‌آورد.
- درک تفاوت شبکه‌های Overlay و BGP برای تضمین کارایی و مانیتورینگ ترافیک در مقیاس سازمانی الزامی است.
- ابزارهایی مانند Falco و kube-bench چشم بینای تیم‌های امنیتی در کشف لحظه‌ای ناهنجاری‌ها و انطباق با بنچمارک‌های جهانی هستند.

فصل ۱۸: واژه‌نامه تخصصی اصطلاحات کوبرنتیز (Glossary & Terminology)

یکی از چالش‌های بزرگ مهندسان در یادگیری کوبرنتیز، حجم انبوه اختصارات، اصطلاحات نوین و واژگان تخصصی دنیای Cloud-Native است. این واژه‌نامه به عنوان یک مرجع سریع و استاندارد، تعاریف اصطلاحات پرتکرار کتاب را به همراه معادل‌های مصطلح فارسی و بینش‌های عملیاتی نایجل پولتون ارائه می‌دهد.

> **📌 نکته مهم: راهنمای استفاده از واژه‌نامه**
>
> اصطلاحات در جدول‌های موضوعی زیر دسته‌بندی شده‌اند تا هنگام مطالعه یا پروژه‌های عملیاتی بتوانید به سرعت مفهوم، نقش و جایگاه هر مفهوم را در معماری کلاستر بازیابی کنید.

۱. مفاهیم پایه، کنترل پلین و معماری کلاستر (Core Architecture)

اصطلاح انگلیسی	معادل فارسی	تعریف و بینش فنی نایجل پولتون
Control Plane	صفحه کنترل / هسته مدیریت	مجموعه سرویس‌های مدیریتی کلاستر (API Server، etcd، Scheduler و Controller Manager) که تصمیمات کلان کلاستر را اتخاذ می‌کنند.
Data Plane (Workers)	صفحه داده / گره‌های کارگر	نودهایی از کلاستر که بارهای کاری واقعی کاربران و کانتینرها را میزبانی و اجرا می‌کنند.
Cluster	کلاستر / خوشه رایانشی	مجموعه‌ای هماهنگ از ماشین‌های فیزیکی یا مجازی (شامل کنترل پلین و ورکرها) که با هم یک سیستم توزیع‌شده یکپارچه را می‌سازند.
kube-apiserver	سرور واسط برنامه‌نویسی	درگاه اصلی و تنها نقطه ورودی تعاملات با کلاستر که واسط RESTful اعلانی را روی HTTPS امن ارائه می‌دهد.
etcd	مخزن داده کلاستر	پایگاه داده کلید-مقدار توزیع‌شده، فوق‌سریع و منسجم که تمام وضعیت مطلوب و جاری کلاستر در آن به صورت ماندگار ذخیره می‌شود.
kube-scheduler	زمان‌بند کلاستر	سرویسی هوشمند که پادهای تازه‌متولدشده را بررسی کرده و بهترین گره را بر اساس منابع، تِینت‌ها و شرایط شبکه برای استقرار آن‌ها برمی‌گزیند.
kube-controller-manager	مدیر کنترلرهای کلاستر	یک باینری یکپارچه متشکل از ده‌ها حلقه کنترل مستقل که مدام وضعیت واقعی کلاستر را با وضعیت اعلام‌شده تطبیق می‌دهند.
cloud-controller-manager	مدیر کنترلرهای ابری	کنترلری اختصاصی برای اتصال کلاستر به APIهای سرویس‌دهنده ابری جهت ساخت خودکار دیسک، لودبالانسر و روتینگ شبکه.

Kubelet	عامل اجرایی گره	عامل هوشمندی که روی تک‌تک نودهای کلاستر اجرا می‌شود و وظیفه گزارش وضعیت گره و گفتگو با ران‌تایم کانتینری برای اجرای پادها را بر عهده دارد.
kube-proxy	پراکسی شبکه گره	پراکسی سبک شبکه روی هر نود که با ویرایش جداول iptables یا IPVS، ترافیک ارسالی به Serviceها را به پادهای پشتیبان هدایت می‌کند.
Container Runtime	محیط زمان اجرای کانتینر	نرم‌افزار سطح پایینی نظیر containerd یا CRI-O که مسئول دریافت ایمیج، ایجاد ایزولاسیون کرنل و اجرای فیزیکی کانتینرها است.
CRI (Container Runtime Interface)	واسط زمان اجرای کانتینر	استاندارد پلاگین‌محور gRPC که به Kubelet اجازه می‌دهد با هر محیط ران‌تایم کانتینری بدون وابستگی به کدی خاص تعامل کند.
Declarative Model	مدل اعلانی	رویکردی که در آن کاربر تنها «وضعیت مطلوب نهایی» (Desired State) را توصیف می‌کند و کلاستر وظیفه دارد آن را بسازد و همواره حفظ کند.

—

۲. بارهای کاری و منابع پردازشی (Workloads & Compute)

اصطلاح انگلیسی	معادل فارسی	تعریف و بینش فنی نایجل پولتون
Pod	پاد / پیله کانتینری	کوچک‌ترین و بنیادی‌ترین واحد محاسباتی قابل استقرار در کوبرنتیز که شامل یک یا چند کانتینر اشتراکی در شبکه و دیسک است.
Deployment	استقرار اعلانی / دیپلویمنت	شیء ناظر بر بارهای کاری بدون‌وضعیت (Stateless) که ارتقای پیوسته، اسکیل خودکار و بازگشت به نسخه‌های قبل را خودکار می‌کند.
ReplicaSet	مجموعه کپی‌ها	کنترلری زیربنایی که تضمین می‌کند همواره تعداد مشخصی از کپی‌های یکسان یک پاد در کلاستر در حال اجرا باقی بمانند.
StatefulSet	مجموعه دارای وضعیت	کنترلری پیشرفته برای استقرار پایگاه‌های داده کلاستری که هویت شبکه، نام‌گذاری ترتیبی و دیسک ماندگار اختصاصی هر پاد را تضمین می‌نماید.
DaemonSet	دیمن‌ست / اجرای همگانی	کنترلری که تضمین می‌کند یک نسخه از یک پاد معین (نظیر جمع‌آوری‌کننده لاگ یا ایجنت مانیتورینگ) روی تک‌تک نودهای کلاستر اجرا شود.
Job / CronJob	وظیفه متوقف‌شونده / جاب	بارهای کاری منقطع که بر خلاف وب‌سرورها، کاری مشخص را اجرا کرده، با موفقیت به پایان می‌رسند و کانتینر خود را متوقف می‌کنند.

Init Container	کانتینر مقدماتی / اول‌ساز	کانتینرهایی ویژه که پیش از اجرای کانتینر اصلی اپلیکیشن اجرا شده و پس از تکمیل وظایفی مانند مقداردهی دیتابیس پایان می‌یابند.
Sidecar Container	کانتینر سایدکار / کمکی	کانتینری جانبی درون همان پاد اصلی که خدماتی نظیر پراکسی شبکه، ارسال لاگ یا هماهنگی امنیتی را به کانتینر اصلی ارائه می‌دهد.

—

۳. شبکه، آدرس‌دهی و سرویس دیسکاوری (Networking)

اصطلاح انگلیسی	معادل فارسی	تعریف و بینش فنی نایجل پولتون
Service	سرویس کوبرنتیز	انتزاعی پایدار از شبکه شامل یک آدرس IP ثابت (ClusterIP) و نام DNS ماندگار که ترافیک کلاینت‌ها را به پادهای پویا لودبالانس می‌کند.
EndpointSlice	برش نقاط انتهایی	ساختار داده فوق‌العاده بهینه‌ای که جایگزین شیء قدیمی Endpoints شد تا امکان اسکیل شدن سرویس‌ها به ده‌ها هزار پاد بدون فشار به etcd فراهم گردد.
Headless Service	سرویس بدون سر	سرویسی با تنظیم `clusterIP: None` که ترافیک را لودبالانس نمی‌کند، بلکه آدرس‌های مستقیم IP تک‌تک پادها را در کوئری‌های DNS برمی‌گرداند.
CoreDNS	سرور نام داخلی کلاستر	سرویس DNS پرسرعت و پلاگین‌محور کلاستر که رجیستری پویا از آدرس‌های تمام سرویس‌ها و پادها نگهداری می‌کند.
Ingress	اینگرس / دروازه لایه ۷	مجموعه‌ای از قوانین اعلانی برای هدایت هوشمند ترافیک HTTP و HTTPS ورودی از بیرون کلاستر به سرویس‌های داخلی بر اساس دامنه و مسیر.
Ingress Controller	کنترلر اینگرس	برنامه و پراکسی معکوسی (نظیر Nginx ،Traefik یا Envoy) که قوانین مانیفست Ingress را در زمان واقعی خوانده و اجرا می‌کند.
NetworkPolicy	خط‌مشی شبکه / فایروال	قوانین فایروال محلی در لایه‌های ۳ و ۴ برای کنترل دسترسی ترافیک ورودی (Ingress) و خروجی (Egress) میان پادها و فضاهای نام.
CNI (Container Network Interface)	واسط شبکه کانتینر	استاندارد پلاگین شبکه که اتصال کارت شبکه مجازی پادها، تخصیص IP و مسیریابی میان نودها را توسط ابزارهایی مانند Calico یا Cilium مدیریت می‌کند.

—

۴. ذخیره‌سازی، پیکربندی و متادیتا (Storage, Config & Metadata)

اصطلاح انگلیسی	معادل فارسی	تعریف و بینش فنی نایجل پولتون
Volume	حجم ذخیره‌سازی	دایرکتوری قابل دسترسی برای کانتینرهای یک پاد با چرخه حیاتی که لزوماً وابسته به مرگ یا ری‌استارت تک‌تک کانتینرها نیست.
PersistentVolume (PV)	دیسک ماندگار کلاستر	قطعه‌ای از حافظه ذخیره‌سازی واقعی در زیرساخت (SAN, NAS, Cloud Disk) که توسط مدیر سیستم به صورت مستقل از پادها تعریف می‌شود.
PersistentVolumeClaim (PVC)	ادعای دیسک ماندگار	درخواست اعلامی توسعه‌دهنده برای دریافت دیسک با ظرفیت و حالت دسترسی مشخص، بدون نیاز به دانستن جزئیات فنی سخت‌افزار.
StorageClass	کلاس ذخیره‌سازی	شیء تعریف‌کننده الگوی تخصیص پویای دیسک (Dynamic Provisioning) بر بستر درایورهای مختلف ذخیره‌سازی.
CSI (Container Storage Interface)	واسط ذخیره‌سازی کانتینر	استاندارد باز رابط میان سیستم‌عامل کوبرنتیز و ارائه‌دهندگان تجهیزات ذخیره‌سازی.
ConfigMap	نقشه پیکربندی	شیء نگهداری تنظیمات غیرحساس متنی در قالب کلید-مقدار جهت تزریق به پاد به صورت متغیر محیطی یا فایل مانت‌شده.
Secret	راز / داده محرمانه	شیء نگهداری اطلاعات حساس نظیر رمزهای عبور، توکن‌ها و کلیدهای خصوصی به صورت امن و ایزوله.
Namespace	فضای نام / قلمرو مجازی	کلاستر مجازی درون کلاستر فیزیکی برای دسته‌بندی پروژه‌ها، تفکیک دسترسی‌ها و سهمیه‌بندی منابع.
Label & Selector	برچسب و انتخاب‌گر	سیستم بنیادین کوبرنتیز برای متصل کردن و گروه‌بندی اشیا به یکدیگر بر اساس زوج‌های کلید-مقدار معنایی.
Annotation	یادداشت متادیتا	اطلاعات تکمیلی غیرقابل‌جست‌وجو متصل به اشیا برای مصرف ابزارهای مانیتورینگ یا ابزارهای شخص ثالث.

—

۵. امنیت، احراز هویت و گسترش‌پذیری (Security & Extensibility)

اصطلاح انگلیسی	معادل فارسی	تعریف و بینش فنی نایجل پولتون
Authentication (AuthN)	احراز هویت	فرآیند اثبات کیستی هویت درخواست‌دهنده از طریق گواهی کلاینت، توکن یا سیستم‌های OIDC.
Authorization (AuthZ)	مجازشماری دسترسی	فرآیند بررسی اینکه آیا هویت تاییدشده اجازه اجرای فعل مورد نظر روی منبع درخواستی را دارد یا خیر.
RBAC (Role-Based Access Control)	کنترل دسترسی نقشی	سیستم امنیتی پیش‌فرض کوبرنتیز بر پایه حداقل اختیارات با استفاده از Roleها و RoleBindingها.
ServiceAccount	حساب سرویس	هویت دیجیتال رسمی برنامه‌ها و پادها برای گفتگو با سرور API کوبرنتیز.
Admission Controller	کنترلر پذیرش	پلاگین‌هایی در سرور API که پس از تأیید هویت و دسترسی، می‌توانند شیء را ویرایش یا اعتبار آن را رد کنند.

CRD (CustomResourceDefinition)	تعریف منبع سفارشی	قابلیت ثبت انواع اشیای جدید و سفارشی در سرور API بدون نیاز به بازکامپایل کوبرنتیز.
Operator Pattern	الگوی اپراتور	ترکیب یک منبع سفارشی (CRD) با یک کنترلر اختصاصی برای اتوماسیون کامل چرخه حیات نرم‌افزارهای پیچیده.
STRIDE	چارچوب مدل‌سازی STRIDE	روش مهندسی مایکروسافت برای تحلیل شش تهدید: جعل هویت، دستکاری، انکار، نشت اطلاعات، DoS و ارتقای دسترسی.
Pod Security Standards (PSS)	استاندارد امنیت پاد	سه پروفایل Baseline، Privileged و Restricted برای حاکمیت امنیتی بر کانتینرها در سطح نیم‌اسپیس.
Seccomp	فیلتر فراخوان‌های سیستمی	مکانیزم کرنل لینوکس برای مسدودسازی Syscallهای غیرضروری کانتینر و کاهش سطح حمله کرنل هاست.
WebAssembly (Wasm)	وب‌اسمبلی در سرور	فرمت باینری فشرده، امن و فوق‌العاده سریع برای اجرای بارهای کاری بدون‌کانتینر در کسری از میلی‌ثانیه.

فصل ۱۹: بخش پایانی و منابع (Outro & References)

۱. یادداشت پایانی نایجل پولتون (Outro)

از اینکه وقت گذاشتید و این کتاب را تا پایان خواندید بسیار سپاسگزارم. اکنون شما به دانش عمیق، مهارت‌های کاربردی و دیدگاه مفهومی مجهّز شده‌اید تا در دنیای جذاب و پرشتاب فناوری‌های بومی ابری (Cloud-Native) نه تنها گلیم خود را از آب بیرون بکشید، بلکه بدرخشید و پروژه‌های بزرگ سازمانی را با موفقیت رهبری کنید.

رازهای پنهان در طراحی جلد کتاب

من عاشق طرح روی جلد ویرایش ۲۰۲۵ این کتاب هستم و از صدها نفری که در نظرسنجی عمومی برای انتخاب این طرح شرکت کردند صمیمانه تشکر می‌کنم. تک‌تک عناصر روی جلد پیامی نمادین دارند:

- **کدهای YAML در سمت چپ:** بیانگر روح فنی، ساختار اعلانی و پیکربندی‌های کد-محور در معماری کوبرنتیز است.
- **چرخ سکان کشتی با ۷ پره در مرکز:** نماد رسمی پروژه کوبرنتیز (برگرفته از واژه یونانی به معنای «سکاندار کشتی») و یادآور پروژه افسانه‌ای Borg در شرکت گوگل است.
- **باران دیجیتالی آیکون‌ها در سمت راست:** سمبل‌های ابری به سبک باران کدهای سبز فیلم‌های خاطره‌انگیز «ماتریکس» طراحی شده‌اند.
- **پیام مخفی:** یک پیام محرمانه به زبان بورگ (Borg) از دنیای استار ترک در میان طرح جلد پنهان شده است که ادای احترامی به خاستگاه تاریخی مهندسی کوبرنتیز به شمار می‌رود!

درباره تصویرسازی‌ها و نمودارهای کتاب

جامعه کوبرنتیز مجموعه آیکون‌های بسیار خوبی در مخزن رسمی خود دارد. با این حال، آن نمادها هنگام چاپ فیزیکی یا مطالعه روی صفحات نمایش تیره کیفیت ایده‌آلی ارائه نمی‌دادند. از این رو، ماه‌ها زمان صرف شد تا یک مجموعه اختصاصی از نمودارها و آیکون‌های گرافیکی برداری طراحی شود تا درک جریان ترافیک، لایه‌های شبکه و رفتار کنترلرها در نگاه اول برای خواننده محقق گردد.

ارتباط با نایجل پولتون

خوشحال می‌شوم با شما در ارتباط باشم و درباره کوبرنتیز و آخرین ترندهای ابری گفتگو کنیم:

- **وب‌سایت رسمی:** `nigelpoulton.com`
- **لینکدین:** `linkedin.com/in/nigelpoulton`
- **توئیتر (X):** `twitter.com/nigelpoulton`
- **کانال یوتیوب:** `youtube.com/nigelpoulton`
- **ایمیل مستقیم جهت ارسال بازخورد، پرسش‌ها یا تصحیحات:** `tkb@nigelpoulton.com`

—

۲. نقشه راه گواهینامه‌های رسمی بنیاد لینوکس و CNCF

برای تثبیت آموخته‌های این کتاب و تقویت رزومه حرفه‌ای در مقیاس بین‌المللی، آزمون‌های رسمی زیر توصیه می‌شوند:

مدرک	عنوان رسمی	مخاطبان و سرفصل‌های آزمون
KCNA	Kubernetes & Cloud Native Associate	مبانی ابری، اصطلاحات کانتینری و آشنایی عمومی با اکوسیستم CNCF (چهارگزینه‌ای).
CKA	Certified Kubernetes Administrator	راهبری کلاستر، معماری، عیب‌یابی شبکه، ذخیره‌سازی، امنیت و کنترل پلین (کاملاً عملی و کنسولی).
CKAD	Certified Kubernetes Application Developer	طراحی و استقرار پادها، دیپلویمنت‌ها، سرویس‌ها، تنظیمات و ساخت کارهای دارای وضعیت برای برنامه‌نویسان.
CKS	Certified Kubernetes Security Specialist	تخصصی‌ترین سطح؛ امن‌سازی کلاستر، ایزولاسیون با AppArmor/Seccomp، اسکن ایمیج با Trivy، آدیت لاگ و ران‌تایم با Falco.

—

۳. محیط‌های تمرینی رایگان برای کارگاه‌های عملی

برای تمرین سناریوهای پیشرفته بدون صرف هزینه در سرورهای ابری: ۱. **Kind (Kubernetes in Docker):** راه‌اندازی سریع کلاسترهای چندگره‌ای روی کامپیوتر شخصی در چند ثانیه. ۲. **Minikube:** مناسب برای آزمایش افزونه‌های توکار نظیر اینگرس، متریگ‌ها و درایورهای ذخیره‌سازی محلی. ۳. **K3s:** توزیع سبک و رسمی کوبرنتیز برای سرورهای کم‌مصرف و محیط‌های توسعه سریع. ۴. **KillerCoda:** پلتفرم مرورگر-محور رایگان برای تمرین تعاملی سناریوهای آزمون CKA و CKS.

—

۴. مراجع و پیوندهای تکمیلی

مرجع تخصصی	پیوند رسمی
مستندات رسمی کوبرنتیز	kubernetes.io/docs
مخزن کدها و مانیفست‌های کتاب نایجل پولتون	github.com/nigelpoulton/thekubernetesbook
چشم‌انداز اکوسیستم ابری CNCF	landscape.cncf.io
استانداردهای امنیت کلاستر CIS	cisecurity.org/benchmark/kubernetes
واژه‌نامه باز بومی ابری (CNCF Glossary)	glossary.cncf.io

💡 ترفند و تجربه عملیاتی: سخن پایانی

کوبرنتیز یک ابزار ساکن نیست؛ یک زبان مشترک در معماری نرم‌افزار مدرن است. با پایان این کتاب، شما کلید تسخیر پیچیده‌ترین سامانه‌های توزیع‌شده را در دست دارید. به سوی کلاسترهای خود گام بردارید و با شجاعت آن‌ها را بسازید، بسنجید و ارتقا دهید!